MODULE 00 · EXAM STRATEGY - ALL DOMAINS

AWS Certified Developer – Associate

DVA-C02. Eleven hours, all four domains at their published weights, 184 worked scenarios.

EXAM **DVA-C02** MODULES **35** SCENARIOS **184**

admin@sthithapragna.com

Almost nobody fails this exam because a service is unfamiliar. They fail because all four options would work, and one clause decides.

The exam

	VALUE
Questions	65 total. 50 scored, 15 unscored and not identified
Time	130 minutes. Exactly 2 minutes per question
Pass mark	720 of 1,000, scaled. Not a percentage
Scoring	Compensatory. No per-domain pass mark
Question types	Multiple choice, and multiple response with no partial credit
Guessing	No penalty. Never leave a question blank
Cost and validity	150 USD. Valid 3 years. 14-day wait to retake

READ THIS TWICE

If English is not your first language, request the ESL plus 30 minutes accommodation in your AWS Certification account BEFORE you schedule. Most who qualify never ask.

The four domains

DOMAIN	WEIGHT AND WHAT IT ASKS
1. Development with AWS Services	32%. Can you build it with the right services?
2. Security	26%. Can you make it reachable only by who should reach it?
3. Deployment	24%. Can you ship it repeatedly and safely?
4. Troubleshooting and Optimization	18%. Can you tell what is wrong when it breaks?

ON THE EXAM

The course matches these weights deliberately: 12 modules on Domain 1, 5 on Domain 2, 5 on Domain 3, 4 on Domain 4.

How the course is arranged

MODULES	WHAT THEY COVER
01 to 02	Foundations: the mental model, and how an SDK call actually works
03 to 14	Domain 1. Lambda, DynamoDB, S3, API Gateway, messaging, containers
15 to 19	Domain 2. IAM, roles, Cognito, KMS, secrets
20 to 24	Domain 3. CloudFormation, SAM and CDK, CI/CD, CodeDeploy, versions
25 to 28	Domain 4. CloudWatch, X-Ray, troubleshooting, cost
29 to 31	Synthesis. Confusion pairs, how questions are written, the revision gap
32 to 34	Three practice exams. 60 questions, no repeats

ON THE EXAM

Watch in order. Later modules assume vocabulary earlier ones established, and the synthesis modules draw on everything before them.

What is in each module

- A **core idea** stated first, so you know what the module is really about.
- Teaching slides, with the examinable facts marked and the limits stated precisely.
- A **tiebreaker** slide wherever two services or features are easily confused.
- **Worked scenarios** at the end, in the shape the exam uses.
- A recap naming the five things to carry forward.

> ON THE EXAM
>
> There are 50 tiebreaker slides across the course. Those are the pairs the exam deliberately puts side by side, and module 29 gathers them.

The scenario convention

- **184 worked scenarios:** 124 in the teaching modules, 60 across three practice exams.
- **Pause and commit to an answer** before the walkthrough. A guess you did not commit to teaches nothing.
- Every walkthrough explains **all four options**, not only the correct one.
- Each ends with **The signal**: the exact clause that decided the answer.
- Several include a deliberate **near miss**: correct engineering, wrong for the scenario.

> READ THIS TWICE
>
> Understanding why a distractor fails is what transfers to an unseen question. Memorising a correct answer does not.

What changed under this exam

- Exam guide **v2.1** was published on 12 December 2024. AWS has not stopped shipping since.
- This course flags **22 specific changes** at the point where each one is taught.
- Each note states the **examinable answer first**, then what is true now, with a date.
- Some answers the exam still marks correct describe things you can no longer buy.
- **Module 31** gathers all of them into one timeline.

> ON THE EXAM
>
> S3 Select has been closed to new customers since July 2024 and remains a correct exam answer. That gap is why the notes exist.

How to work through it

1. Watch modules 01 to 31 in order. Pause and commit at every scenario.
2. Take practice exam 1 and score it **by domain**, not just overall.
3. Review every wrong answer until you can say why all four options are right or wrong.
4. Take exam 2. Leave several days before exam 3.
5. Rewatch **Confusion Pairs** and **The Revision Gap** the night before you sit.

> ON THE EXAM
>
> Recall after a gap predicts a pass far better than recall immediately after watching. The days between exams 2 and 3 are doing real work.

Reading your practice scores

- **17 or more out of 20, consistently:** you are ready to book.
- **14 to 16:** passing territory, with a domain or two to firm up.
- **Below 14:** average your domain scores and revisit the weakest two modules.
- A domain weak in **all three** exams is a real gap. Weak in one is sampling noise.
- Any answer you got right but could not justify is a guess, not knowledge.

> ON THE EXAM
>
> Twenty questions is a small sample. Comparing the three exams by domain tells you far more than any single total.

Who this is for

- AWS describes the candidate as having **one or more years** of development on AWS.
- You should be comfortable in **at least one programming language**. Snippets are mostly Python.
- No prior AWS certification is required, and none is assumed here.
- You do not need to have used every service. Each is introduced before it is tested.
- If you are coming from servers rather than serverless, **module 01 matters most**.

> ON THE EXAM
>
> The most common background gap is not technical. It is the habit of thinking in machines rather than services, which module 01 addresses directly.

What this exam does not ask

- It does not ask you to **write code**. You read snippets and say what they do.
- It does not ask for **pricing figures**, only relative cost.
- It does not test **console click paths** or CLI flags.
- It does not go deep on **networking**. VPCs appear as constraints, not as subjects.
- It does expect **limits, defaults and which service fits**, repeatedly.

> ON THE EXAM
>
> People routinely over-prepare on networking and under-prepare on limits. Roughly thirty numbers decide a meaningful share of the questions.

Before you sit

- Check the **in-scope service list** in the current exam guide before you book.
- Request the **ESL accommodation** before scheduling if it applies to you.
- Decide on **testing centre or online proctored** early. Online has room requirements.
- Two minutes per question. Flag at two and a half and move on.
- **Never leave a question blank.** There is no penalty for a wrong answer.

> ON THE EXAM
>
> Module 30 covers exam technique in full: reading order, the qualifier words, and the six kinds of wrong answer.

Every scenario in this course is original.
None of it is a reproduced question bank.

MODULE 01 · EXAM STRATEGY - ALL DOMAINS

The Developer's Mental Model

Eight ideas that make the rest of this course make sense – and that most exam questions quietly assume.

TYPE Foundations MODULE 1 of 34 SCENARIOS 2

admin@sthithapragna.com

You are not renting machines. You are **calling services**, and the services are already running.

admin@sthithapragna.com

The eight ideas

IDEA	WHAT IT MEANS FOR YOU
Everything is an API	Anything the console does, your code can do
Identity is the perimeter	Access is decided by IAM, not by network location
Managed means operated, not free	AWS runs it, and you still configure and pay for it
Compute is stateless	Anything you need later must be written somewhere
Everything fails	Design for retry, and therefore for idempotency
Consistency has a price	Strong costs more and is not always needed
Limits exist everywhere	Every service throttles, and quotas are per account and Region
Loose coupling is the default	Components communicate through queues, topics and events

READ THIS TWICE

Worth screenshotting now and revisiting at the end. Every module in this course is an application of one or more of these.

One: everything is an API

- Every AWS service is an **HTTPS API**. The console is a client of it, like your code.
- So anything you can do by clicking, you can do from a script or a template.
- **CloudTrail records those API calls**, which is why it can tell you who changed what.
- It also means infrastructure can be code, because creating a resource is just a call.
- The exam almost never asks about the console. It asks what your code or template does.

> ON THE EXAM
>
> This is why 'do it in the console' is rarely the intended answer. The console is for looking; the API is for building.

Two: identity is the perimeter

- There is no inside. A request from your own Lambda function is authenticated like any other.
- **Every API call is signed**, and every call is authorised against IAM policy.
- So 'it is in our VPC' grants nothing. A role and a policy do.
- Roles give **temporary credentials**, which is why long-lived keys are almost always wrong.
- Security is 26% of this exam, and nearly all of it flows from this idea.

> ON THE EXAM
>
> The habit worth building now: when something cannot access something else, ask which identity is making the call before you look at anything else.

Three: managed means operated

- AWS handles the **undifferentiated** work: patching, replacement, replication, scaling machinery.
- You still choose the **configuration**, and the configuration is what the exam tests.
- AWS runs DynamoDB. You choose the partition key, and a bad one still throttles.
- AWS runs Lambda. You choose the memory, timeout and concurrency.
- **Managed does not mean you cannot misuse it.** Most exam scenarios are misconfiguration.

> ON THE EXAM
>
> This is the shape of nearly every question: the service is working correctly, and the configuration is wrong. Look at the settings before the service.

Four: compute is stateless

- A Lambda execution environment, a container or an instance can disappear at any moment.
- **Anything you need later must be written somewhere durable.** Memory and local disk are not.
- Sessions go to DynamoDB or ElastiCache, not into process memory.
- Uploads go to S3, not to a local directory.
- Reuse across invocations is an **optimisation** you may benefit from, never a guarantee.

> ON THE EXAM
>
> The subtle version: Lambda often reuses an environment, so cached state usually survives. Code that depends on that works in testing and fails unpredictably later.

Five: everything fails

- Networks drop, services throttle, and instances are replaced. This is normal operation.
- AWS SDKs **retry automatically**, so your code will be called more than once eventually.
- **That makes idempotency a requirement, not a refinement.**
- Queues deliver at least once. Events can arrive twice. Timeouts leave you genuinely unsure.
- Design so that processing the same message twice is harmless.

> ON THE EXAM
>
> The timeout is the case that forces this: the operation may have succeeded and the response been lost, so a retry without protection charges the customer twice.

Six: consistency has a price

- **Durability** is whether data survives. **Availability** is whether you can reach it now.
- **Strongly consistent** reads always see the latest write, and cost more.
- **Eventually consistent** reads may briefly lag, and cost half as much in DynamoDB.
- Most reads tolerate a lag of milliseconds. A balance check does not.
- Distributed replicas are why the choice exists. It is not an implementation detail.

> ON THE EXAM
>
> The habit: ask whether this particular read would be harmed by data a few milliseconds old. Usually not, and the default is the cheaper one for a reason.

Seven: limits exist everywhere

- Every service has quotas, and every service throttles when you exceed them.
- Most quotas are **per account and per Region**, so workloads share them.
- One noisy workload can therefore starve another in the same account.
- Some quotas are **adjustable** by request. Some are hard and cannot be raised.
- Hard numbers decide exam questions: 15 minutes, 29 seconds, 400 KB, 5 GB.

> ON THE EXAM
>
> Knowing roughly thirty numbers is worth more than knowing a hundred features, because a stated size or duration often eliminates options outright.

Eight: loose coupling is the default

- Components communicate through **queues, topics and events** rather than direct calls.
- A queue lets a slow consumer fall behind without the producer failing.
- A topic lets you add a consumer without changing the producer.
- That is why the integration services carry so much weight on this exam.
- Direct synchronous calls between components are usually the wrong answer.

> ON THE EXAM
>
> When two options both work, the more loosely coupled one is usually intended. That heuristic is genuinely predictive on this exam.

Regions and Availability Zones

- A **Region** is a geographic area. Resources and most quotas are Regional.
- An **Availability Zone** is one or more isolated data centres within a Region.
- Managed services such as S3, DynamoDB and Lambda span AZs for you automatically.
- Data does **not** move between Regions unless you configure it to.
- A few services are global: **IAM, CloudFront and Route 53**, with endpoints in us-east-1.

> ON THE EXAM
>
> For a developer the practical point is that Region is part of a resource's identity. The same bucket name in another Region is a different bucket.

The account as a boundary

- An AWS account is the hardest boundary AWS offers, for security, quotas and billing.
- Separate accounts per environment is the common pattern, and prevents dev affecting prod.
- Nothing crosses an account boundary **by default**. Every crossing is explicit.
- Cross-account access needs **both** sides to agree: a role or resource policy, and an identity policy.
- Within one account, either side granting access is enough.

> ON THE EXAM
>
> That last pair is worth remembering from the start: same account is OR, cross-account is AND. It resolves a surprising number of questions.

The layers of an AWS application

LAYER	WHAT YOU WILL MEET IN THIS COURSE
Compute	Lambda, ECS and Fargate, Elastic Beanstalk
Storage and data	S3, DynamoDB, RDS and Aurora, ElastiCache
Integration	API Gateway, SQS, SNS, EventBridge, Step Functions
Security	IAM, Cognito, KMS, Secrets Manager
Deployment	CloudFormation, SAM, CDK, CodePipeline, CodeDeploy
Observability	CloudWatch, X-Ray, Application Signals

ON THE EXAM

Six layers, and the exam draws from all six. Recognising which layer a question is about is often the fastest way to eliminate options.

The cost model

- **Pay per use** charges for what you consume, and nothing while idle.
- **Provisioned** charges for capacity whether or not you use it, and is cheaper when steady.
- The exam asks for **relative** cost, never dollar figures.
- Predictability decides between them, not volume.
- In serverless you pay for **time**, so faster is frequently cheaper.

> ON THE EXAM
>
> That last line is genuinely counterintuitive and worth carrying: giving a Lambda function more memory often reduces the bill, because the duration falls further than the rate rises.

Request-response or event-driven?

REQUEST-RESPONSE

The caller waits

- A client needs an answer now
- API Gateway in front of Lambda
- Latency is user-visible
- Failure is returned to the caller
- Bounded by an integration timeout

EVENT-DRIVEN

The caller does not wait

- Work happens after the response
- A queue, topic or event bus
- Latency is absorbed by the buffer
- Failure retried, then a DLQ
- Bounded by the consumer, not the client

THE TIEBREAKER

Most scenarios contain both. The question is which parts genuinely need an answer before the user sees a response, and the answer is usually fewer than you think.

The four domains as a workflow

DOMAIN	THE QUESTION IT ASKS
1. Development, 32%	Can you build it with the right services?
2. Security, 26%	Can you make it accessible only to who should reach it?
3. Deployment, 24%	Can you ship it repeatedly and safely?
4. Troubleshooting, 18%	Can you tell what is wrong when it breaks?

ON THE EXAM

That is a developer's working week in four lines. The weights are not arbitrary: building is the largest, and troubleshooting the smallest but the most diagnostic.

Answering a which-service question

- **Find the constraint**, not the topic. The constraint is usually one clause.
- Duration, size, ordering, consistency, cost, or who must be able to do it.
- **Eliminate on hard limits first.** A stated duration or size often removes options outright.
- Then apply the qualifier: most cost-effective, least operational overhead.
- If two options still both work, you have missed a clause. Reread rather than guessing.

> ON THE EXAM
>
> Every worked scenario in this course names the deciding clause explicitly. That is deliberate, and module 30 makes the technique explicit.

What this exam does not ask

- It does not ask you to **write code**. Snippets appear, usually Python with boto3.
- It does not ask for **pricing figures**, only relative cost.
- It does not test **console click paths** or CLI flags.
- It does not go deep on **networking**. VPCs appear as constraints, not as subjects.
- It does expect **limits, defaults and which service fits**, repeatedly.

> ON THE EXAM
>
> People routinely over-prepare on networking and under-prepare on limits. The last line is where revision time actually pays.

How this course is arranged

- **Modules 2 to 14** cover Domain 1: the SDK, then the services you build with.
- **15 to 19** cover security. **20 to 24** cover deployment. **25 to 28** cover troubleshooting.
- **29 to 31** are synthesis: confusion pairs, question anatomy, and what changed since the guide.
- **32 to 34** are three practice exams, sixty questions with no repeats.
- Every teaching module ends with worked scenarios. Pause and commit before each one.

> ON THE EXAM
>
> The modules build on each other, so the order is worth following. Later modules assume the vocabulary earlier ones established.

Two scenarios. Neither needs a service you have met yet - only the ideas from this module.

> A team is moving a web application to AWS. It currently stores uploaded files on the web server's local disk and keeps user sessions in the application process's memory. They plan to run it on a fleet that scales in and out with demand.
>
> They report that after scaling, some users lose their session and some uploaded files cannot be found.

What explains both symptoms?

A The instances are stateless, so anything held in memory or on local disk is lost when one is replaced

B The load balancer is misconfigured and routing to unhealthy instances

C The instances need larger disks to hold the uploaded files

D Session data must be replicated between instances by the application

◆ PAUSE AND DECIDE

What explains both symptoms?

A Compute is stateless

Both symptoms have one cause. Scaling in terminates instances, taking their memory and local disk with them. Sessions belong in DynamoDB or ElastiCache, and files in S3.

B Load balancer misconfigured

Would cause failed requests, not selectively missing sessions and files. It also does not explain why the symptoms follow scaling events.

C Larger disks

The files are not too large, they are on an instance that no longer exists. A bigger disk on a terminated instance helps nobody.

D Replicate sessions between instances

The near miss. It addresses sessions and not files, and it builds a distributed state system rather than using a service designed for it.

THE SIGNAL

One idea explains two symptoms, which is the tell that you have found the cause rather than a cause. Option D fixes half the problem the hard way.

> An order service calls a payment provider synchronously, then an inventory service, then an email service, all within one request. When the email service is slow, order submission times out and customers see failures even though payment succeeded.
>
> Only payment confirmation must be known before responding to the customer.

What is the appropriate change?

A. Increase the timeout on the order service so slow calls complete

B. Retry the email call until it succeeds before responding

C. Respond after payment, and publish an event that inventory and email consume independently

D. Run all three calls in parallel to reduce total latency

◆ PAUSE AND DECIDE

What is the appropriate change?

A Longer timeout

Makes customers wait longer for something they do not need. The coupling that causes the failure is untouched.

B Retry email inline

Makes it worse. The customer now waits through several attempts at the slowest and least important step.

C Respond after payment, publish an event

The scenario states only payment must be known before responding. Everything else becomes an event, so a slow email cannot fail an order.

D Parallel calls

Reduces total latency and keeps the coupling: the response still waits for the slowest of the three, which is the email service.

THE SIGNAL

The deciding clause is 'only payment confirmation must be known before responding'. It tells you exactly where the synchronous boundary belongs.

What to carry forward

- **Identity is the perimeter.** When access fails, ask which identity is calling.
- **Managed means operated, not foolproof.** Most scenarios are misconfiguration.
- **Compute is stateless and everything fails**, so idempotency is a requirement.
- Same account is OR, cross-account is AND. Limits are per account and Region.
- When two options both work, prefer the more loosely coupled one.

> ON THE EXAM
>
> Next: the AWS SDK and the API request lifecycle, which is how every one of these ideas reaches actual code.

MODULE 02 · DOMAIN 1 - DEVELOPMENT WITH AWS SERVICES

The AWS SDK and the API Request Lifecycle

What actually happens between your function call and the AWS service - and the five behaviours that surprise people.

DOMAIN 1 of 4 EXAM WEIGHT 32% MODULE 2 of 34

admin@sthithapragna.com

Every AWS service is an **HTTPS API**. The SDK is a convenience layer over it, and the convenience is where the surprises live.

What happens on one API call

01

Resolve

Work out the Region, the endpoint and the credentials to use.

02

Sign

Build a SigV4 signature over the request, using the secret key.

03

Send

Transmit over HTTPS and wait for a response.

04

Handle

Parse the response, or classify the error and decide whether to retry.

ON THE EXAM

Four stages, and a failure can come from any of them. Knowing which stage failed narrows a problem faster than reading the stack trace.

The SDKs

- Official SDKs for **Python, JavaScript, Java, .NET, Go, Ruby, PHP, C++, Rust, Kotlin and Swift**.
- All are generated from the same service definitions, so behaviour is consistent across them.
- Python's is **boto3**, built on **botocore**, which holds the API definitions as JSON.
- That generated design is why a new AWS API appears in the SDK almost immediately.
- Exam code snippets are usually **Python with boto3**, with some Java and JavaScript.

> ON THE EXAM
>
> You are not expected to write code in the exam. You are expected to read a snippet and say what it does or why it fails.

Constructing a client

- A **client** is created per service, and is where region and credentials are resolved.
- **Creating a client is not free.** It loads definitions and resolves credentials.
- So create clients **once**, outside a Lambda handler, and reuse them.
- Clients are generally thread-safe and designed to be shared.
- Creating one per invocation is a measurable and entirely avoidable cost.

> ON THE EXAM
>
> This is the single most repeated piece of Lambda advice, and it originates here: client construction is the expensive part, not the API call.

Which Region

- An explicit **region** argument on the client wins over everything.
- Then **AWS_REGION**, which Lambda sets automatically for you.
- Then the **region** setting in the shared config file, under the active profile.
- With none of those, the SDK raises an error rather than guessing.
- **IAM, CloudFront and Route 53 are global**, and their endpoints live in us-east-1.

> ON THE EXAM
>
> The last line explains a recurring confusion: a CloudFront or ACM call that must be made in us-east-1 regardless of where your application runs.

Which credentials

- The SDK walks a **provider chain** and uses the first source that returns credentials.
- Roughly: environment variables, then a web identity token, then the shared files, then the container endpoint, then the instance profile.
- Correctly written code therefore needs **no credential configuration at all**.
- Passing keys explicitly to a client constructor **bypasses the chain**, and is almost always wrong.
- Module 16 covers this properly, including why order matters.

> ON THE EXAM
>
> The chain is why the same code runs on a laptop, in Lambda and on EC2 without change. That portability is the whole point.

Configuration precedence

- **Explicit client configuration in code** beats everything else.
- Then **environment variables**, such as AWS_REGION or AWS_MAX_ATTEMPTS.
- Then the **shared config file**, under the profile named by AWS_PROFILE.
- A stray environment variable silently overrides your config file, with no warning.
- The same order applies to region, credentials, retries and endpoint settings alike.

> ON THE EXAM
>
> One order governs everything: code, environment, file. When behaviour differs between two machines running the same code, this is the first thing to check.

Signing

- Every request is signed with **SigV4**, over the method, URI, headers, payload hash and timestamp.
- **The secret key is never transmitted.** Only the derived signature travels.
- The signature is Region-specific and service-specific, which is why both must be resolved first.
- A clock more than about **5 minutes** out causes every call to fail on signature.
- The SDK does all of this. You sign manually only when calling AWS without an SDK.

> ON THE EXAM
>
> The clock skew case is worth remembering: a container or device with a wrong clock fails every call, and the error mentions the signature rather than the time.

Pagination

- Many list and query operations return a **truncated** result set, not everything.
- The response carries a **NextToken**, or IsTruncated, or LastEvaluatedKey depending on the service.
- **Ignoring it silently returns only the first page.** No error is raised.
- S3 ListObjectsV2 caps at 1,000 keys. DynamoDB Query and Scan cap at 1 MB.
- Code that works in development and misses records in production is usually this.

> READ THIS TWICE
>
> This is the most dangerous default in the SDK, because it fails silently and correctly. You get a valid response containing part of the answer.

Paginators

- A **paginator** wraps a truncated operation and iterates the whole result set for you.
- In boto3: **client.get_paginator('list_objects_v2')**, then iterate **paginate()**.
- Other SDKs offer the same idea as async iterators or auto-paginating methods.
- It handles the token, the loop and the termination condition. You handle the items.
- A manual loop works, and is one forgotten condition away from an infinite loop or a silent truncation.

> ON THE EXAM
>
> Paginators exist because the manual loop is easy to write and easy to write slightly wrong. Prefer them wherever the SDK offers one.

Waiters

- Many AWS operations return immediately while the work continues asynchronously.
- Starting an EC2 instance returns before it is running. Creating a table returns before it is ACTIVE.
- A **waiter** polls until a resource reaches a state, then returns.
- In boto3: **client.get_waiter('instance_running').wait(...)**.
- Waiters have sensible built-in intervals and a maximum attempt count, and are configurable.

> ON THE EXAM
>
> The alternative is a hand-written sleep-and-poll loop, which is where fixed delays and missing timeouts creep in. A waiter has both already.

Reading errors

- Service errors surface as exceptions carrying an **error code** and a message.
- In boto3, **ClientError** with **e.response['Error']['Code']** holding the code.
- Modelled exceptions exist too, such as **ConditionalCheckFailedException**.
- **Catch on the error code**, not on the message. Messages change; codes are contractual.
- The response metadata also carries the **request ID**, which AWS Support will ask for.

> ON THE EXAM
>
> Matching on message text is fragile and common. The error code is part of the API contract and is what you should branch on.

Retries, briefly

- The SDK **automatically retries** throttling and transient errors, with exponential backoff and jitter.
- So an error you actually see has usually been retried two or three times already.
- Wrapping SDK calls in your own retry loop **multiplies** the attempts rather than adding to them.
- Configure the SDK instead: retry mode and max attempts are both settings.
- Module 27 covers which errors are retryable and why jitter matters.

> ON THE EXAM
>
> Worth knowing from the start: the SDK is already retrying. A failure that reaches your code is not a one-off blip.

Idempotency tokens

- A retried creation call can create the resource **twice** if the first response was lost.
- Many create operations accept a **ClientToken** or similar idempotency parameter.
- AWS recognises a repeat with the same token and returns the original result.
- Some SDKs generate one automatically. Others require you to supply it.
- Without one, a timeout leaves you genuinely unsure whether the resource exists.

> ON THE EXAM
>
> The timeout case is the reason this exists. You never learn whether the request succeeded, so a blind retry is how duplicates get created.

Client or resource interface?

CLIENT

The full API

- One method per API operation
- Every service, every operation
- Returns dictionaries
- Paginators and waiters live here
- The one to learn

RESOURCE, BOTO3 ONLY

Object-oriented wrapper

- Objects with attributes and methods
- Only some services, some operations
- Reads more naturally
- In maintenance mode, no new features
- Reach for meta.client when it lacks something

THE TIEBREAKER

The resource interface is Python-only and no longer being extended. Learn the client interface, which exists in every SDK and covers everything.

Endpoints

- The SDK builds a regional endpoint from the service and Region: **service.region.amazonaws.com**.
- A custom **endpoint URL** can be set, for VPC endpoints, FIPS, dualstack or local testing.
- **Interface VPC endpoints** keep traffic on the AWS network without internet egress.
- **FIPS endpoints** exist where a compliance regime requires validated cryptography.
- A custom endpoint changes where the request goes, not what it is signed for.

> ON THE EXAM
>
> Overriding the endpoint is how local emulators are used in testing. The request is still signed as though it were going to the real service.

The five that surprise people

- **Results are paginated**, and ignoring the token silently returns one page.
- **Operations return before the work finishes**, which is what waiters are for.
- **The SDK is already retrying**, so your own loop multiplies attempts.
- **Environment variables beat your config file**, silently.
- **Client construction is the expensive part**, not the API call.

> ON THE EXAM
>
> Five defaults, and each produces behaviour that looks like a bug elsewhere. Knowing them is most of what this module is for.

Each of these is a default doing exactly what it is documented to do.

> A script lists the objects in an S3 bucket and reports 1,000 objects. The console shows the bucket contains roughly 47,000. The script raises no errors and the same code has been in production for months.
>
> It calls list_objects_v2 once and iterates the Contents array.

What is wrong?

A The IAM policy limits how many objects may be listed

B The script needs a longer timeout to retrieve all the objects

C The response is truncated at 1,000 keys and the continuation token is being ignored

D S3 eventual consistency means recent objects are not yet listed

◆ PAUSE AND DECIDE

What is wrong?

A **IAM limits the count**

IAM grants or denies an action. It has no concept of allowing a limited number of results.

B **Longer timeout**

The call completes successfully and quickly. Nothing is timing out.

C **Truncated at 1,000 keys**

ListObjectsV2 returns at most 1,000 keys per call, with IsTruncated and a continuation token. Ignoring them returns exactly one page, silently.

D **Eventual consistency**

S3 has had strong read-after-write consistency since December 2020, and 46,000 missing objects is not a consistency window.

THE SIGNAL

Exactly 1,000 is the tell. A round number matching a documented page size is a pagination limit, not a coincidence.

> A deployment script creates a DynamoDB table and immediately writes seed data into it. The write intermittently fails with ResourceNotFoundException, more often on larger tables.
>
> The team has added a fixed 10-second sleep after table creation, which usually works.

What is the correct fix?

A Use the table_exists waiter after CreateTable, before writing

B Increase the sleep to 60 seconds

C Catch the exception and retry the write in a loop

D Create the table in a separate script run earlier

◆ PAUSE AND DECIDE

What is the correct fix?

A The table_exists waiter

CreateTable returns while the table is still CREATING. A waiter polls until it is ACTIVE, with a sensible interval and a maximum attempt count already built in.

B Longer sleep

Trades correctness for a guess. Too short still fails, too long wastes time on every run, and neither is reliable.

C Catch and retry

Rebuilds a waiter by hand, and a naive loop has no backoff and no maximum, so a genuinely missing table loops forever.

D Separate earlier run

Avoids the race by splitting the script, and the same race returns whenever the two runs are close together.

THE SIGNAL

A fixed sleep is the tell that a waiter is missing. The operation returned before the work finished, which is exactly the case waiters exist for.

> A developer runs a deployment script that works on their colleague's machine but fails on theirs, writing to the wrong AWS account. Both have identical code and an identical ~/.aws/config naming the same profile.
>
> The developer previously exported credentials into their shell to debug an unrelated issue.

What is the most likely cause?

A The shared config file is corrupt and must be recreated

B The two machines are running different SDK versions

C The profile in the config file names a different Region

D Exported environment variables take precedence over the shared config file

◆ PAUSE AND DECIDE

What is the most likely cause?

A **Corrupt config file**

The scenario states the config files are identical, and a corrupt file would produce a parse error rather than silently using another account.

B **Different SDK versions**

Precedence order is consistent across SDK versions and languages. It is not a version-dependent behaviour.

C **Different Region**

A wrong Region writes to the wrong Region, not the wrong account. The symptom names the account.

D **Environment variables win**

Code beats environment beats file. Exported credentials silently override the profile, and remain in the shell until it is closed.

THE SIGNAL

The last sentence is the planted clue. Exported credentials outlive the task they were exported for, and nothing warns you they are being used.

> A provisioning service creates infrastructure on customer request. Occasionally two identical resources appear for one request. Logs show the creating API call timed out from the client's perspective, and the SDK retried it.
>
> The operation supports a ClientToken parameter, which the code does not currently supply.

What should be changed?

A Disable SDK retries for this call by setting max attempts to 1

B Supply a ClientToken derived from the customer request, so a retry is recognised as a repeat

C Increase the client timeout so the call does not time out

D Check whether the resource exists before creating it

◆ PAUSE AND DECIDE

What should be changed?

A Disable retries

Removes duplicates by removing resilience. A genuinely transient failure now fails the customer request outright.

B Supply a ClientToken

AWS recognises a repeat carrying the same token and returns the original result rather than creating a second resource. Retries stay safe.

C Longer timeout

Reduces how often it happens without removing the case. A timeout can still occur, and the retry still duplicates.

D Check before creating

A check-then-act race. Two concurrent requests can both see nothing and both proceed, which is the bug idempotency tokens remove.

THE SIGNAL

The scenario names the parameter, which is the giveaway. Options A and C both make retries less likely rather than making them safe.

What to carry forward

- **Every service is an HTTPS API.** The SDK resolves, signs, sends and handles.
- **Results are paginated.** Use a paginator, and never assume one call returned everything.
- **Operations return before the work finishes.** Use a waiter rather than a sleep.
- Precedence is code, then environment, then file, for every setting alike.
- Create clients once and reuse them. Construction is the expensive part.

> ON THE EXAM
>
> Next: AWS Lambda, which is where most of this course lives - and where every one of these behaviours shows up again.

MODULE 03 · DOMAIN 1 - DEVELOPMENT WITH AWS SERVICES

Lambda I – The Execution Model

How an execution environment is born, reused and destroyed - and why almost every Lambda question turns on that.

DOMAIN 1 of 4 EXAM WEIGHT 32% MODULE 3 of 34

admin@sthithapragna.com

Lambda does not run your function. It builds an **execution environment**, and then runs your function inside it, possibly many times.

The three phases of an execution environment

01

INIT

Download code, start the runtime, run your initialisation code. Only on a cold start.

02

INVOKE

Run the handler. Repeats for every request the warm environment serves.

03

SHUTDOWN

Environment is reclaimed. Lambda decides when, not you.

ON THE EXAM

INIT happens once per environment. INVOKE happens many times. Knowing which phase code runs in is the most useful Lambda skill you can have.

What actually happens during INIT

- **Extension init.** Any external extensions start first and register themselves.
- **Runtime init.** The language runtime bootstraps. Node, Python, Java and so on.
- **Function init.** Your module-level code runs. Imports, clients, config, connections.
- The whole INIT phase gets up to **10 seconds** before Lambda restarts it.
- A cold start is INIT plus the first INVOKE. Only INIT is avoidable.

> ON THE EXAM
>
> Heavy imports and eager client construction are function init work. That is where cold start latency is won or lost.

The most important line in your code

OUTSIDE THE HANDLER

Runs once per environment

- Imports and module-level code
- SDK clients and database connections
- Config and secrets fetched at startup
- Expensive setup you want amortised
- Billed as INIT duration

INSIDE THE HANDLER

Runs on every single request

- Per-request business logic
- Anything that depends on the event
- Anything that must not be shared
- Billed as INVOKE duration, every time
- Runs thousands of times, not once

THE TIEBREAKER

If it is the same for every request, put it outside. If it depends on the event, it must go inside. Exam questions about latency or cost very often hinge on code sitting on the wrong side of this line.

What survives between invocations

- **Global and static variables.** Values set during INIT are still there.
- **Open connections.** Database and HTTP connections stay alive and reusable.
- **Files written to /tmp.** They persist for the life of the environment.
- **Background processes are frozen**, not killed, and resume on the next invoke.
- None of this is guaranteed. Lambda may hand you a fresh environment at any time.

> ON THE EXAM
>
> Reuse is an optimisation you may exploit but never depend on. Correct code works identically warm or cold.

The traps this creates

- **Leaked state.** A global mutated by request A is visible to request B.
- **Cached credentials or tenant context** bleeding across users. A real security bug.
- **/tmp filling up** over a long-lived environment until writes start failing.
- **Concurrency multiplies everything.** 200 concurrent requests means 200 environments, each with its own globals, its own /tmp and its own connections.
- A connection pool of 10 in code becomes 2,000 connections at 200 concurrency.

> ON THE EXAM
>
> The fourth point is the one that catches people. Environment reuse and concurrency pull in opposite directions.

Ephemeral storage: the /tmp directory

PROPERTY	VALUE
Path	/tmp - the only writable location in the filesystem
Default size	512 MB
Configurable	512 MB to 10,240 MB, in 1 MB increments
Cost	Free up to 512 MB, charged per GB-second above that
Lifetime	Lives and dies with the execution environment
Shared?	No. Each concurrent environment has its own /tmp

ON THE EXAM

Scratch space for one invocation, sized up to 10 GB. Not shared storage, not durable storage, not a cache you can rely on.

/tmp or Amazon EFS?

CHOOSE /TMP WHEN

Scratch space for one invocation

- Temporary files nobody else needs
- Up to 10 GB is enough
- You want zero extra configuration
- No VPC required
- Fastest option - it is local disk

CHOOSE EFS WHEN

Shared or persistent state

- Several functions need the same files
- You need more than 10 GB
- Data must outlive the environment
- Requires VPC configuration and a mount target
- Adds cold start and networking overhead

THE TIEBREAKER

Temporary and private means /tmp. Shared or persistent means EFS. If the scenario says the files are not needed afterwards, /tmp is almost always the answer.

Memory is the only performance dial

- **128 MB to 10,240 MB**, in 1 MB increments.
- **CPU scales in proportion to memory.** There is no separate CPU setting.
- At **1,769 MB** a function gets the equivalent of one full vCPU.
- Network bandwidth scales with memory too.
- Under 1,769 MB you have a fraction of a vCPU, so multi-threading buys you nothing.

> ON THE EXAM
>
> More memory can make a function cheaper. If doubling memory more than halves duration, the bill goes down.

arm64 and Graviton

- Lambda runs on **x86_64** or **arm64**, the latter backed by AWS Graviton.
- arm64 is typically **cheaper per GB-second** and often faster for the same work.
- Switching is a configuration change for interpreted runtimes like Python and Node.js.
- Native binaries and compiled dependencies must be rebuilt for the target architecture.
- Layers and container images are architecture-specific. They must match the function.

> ON THE EXAM
>
> When a question asks for cheaper compute with no code rewrite, arm64 is a strong candidate - unless native dependencies are mentioned.

What you actually pay for

- **Requests.** A flat charge per invocation, regardless of duration.
- **Duration in GB-seconds.** Memory configured, multiplied by time billed.
- **Ephemeral storage** above the free 512 MB, also per GB-second.
- Billed in **1 ms increments**, so shaving milliseconds genuinely pays at scale.
- **CloudWatch Logs is billed separately** and is a real cost on chatty functions.

> ON THE EXAM
>
> GB-seconds is the number that matters. Memory x duration, which is why the two dials trade against each other.

The hard limits that decide questions

LIMIT	VALUE
Function timeout	900 seconds, that is 15 minutes. Default 3 seconds
Memory	128 MB to 10,240 MB
Ephemeral storage	512 MB to 10,240 MB
Environment variables	4 KB total, all variables combined
Layers per function	5
Deployment package	50 MB zipped upload, 250 MB unzipped including layers
Container image	10 GB
Payload, synchronous	6 MB request and 6 MB response
Payload, asynchronous	256 KB

ON THE EXAM

These are hard limits. No support ticket raises them. Concurrency is different - that is a soft limit and can be increased.

Timeout behaviour

- **Default is 3 seconds.** Easy to forget, and a common cause of mystery failures.
- **Maximum is 900 seconds.** Fifteen minutes. Not negotiable.
- On timeout the invocation is killed. No graceful shutdown, no cleanup handler.
- Set the timeout slightly above expected duration, not at the maximum.
- A timeout downstream of API Gateway still costs you the full billed duration.

> ON THE EXAM
>
> Work longer than 15 minutes means Step Functions, ECS, Fargate or AWS Batch. Module 13 covers choosing between them.

Environment variables

- **4 KB total** across every variable on the function, combined.
- Encrypted at rest by default with an AWS managed KMS key.
- Use a **customer managed key** when you need to control or audit access.
- Values are **visible in the console** to anyone with GetFunctionConfiguration.
- So they are fine for config, and wrong for secrets.

> ON THE EXAM
>
> Endpoint URLs and table names belong here. Database passwords belong in Secrets Manager. Module 19 covers the split properly.

Lambda layers

- **Up to 5 layers** per function, extracted to **/opt** at startup.
- Layers count towards the **250 MB unzipped** package limit. They do not bypass it.
- Versioned and immutable. A function pins a specific layer version ARN.
- Shareable across accounts, or made public, via layer permissions.
- Layers do not reduce cold start time. The content still has to be downloaded.

> ON THE EXAM
>
> Layers are for sharing and for keeping deployment packages small. They are not a workaround for the size limit.

ZIP archive or container image?

CHOOSE ZIP WHEN

The default

- Package fits inside 250 MB unzipped
- You want the simplest deployment
- You want to use layers
- Generally faster cold starts
- Editable in the console for small functions

CHOOSE CONTAINER IMAGE WHEN

Big, or already containerised

- Up to 10 GB uncompressed
- Large ML models or heavy dependencies
- Your team already builds images
- Stored in Amazon ECR, not S3
- Layers are not supported

THE TIEBREAKER

The 250 MB ceiling is the usual decider. Above it, container images are the only option - and you give up layers to get there.

Lambda extensions

- **Internal extensions** run inside the runtime process, altering how it starts.
- **External extensions** run as separate processes alongside your function.
- They start before INIT and can keep running after the handler returns.
- Typical uses: monitoring agents, secret caching, configuration fetching.
- They share the function's memory, CPU and cost. They are not free.

> ON THE EXAM
>
> Extensions are the supported way to run an agent inside Lambda without embedding it in your handler code.

The Parameters and Secrets extension

- AWS-provided layer that caches **Secrets Manager** and **Parameter Store** values.
- Your function calls a **localhost HTTP endpoint** instead of the AWS API.
- Cuts latency and cuts API calls, which cuts cost and throttling risk.
- Cache TTL is configurable, so rotated secrets are picked up.
- Without it, fetching a secret inside the handler means an API call on every request.

> ON THE EXAM
>
> When a scenario complains about latency or cost from repeated Secrets Manager calls, this extension is very often the intended answer.

Execution role or resource-based policy?

EXECUTION ROLE

What the function may do

- An IAM role the function assumes
- Grants access to DynamoDB, S3, KMS and so on
- Always required, even for logging
- Credentials appear in the environment automatically
- Outbound permissions

RESOURCE-BASED POLICY

Who may invoke the function

- A policy attached to the function itself
- Lets S3, EventBridge or API Gateway call it
- Enables cross-account invocation
- Limited to 20 KB
- Inbound permissions

THE TIEBREAKER

Outbound is the execution role. Inbound is the resource-based policy. An 'access denied' when a service tries to trigger your function is almost always a missing resource-based policy.

How a function logs

- Everything written to stdout or stderr lands in **/aws/lambda/<function-name>**.
- **The execution role must allow it.** No permission means no logs, and no error either.
- Advanced logging controls set the **log format** to JSON and a **log level**, without code changes.
- JSON format lets Logs Insights query fields directly instead of parsing text.
- You can send logs to a **custom log group**, useful for separating environments.

> ON THE EXAM
>
> A function that appears to produce no logs at all is usually a missing CreateLogStream or PutLogEvents permission, not a broken function.

Lambda inside a VPC

- By default a function runs **outside your VPC** with internet access.
- Attach it to a VPC to reach private resources like RDS or ElastiCache.
- **A VPC-attached function loses internet access** unless you add a NAT gateway.
- For AWS services, **VPC endpoints** are cheaper and faster than a NAT gateway.
- Hyperplane ENIs are shared across environments, so VPC attachment no longer adds much cold start cost.

> ON THE EXAM
>
> The classic failure: a developer attaches a function to a VPC to reach RDS, and its calls to S3 or a public API start timing out.

Reading the REPORT line

FIELD	WHAT IT TELLS YOU
Duration	How long the handler actually ran
Billed Duration	What you pay for. Now includes Init Duration
Memory Size	What you configured
Max Memory Used	What was actually consumed - the right-sizing signal
Init Duration	Cold start work. Present only on a cold start

ON THE EXAM

Max Memory Used far below Memory Size means you are over-provisioned. Init Duration appearing often means cold starts are frequent.

Currency note: INIT is now billed

- Before, functions packaged as **ZIP with a managed runtime** did not pay for INIT on on-demand invocations.
- From **1 August 2025**, INIT duration is billed across every configuration.
- Provisioned Concurrency, custom runtimes and container images always billed it.
- AWS reports INIT occurs in under 1% of invocations for most workloads.
- Mitigations: smaller packages, SnapStart, or Provisioned Concurrency.

> ON THE EXAM
>
> Older cost material and older practice questions predate this. Cold starts used to be partly free. They are not any more.

Runtimes and deprecation

- Managed runtimes follow the upstream language's support lifecycle.
- After deprecation you **cannot create** functions on that runtime.
- Later, you cannot **update** existing ones either. They keep running, unpatched.
- AWS emails owners and posts dates well ahead. Notices land in Health Dashboard.
- A **custom runtime** via the Runtime API is the escape hatch for unsupported languages.

> ON THE EXAM
>
> The examinable point is the order: create is blocked first, update second, execution never. Existing functions are not switched off.

Powertools for AWS Lambda

- An AWS-maintained library for Python, TypeScript, Java and .NET.
- **Structured logging** in JSON, with the request ID attached automatically.
- **Custom metrics** via embedded metric format, with no extra API call.
- **Tracing** helpers, and an **idempotency** utility backed by DynamoDB.
- Distributed as a Lambda layer, so adding it is a configuration change.

> ON THE EXAM
>
> When a question wants structured logs, custom metrics or idempotency with the least development effort, Powertools is the low-effort answer.

Answer each one **before** the walkthrough plays. Reading a solution you did not commit to teaches almost nothing.

A developer builds a Lambda function that reads from an Amazon RDS for PostgreSQL database. Following best practice, they create the database connection outside the handler so it is reused across invocations. In testing with light traffic everything works and latency is good.

In production the function is invoked heavily and concurrency reaches around 900. The database begins rejecting connections with 'too many clients already', and the function starts failing. The database instance has a max_connections setting of 500.

What explains the failure, and what is the best fix?

A The connection outside the handler is a bug. Move it inside so each invocation closes its own connection

B Each concurrent execution environment holds its own connection, so ~900 environments means ~900 connections. Put Amazon RDS Proxy in front of the database

C Increase the function's memory so connections are established faster

D Increase the RDS instance size so max_connections rises above 900

◆ PAUSE AND DECIDE

What explains the failure, and what is the best fix?

A Move the connection inside the handler

This makes it worse, not better. You still get one connection per concurrent execution, but now you pay to establish it on every single request.

B One connection per environment; add RDS Proxy

Correct diagnosis and the purpose-built fix. RDS Proxy pools and multiplexes connections so thousands of Lambda environments share a small pool.

C Increase memory

Memory buys CPU and bandwidth. It has no effect on how many connections exist.

D Scale the database up

It would work today and fail again at higher concurrency. It treats the symptom and pays for a bigger instance to do it.

THE SIGNAL

Reuse is per environment, not per function. Concurrency multiplies every per-environment resource.

A media company runs a Lambda function that downloads a source video from Amazon S3, transcodes it, and uploads the result back to S3. Source files are up to 6 GB, and the transcoding library requires the whole file on a local filesystem. It cannot stream.

The work completes in about 9 minutes. The intermediate files are useless once the upload finishes. The team wants the simplest configuration that works, and does not want to manage a VPC.

How should the developer provide the working storage?

A. Configure the function's ephemeral storage to 10,240 MB and work in /tmp

B. Attach an Amazon EFS file system to the function and write there

C. Attach an Amazon EBS volume to the function

D. Stream the file from S3 in chunks and keep only a small buffer in /tmp

◆ PAUSE AND DECIDE

How should the developer provide the working storage?

A Ephemeral storage at 10,240 MB

6 GB fits inside the 10 GB maximum, the files are temporary and private, and it needs no VPC. Simplest thing that works.

B Amazon EFS

It would work, but it requires VPC configuration and a mount target, which the scenario explicitly rules out. Reach for EFS when files are shared or must persist.

C Amazon EBS

EBS cannot be attached to a Lambda function at all. It is block storage for EC2.

D Stream in chunks

Directly contradicts a stated constraint - the library requires the whole file on disk and cannot stream.

THE SIGNAL

Two sentences decided this: files are not needed afterwards, and no VPC. Both point at /tmp.

A data-processing Lambda function is configured with 512 MB of memory and takes 12 seconds per invocation. It is CPU-bound: it parses and compresses a large payload in memory. It runs 5 million times a month and the bill has become the team's largest serverless line item.

A developer tests the same function at 2,048 MB and finds it completes in 2.5 seconds. Max Memory Used reports 340 MB in both configurations.

What should the team conclude?

A Keep 512 MB. Max Memory Used is only 340 MB, so 2,048 MB wastes memory they do not need

B Move to 2,048 MB for the speed, and accept that the bill will rise roughly fourfold

C Move to 2,048 MB. It is faster and cheaper, because 4x the memory for less than a quarter of the duration lowers total GB-seconds

D Enable Provisioned Concurrency, since the slowness is caused by cold starts

◆ PAUSE AND DECIDE

What should the team conclude?

A Keep 512 MB, memory is barely used

The trap. Memory allocation buys CPU, not just RAM. Max Memory Used tells you nothing about how much CPU the function needs.

B Move up and pay 4x

Right action, wrong arithmetic. Cost is memory multiplied by duration, and the duration fell by more than the memory rose.

C Move up: faster and cheaper

512 MB x 12s = 6 GB-seconds. 2,048 MB x 2.5s = 5 GB-seconds. Lower cost and a 5x speed-up.

D Provisioned Concurrency

Solves cold starts. Nothing here points to cold starts - the duration is consistent, and the fix worked by adding CPU.

THE SIGNAL

Cost is memory multiplied by duration. For CPU-bound work, more memory routinely costs less.

> An API-backed Lambda function retrieves a database credential from AWS Secrets Manager. The call sits at the top of the handler. The function is invoked about 400 times per second at peak.
>
> The team observes two problems: roughly 60 ms of added latency on every request, and intermittent ThrottlingException errors from Secrets Manager. The credential is rotated every 30 days. Security requires that a rotated credential is picked up without redeploying.

Which change best addresses both problems?

A. Move the secret into a Lambda environment variable encrypted with a customer managed KMS key

B. Move the Secrets Manager call outside the handler so it runs once per execution environment

C. Cache the secret in a module-level variable with no expiry, populated on first use

D. Add the AWS Parameters and Secrets Lambda Extension and read the secret from its local endpoint, with a cache TTL shorter than the rotation period

◆ PAUSE AND DECIDE

Which change best addresses both problems?

A Encrypted environment variable

Environment variables are readable by anyone with GetFunctionConfiguration, and the value would freeze at deploy time, breaking the rotation requirement.

B Move the call outside the handler

A real improvement and the closest wrong answer. But a long-lived environment caches the value indefinitely, so a rotated secret may never be picked up.

C Module-level cache, no expiry

Same defect as B, stated explicitly. No expiry means no rotation. It also still calls Secrets Manager once per cold start.

D Parameters and Secrets extension with a TTL

Removes the per-request API call, so latency and throttling both go away, and the TTL guarantees rotated values are collected without a deployment.

THE SIGNAL

Three constraints, not two: latency, throttling, and rotation without redeploy. Only the TTL satisfies the third.

What to carry into Lambda II

- Environment reuse is **per environment**. Concurrency multiplies every per-environment resource.
- The handler boundary decides whether code runs **once** or **every time**.
- Memory buys CPU. More memory is often **cheaper**, not dearer.
- Hard limits: 15 minutes, 10,240 MB memory, 10 GB /tmp, 4 KB env vars, 5 layers.
- Outbound permission is the execution role. Inbound is the resource-based policy.

> ON THE EXAM
>
> Next: concurrency, scaling and cold starts - where reserved and provisioned concurrency, SnapStart and throttling all live.

MODULE 04 · DOMAIN 1 - DEVELOPMENT WITH AWS SERVICES

Lambda II – Concurrency and Cold Starts

How Lambda scales, what throttling looks like, and the three ways to make a cold start stop hurting.

DOMAIN 1 of 4 EXAM WEIGHT 32% MODULE 4 of 34

admin@sthithapragna.com

Concurrency is not requests per second. It is **how many requests are in flight at the same instant**.

The formula worth memorising

- **Concurrency = requests per second x average duration in seconds**
- 100 requests/sec at 200 ms = 100 x 0.2 = **20 concurrent executions**.
- 100 requests/sec at 3 seconds = 100 x 3 = **300 concurrent executions**.
- Making a function faster reduces concurrency, which reduces throttling risk.
- A slow dependency raises concurrency without any change in traffic.

> ON THE EXAM
>
> Exam questions give you a request rate and a duration and expect you to produce a number. Practise the multiplication until it is automatic.

The account concurrency limit

- **1,000 concurrent executions** by default, per account, per Region.
- This is a **soft limit**. Raise it through Service Quotas or a support case.
- **Shared across every function in the account.** One noisy function can starve the rest.
- Provisioned concurrency is drawn from this same pool.
- At least **100** must remain unreserved for functions with no reservation of their own.

> ON THE EXAM
>
> Soft limit, so 'request a quota increase' is a legitimate answer here - unlike the 15-minute timeout, where it never is.

How fast Lambda can scale

- **1,000 new execution environments every 10 seconds**, per function, per Region.
- Equivalently, 10,000 additional requests per second every 10 seconds.
- **It is per function.** One function's spike no longer steals headroom from another.
- It does **not accrue**. Idling does not bank capacity for a later burst.
- Going from 0 to 30,000 concurrency takes roughly five minutes of sustained demand.

> ON THE EXAM
>
> Having concurrency available and being allowed to use it instantly are different things. Lambda rate-limits how fast environments appear.

Currency note: scaling was rebuilt

- **Old model:** an account-wide burst pool of 500, 1,000 or 3,000 depending on Region, refilling at 500 per minute.
- That pool was **shared**, so functions competed. A classic noisy-neighbour problem.
- **New model, since 6 December 2023:** 1,000 every 10 seconds, **per function**. Up to 12x faster.
- The noisy-neighbour effect on **scaling rate** is gone entirely.
- The account **concurrency limit** is still shared. Only the scaling rate became per-function.

> ON THE EXAM
>
> Plenty of question banks and courses still teach the old burst-pool model with its 500-per-minute refill. If you meet those numbers, they are out of date.

What happens when a request arrives

01

Warm?

An idle environment exists. Reuse it. No cold start.

02

Scale?

None free, but concurrency and scaling rate allow more. Build one. Cold start.

03

Throttle

At the limit, or scaling too fast. Reject with a 429.

ON THE EXAM

Only the middle path costs you a cold start, and only the third path produces an error. Most invocations take the first path.

What throttling actually does

INVOCATION TYPE	BEHAVIOUR WHEN THROTTLED
Synchronous, for example API Gateway	Immediate 429 TooManyRequestsException. The caller must retry
Asynchronous, for example S3 or SNS	Lambda retries internally for up to 6 hours with backoff
Event source mapping, SQS	The message stays on the queue and is retried
Event source mapping, Kinesis or DynamoDB	The shard blocks and retries until success or expiry

ON THE EXAM

Synchronous throttling is visible to the user immediately. Asynchronous throttling is usually invisible until you look at the metrics.

Maximum concurrency is not reserved concurrency

- **Reserved concurrency** is set on the **function**. It caps and reserves account capacity.
- **Maximum concurrency** is set on the **event source mapping**. It caps only that source.
- One function with several queues can give each queue its own maximum.
- On reaching the maximum, Lambda simply stops reading more messages. Nothing is thrown away.
- Reserved concurrency at the same value would throttle, sending messages back for retry.

> ON THE EXAM
>
> Throttling a queue-driven function wastes retries. Maximum concurrency slows the poller instead, which is the cleaner control for SQS.

Concurrency for event source mappings

- With SQS, Lambda **polls** rather than being pushed. The poller decides how fast to scale.
- Standard queues start at **5 concurrent batches** and add up to **300 instances per minute**.
- **FIFO queues scale to the number of active message groups**, and no further.
- So a FIFO queue with one message group gives you a concurrency of exactly one.
- Lambda deliberately slows scaling while invocations are failing, then resumes once they succeed.

> ON THE EXAM
>
> The FIFO point decides questions. If ordering matters and every message shares a group ID, no amount of concurrency configuration will parallelise it.

Reserved concurrency

- **Caps** the function. It can never exceed the reserved number.
- **Guarantees** the function. That capacity is carved out and nobody else can take it.
- Both jobs, one setting. That is what makes it confusing and examinable.
- It is subtracted from the account's unreserved pool the moment you set it.
- **No extra cost.** You are partitioning capacity, not buying it.

> ON THE EXAM
>
> Use it to protect a downstream database from too many connections, and to stop a batch function starving a customer-facing one.

Reserved concurrency of zero

- Setting reserved concurrency to **0** throttles every invocation of that function.
- The fastest way to stop a misbehaving function without deleting it.
- Useful during an incident, or to pause a consumer while you fix a downstream system.
- Configuration, event source mappings and code all stay intact. Reversible instantly.
- Asynchronous events still arrive and are retried, so they can pile up while it is off.

> ON THE EXAM
>
> When a question asks how to immediately stop a function without losing its configuration, this is the answer.

Provisioned concurrency

- Lambda keeps N environments **initialised and waiting**. INIT has already run.
- Configured on a **version or an alias**. Never on $LATEST.
- **You pay for it by the hour**, whether or not any request arrives.
- Traffic beyond the provisioned amount **spills over** to on-demand, with cold starts.
- Watch **ProvisionedConcurrencySpilloverInvocations** to see if you under-provisioned.

> ON THE EXAM
>
> The only option that eliminates cold starts outright. It is also the only one with a standing bill.

Reserved or provisioned concurrency?

RESERVED CONCURRENCY

Controls how many

- Caps and guarantees a maximum
- Protects downstream systems
- Stops noisy functions starving others
- Free - it partitions existing capacity
- Does nothing about cold starts

PROVISIONED CONCURRENCY

Controls how fast

- Pre-initialises environments
- Removes cold start latency
- Set on a version or alias only
- Billed hourly, used or not
- Does nothing about capacity limits

THE TIEBREAKER

Reserved is about quantity. Provisioned is about latency. The names sound alike and they solve unrelated problems - this pair appears constantly.

Scaling provisioned concurrency automatically

- **Scheduled scaling** raises and lowers it at known times. Ideal for predictable daily peaks.
- **Target tracking** adjusts against a utilisation target as traffic moves.
- Together they avoid paying peak rates around the clock.
- Utilisation is ProvisionedConcurrentExecutions divided by allocated.
- Scaling up is not instant. Provisioning environments takes time, so schedule ahead of the spike.

> ON THE EXAM
>
> A scenario with a known daily or weekly traffic pattern and a cost constraint is usually pointing at scheduled scaling.

What actually causes a cold start

- **Scaling up.** More concurrent requests than warm environments.
- **Idle reaping.** Lambda reclaims environments after a period with no traffic.
- **Any deployment.** Publishing new code discards every warm environment.
- Large deployment packages and heavy imports make each one longer.
- Java and .NET suffer most. Python and Node.js are far quicker to start.

> ON THE EXAM
>
> Cold starts are a small fraction of invocations in steady traffic. They matter most for spiky traffic and latency-sensitive APIs.

Lambda SnapStart

- Lambda runs INIT once, snapshots the initialised environment, and **restores from the snapshot** on later cold starts.
- Cold starts for heavy Java frameworks can fall from seconds to a few hundred milliseconds.
- **No additional charge** for the feature itself.
- Runtimes: **Java 11+, Python 3.12+, .NET 8+**. Not Node.js, not Ruby.
- Works only on **published versions and aliases**. Never on $LATEST.

> ON THE EXAM
>
> The middle ground: most of the latency benefit of provisioned concurrency, without the standing bill. But only on supported runtimes.

What SnapStart will not do

CONSTRAINT	DETAIL
Provisioned concurrency	Mutually exclusive. You cannot use both on the same version
Amazon EFS	Not supported
Ephemeral storage	Cannot exceed the default 512 MB
$LATEST	Not supported. Published versions and aliases only
Node.js and Ruby	Not supported runtimes

ON THE EXAM

The first row is the highest-value fact here. A scenario that needs both is a scenario where one of them has to go.

The SnapStart uniqueness trap

- One snapshot becomes the starting state for **many** environments.
- So anything unique generated during INIT is **duplicated** across all of them.
- Random seeds, UUIDs, cached credentials and timestamps are all at risk.
- **Fix one:** generate unique values inside the handler, not at module load.
- **Fix two:** use a runtime hook that runs after restore, or a CSPRNG like SecureRandom.

> ON THE EXAM
>
> Symptoms are intermittent and nasty: duplicate IDs, identical 'random' values, stale credentials. AWS ships an open-source scanning tool for Java to catch it.

SnapStart or provisioned concurrency?

CHOOSE SNAPSTART WHEN

Latency matters, cost matters more

- Runtime is Java, Python or .NET
- No standing charge is acceptable
- Sub-second is good enough
- Traffic is spiky or unpredictable
- No EFS and /tmp stays at 512 MB

CHOOSE PROVISIONED CONCURRENCY WHEN

Latency is non-negotiable

- Any runtime, including Node.js
- Double-digit milliseconds required
- You will pay hourly for it
- Traffic is predictable enough to size
- You need EFS or larger /tmp

THE TIEBREAKER

SnapStart is free but restricted. Provisioned concurrency works everywhere and costs money. If the scenario stresses cost, lean SnapStart; if it stresses a hard latency SLA, lean provisioned.

The concurrency metrics that matter

METRIC	WHAT IT TELLS YOU
ConcurrentExecutions	How much concurrency is in use right now
UnreservedConcurrentExecutions	What is left in the shared pool
Throttles	Requests rejected. Any sustained value is a problem
ProvisionedConcurrentExecutions	How much of your provisioned capacity is busy
ProvisionedConcurrencySpilloverInvocations	Requests that overflowed to on-demand
ClaimedAccountConcurrency	Total claimed against the account quota

ON THE EXAM

Throttles above zero plus ConcurrentExecutions flat at a round number means you have hit a limit, not a bug.

Four scenarios. Commit to an answer **before** the walkthrough, including which options you eliminated and why.

> An account runs two Lambda functions. A customer-facing checkout function sits behind API Gateway and must never fail. A nightly report function is triggered by Amazon S3 and processes thousands of objects, scaling to hundreds of concurrent executions.
>
> During the nightly run, checkout requests start returning 429 errors. The account concurrency limit is the default 1,000. The business wants checkout protected, and is content for the report job to take longer if necessary.

What should the developer configure?

A. Request an account concurrency limit increase from 1,000 to 5,000

B. Configure provisioned concurrency on the checkout function

C. Configure reserved concurrency on the report function, capping it well below 1,000

D. Move the report function to a separate AWS Region

◆ PAUSE AND DECIDE

What should the developer configure?

A Raise the account limit

Buys headroom but fixes nothing structurally. The report function can still consume whatever it likes and starve checkout at a higher number.

B Provisioned concurrency on checkout

Solves cold starts, not capacity. Provisioned capacity still draws from the same account pool, and 429s are a concurrency problem.

C Reserved concurrency on the report function

Caps the noisy function so it can never consume the whole pool, and the business has already accepted a slower report.

D Separate Region

Concurrency limits are per Region so this would technically work, but it is a large architectural change to solve a configuration problem.

THE SIGNAL

Throttling is a capacity problem, so the answer is a capacity control. The sentence granting permission for a slower report is what makes capping acceptable.

> A Java Spring Boot application runs as a Lambda function behind API Gateway. Cold starts take between 4 and 6 seconds and users notice. Traffic is unpredictable, arriving in bursts several times a day with long quiet periods between them.
>
> The team's p99 latency target is under one second. Finance has rejected a proposal that adds a fixed monthly cost. The function writes small temporary files and does not use Amazon EFS.

What is the most appropriate way to reduce the cold start latency?

A. Configure provisioned concurrency sized for peak traffic

B. Increase the function's memory to 10,240 MB

C. Create a scheduled EventBridge rule that invokes the function every 5 minutes to keep it warm

D. Enable Lambda SnapStart on a published version and point the alias at it

◆ PAUSE AND DECIDE

What is the most appropriate way to reduce the cold start latency?

A Provisioned concurrency at peak

It would meet the latency target, but it bills hourly whether traffic arrives or not - exactly the fixed cost finance rejected, and wasteful given long quiet periods.

B Maximum memory

More CPU shortens INIT somewhat, but it will not turn a 5-second JVM start into under a second, and it raises the cost of every invocation.

C EventBridge warming

The classic hack. It keeps roughly one environment warm, so it does nothing for a burst that needs fifty, and it bills for invocations that do no work.

D SnapStart on a published version

Purpose-built for exactly this: heavy JVM initialisation, no additional charge, and every restored environment skips INIT rather than only the first.

THE SIGNAL

Three constraints line up: Java runtime, no fixed cost, unpredictable bursts. SnapStart is the only option that satisfies all three.

A team enables SnapStart on a Java function that generates a correlation ID for each execution environment at startup and uses java.util.Random, seeded once during initialisation, to produce short reference codes for customer orders.

After deployment, support reports that different customers are occasionally receiving the same order reference code. The problem is intermittent and never reproduces in a single-user test.

What is the cause, and what is the correct fix?

A SnapStart is corrupting memory. Disable SnapStart and use provisioned concurrency instead

B The seed and correlation ID are captured in the snapshot and shared by every restored environment. Generate them in the handler, or refresh them in an afterRestore runtime hook

C The function needs more memory so each environment gets its own random state

D Add a DynamoDB conditional write to detect duplicate codes and retry

◆ PAUSE AND DECIDE

What is the cause, and what is the correct fix?

A Disable SnapStart

Misdiagnoses it as a bug in Lambda. It is documented behaviour, and abandoning SnapStart gives up the benefit rather than fixing the code.

B Snapshot captured the seed; move generation after init

The correct diagnosis and the documented fix. One snapshot seeds many environments, so uniqueness must be created after restore.

C More memory

Memory has no bearing on random state. Every restored environment gets an identical copy regardless of size.

D Detect duplicates in DynamoDB

Papers over the defect. The generator is still producing colliding sequences, so collisions grow with concurrency and every order now costs an extra write.

THE SIGNAL

'Intermittent, and never in single-user testing' is the tell. It only appears once several environments restore from the same snapshot.

> A payments API on Node.js 22 must respond within 100 ms at p99, including cold starts. Traffic follows a strict and well-understood pattern: near zero overnight, rising sharply at 08:00, steady all day, falling after 18:00.
>
> The team has budget for the latency guarantee but does not want to pay peak rates for the sixteen quiet hours each day.

Which approach best meets both requirements?

A. Provisioned concurrency on an alias, with Application Auto Scaling scheduled scaling raising it before 08:00 and lowering it after 18:00

B. Enable SnapStart on a published version and point the alias at it

C. Reserved concurrency set to the expected daytime peak

D. Provisioned concurrency on $LATEST, fixed at the daytime peak all day

◆ PAUSE AND DECIDE

Which approach best meets both requirements?

A Provisioned concurrency plus scheduled scaling

Meets a 100 ms target that only pre-warmed environments can hit, and the schedule removes the cost during the sixteen quiet hours.

B SnapStart

Node.js is not a supported SnapStart runtime. Even if it were, restore latency does not reliably reach double-digit milliseconds.

C Reserved concurrency

Reserves capacity but pre-warms nothing. Every scale-up still cold starts, so the latency target is missed.

D Provisioned concurrency on $LATEST

Not possible - provisioned concurrency requires a version or alias. It also pays peak rates overnight, which the scenario rules out.

THE SIGNAL

Compare with scenario 2. Same problem, opposite answer, because the runtime and the cost constraint both flipped.

Currency note: Lambda Managed Instances

- Runs Lambda functions on **dedicated EC2-backed instances** in your account, keeping the Lambda programming model.
- Multiple concurrent invocations share an instance, rather than one environment per request.
- Aimed at **steady, high-volume** workloads where per-invocation pricing stops making sense.
- Compatible with EC2 Savings Plans, and gives access to newer processors.
- **Not in exam guide v2.1.** Answer the exam from the classic model: one environment, one concurrent request.

> ON THE EXAM
>
> Worth knowing because it changes the standard 'Lambda gets expensive at constant high load, so move to Fargate' answer. On the exam, that answer still stands.

What to carry into Lambda III

- Concurrency = requests per second x duration. Slower functions cost more concurrency.
- **Reserved is quantity. Provisioned is latency.** Free versus billed hourly.
- SnapStart: free, Java/Python/.NET, published versions only, and never with provisioned concurrency.
- Scaling is **1,000 per 10 seconds, per function**. The old shared burst pool is gone.
- Throttling means different things for sync, async and event source mappings.

> ON THE EXAM
>
> Next: event sources and failure handling - where that last point gets its own module, along with retries, DLQs and idempotency.

MODULE 05 · DOMAIN 1 - DEVELOPMENT WITH AWS SERVICES

Lambda III – Event Sources and Failures

Three invocation models, three completely different retry stories – and the one question that decides which you are looking at.

DOMAIN 1 of 4 EXAM WEIGHT 32% MODULE 5 of 34

admin@sthithapragna.com

Before you can answer a Lambda failure question, you must know **how the function was invoked**. Everything else follows from that.

The three invocation models

MODEL	WHO CALLS IT, AND WHAT HAPPENS ON FAILURE
Synchronous	Caller waits for the response. Lambda does not retry. The caller must
Asynchronous	Lambda queues the event and returns 202 immediately. Lambda retries
Event source mapping	Lambda polls the source. The poller retries, and the source holds the data

ON THE EXAM

Synchronous: the caller owns the retry. Asynchronous: Lambda owns it. Event source mapping: the poller owns it, and the record survives in the source until it succeeds or expires.

Synchronous invocation

- API Gateway, Application Load Balancer, function URLs, direct **Invoke** calls.
- **Lambda performs no retries.** The error goes straight back to the caller.
- Dead-letter queues and destinations do **not** apply. There is nowhere to send it.
- Retry logic belongs in the client, with exponential backoff and jitter.
- A throttled synchronous call returns 429 immediately.

> ON THE EXAM
>
> If a question describes a synchronous trigger and asks how to capture failed events automatically, the honest answer is that you cannot - you handle it in the caller or change the model.

Asynchronous invocation

- S3 events, SNS, EventBridge, and **Invoke** with the Event invocation type.
- Lambda returns **202 Accepted** at once and queues the event internally.
- On a function error, Lambda retries **twice more**: after 1 minute, then after 2 minutes.
- The caller never learns the outcome. Success and failure look identical to it.
- So an unhandled failure is **silently discarded** unless you configure somewhere for it to go.

> ON THE EXAM
>
> Three attempts in total, not three retries. The silent-discard behaviour is why on-failure destinations exist.

Two kinds of failure, two retry behaviours

- **Function errors** – your code threw, or the invocation timed out. Retried **twice**, at 1 and 2 minutes.
- **Throttling (429) and system errors (5xx)** – the function never ran.
- Those are returned to the queue and retried for up to **6 hours**, with exponential backoff.
- Backoff grows from 1 second up to a maximum of 5 minutes between attempts.
- With a long queue, Lambda slows its own read rate and events can age out before they run.

> ON THE EXAM
>
> 'Retried twice' applies to code failures. A throttled event gets hours of attempts, because it never had a chance to run in the first place.

Controlling asynchronous retries

- **Maximum retry attempts:** 0, 1 or 2. Default 2.
- **Maximum event age:** 60 seconds to 6 hours. Default 6 hours.
- Whichever limit is reached first ends the attempts.
- Lower both when stale events are worse than lost ones - time-sensitive work, for example.
- Each retry is a full billed invocation. Retrying a doomed event three times costs three times.

> ON THE EXAM
>
> Set maximum event age low when processing an hour-old event would be wrong. A price update or a fraud check is worse late than never.

Dead-letter queue or destination?

DEAD-LETTER QUEUE

The original mechanism

- Failure only. No success path
- Targets SQS or SNS only
- Captures the event payload
- Error detail in message attributes
- Part of the function configuration

ON-FAILURE DESTINATION

What AWS recommends now

- Both onSuccess and onFailure
- Targets SQS, SNS, Lambda or EventBridge
- Captures the full invocation record
- Error message, stack trace and payloads
- Configured per version or alias

THE TIEBREAKER

Destinations are a superset. Choose them unless a question specifically describes a DLQ. When a scenario asks for the reason a failure happened, only destinations carry the stack trace.

What event source mappings are

- A Lambda-managed **poller** reads from the source and invokes your function with a batch.
- The **execution role** needs read permission on the source. No resource-based policy required.
- Records stay in the source until processed successfully, so nothing is lost mid-failure.
- Sources: SQS, Kinesis Data Streams, DynamoDB Streams, Amazon MSK, self-managed Kafka, Amazon MQ, DocumentDB.
- The mapping can be disabled without touching the function - a useful pause switch.

> ON THE EXAM
>
> This is the one model where permission flows the other way. The function reads the source, so the permission goes on the execution role, not on the function.

Batch size and batching window

- **Batch size** is the maximum records per invocation. SQS standard allows up to 10,000.
- **Maximum batching window** waits up to 300 seconds to fill a batch before invoking.
- Bigger batches mean fewer invocations, so lower cost and better throughput.
- Bigger batches also mean higher latency and a larger blast radius when one record fails.
- The invocation payload limit still applies, so a batch may arrive smaller than configured.

> ON THE EXAM
>
> Batching trades latency for cost. A low-traffic queue with a large batch size and no batching window can leave records waiting a long time.

SQS as an event source

- Successfully processed messages are **deleted by Lambda** when the invocation succeeds.
- On failure, nothing is deleted. The whole batch reappears after the visibility timeout.
- Retries are governed by the queue's **maxReceiveCount**, not by Lambda.
- The **DLQ belongs to the queue**, configured in its redrive policy.
- A Lambda DLQ does nothing here. That is a function-level asynchronous setting.

> ON THE EXAM
>
> The last point is a classic distractor. For an SQS-triggered function, the dead-letter queue is configured on the queue, not on the function.

The visibility timeout rule

- AWS recommends **visibility timeout = 6 x function timeout + MaximumBatchingWindowInSeconds**.
- Too short, and the message reappears while your function is still working on it.
- A second invocation then processes the same message. That is duplicate processing.
- The 6x allowance exists to leave room for retries after a throttle.
- The SQS default visibility timeout is 30 seconds - shorter than many function timeouts.

> ON THE EXAM
>
> Reports of messages being processed twice, with no errors in the logs, point at the visibility timeout before they point at anything else.

Partial batch responses

- By default one bad record fails the whole batch, so **successful records are reprocessed too**.
- Set **ReportBatchItemFailures** in FunctionResponseTypes on the event source mapping.
- The function then returns a list of failed item identifiers, and only those become visible again.
- If the function **throws** instead of returning, the entire batch still fails.
- The Powertools Batch Processor utility implements this for you.

> ON THE EXAM
>
> This is the standard fix for 'nine records succeeded, one failed, and all ten ran again'. It needs both the mapping setting and a code change.

Starting position and iterator age

- **TRIM_HORIZON** starts at the oldest record still in the stream.
- **LATEST** starts just after the newest record, so existing data is skipped.
- **AT_TIMESTAMP** starts from a point in time you supply.
- Creating or updating a mapping is eventually consistent, so **LATEST can miss events** during the change.
- **IteratorAge** is the age of the last record processed. Rising means the consumer is falling behind.

> ON THE EXAM
>
> Iterator age climbing steadily is the single clearest sign of a struggling stream consumer. It never applies to SQS, which has no ordered position.

Streams behave differently

- Records are **ordered within a shard**, and Lambda preserves that order.
- By default **one concurrent invocation per shard**. Concurrency is shard count, not traffic.
- A failing batch is **retried until it succeeds or the records expire** - blocking that shard.
- That is the poison-pill problem. One bad record can stall a shard for its whole retention period.
- Records are not deleted after processing. They stay until retention expires.

> ON THE EXAM
>
> Ordering is the reason for the blocking. Lambda cannot skip a record without breaking the guarantee it is giving you.

Escaping a poison pill

- **BisectBatchOnFunctionError** splits a failed batch in half repeatedly to isolate the bad record.
- **MaximumRetryAttempts** caps retries so the shard eventually moves on.
- **MaximumRecordAgeInSeconds** discards records older than a threshold.
- **On-failure destination** sends metadata about skipped records to SQS or SNS.
- Without a destination configured, skipped records are dropped and gone.

ON THE EXAM

Bisecting narrows the blast radius. Maximum retries and record age unblock the shard. The destination is what stops you losing the data while doing it.

Parallelisation factor

- Raises concurrent invocations **per shard**, from 1 up to 10.
- Ordering is still preserved **within each partition key**, so the guarantee survives.
- The cheaper way to scale a stream consumer than resharding.
- It does not help if the bottleneck is a downstream system rather than Lambda.
- Applies to Kinesis Data Streams and DynamoDB Streams.

> ON THE EXAM
>
> When a scenario says the consumer cannot keep up and iterator age is climbing, this is usually cheaper and faster than adding shards.

Tumbling windows

- Fixed time windows, **up to 15 minutes**, that let state pass between invocations.
- The event carries **window**, **state** and **isFinalInvokeForWindow**.
- Lets you aggregate across batches without an external database.
- State is capped at **1 MB per shard**. Exceed it and the window is terminated early.
- Not supported across resharding. A closing shard ends its window.

> ON THE EXAM
>
> The usual answer for running totals over a stream is DynamoDB or ElastiCache. Tumbling windows are the answer when the aggregation fits in a window and 1 MB.

Event filtering

- **Filter criteria** on the event source mapping discard non-matching records **before invocation**.
- You are not billed for events that are filtered out. There is no invocation.
- Available for SQS, Kinesis, DynamoDB Streams, MSK, Kafka and DocumentDB.
- Up to 5 filter patterns per mapping, using EventBridge pattern syntax.
- Filtering in your handler still costs a full invocation. The saving comes from filtering earlier.

> ON THE EXAM
>
> When a scenario says a function is invoked constantly but discards most events, this is the intended answer - lower cost and no code change.

S3 events: direct or via EventBridge

- **S3 event notifications** invoke Lambda directly. Simple, and asynchronous.
- Only one destination per event type and prefix combination. Fan-out needs SNS in between.
- **S3 to EventBridge** emits every event to the default bus instead.
- EventBridge gives many targets, richer filtering, archive and replay.
- It also covers more event types than notifications do.

> ON THE EXAM
>
> One consumer, keep it simple: notifications. Several consumers, or you want filtering and replay: EventBridge.

Every retry mechanism in this module can deliver the same event twice. Lambda gives you **at-least-once**, never exactly-once.

Building idempotency

- Pick a stable **idempotency key**: an order ID or message ID, not a timestamp or a random value.
- Record it in DynamoDB with a **conditional write** so a duplicate fails the condition.
- Use **TTL** on those records so the table does not grow without bound.
- The **Powertools idempotency utility** implements the whole pattern as a decorator.
- Naturally idempotent operations need nothing. Setting a status to 'shipped' twice is harmless.

> ON THE EXAM
>
> The distinction to make quickly: does running this twice do damage? Charging a card, yes. Overwriting a record with the same value, no.

Recursive loop detection

- The classic accident: a function triggered by an S3 bucket that writes back to the same bucket.
- Lambda tracks a **Lineage** counter in the X-Ray trace header across the chain.
- After roughly **16** invocations of the same event chain, Lambda stops the next invocation.
- It emits **RecursiveInvocationsDropped** and notifies you by email and Health Dashboard.
- On by default, covering Lambda, SQS, SNS and S3. Requires a supported SDK version.

> ON THE EXAM
>
> It limits the damage, it does not fix the design. If you hit it, set reserved concurrency to zero, then fix the loop.

Failure handling, by source

TRIGGER	WHERE FAILURES GO
API Gateway, ALB, function URL	Nowhere. The caller sees the error and must retry
S3, SNS, EventBridge	Lambda DLQ or on-failure destination, after 2 retries
SQS	The queue's own DLQ, via maxReceiveCount in the redrive policy
Kinesis, DynamoDB Streams	On-failure destination, after retry and record-age limits

ON THE EXAM

One table, four rows, and a large slice of the troubleshooting domain. The SQS row is the one most often answered wrongly.

EventBridge Pipes

- A managed point-to-point integration: **source, filter, enrichment, target**.
- Sources include SQS, Kinesis, DynamoDB Streams, MSK and Amazon MQ.
- Removes the Lambda function that existed only to read from one service and write to another.
- Filtering and simple transformation happen without any code at all.
- Real business logic still belongs in a function. Pipes is for plumbing, not processing.

> ON THE EXAM
>
> When a scenario describes a function that only reads from a queue and forwards to another service, and asks for the least operational overhead, Pipes is often the intended answer.

SQS or Kinesis for this consumer?

CHOOSE SQS WHEN

Independent work items

- Order does not matter
- Each message is consumed once
- You want concurrency to follow load
- One bad message must not block others
- Simple retry and DLQ semantics

CHOOSE KINESIS WHEN

An ordered record stream

- Order within a key matters
- Several consumers read the same data
- You may need to replay history
- Concurrency is bounded by shards
- A bad record can block a shard

THE TIEBREAKER

Ordering and replay push you to Kinesis. Independent tasks and non-blocking failures push you to SQS. 'Multiple consumers of the same events' is the clearest single signal for Kinesis.

For each one, identify the **invocation model** first. Then decide.

> An order-processing Lambda function reads from an Amazon SQS standard queue with a batch size of 10. It charges a payment provider for each order in the batch.
>
> Customers report occasional duplicate charges. Investigation shows that when one message in a batch fails, all ten become visible again and the whole batch is reprocessed. The failing message is usually a genuine data problem affecting a single order.

What combination best prevents the duplicate charges?

A. Reduce the batch size to 1 so a failure can only ever affect one message

B. Increase the queue's visibility timeout to six times the function timeout

C. Configure a Lambda on-failure destination to capture the failed batch

D. Enable ReportBatchItemFailures on the event source mapping, return the failed message IDs, and make the charge operation idempotent using the order ID

◆ PAUSE AND DECIDE

What combination best prevents the duplicate charges?

A Batch size of 1

It would work, at ten times the invocations and ten times the cost, and it throws away batching entirely. A blunt instrument for a solved problem.

B Longer visibility timeout

Fixes a different duplicate-processing cause - messages reappearing mid-execution. Here the batch is failing outright, not timing out.

C On-failure destination

Destinations are for asynchronous invocation. This is an event source mapping, so this setting does not apply.

D Partial batch responses plus idempotency

Stops the nine good messages being returned to the queue, and idempotency protects against the duplicates that at-least-once delivery can still produce.

THE SIGNAL

Two defences, deliberately. Partial batch responses remove the cause; idempotency covers every other retry path that could still deliver twice.

A Lambda function consumes a Kinesis Data Stream carrying device telemetry. One device began emitting a malformed record, and the function throws on it. Processing for that shard has stopped completely, and iterator age has been climbing for hours.

The team needs processing to resume, wants the surrounding good records processed rather than discarded, and must keep any skipped records for later investigation.

Which configuration meets all three requirements?

A Increase the number of shards in the stream

B Enable BisectBatchOnFunctionError, set MaximumRetryAttempts, and configure an on-failure destination

C Set the parallelisation factor to 10

D Configure a dead-letter queue on the Lambda function

◆ PAUSE AND DECIDE

Which configuration meets all three requirements?

A More shards

Resharding does not remove the bad record. That shard's records are redistributed but the poison pill still blocks whichever shard holds it.

B Bisect, retry limit, and destination

Bisecting isolates the bad record so good ones proceed, the retry cap unblocks the shard, and the destination preserves what was skipped. All three requirements.

C Parallelisation factor

Adds throughput, not fault tolerance. The blocked partition key is still blocked, so nothing here resumes processing.

D Lambda DLQ

A function-level asynchronous setting. Stream event source mappings use an on-failure destination instead.

THE SIGNAL

Three requirements, three settings. Only B addresses all of them, and the third one - keeping the skipped records - is what eliminates everything else.

A Lambda function is triggered by S3 object-created events to generate thumbnails. Occasionally an upload produces no thumbnail and nobody notices for days. There is no record of what happened.

The team wants every permanent failure captured automatically, including the error message and stack trace, so a separate function can attempt recovery and raise an alert. They do not want to modify the thumbnail function's code.

What should the team configure?

A. An on-failure destination pointing at a Lambda function that handles recovery and alerting

B. A dead-letter queue on the function, pointing at an SQS queue

C. Wrap the handler in a try/except block that publishes failures to SNS

D. Enable ReportBatchItemFailures on the S3 event source mapping

◆ PAUSE AND DECIDE

What should the team configure?

A On-failure destination to a Lambda function

Destinations carry the full invocation record including the stack trace, and can target a Lambda function directly. No code change needed.

B Dead-letter queue to SQS

The closest wrong answer. It captures the event, but only limited error detail in message attributes, and it cannot invoke a function - something must poll the queue.

C try/except publishing to SNS

Explicitly ruled out, since the team will not modify the code. It also misses timeouts and out-of-memory kills, which never reach an exception handler.

D ReportBatchItemFailures

S3 invokes Lambda asynchronously. There is no event source mapping here, so the setting does not exist to enable.

THE SIGNAL

Two constraints do the work: the stack trace is needed, and the code cannot change. Those together select destinations over a DLQ.

A Lambda function is attached to a DynamoDB Stream on a large table. It exists solely to send a notification when an order's status changes to CANCELLED. Status changes to CANCELLED are about 2% of all writes.

The function is invoked roughly 40 million times a month. Its first line inspects the record and returns immediately for anything that is not a cancellation. The Lambda bill has become a problem and the team wants it reduced without changing behaviour.

What is the most effective change?

A Reduce the function's memory to 128 MB so each invocation costs less

B Increase the batch size so fewer invocations are needed

C Add filter criteria to the event source mapping so only CANCELLED records invoke the function

D Move the status check into a Step Functions Choice state before the Lambda function

◆ PAUSE AND DECIDE

What is the most effective change?

A Lower the memory

Shaves a little from each invocation while still paying for all 40 million. It also risks making the function slower.

B Larger batches

A real improvement, and the second-best answer. But it still delivers every record to the function and still pays per request.

C Filter criteria on the event source mapping

Non-matching records are discarded before invocation, so 98% of those invocations stop happening and stop being billed. No code change.

D Step Functions Choice state

Adds a state machine and its own per-transition charge to every event. It makes the bill worse while adding a component.

THE SIGNAL

The cheapest invocation is the one that never happens. Filtering at the event source is the only option that removes invocations rather than making them cheaper.

What to carry into Lambda IV

- **Identify the invocation model first.** It determines everything about failure handling.
- Async retries twice on function errors, but up to 6 hours on throttles and system errors.
- **SQS failures go to the queue's DLQ**, never to a Lambda DLQ.
- Streams block on failure. Bisect, cap retries, and add an on-failure destination.
- Delivery is **at-least-once**. Idempotency is a requirement, not a refinement.

> ON THE EXAM
>
> Next: response streaming, function URLs, edge compute, and the new Lambda capabilities that are changing what belongs in a function at all.

MODULE 06 · DOMAIN 1 - DEVELOPMENT WITH AWS SERVICES

Lambda IV – Streaming, Edge and What Changed

Function URLs, response streaming, edge compute – and four rules the exam still teaches that AWS has quietly changed.

DOMAIN 1 of 4 EXAM WEIGHT 32% MODULE 6 of 34

admin@sthithapragna.com

A Lambda function does nothing until something calls it. This module is about the front door - and there are now four of them.

Lambda function URLs

- A dedicated HTTPS endpoint on the function: **https://<id>.lambda-url.<region>.on.aws**
- **No additional charge.** You pay only for the Lambda invocation.
- Built-in CORS configuration, no code required.
- Can be attached to a specific **alias or version**, using the qualifier.
- One function, one URL. It is not an API, and it has no routing.

> ON THE EXAM
>
> Function URLs are for a single-purpose endpoint. The moment you need paths, methods and routing, you want API Gateway.

The function URL auth model

- **AWS_IAM** - the caller must sign the request with SigV4 and hold lambda:InvokeFunctionUrl.
- **NONE** - completely public. Anyone with the URL can invoke it.
- No API keys, no usage plans, no per-caller throttling, no Cognito authoriser.
- No Lambda authorisers either. Custom auth must live inside your handler.
- For WAF or per-caller rate limiting, put CloudFront or API Gateway in front.

> ON THE EXAM
>
> The absence of API keys and usage plans is the fact that decides questions. Any scenario mentioning tiered access or per-customer quotas rules function URLs out immediately.

Function URL or API Gateway?

FUNCTION URL

One endpoint, no management

- No extra charge at all
- IAM or public. Nothing else
- No routing or path matching
- Supports response streaming
- Set up in one configuration step

API GATEWAY

A managed API surface

- Charged per request
- API keys, usage plans, authorisers
- Routing, stages, versioning
- Request validation and caching
- WAF attaches directly

THE TIEBREAKER

If the scenario names throttling per customer, API keys, request validation or several routes, it is API Gateway. If it says least operational overhead for one internal endpoint, it is a function URL.

CloudFront in front of a function URL

- CloudFront fixes most of what a function URL lacks: caching, **WAF**, Shield, a custom domain.
- **Origin access control** for function URLs, since April 2024, signs origin requests with SigV4.
- That lets the function URL use **AWS_IAM** and stay unreachable directly.
- Without OAC, the URL had to be public and the function had to check a shared secret itself.
- Streaming still works through CloudFront, so you keep both.

> ON THE EXAM
>
> A function URL with AuthType NONE is genuinely open to the internet. CloudFront with OAC is how you front one safely without moving to API Gateway.

Response streaming

- The function sends the response progressively instead of buffering it whole.
- **Time to first byte** drops sharply. The user sees output while work continues.
- Node.js requires wrapping the handler in **streamifyResponse()**.
- Set **InvokeMode: RESPONSE_STREAM** on the function URL, or call InvokeWithResponseStream.
- Not supported on function URLs when the function is attached to a VPC.

> ON THE EXAM
>
> Two use cases dominate: generative AI token-by-token output, and payloads that exceed the 6 MB buffered limit.

The streaming numbers

- **Buffered response: 6 MB.** The classic Lambda synchronous limit.
- **Streamed response: 20 MB** by default, and this one can be raised on request.
- The first **6 MB** streams at uncapped bandwidth.
- Beyond 6 MB, the remainder is capped at roughly **2 MB per second**.
- A disconnecting client does not stop the function. You pay for the full duration.

> ON THE EXAM
>
> Streaming raises the payload ceiling as well as improving latency. That second benefit is easy to forget and often the one a scenario is testing.

Which front door actually streams

FRONT DOOR	STREAMING SUPPORT
Lambda function URL	Yes, with InvokeMode RESPONSE_STREAM. Not inside a VPC
API Gateway REST API	Yes, since November 2025. See the next slide
API Gateway HTTP API	No. Buffered only
Application Load Balancer	No streaming path for Lambda targets
CloudFront with a function URL origin	Yes, and it adds caching and WAF

ON THE EXAM

The HTTP API row surprises people, because HTTP APIs are usually the more modern option. On streaming, REST is the capable one.

Currency note: REST APIs can now stream

- **The examinable answer:** API Gateway has a 29-second integration timeout and a 10 MB payload cap.
- Since **November 2025**, REST APIs support response streaming.
- That extends integration timeouts to **15 minutes** and allows payloads **over 10 MB**.
- Works with Lambda, HTTP proxy and private integrations. REST only, not HTTP APIs.
- **Answer 29 seconds and 10 MB on the exam.** Exam guide v2.1 predates this by a year.

> ON THE EXAM
>
> Two of the most reliable API Gateway facts became conditional. Before November 2025 the only way to stream from a REST surface was a function URL behind an HTTP_PROXY integration.

Lambda@Edge

- Node.js or Python, running at CloudFront **regional edge caches**.
- All **four** trigger points, including origin request and origin response.
- Network access, filesystem access, and it can call other AWS services.
- Must be created in **us-east-1** and deployed from there to the edge.
- No environment variables, no VPC access, no layers. Deployment takes minutes.

> ON THE EXAM
>
> The three missing features catch people out. Configuration cannot come from environment variables, so it is hardcoded or fetched at runtime and cached.

CloudFront Functions

- JavaScript only, running at **CloudFront edge locations** themselves.
- **Viewer request and viewer response only.** No origin triggers.
- Sub-millisecond execution, and roughly a sixth the price of Lambda@Edge.
- **2 MB memory, 10 KB of code, under 1 ms.** Severe by design.
- No network access, no filesystem, no access to the request body.

> ON THE EXAM
>
> The 'no network access' limit is the cleanest eliminator in the pair. If the code must look anything up, it cannot be a CloudFront Function.

CloudFront Functions or Lambda@Edge?

CLOUDFRONT FUNCTIONS

Rewrite, redirect, header work

- URL rewrites and redirects
- Header manipulation
- Cache key normalisation
- Simple token checks
- Millions of requests, lowest cost

LAMBDA@EDGE

Anything that needs to think

- Calls another service or API
- Reads or changes the request body
- Origin request or origin response
- Image resizing, content generation
- Needs more than 10 KB of code

THE TIEBREAKER

Three questions decide it: does it need the network, does it need an origin trigger, and is the code over 10 KB? Any yes means Lambda@Edge.

The four CloudFront trigger points

TRIGGER	WHEN IT RUNS, AND WHAT IT IS FOR
Viewer request	Every request, before the cache is checked. Auth, rewrites, cache keys
Origin request	Only on a cache miss, before going to the origin. Origin selection, lookups
Origin response	Only on a cache miss, after the origin replies. Transforming content
Viewer response	Every response, before it reaches the user. Security headers

ON THE EXAM

Viewer triggers run on every request. Origin triggers run only on cache misses, so they are cheaper and can safely do more work.

Lambda behind an Application Load Balancer

- A Lambda function can be a **target group** on an ALB, alongside EC2 and containers.
- Useful when routing rules already live in an ALB, or during a gradual migration.
- **1 MB payload limit** in both directions – far smaller than API Gateway's 10 MB.
- No response streaming, no API keys, no usage plans, no request validation.
- The ALB is billed hourly, so it only makes sense if one already exists.

> ON THE EXAM
>
> The realistic use case is a migration: existing traffic already flows through an ALB, and one path is being moved to Lambda without redesigning the routing.

Currency note: Lambda Durable Functions

- **The examinable rule:** Lambda stops at 15 minutes, so longer workflows need Step Functions.
- Durable Functions let a function run or wait for **up to one year**.
- You write ordinary sequential code with a durable context object and checkpoints.
- **context.wait()** suspends the execution, and waiting time is not charged.
- **Not in exam guide v2.1.** Answer 15 minutes and Step Functions on the day.

> READ THIS TWICE
>
> This is the single most useful thing to know about modern Lambda and the single most dangerous thing to answer with. The exam's rule is unchanged.

Durable Functions or Step Functions?

STEP FUNCTIONS

The examinable answer

- Workflow defined outside your code
- Visual state machine and history
- Service integrations without code
- Priced per state transition
- What the exam expects

DURABLE FUNCTIONS

The 2026 reality

- Workflow written as ordinary code
- Automatic checkpoint and replay
- Up to one year, idle time unbilled
- Node.js and Python at launch
- Not examinable on DVA-C02

THE TIEBREAKER

Included so you are not confused when you meet it at work. On the exam, a long-running or multi-step workflow is Step Functions. Module 13 covers that properly.

The revision gap, in one table

THE EXAM STILL SAYS	AWS NOW ALSO OFFERS
Lambda hard-stops at 15 minutes	Durable Functions: up to one year, idle unbilled
API Gateway integration timeout is 29 seconds	REST response streaming: up to 15 minutes
API Gateway payloads cap at 10 MB	REST response streaming: larger payloads
One execution environment per concurrent request	Managed Instances: shared, EC2-backed

ON THE EXAM

Answer the left column on exam day. Expect the right column when you open current AWS documentation. Module 31 collects every one of these.

Two of these turn on a constraint you have to **notice** rather than calculate. Read the second paragraph carefully.

A company runs a customer support assistant on Amazon Bedrock behind a Lambda function. Responses take 20 to 40 seconds to generate fully, and users currently stare at a spinner until the whole answer arrives. The team wants tokens to appear as they are produced.

The assistant is one route on an existing API Gateway REST API. Access is authorised by a Cognito user pool authoriser, and each customer tier has a different request quota enforced by a usage plan. Both must be preserved.

How should the team deliver the streamed response?

A. Replace the route with a Lambda function URL using InvokeMode RESPONSE_STREAM

B. Enable response streaming on the existing API Gateway REST API route

C. Migrate the API to an API Gateway HTTP API and enable response streaming

D. Put the response in Amazon S3 and return a presigned URL for the client to poll

◆ PAUSE AND DECIDE

How should the team deliver the streamed response?

A Function URL with RESPONSE_STREAM

Streams correctly, but function URLs support only IAM or public auth and have no usage plans. Both stated requirements would be lost.

B Response streaming on the REST API

Keeps the Cognito authoriser and the usage plans, keeps the existing route, and REST APIs have supported response streaming since November 2025.

C Migrate to an HTTP API

HTTP APIs do not support response streaming at all, and migrating would also lose usage plans, which HTTP APIs lack.

D S3 plus polling

A pre-streaming workaround. It does not improve time to first byte at all - the user still waits for the whole answer, now with polling on top.

THE SIGNAL

Three constraints: stream, keep Cognito, keep usage plans. Only one option holds all three - and only because of a change made in November 2025.

A media site serves images through Amazon CloudFront. The team wants to normalise the cache key by lowercasing the path and stripping tracking query parameters, and to add a set of security headers to every response.

The distribution serves about 2 billion requests a month. The logic is roughly 40 lines of JavaScript, needs no external data, and must add as little latency as possible.

How should this be implemented?

A. A Lambda@Edge function on the origin request and origin response triggers

B. A Lambda@Edge function on the viewer request and viewer response triggers

C. CloudFront Functions on the viewer request and viewer response triggers

D. A CloudFront response headers policy plus a Lambda@Edge viewer request function

◆ PAUSE AND DECIDE

How should this be implemented?

A Lambda@Edge on origin triggers

Origin triggers only fire on cache misses. Cache key normalisation must happen before the cache is checked, so it would never run for cached requests.

B Lambda@Edge on viewer triggers

It would work, at roughly six times the cost across 2 billion requests, with higher latency and no capability actually being used.

C CloudFront Functions on viewer triggers

No network access needed, code well under 10 KB, viewer triggers only - it fits every constraint, at the lowest cost and latency.

D Headers policy plus Lambda@Edge

The headers policy half is genuinely good practice. Pairing it with Lambda@Edge for a simple rewrite still overpays for the rewrite.

THE SIGNAL

Option A is the real trap. Cache key normalisation has to happen at viewer request, because by origin request the cache has already been consulted.

> An internal reporting tool needs to trigger a Lambda function over HTTPS. The only callers are three other services inside the same AWS account, all of which already use the AWS SDK and have IAM roles.
>
> There are no plans for external access, no need for routing beyond a single endpoint, and no per-caller quotas. The platform team has asked for the option with the least operational overhead and the lowest cost.

What should the developer implement?

A. An API Gateway REST API with IAM authorisation

B. An API Gateway HTTP API with a JWT authoriser

C. An Application Load Balancer with the function as a target group

D. A Lambda function URL with AuthType AWS_IAM

◆ PAUSE AND DECIDE

What should the developer implement?

A REST API with IAM auth

Correct and secure, but you pay per request for routing, stages and usage plans that the scenario explicitly says are not needed.

B HTTP API with JWT authoriser

Introduces a token issuer the callers do not have. They already hold IAM roles, so JWT adds a moving part for nothing.

C ALB target group

An ALB bills hourly whether used or not, and none is mentioned as existing. The most expensive option here by some distance.

D Function URL with AWS_IAM

One endpoint, SigV4 auth the callers can already produce, no extra charge, and a single configuration step.

THE SIGNAL

This scenario is the mirror image of scenario 1. Same choice, opposite answer, because every managed API feature was explicitly ruled out.

A retailer serves product pages through CloudFront. When a product is discontinued, requests for its URL must be redirected to the replacement product. The mapping of old URL to new URL lives in a DynamoDB table and changes several times a day.

Redirects apply to roughly 3% of traffic. The team wants the lookup to happen only when CloudFront does not already have the page cached.

Where should the redirect logic run?

A A Lambda@Edge function on the origin request trigger

B A CloudFront Function on the viewer request trigger

C A Lambda@Edge function on the viewer request trigger

D A CloudFront Function on the viewer request trigger, with the mapping in the function code

◆ PAUSE AND DECIDE

Where should the redirect logic run?

A Lambda@Edge on origin request

Only Lambda@Edge can query DynamoDB, and the origin request trigger fires only on cache misses - exactly the behaviour asked for.

B CloudFront Function on viewer request

CloudFront Functions have no network access, so the DynamoDB lookup is impossible. This is the cleanest disqualification in the pair.

C Lambda@Edge on viewer request

Capable of the lookup, but viewer request fires on every request including cache hits. It contradicts the stated requirement and costs far more.

D CloudFront Function with the mapping in code

Sidesteps the network limit, but the 10 KB code limit will not hold a product mapping, and it must be redeployed several times a day.

THE SIGNAL

Two constraints, two eliminations. 'Lives in DynamoDB' removes CloudFront Functions. 'Only when not cached' removes viewer triggers.

What to carry forward

- Function URLs offer **IAM or public**. No API keys, no usage plans, no authorisers.
- Streaming: 6 MB buffered, 20 MB streamed. REST APIs stream, HTTP APIs do not.
- Edge: **network access, origin trigger, or over 10 KB** means Lambda@Edge.
- Viewer triggers run on every request. Origin triggers run only on cache misses.
- Four exam rules now have exceptions in the live service. **Answer the exam rule.**

> ON THE EXAM
>
> That completes Lambda. Next: DynamoDB, starting with keys, partitions and indexes – the second most tested service on this exam.

MODULE 07 · DOMAIN 1 - DEVELOPMENT WITH AWS SERVICES

DynamoDB I – Keys, Partitions and Indexes

The one decision you cannot change later, and the two index types the exam will not stop asking about.

DOMAIN 1 of 4 EXAM WEIGHT 32% MODULE 7 of 34

admin@sthithapragna.com

In a relational database you model the data and query it later. In DynamoDB you model the **queries**, and the data arrangement follows.

The vocabulary

- **Table** holds **items**, and each item is a set of **attributes**. There is no fixed schema beyond the key.
- **Partition key** decides which physical partition stores the item. Also called the hash key.
- **Sort key** is optional. It orders items that share a partition key. Also called the range key.
- **Item collection** is every item sharing one partition key value.
- Keys must be scalar: **String, Number or Binary**. Never a list, map or boolean.

> ON THE EXAM
>
> Only the key attributes need to exist on every item. Everything else varies item to item, which is what makes single-table design possible.

Attribute data types

- **Scalar:** String, Number, Binary, Boolean, Null. Only the first three can be keys.
- **Document:** List and Map, nestable up to 32 levels deep.
- **Set:** String Set, Number Set, Binary Set. Unordered, no duplicates, cannot be empty.
- Numbers hold up to 38 digits of precision, so pass them as strings when precision matters.
- Attribute names count towards the 400 KB item size, so short names save real money at scale.

> ON THE EXAM
>
> The examinable line is that keys must be String, Number or Binary. A boolean or a list can never be a partition or sort key.

Simple key or composite key?

PARTITION KEY ONLY

One item per key value

- The key must be unique per item
- GetItem by key is the access pattern
- No Query within a key
- Suits lookup tables and sessions
- Simplest possible design

PARTITION KEY PLUS SORT KEY

Many items per key value

- Uniqueness is the pair, not the key
- Query returns a range of items
- Items stored sorted by sort key
- Suits one-to-many relationships
- Enables nearly all rich patterns

THE TIEBREAKER

If a partition key value ever needs to hold more than one item, you need a sort key. Adding one later means rebuilding the table, so decide early.

How a write finds its home

01

Hash

DynamoDB hashes the partition key value.

02

Route

The hash selects one physical partition.

03

Store

The item is written, ordered by sort key within that partition.

ON THE EXAM

Everything about hot partitions, throttling and key design follows from this one mechanism. The sort key plays no part in choosing the partition.

Choosing a partition key

- **High cardinality.** Many distinct values, so data spreads across many partitions.
- **Uniform access.** Requests should be spread roughly evenly across those values.
- Good: customer ID, order ID, device ID, session ID.
- Bad: status, country, a boolean, today's date, or any fixed value.
- Cardinality alone is not enough. A million keys where one is 90% of traffic is still hot.

> ON THE EXAM
>
> Two properties, and questions test both. 'Many distinct values' is necessary. 'Accessed evenly' is what actually prevents throttling.

Hot partitions

- Each physical partition is capped at roughly **3,000 RCU** and **1,000 WCU** per second.
- That cap is **per partition**, so table-level capacity cannot rescue a single hot key.
- The symptom: **throttling while consumed capacity sits well below provisioned**.
- **Adaptive capacity** shifts throughput toward busy partitions and can split them automatically.
- Splitting takes minutes, and it cannot lift a single partition past the hard cap.

> ON THE EXAM
>
> That symptom line is the giveaway. Throttling with capacity to spare is a key design problem, never a capacity problem. Raising provisioned capacity will not help.

Write sharding

- Append a suffix to the partition key: **2026-09-04#1** through **2026-09-04#10**.
- Ten shards means ten partitions, so roughly ten times the write ceiling.
- A **random** suffix spreads writes best. A **calculated** one keeps reads targeted.
- Reads now need a query per shard, and the results merged in your code.
- Choose the shard count deliberately. Changing it later means rewriting historical keys.

> ON THE EXAM
>
> This is the standard answer for time-series data keyed by date. It trades read simplicity for write throughput, and the exam expects you to name that trade.

Four ways to model one-to-many

- **Sort key on the same partition.** One Query returns the parent and all children.
- **A nested list or map** inside the parent item. Simple, but bounded by 400 KB.
- **A GSI** where the child's foreign key becomes the index partition key.
- **Composite sort key** when the relationship is hierarchical rather than flat.
- Never a second table you join in application code. That is the pattern to avoid.

> ON THE EXAM
>
> Unbounded children means separate items. A small, bounded set that is always read with the parent can live inside the item.

Sort keys do more than sort

- Query supports **begins_with**, **between**, and the comparison operators on the sort key.
- A **composite sort key** encodes a hierarchy: **COUNTRY#STATE#CITY**.
- begins_with then queries at any level of that hierarchy with one index.
- An ISO timestamp as sort key gives you date ranges for free, since it sorts lexically.
- Sort key conditions are exact prefixes. There is no contains and no wildcard in the middle.

> ON THE EXAM
>
> Most 'I need another index' problems are really 'I need a better sort key' problems. Hierarchical sort keys serve several access patterns from one structure.

The constraints worth memorising

LIMIT	VALUE
Item size	400 KB, including attribute names
Partition key value	1 to 2,048 bytes
Sort key value	1 to 1,024 bytes
Query or Scan page	1 MB, then a LastEvaluatedKey to continue
Global secondary indexes	20 per table by default, and it can be raised
Local secondary indexes	5 per table, fixed, and only at creation
Item collection with an LSI	10 GB per partition key value

ON THE EXAM

The 400 KB limit drives the standard pattern: store large objects in S3 and keep the key in DynamoDB. The last row is a real design trap.

Secondary indexes exist for one reason: to answer a question your primary key cannot.

Global secondary indexes

- **Any attribute** can be the index partition key and sort key. Both differ from the table.
- Physically separate, with **its own capacity**, its own partitions and its own throttling.
- **Create or delete at any time**, on a live table, without downtime.
- **Eventually consistent reads only.** Strongly consistent reads are impossible on a GSI.
- Every write to the table becomes an extra write to each GSI, and you pay for it.

> ON THE EXAM
>
> GSIs are the default answer for a new access pattern. Their two real costs are eventual consistency and duplicated write capacity.

Local secondary indexes

- **Same partition key** as the table, and a different sort key. That is the whole idea.
- Stored alongside the item in the **same partition**, which is what 'local' means.
- **Supports strongly consistent reads.** The only reason to choose one.
- Shares the table's capacity rather than having its own.
- **Must be created with the table.** You cannot add or remove one afterwards.

> ON THE EXAM
>
> And the sting: any table with an LSI caps each item collection at 10 GB. Without an LSI there is no such ceiling.

GSI overloading

- Generic index keys, **GSI1PK** and **GSI1SK**, holding different meanings per item type.
- An order item might set GSI1PK to a status; a customer item might set it to an email.
- One index then serves several unrelated access patterns.
- Keeps you well inside the 20-index default quota.
- Queries must know which item type they expect, since results are mixed.

> ON THE EXAM
>
> The companion to single-table design. When you see GSI1PK in a schema, this is what is happening.

When a GSI throttles the table

- **GSI read** throttling does not affect the base table at all.
- **GSI write** throttling throttles writes to the **base table**, and to its other indexes.
- DynamoDB applies **back pressure** so the index cannot fall too far behind the table.
- So provision each GSI's write capacity at or above the table's.
- A GSI has its own partitions, so a well-spread table can still have a hot index.

> ON THE EXAM
>
> This is genuinely counterintuitive: writes to a healthy table fail because an index is short of capacity. Look for IndexWriteProvisionedThroughputExceeded.

Time-series data

- A date as partition key concentrates every write of that day on one partition.
- **Shard the key**, or use a higher-cardinality entity like device or user as the partition key.
- **A table per period** - one per month - keeps the current table small and hot.
- Old tables can then be exported and deleted cheaply, instead of deleting item by item.
- Or use **TTL** to expire old items automatically at no write cost.

> ON THE EXAM
>
> Deleting millions of expired rows with DeleteItem costs a WCU each. A table per period, or TTL, avoids that entirely.

Key design anti-patterns

ANTI-PATTERN	WHAT HAPPENS
A low-cardinality partition key	Few partitions, so throttling well below provisioned capacity
A date or timestamp as partition key	All of today's traffic hits one partition
Adding variety to the sort key to fix heat	No effect. The sort key does not choose the partition
Scanning with a filter as a query	Pays to read everything, returns a fraction
A GSI with a low-cardinality partition key	A hot index, which then throttles the base table

ON THE EXAM

Row three is the one worth remembering. Diversifying a sort key never fixes a hot partition, and it appears as a distractor.

Three ways to read, at a glance

OPERATION	WHAT IT DOES, AND WHAT IT COSTS
`GetItem`	One item by full primary key. The cheapest possible read
`Query`	Items sharing one partition key, optionally filtered by sort key. Efficient
`Scan`	Every item in the table or index. Expensive, and rarely correct

ON THE EXAM

Query needs the partition key value. If you cannot supply one, you need a different index - not a Scan.

GSI or LSI?

GLOBAL SECONDARY INDEX

Almost always this one

- Any partition key and sort key
- Added or removed at any time
- Its own capacity and throttling
- Eventually consistent only
- No size limit on collections

LOCAL SECONDARY INDEX

Only for strong consistency

- Same partition key, new sort key
- Created with the table only
- Shares the table's capacity
- Strongly consistent reads possible
- Caps item collections at 10 GB

THE TIEBREAKER

Choose an LSI only when you need strongly consistent reads on an alternate sort order, and you know it at table creation. Every other case is a GSI.

Projections

- **KEYS_ONLY** – table keys plus index keys. Smallest and cheapest.
- **INCLUDE** – keys plus a named list of attributes you choose.
- **ALL** – every attribute. Largest storage and highest write cost.
- A GSI query returning a non-projected attribute **does not fetch it**. It is simply absent.
- An LSI query can fetch non-projected attributes, at the cost of extra reads.

> ON THE EXAM
>
> The GSI behaviour catches people. An attribute missing from a query result is usually a projection problem, not a data problem.

Sparse indexes

- An item appears in an index **only if it has the index key attribute**.
- So write that attribute only on items you want indexed, and the index stays tiny.
- Classic use: an **unprocessed** flag, deleted once the item is handled.
- Querying a handful of items in a billion-item table, with no scan at all.
- Setting the attribute to null or empty does not remove it. It must be absent.

> ON THE EXAM
>
> When a scenario needs to find the small set of items in some state across a huge table, a sparse GSI is usually the intended answer.

Single-table design, briefly

- Generic key names, **PK** and **SK**, holding different meanings per item type.
- Prefixes keep types apart: **CUSTOMER#123**, **ORDER#456**, **PRODUCT#789**.
- Related items share a partition key, so one Query fetches a customer and their orders.
- That replaces the join DynamoDB does not have.
- Harder to read and to change. Not required, and not always right.

> ON THE EXAM
>
> The exam rarely asks you to design a single-table schema. It does expect you to recognise PK and SK with prefixed values and understand why they look like that.

Three of these are design questions in disguise. Ask what the **access pattern** is before you look at the options.

An IoT platform writes sensor readings to a DynamoDB table. The partition key is the date in YYYY-MM-DD format and the sort key is a timestamp. About 40,000 readings arrive per second, spread across 50,000 devices.

The table is provisioned with 60,000 WCUs but writes are being throttled heavily. CloudWatch shows consumed write capacity at roughly 5% of provisioned. Queries always ask for one device's readings over a time range.

What is the root cause, and what should the developer change?

A Increase provisioned write capacity, since 40,000 writes per second is close to the limit

B Change the partition key to the device ID and the sort key to the timestamp

C Add a global secondary index with device ID as the partition key

D Switch the table to on-demand capacity mode

◆ PAUSE AND DECIDE

What is the root cause, and what should the developer change?

A More provisioned capacity

The table already has twelve times the capacity it is consuming. Per-partition limits are the constraint, and table capacity cannot raise them.

B Device ID as partition key

Fifty thousand devices spreads writes across many partitions, and it matches the stated query pattern exactly. It fixes the cause rather than the symptom.

C GSI on device ID

Improves reads, but every write still lands on today's date partition. The write throttling is untouched, and the GSI adds write cost.

D On-demand mode

Removes capacity planning, not per-partition limits. A single hot key throttles in on-demand mode too.

THE SIGNAL

Consumed capacity at 5% of provisioned while throttling is the fingerprint of a hot partition. One date means one partition takes every write.

> An existing orders table uses customer ID as the partition key and order date as the sort key. It has been in production for two years and holds several hundred gigabytes.
>
> A new feature must list all orders with a given status, such as PENDING or SHIPPED, across all customers. Status is already stored on every order item. Eventually consistent results are acceptable, and pending orders are under 1% of the table.

How should the developer support the new access pattern?

A. Add a local secondary index with status as the sort key

B. Scan the table with a filter expression on status

C. Create a global secondary index with status as the partition key and order date as the sort key

D. Create a new table keyed by status and dual-write to both tables

◆ PAUSE AND DECIDE

How should the developer support the new access pattern?

A Add an LSI

Impossible twice over. LSIs can only be created with the table, and an LSI keeps the table's partition key, so it could never search across customers.

B Scan with a filter

A filter is applied after reading, so you pay to read hundreds of gigabytes to return under 1% of it. Slow and expensive.

C GSI on status

A different partition key on a live table is exactly what a GSI is for, eventual consistency is accepted, and the sort key gives date ordering for free.

D Second table with dual writes

Hand-building what a GSI does automatically, and introducing a consistency bug the moment one of the two writes fails.

THE SIGNAL

Two sentences settle it: the table already exists, and eventual consistency is fine. The first kills the LSI, the second removes the only reason to want one.

A document management system stores metadata in DynamoDB. Each document item holds a title, owner, tags and a full extracted text body. Text bodies range from a few kilobytes to about 12 MB.

Writes are failing intermittently for larger documents. The team needs full-text bodies retrievable by document ID, and wants to keep querying metadata efficiently.

How should the developer restructure the storage?

A Split the text body across multiple items using a chunk number in the sort key

B Compress the text body with gzip before writing it to the item

C Request an item size limit increase from AWS Support

D Store the text body in Amazon S3 and keep the object key in the DynamoDB item

◆ PAUSE AND DECIDE

How should the developer restructure the storage?

A Chunk across items

A documented pattern, but 12 MB needs about thirty items, and every read reassembles them. It also inflates the item collection considerably.

B Compress with gzip

Helps, and does not solve it. Text compressing 4:1 still leaves 3 MB, well past 400 KB. It also makes the body unreadable to queries.

C Request a limit increase

The 400 KB item size is a hard limit. No support request changes it.

D Body in S3, key in DynamoDB

The standard pattern. Metadata stays small and fast to query, S3 handles objects of any size, and the item holds a pointer.

THE SIGNAL

'Intermittent failures on larger items' names the 400 KB ceiling without saying it. Large object in S3, pointer in DynamoDB, is the answer AWS documents.

A billing table holds about 900 million invoice items. A nightly job must find every invoice in the DISPUTED state and route it for review. Disputed invoices number in the low thousands at any time, and the flag is cleared once an invoice is resolved.

The current job scans the entire table each night, takes over four hours, and consumes most of the account's read capacity.

What is the most efficient way to find the disputed invoices?

A Create a GSI on a disputedAt attribute that is written only while an invoice is disputed, and removed when resolved

B Create a GSI with status as the partition key and query for DISPUTED

C Keep the scan but add a filter expression on status and run it in parallel segments

D Enable DynamoDB Streams and maintain a separate list of disputed invoice IDs in ElastiCache

◆ PAUSE AND DECIDE

What is the most efficient way to find the disputed invoices?

A Sparse GSI on disputedAt

Only disputed invoices carry the attribute, so the index holds a few thousand items instead of 900 million. Removing the attribute on resolution removes the item from the index.

B GSI on status

Works, and it indexes all 900 million items, duplicating storage and adding a write to every invoice change. It also makes DISPUTED a low-cardinality partition key.

C Parallel scan with a filter

Faster in wall-clock time, and it still reads all 900 million items and consumes the same capacity. Filters do not reduce what is read.

D Streams into ElastiCache

Builds and operates an index by hand, adds a cache to keep consistent, and loses the data if the cache is flushed.

THE SIGNAL

Compare with scenario 2, where a full GSI on status was correct. Here the target set is a millionth of the table and the flag is cleared - which is exactly when sparse wins.

What to carry into DynamoDB II

- Model the **access patterns** first. The primary key cannot be changed later.
- Only the partition key spreads load. Per-partition caps are ~3,000 RCU and ~1,000 WCU.
- **Throttling with capacity to spare** means a hot partition, not a capacity shortage.
- **LSI only for strong consistency, and only at table creation.** Otherwise GSI.
- Sparse indexes find a small set in a huge table without ever scanning.

> ON THE EXAM
>
> Next: reading and writing. Query against Scan, consistency, conditional writes, transactions, TTL, capacity modes and DAX.

MODULE 08 · DOMAIN 1 - DEVELOPMENT WITH AWS SERVICES

DynamoDB II – Reading, Writing and Capacity

Query against Scan, consistency, conditional writes, transactions, capacity modes and DAX.

DOMAIN 1 of 4 EXAM WEIGHT 32% MODULE 8 of 34

admin@sthithapragna.com

DynamoDB charges you for what it **reads**, not for what it returns. Almost every cost surprise comes from forgetting that.

The three read operations

OPERATION	WHAT IT READS
`GetItem`	One item, by complete primary key. The cheapest read there is
`Query`	Items under one partition key. Reads only the matching range
`Scan`	Every item in the table or index, page by page

ON THE EXAM

Query requires a partition key value. If your access pattern cannot supply one, the fix is an index, not a Scan.

Query in detail

- **KeyConditionExpression** selects what is read. Equality on the partition key, plus an optional sort key condition.
- **FilterExpression** discards items **after** they are read. It reduces the response, never the cost.
- **ProjectionExpression** chooses which attributes come back. It does not reduce read cost either.
- **ScanIndexForward: false** returns items in descending sort key order. The way to get 'most recent'.
- The partition key condition must be equality. No ranges, no begins_with.

> ON THE EXAM
>
> Only the key condition affects what you pay. Getting FilterExpression and KeyConditionExpression the wrong way round is a standard exam trap.

Query cost, worked

- A partition holds 1,000 items of 2 KB each. You query it with a filter matching 10 of them.
- DynamoDB reads all 1,000 items: 2,000 KB, which is 500 blocks of 4 KB.
- Eventually consistent, that is **250 RCUs** – to return 10 items.
- A sort key condition instead of a filter would read only the matching range.
- The 1 MB page limit also applies **before** filtering, so results arrive in pages.

> ON THE EXAM
>
> Same result set, wildly different cost. This is the single most useful calculation to be able to do quickly.

Scan, and when it is acceptable

- Reads every item, then applies any filter. Cost is proportional to **table size**, not results.
- Fine on genuinely small tables, or for one-off administrative work.
- **Parallel scan** splits the table into segments read concurrently. Faster, not cheaper.
- A parallel scan can consume all your capacity and throttle production traffic.
- Analytics over a whole table belongs in S3 or Athena, via an export.

> ON THE EXAM
>
> Parallel scan reduces wall-clock time and consumes capacity faster. If a scenario complains about cost, parallel scan is the wrong answer.

Pagination

- Query and Scan return at most **1 MB** per call, then stop.
- The response carries **LastEvaluatedKey** if more data exists.
- Pass it back as **ExclusiveStartKey** to fetch the next page.
- **Limit** caps items **read**, before filtering. So a filtered page can come back empty.
- No LastEvaluatedKey means you are done. An empty page does not.

> ON THE EXAM
>
> An empty page with a LastEvaluatedKey is normal, not a bug. Code that stops on the first empty result silently misses data.

Eventually or strongly consistent?

EVENTUALLY CONSISTENT

The default

- May not reflect a very recent write
- Costs half an RCU per 4 KB
- The only option on a GSI
- Reads can be served by any replica
- Right for the large majority of reads

STRONGLY CONSISTENT

Ask for it explicitly

- Always reflects all prior writes
- Costs a full RCU per 4 KB
- Set ConsistentRead to true
- Not available on a GSI, ever
- Higher latency, and less available

THE TIEBREAKER

Twice the price for the guarantee. Read-after-write on the same item is the case that needs it - and if the read goes through a GSI, it is simply impossible.

Capacity unit arithmetic

OPERATION	UNITS CONSUMED
Strongly consistent read	1 RCU per 4 KB, rounded up
Eventually consistent read	0.5 RCU per 4 KB, rounded up
Transactional read	2 RCU per 4 KB
Write	1 WCU per 1 KB, rounded up
Transactional write	2 WCU per 1 KB

ON THE EXAM

Reads round to 4 KB, writes round to 1 KB, and everything rounds up. A 0.5 KB write still costs a full WCU.

Provisioned or on-demand?

PROVISIONED

You set the capacity

- Cheapest for steady, predictable load
- Auto scaling adjusts within bounds
- Reserved capacity gives deeper discounts
- Throttles when you exceed it
- Needs capacity planning

ON-DEMAND

Pay per request

- No capacity planning at all
- Instant response to spikes
- Notably dearer per request
- Ideal for new or spiky workloads
- Still bound by per-partition limits

THE TIEBREAKER

Switch modes as often as once every 24 hours. On-demand removes planning, not physics - a hot key throttles in either mode.

Auto scaling, burst and reserved capacity

- **Auto scaling** tracks a target utilisation, typically 70%, between a floor and a ceiling you set.
- It reacts to CloudWatch alarms, so it takes **minutes**. It cannot absorb a sudden spike.
- **Burst capacity** banks up to 5 minutes of unused capacity, which covers short bursts.
- Burst capacity is best effort and can be consumed by background processes.
- **Reserved capacity** is a 1 or 3 year commitment for a large discount. Provisioned mode only.

> ON THE EXAM
>
> Auto scaling handles trends, burst capacity handles seconds, and neither handles a flash sale. That gap is what on-demand is for.

Writing items

- **PutItem** replaces the entire item. Attributes you omit are deleted.
- **UpdateItem** modifies named attributes and creates the item if it is absent.
- Update actions: **SET**, **REMOVE**, **ADD**, **DELETE**.
- **ADD** on a number is an **atomic counter**. Concurrent increments never lose one.
- **ReturnValues** can hand back the item before or after the change, saving a second read.

> ON THE EXAM
>
> The PutItem behaviour surprises people. Reading an item, changing one field and putting it back is how concurrent updates get lost.

Conditional writes

- **ConditionExpression** applies the write only if the condition holds.
- If it fails, you get **ConditionalCheckFailedException** and nothing is written.
- **attribute_not_exists(pk)** turns PutItem into insert-if-absent, preventing overwrites.
- **attribute_exists(pk)** turns it into update-only.
- A failed condition still costs capacity. You paid for the check.

> ON THE EXAM
>
> Conditional writes are how idempotency is implemented on DynamoDB. Module 5's idempotency key pattern is exactly this.

Expression attribute names and values

- **#name** placeholders substitute attribute names. **:value** placeholders substitute values.
- DynamoDB has **reserved words**, including name, status, size, timestamp and count.
- Using one directly raises a ValidationException, which reads confusingly.
- ExpressionAttributeNames sidesteps the whole problem: **#s** mapped to **status**.
- Values must always be placeholders. You cannot inline a literal in an expression.

> ON THE EXAM
>
> A ValidationException on a query that looks correct is very often a reserved word. Status and name catch nearly everyone once.

Optimistic locking

- Keep a **version** attribute on the item and increment it on every write.
- Condition the write on the version you read. If it moved, someone else got there first.
- The application then re-reads and retries, or reports a conflict to the user.
- No locks are held, so nothing blocks and nothing deadlocks.
- Use an atomic counter instead when the update is a pure increment. It never conflicts.

> ON THE EXAM
>
> Optimistic locking is for read-modify-write on a whole item. Atomic counters are for arithmetic. Choosing the wrong one shows up as either lost updates or needless retries.

Batch and transaction limits

OPERATION	LIMITS AND BEHAVIOUR
`BatchGetItem`	Up to 100 items, 16 MB. Reads in parallel across tables
`BatchWriteItem`	Up to 25 puts or deletes, 16 MB. No updates, no conditions
`TransactGetItems`	Up to 100 items. All-or-nothing, 2x read cost
`TransactWriteItems`	Up to 100 actions. All-or-nothing, 2x write cost

ON THE EXAM

25 is the odd one out. Every other bulk operation caps at 100, because BatchWriteItem never received the increase the others did in 2022.

Batch or transaction?

BATCHWRITEITEM

Throughput

- Individual writes are independent
- Partial failure is normal
- Failures return in UnprocessedItems
- You must retry them yourself
- Standard capacity cost

TRANSACTWRITEITEMS

Correctness

- All succeed or none do
- Supports conditions per action
- Can span tables in one Region
- Fails whole if any condition fails
- Double the capacity cost

THE TIEBREAKER

Batch is a convenience wrapper around many independent writes. A transaction is an atomicity guarantee, and you pay double for it.

DynamoDB Streams

- An ordered log of item-level changes, retained for **24 hours**.
- View types: **KEYS_ONLY**, **NEW_IMAGE**, **OLD_IMAGE**, **NEW_AND_OLD_IMAGES**.
- Ordered per partition key, and read by Lambda through an event source mapping.
- Powers replication, aggregation, search indexing and audit trails.
- 24 hours is the whole retention. Miss the window and the change is gone.

> ON THE EXAM
>
> NEW_AND_OLD_IMAGES is what you need to compute what actually changed. KEYS_ONLY tells you only that something did.

Time to Live

- Name a Number attribute holding a **Unix epoch in seconds**. Not milliseconds.
- **Deletion consumes no write capacity.** It is genuinely free.
- Deletion is best effort, **typically within 48 hours** of expiry. It is not immediate.
- So expired items remain readable, and still cost RCUs, until removed.
- TTL deletions appear in Streams as REMOVE events, which is how you archive before losing them.

> ON THE EXAM
>
> Two traps: milliseconds instead of seconds means items never expire, and queries must filter out expired items themselves during the deletion window.

DynamoDB Accelerator

- A fully managed, in-memory cache in front of DynamoDB. Microsecond reads.
- **API-compatible**, so adopting it is a client change, not a code rewrite.
- Two caches: an **item cache** for GetItem, and a **query cache** for Query and Scan.
- **Write-through**: writes go to DynamoDB and the cache together.
- Runs in a VPC, and it **cannot serve strongly consistent reads**. Those pass through.

> ON THE EXAM
>
> DAX for read-heavy DynamoDB traffic; ElastiCache when you are caching something computed, or not from DynamoDB at all.

DAX or ElastiCache?

CHOOSE DAX WHEN

Caching DynamoDB itself

- The data source is DynamoDB
- You want no application changes
- Read-heavy, repetitive access
- Write-through keeps it fresh
- Microsecond reads are the goal

CHOOSE ELASTICACHE WHEN

Caching anything else

- Caching computed or aggregated results
- The source is RDS or an external API
- You need data structures like sorted sets
- You control invalidation yourself
- Also used for sessions and rate limiting

THE TIEBREAKER

If the scenario says 'reduce DynamoDB read latency with minimal code change', it is DAX. Anything requiring lists, counters or leaderboards is ElastiCache.

Choosing a data store

- **Ad hoc queries** you cannot predict. DynamoDB needs the access pattern up front.
- **Complex joins and aggregations** across entities. That is a relational problem.
- **Full-text search.** Use OpenSearch, fed from DynamoDB Streams.
- DynamoDB wins on **predictable single-digit millisecond** access at any scale.
- It also wins when you want no servers, no patching and no connection pool.

> ON THE EXAM
>
> An exam scenario describing reporting, analytics or arbitrary filtering is usually steering you to Aurora, Athena or OpenSearch, not to a bigger DynamoDB table.

Errors and retries

- **ProvisionedThroughputExceededException** means throttling. Retry with backoff.
- **ConditionalCheckFailedException** means your condition was false. **Do not retry blindly.**
- **ValidationException** is a malformed request. Retrying will never help.
- The AWS SDKs retry 5xx and throttling errors automatically, with exponential backoff and jitter.
- 4xx errors other than throttling are your bug, and the SDK will not retry them.

> ON THE EXAM
>
> The dividing line: 5xx and throttling are worth retrying, other 4xx errors are not. Conditional check failures need application logic, not a retry loop.

Two things worth recognising

- **PartiQL** gives DynamoDB a SQL-like query syntax over the same underlying operations.
- It does **not** add joins, and a badly written statement still becomes a Scan.
- **Global tables** replicate a table across Regions, active-active, in about a second.
- Conflicts resolve **last writer wins**, which is not always what you want.
- Global tables require Streams to be enabled on the table.

> ON THE EXAM
>
> PartiQL is a syntax, not a capability. Familiar SQL does not make an inefficient access pattern efficient.

Atomicity, at a glance

WHAT YOU NEED	WHAT TO USE
Increment a number safely	UpdateItem with ADD. An atomic counter
Write only if absent	PutItem with attribute_not_exists
Write only if unchanged	ConditionExpression on a version attribute
Several items, all or nothing	TransactWriteItems
Many independent writes, fast	BatchWriteItem, and retry UnprocessedItems

ON THE EXAM

Five different guarantees, five different tools. Reaching for a transaction when an atomic counter would do costs you double for nothing.

Two of these need arithmetic. Do it properly rather than estimating - the wrong answers are built from plausible miscalculations.

> A product catalogue table serves 2,000 reads per second. Each item is about 6 KB. The application tolerates data being a second or two out of date.
>
> The team is sizing provisioned read capacity and wants the lowest correct figure.

How many read capacity units should be provisioned?

A 1,000 RCUs

B 2,000 RCUs

C 4,000 RCUs

D 6,000 RCUs

◆ PAUSE AND DECIDE

How many read capacity units should be provisioned?

A 1,000 RCUs

Halves correctly but forgets the rounding. 6 KB is two 4 KB blocks, not one, so this is exactly half what is needed.

B 2,000 RCUs

6 KB rounds up to two 4 KB blocks. Eventually consistent costs 0.5 RCU per block, so 1 RCU per read. 2,000 reads per second needs 2,000 RCUs.

C 4,000 RCUs

Correct rounding, but priced as strongly consistent. The scenario allows stale data, so this overpays by exactly double.

D 6,000 RCUs

Treats each kilobyte as one unit. No rounding rule in DynamoDB works that way.

THE ARITHMETIC

6 KB rounds up to 8 KB, which is 2 blocks. Eventually consistent halves each block to 0.5, giving 1 RCU per read. 2,000 reads per second x 1 = 2,000 RCUs.

> A ticketing application must reserve a seat and decrement the remaining inventory for that event. Both live in the same DynamoDB table. If either write fails, neither may be applied, or the system will sell a seat it does not have.
>
> Seats are also occasionally released. The team needs a guarantee, not best-effort behaviour.

How should the developer implement the reservation?

A. Use BatchWriteItem with both the seat item and the inventory item

B. Use two UpdateItem calls in sequence and roll back manually if the second fails

C. Use TransactWriteItems with a condition that the seat is unreserved and inventory is above zero

D. Use UpdateItem with an atomic counter on the inventory item

◆ PAUSE AND DECIDE

How should the developer implement the reservation?

A BatchWriteItem

Not atomic. Individual writes can fail independently, which is exactly the outcome the scenario forbids. It also cannot take conditions.

B Sequential updates with manual rollback

The rollback itself can fail, and the process can die between the two writes. This is what transactions exist to replace.

C TransactWriteItems with conditions

All-or-nothing across both items, with per-action conditions preventing double booking and negative inventory. Costs double, which the guarantee justifies.

D Atomic counter

Handles the inventory decrement safely, and does nothing about the seat item. Half the problem, and inventory could go negative.

THE SIGNAL

'If either fails, neither may be applied' is the definition of a transaction. When a scenario states that requirement, it has named the answer.

A session table stores about 40 million active sessions. Sessions expire after 24 hours. A nightly Lambda function currently queries expired sessions and issues DeleteItem for each one, consuming roughly 8 million WCUs a night and often being throttled.

The team wants expired sessions removed without that write cost, and wants an archive record kept in Amazon S3 for compliance.

What should the team implement?

A Enable TTL on an expiry attribute, and process the resulting REMOVE events from DynamoDB Streams into S3

B Enable TTL on an expiry attribute and export the table to S3 nightly

C Replace DeleteItem with BatchWriteItem, deleting 25 sessions per call

D Switch the table to on-demand capacity so the deletes are never throttled

◆ PAUSE AND DECIDE

What should the team implement?

A TTL plus Streams to S3

TTL deletion consumes no write capacity at all, and TTL deletions surface in Streams as REMOVE events carrying the old image - which is exactly the archive record.

B TTL plus nightly export

TTL solves the cost, but an export captures what still exists. Sessions deleted between exports are never archived.

C BatchWriteItem

Fewer API calls, identical capacity cost. Batching saves network round trips, never write units.

D On-demand mode

Stops the throttling by paying for all 8 million writes instead. It makes the bill worse, not better.

THE SIGNAL

Two requirements: no write cost, and an archive. TTL alone gives the first. Streams with the old image gives the second, on the same mechanism.

Monitoring a table

- **ConsumedReadCapacityUnits** and **ConsumedWriteCapacityUnits** against what you provisioned.
- **ThrottledRequests**, plus **ReadThrottleEvents** and **WriteThrottleEvents** per index.
- **SuccessfulRequestLatency**, split by operation, for the DynamoDB side of your latency.
- **CloudWatch Contributor Insights** names the most accessed and most throttled **keys**.
- Metrics are per minute. Second-level throttling can hide inside a healthy-looking minute.

> ON THE EXAM
>
> Contributor Insights is the tool that turns 'the table is throttling' into 'this specific partition key is throttling'. Reach for it before changing capacity.

A dashboard reads the same few thousand configuration items from DynamoDB on nearly every page load, at about 30,000 reads per second. The DynamoDB bill is dominated by these reads, and p99 latency of 9 ms is now the slowest part of the page.

The configuration changes a handful of times per day. The team wants a fix that does not require rewriting the data access layer, and reads must reflect a change within a second or two.

What should the team implement?

A. Add a global secondary index on the configuration attributes

B. Put Amazon ElastiCache for Redis in front of DynamoDB with a 60-second TTL

C. Switch the reads to strongly consistent so they are served faster

D. Deploy a DAX cluster and point the existing client at the DAX endpoint

◆ PAUSE AND DECIDE

What should the team implement?

A Add a GSI

Indexes create new access patterns. The access pattern here already works - it is simply hot. A GSI adds cost and changes nothing.

B ElastiCache with a 60-second TTL

The strongest wrong answer. It would cut cost and latency, but it needs cache-aside code in the data layer, and a 60-second TTL breaks the two-second freshness requirement.

C Strongly consistent reads

Doubles the read cost and increases latency. It is the opposite of both requirements.

D DAX at the existing endpoint

API-compatible so the data layer is unchanged, microsecond reads, and write-through keeps it fresh on every change rather than on a timer.

THE SIGNAL

Compare with the DAX or ElastiCache tiebreaker. 'No rewrite' and 'fresh within seconds' both point at DAX - write-through beats a TTL for freshness.

What to carry forward

- You pay for what is **read**. Filters and projections shrink the response, never the bill.
- Reads round to **4 KB**, writes to **1 KB**. Eventually consistent is half price. Transactions are double.
- **BatchWriteItem is 25 and not atomic.** Transactions are 100 and all-or-nothing.
- TTL is free but best-effort, typically within 48 hours, and shows up in Streams.
- **DAX for DynamoDB, ElastiCache for everything else.** DAX cannot do strongly consistent reads.

> ON THE EXAM
>
> That completes DynamoDB. Next: Amazon S3 for developers - presigned URLs, multipart uploads, conditional writes and event notifications.

MODULE 09 · DOMAIN 1 - DEVELOPMENT WITH AWS SERVICES

Amazon S3 for Developers

Presigned URLs, multipart uploads, conditional writes and events - the parts of S3 a developer actually writes code against.

DOMAIN 1 of 4 EXAM WEIGHT 32% MODULE 9 of 34

admin@sthithapragna.com

For a developer, S3 is not a filesystem. It is an **HTTP API with an object in it**, and almost everything follows from that.

Buckets, keys and prefixes

- A bucket name is **globally unique** across all AWS accounts, and lives in one Region.
- The **key** is the full path-like string. **invoices/2026/09/inv-1.pdf** is one key, not four levels.
- A **prefix** is just the leading portion of a key. It is what ListObjectsV2 filters on.
- There are no directories, so there is no atomic rename. Copy then delete.
- Object metadata is fixed at write time. Changing it means copying the object over itself.

> ON THE EXAM
>
> The flat namespace matters for performance too: request rates scale per prefix, so key design affects throughput exactly as it does in DynamoDB.

The size limits

LIMIT	VALUE
Object size	0 bytes to 5 TB on the exam. See the next slide
Single PUT upload	5 GB maximum
Recommended multipart threshold	About 100 MB
Multipart part size	5 MB minimum, 5 GB maximum. Last part may be smaller
Parts per upload	10,000
Console upload	160 GB

ON THE EXAM

5 GB is the number that decides questions. Anything larger than 5 GB cannot be uploaded in one PUT, so multipart is mandatory rather than optional.

Currency note: objects can now be 50 TB

- **The examinable answer:** the maximum S3 object size is **5 TB**.
- On **2 December 2025** AWS raised it to **50 TB**, a tenfold increase, in all Regions.
- It applies to every storage class and works with all S3 features.
- Above 5 TB, AWS directs you to the S3 Transfer Manager and the AWS Common Runtime.
- **Answer 5 TB on the exam.** Exam guide v2.1 predates this by a year.

> ON THE EXAM
>
> The 5 GB single-PUT limit did not change, and that is the one questions actually turn on. 5 TB against 50 TB is trivia by comparison - but you will meet 50 TB in the documentation.

Multipart upload

- Three steps: **CreateMultipartUpload**, then **UploadPart** repeatedly, then **CompleteMultipartUpload**.
- Parts upload **independently, in any order, in parallel**. A failed part retries alone.
- Each part returns an **ETag**, and completion submits the part number and ETag list.
- The object does not exist until completion. There is no partially visible object.
- The high-level SDK transfer managers do all of this for you automatically.

> ON THE EXAM
>
> The parallel retry is the real benefit. On a flaky connection, a failed 10 MB part costs one retry rather than restarting a 5 GB upload.

Incomplete multipart uploads

- An abandoned upload leaves its parts in the bucket, **billed as storage**, indefinitely.
- They are **invisible to ListObjectsV2**, because no object exists yet.
- Find them with **ListMultipartUploads**, remove them with **AbortMultipartUpload**.
- The real fix is a lifecycle rule: **AbortIncompleteMultipartUpload** after N days.
- AWS recommends this rule on every bucket that receives multipart uploads.

> ON THE EXAM
>
> A classic 'why is my bill higher than the sum of my objects' scenario. The storage is real, the objects are not listed, and only a lifecycle rule cleans it up automatically.

Presigned URLs

- A normal S3 URL plus a signature, granting one specific operation for a limited time.
- Works for **GET** and for **PUT**, so clients can upload directly to S3.
- The URL carries the permissions of **whoever signed it**, not of whoever uses it.
- The bucket stays entirely private. Nothing is made public.
- Anyone holding the URL can use it, as often as they like, until it expires.

> ON THE EXAM
>
> The standard upload pattern: the browser asks your API for a presigned PUT URL, then uploads straight to S3. Your compute never touches the bytes.

Why presigned URLs expire early

- A presigned URL dies when the **credentials that signed it** expire, whatever expiry you set.
- **IAM user credentials:** up to **7 days**, the SigV4 maximum.
- **An assumed role:** only as long as that session, often 1 hour.
- So a Lambda function signing a 7-day URL produces one that dies in about an hour.
- For genuinely long-lived access, use a **CloudFront signed URL** instead.

> ON THE EXAM
>
> A URL that expires far sooner than requested is almost always this. Lambda uses temporary role credentials, and no expiry setting can outlive them.

Presigned URL or CloudFront signed URL?

S3 PRESIGNED URL

Simple and direct

- Signed with IAM credentials
- Works for upload and download
- Maximum 7 days, often much less
- No CDN, no edge caching
- One line of SDK code

CLOUDFRONT SIGNED URL

Distribution and control

- Signed with a key pair you manage
- Download only
- Expiry can be far longer
- Cached at edge locations
- Can restrict by IP address

THE TIEBREAKER

Uploads always mean presigned URL - CloudFront cannot sign an upload. Long-lived or globally distributed downloads mean CloudFront.

Transfer Acceleration

- Uploads enter at the nearest **CloudFront edge location** and travel the AWS backbone.
- Enabled per bucket, then used via a distinct **s3-accelerate** endpoint.
- Helps most when clients are far from the bucket's Region.
- It costs extra, and adds nothing when the client is already near the Region.
- AWS provides a speed comparison tool to check it helps before you pay for it.

> ON THE EXAM
>
> 'Users worldwide uploading to a bucket in one Region, and it is slow' is the shape. Combine with multipart for large files.

Consistency and conditional writes

- S3 has had **strong read-after-write consistency** since December 2020, for all operations.
- Older material describes eventual consistency for overwrites and deletes. That is out of date.
- **If-None-Match: *** on PutObject writes only if the key does not exist. A conflict returns **412**.
- **If-Match** with an ETag writes only if the object is unchanged. Compare-and-swap.
- Extended to **CopyObject** in October 2025, with matching bucket policy condition keys.

> ON THE EXAM
>
> Conditional writes removed the old pattern of using DynamoDB as a lock table to coordinate S3 writers. Both patterns still appear as options.

Versioning

- Every write creates a new **version ID**. The old version remains and is still billed.
- A delete without a version ID writes a **delete marker** and hides the object.
- Removing the delete marker restores the object. That is the undelete.
- Deleting permanently means deleting a **specific version ID**.
- Versioning can be suspended but **never turned off**, and existing versions remain.

> ON THE EXAM
>
> Versioning plus a lifecycle rule on noncurrent versions is the standard pairing. Without the rule, storage grows forever.

Lifecycle rules

- **Transition** actions move objects between storage classes after N days.
- **Expiration** actions delete objects, or noncurrent versions, after N days.
- Rules can be scoped by **prefix**, by **object tag**, or by object size.
- **AbortIncompleteMultipartUpload** is a lifecycle action too. Set it everywhere.
- Lifecycle deletions are free and asynchronous. They are not immediate.

> ON THE EXAM
>
> Three separate cost leaks are all fixed here: old versions, forgotten objects, and abandoned multipart parts.

Object tags and cost attribution

- Up to **10 tags** per object, set at upload or added later.
- Lifecycle rules and replication rules can be **scoped by tag**, not just by prefix.
- IAM and bucket policies can condition on **s3:ExistingObjectTag** for fine-grained access.
- Tags appear in cost allocation reports, so one bucket can be billed by team.
- Tags are separate from user metadata, and are the only part you can change without copying.

> ON THE EXAM
>
> Tags are the way to apply different lifecycle or access rules inside one bucket when prefixes do not line up with the policy you want.

Encryption at rest

OPTION	WHO HOLDS AND MANAGES THE KEY
SSE-S3	S3 manages everything. The default on all new objects
SSE-KMS	A KMS key you choose. Auditable in CloudTrail, revocable
DSSE-KMS	Two independent layers of KMS encryption. For strict compliance
SSE-C	You supply the key on every request. AWS never stores it
Client-side	You encrypt before upload. S3 only ever sees ciphertext

ON THE EXAM

All new objects are encrypted with SSE-S3 by default. The question is never whether to encrypt, only which key and who controls it.

Access control

- **Block Public Access is on by default** and **ACLs are disabled by default** on new buckets.
- **Bucket policies** are the resource-based policy. They are how cross-account access is granted.
- IAM policies grant your principals access. Bucket policies grant others access.
- Block Public Access **overrides** any policy that would make the bucket public.
- For CloudFront in front of S3, use **origin access control** rather than a public bucket.

> ON THE EXAM
>
> Any option that solves a problem by making a bucket public is almost certainly wrong. Presigned URLs or OAC are the intended answers.

CORS

- A JSON configuration on the bucket listing allowed origins, methods and headers.
- Without it, a browser on another domain is blocked **by the browser**, not by S3.
- The browser sends an **OPTIONS** preflight first for anything beyond a simple request.
- **ExposeHeaders** is needed for JavaScript to read response headers like ETag.
- Presigned PUT uploads from a browser require CORS. This is the usual cause of failure.

> ON THE EXAM
>
> A presigned URL that works in curl but fails in a browser is a CORS problem every time. The signature is fine; the browser is enforcing policy.

Events out of S3

- **Event notifications** send directly to Lambda, SQS or SNS. Asynchronous invocation.
- One destination per event type and prefix. Fan-out needs SNS in the middle.
- **EventBridge**, enabled per bucket, publishes every event to the default bus.
- EventBridge gives many targets, richer filtering, plus archive and replay.
- Notification delivery is typically seconds but is **not guaranteed to be ordered**.

> ON THE EXAM
>
> One consumer, keep it simple: notifications. Several consumers, or you need filtering and replay: EventBridge. Module 5 covered the Lambda side.

Reading less than the whole object

- **Byte-range fetches** use the HTTP **Range** header to read part of an object.
- Useful for headers of large files, and for parallel downloads of separate ranges.
- **S3 Object Lambda** runs a function that transforms an object as it is retrieved.
- One stored object can then be redacted, converted or filtered per caller.
- Object Lambda adds Lambda cost and latency to every GET that goes through it.

> ON THE EXAM
>
> Object Lambda is the answer when different consumers need different views of the same object and you do not want to store several copies.

Currency note: S3 Select is closed

- **The examinable answer:** S3 Select runs SQL against a single object without downloading it.
- **S3 Select and S3 Glacier Select closed to new customers on 25 July 2024.**
- Existing customers continue to use it, with security and availability updates.
- AWS now directs you to **Amazon Athena**, **S3 Object Lambda**, or client-side filtering.
- It remains a correct exam answer even though you cannot sign up for it.

> ON THE EXAM
>
> The clearest example of the revision gap in this course: a service you cannot purchase, that is still the right answer on the exam.

S3 performance

- At least **3,500 PUT, COPY, POST or DELETE** requests per second per partitioned prefix.
- At least **5,500 GET or HEAD** requests per second per partitioned prefix.
- The limits are **per prefix**, so spreading keys across prefixes multiplies throughput.
- S3 also repartitions automatically as request patterns grow.
- Sequential key prefixes such as timestamps concentrate load, exactly as in DynamoDB.

> ON THE EXAM
>
> The same lesson as module 7. Load spreads across the leading portion of the key, so keys beginning with a date concentrate traffic.

Two of these are diagnosis: something works in one context and fails in another. Ask **what is different** about the failing case.

A web application lets users upload video files of up to 4 GB. Currently the browser posts the file to an EC2-backed API, which buffers it and forwards it to Amazon S3. The API servers are running out of memory and disk, and upload latency is poor for distant users.

The S3 bucket must remain private. The team wants uploads to stop passing through their compute entirely.

How should the upload be redesigned?

A. Make the S3 bucket public for writes and have the browser PUT directly to it

B. Increase the EC2 instance size and add an Application Load Balancer

C. Store the uploads in Amazon EFS mounted on the EC2 instances, then sync to S3

D. Have the API return a presigned PUT URL, and let the browser upload directly to S3 using multipart

◆ PAUSE AND DECIDE

How should the upload be redesigned?

A Public bucket

Directly contradicts the requirement that the bucket stay private, and a publicly writable bucket is a serious security failure. Almost never a correct answer.

B Bigger instances plus an ALB

Scales the problem rather than removing it. The bytes still flow through your compute, and you pay for that traffic twice.

C EFS then sync

Adds a second storage system and still routes every byte through EC2. More moving parts, same bottleneck.

D Presigned PUT plus multipart

The bucket stays private, the bytes never touch your compute, and multipart handles 4 GB with resumable parts.

THE SIGNAL

'Must remain private' and 'stop passing through our compute' look contradictory until you remember a presigned URL grants access without making anything public.

A Lambda function generates presigned GET URLs for report files and emails them to customers, with an expiry set to 7 days. Customers report that the links stop working after about an hour, long before the stated expiry.

The same code, run locally by a developer using their own IAM user access keys, produces URLs that work for the full 7 days.

What explains the difference, and what should the developer do?

A The Lambda execution role provides temporary credentials that expire with the session, so the URL dies with them. Use CloudFront signed URLs for long-lived links

B The Lambda function's timeout is too short, so the URL is not fully signed

C S3 caps all presigned URLs at one hour regardless of the requested expiry

D The bucket lifecycle policy is expiring the objects after one hour

◆ PAUSE AND DECIDE

What explains the difference, and what should the developer do?

A Role credentials expire; use CloudFront

A presigned URL cannot outlive the credentials that signed it. Lambda uses an assumed role, so the URL dies with the session. CloudFront signed URLs use a managed key pair and have no such ceiling.

B Function timeout

Signing is a local cryptographic operation taking milliseconds. A timeout would fail the invocation outright, not produce a short-lived URL.

C S3 caps at one hour

No such cap exists. SigV4 allows up to 7 days, which the developer's local run demonstrates.

D Lifecycle expiry

That would delete the objects, so the developer's own URLs would fail too. They do not.

THE SIGNAL

The local run working is the whole clue. Same code, same bucket, same expiry - only the credentials differ, so the credentials are the cause.

A distributed job writes result files to S3. Several workers can occasionally be assigned the same job, and both then write to the same key. The team needs the first write to win, and later writers to detect that the result already exists rather than overwrite it.

They do not want to introduce another service, and the workers have no shared coordination mechanism.

How should this be implemented?

A Enable versioning on the bucket so every write is preserved

B Use PutObject with the If-None-Match header set to *, and handle the 412 response

C Have each worker call HeadObject first and skip the write if the object exists

D Write to a DynamoDB lock table before writing to S3

◆ PAUSE AND DECIDE

How should this be implemented?

A Versioning

Preserves both writes rather than preventing the second. The last writer is still the current version, so readers get the wrong result.

B If-None-Match on PutObject

S3 evaluates the condition atomically. The first write succeeds, the second gets 412 Precondition Failed and knows it lost. No extra service.

C HeadObject then write

A check-then-act race. Both workers can see no object and both proceed. This is the bug conditional writes were built to remove.

D DynamoDB lock table

It works, and it was the standard pattern before August 2024. It adds the service the scenario explicitly rules out.

THE SIGNAL

Option C is the tempting one, and it is a textbook race condition. Only a condition evaluated inside S3 is atomic.

A finance team reports that the S3 bill for a media bucket is roughly 40% higher than the total size of the objects it contains. The bucket receives large uploads from a mobile app, which frequently loses connectivity mid-upload. Versioning is not enabled.

Listing the bucket accounts for only the expected object storage. The team wants the excess removed and prevented from recurring.

What is the cause, and what should the team configure?

A. Old object versions are accumulating. Add a lifecycle rule to expire noncurrent versions

B. The objects are in the wrong storage class. Add a lifecycle transition to S3 Standard-IA

C. Abandoned multipart upload parts are being stored. Add a lifecycle rule with AbortIncompleteMultipartUpload

D. Request charges account for the difference. Enable S3 Transfer Acceleration to reduce them

◆ PAUSE AND DECIDE

What is the cause, and what should the team configure?

A **Expire noncurrent versions**

The scenario states versioning is not enabled, so there are no noncurrent versions to expire. Eliminated by one sentence.

B **Transition to Standard-IA**

Would reduce cost per gigabyte, and leaves the mystery storage in place. It treats the symptom while the leak keeps growing.

C **AbortIncompleteMultipartUpload**

Interrupted uploads leave parts that are billed but never listed, which explains storage that listing cannot account for. The lifecycle rule removes them and prevents recurrence.

D **Transfer Acceleration**

Speeds uploads and costs more, not less. Request charges are also unrelated to a storage discrepancy.

THE SIGNAL

Three clues converge: storage that listing cannot see, large uploads, and frequent disconnections. Only incomplete multipart parts fit all three.

What to carry forward

- **5 GB** is the single PUT limit. Above that, multipart is mandatory, not optional.
- A presigned URL **cannot outlive the credentials that signed it**.
- Conditional writes give atomic create-if-absent and compare-and-swap, since 2024.
- Lifecycle rules fix three cost leaks: old versions, stale objects, abandoned parts.
- Making a bucket public is almost never the answer. Presigned URLs or OAC are.

> ON THE EXAM
>
> Next: caching, relational data, and choosing between DynamoDB, RDS, Aurora and ElastiCache when several would work.

MODULE 10 · DOMAIN 1 - DEVELOPMENT WITH AWS SERVICES

Caching, Relational Data and Choosing a Store

RDS, Aurora, ElastiCache – and the constraints that decide between them when two would work.

DOMAIN 1 of 4 EXAM WEIGHT 32% MODULE 10 of 34

admin@sthithapragna.com

In a selection question, the wrong answers usually **work**. They lose on a constraint the scenario stated in passing.

Amazon RDS for developers

- Managed relational engines: MySQL, PostgreSQL, MariaDB, Oracle, SQL Server and Aurora.
- **Multi-AZ** is for availability. A standby you cannot read from, failover by DNS.
- **Read replicas** are for scale. Readable, asynchronous, and up to 15 on Aurora.
- **IAM database authentication** issues a 15-minute token instead of a stored password.
- The connection is TCP and stateful, which is where serverless compute starts to struggle.

> ON THE EXAM
>
> Multi-AZ against read replicas is the classic pairing: availability against scale. On Aurora you get both, from the same cluster.

The Lambda and RDS problem

- Each concurrent execution environment opens its **own connection**. Concurrency multiplies them.
- Opening a connection costs tens of milliseconds, on every cold start.
- Databases cap connections. Exceed it and you get 'too many clients', not throttling.
- Reaching a private database means attaching the function to a **VPC**, losing internet access.
- Three answers exist: RDS Proxy, the Aurora Data API, or not using a relational database.

> ON THE EXAM
>
> This is module 3's scenario 1 restated as a design problem. Connection pooling assumes long-lived processes, and Lambda has none.

Amazon RDS Proxy

- A managed pool sitting between your application and RDS or Aurora.
- Thousands of Lambda environments share a small number of real database connections.
- Credentials come from **Secrets Manager**, and IAM authentication is supported.
- Failover is faster, because the proxy holds the connection while the database moves.
- It runs in a VPC, is billed per vCPU-hour of the database, and adds a little latency.

> ON THE EXAM
>
> The answer when the constraint is 'keep the existing code and the existing SQL'. The application still speaks the native protocol.

Amazon Aurora for developers

- MySQL and PostgreSQL compatible, with storage that grows automatically to 128 TB.
- **Cluster endpoints:** a writer endpoint, a reader endpoint that load-balances, and custom endpoints.
- Up to **15 replicas**, with replica lag usually in the low milliseconds.
- **Aurora Serverless v2** scales capacity in fine increments, and can now scale to zero.
- **Aurora Serverless v1 reached end of life on 31 March 2025.** Remaining clusters were upgraded to v2.

> ON THE EXAM
>
> Pointing read traffic at the writer endpoint wastes 15 replicas. Using the reader endpoint is the single easiest Aurora scaling win.

The Aurora Data API

- An **HTTPS endpoint** for running SQL. No driver, no connection, no pool.
- **No VPC required**, so the Lambda function keeps its internet access and its fast cold start.
- Authorised by **IAM**, with credentials read from Secrets Manager.
- Higher per-query latency than a native connection, and result size is limited.
- It is not a drop-in replacement. Existing code using a SQL driver must be rewritten.

> ON THE EXAM
>
> The Data API removes the connection problem rather than managing it. That is why it wins on 'no VPC' and loses on 'no code changes'.

RDS Proxy or the Data API?

RDS PROXY

Keep the code, pool the connections

- Native protocol and existing drivers
- No application changes at all
- Requires the function to be in a VPC
- Billed per vCPU-hour, always on
- Works with RDS and Aurora

AURORA DATA API

Remove connections entirely

- SQL over an HTTPS endpoint
- Application code must be rewritten
- No VPC needed for the function
- No standing charge
- Aurora only, not plain RDS

THE TIEBREAKER

'Minimal code change' points at RDS Proxy. 'The function must stay out of a VPC' or 'no idle cost' points at the Data API.

Amazon ElastiCache engines

- **The examinable pairing:** Redis against Memcached.
- **Valkey**, added October 2024, is a Linux Foundation fork of Redis 7.2 and a drop-in replacement.
- AWS now recommends Valkey for new deployments. It is around 20% cheaper on nodes.
- It exists because Redis changed its licence away from open source in March 2024.
- **Answer Redis on the exam.** Everything true of Redis is true of Valkey.

> ON THE EXAM
>
> This one is painless: Valkey is API-compatible with Redis, so the exam's Redis answers remain correct. Only the name and the price changed.

Redis or Memcached?

REDIS AND VALKEY

A data structure server

- Lists, sets, sorted sets, hashes
- Persistence and snapshots
- Replication and Multi-AZ failover
- Pub/sub and transactions
- The default choice

MEMCACHED

A simple object cache

- Strings and objects only
- No persistence at all
- No replication or failover
- Multi-threaded, so scales up well
- Scales out by adding nodes

THE TIEBREAKER

Any requirement beyond a plain key-value cache selects Redis. Memcached wins only on multi-threaded simplicity and pure horizontal scale-out.

Caching strategies

- **Lazy loading:** read the cache, and on a miss read the database and populate it.
- Only requested data is cached, and a cache failure is survivable. Every miss costs latency.
- **Write-through:** update the cache whenever you update the database.
- Data is never stale, and reads always hit. But you cache data nobody asks for.
- **A TTL** on every key bounds staleness whichever strategy you choose. Use both together.

> ON THE EXAM
>
> Lazy loading serves stale data until eviction; write-through wastes writes on cold data. Real systems use write-through plus a TTL, which is what DAX does.

ElastiCache beyond caching

- **Session store** for a stateless web tier, so any instance can serve any user.
- **Leaderboards**, using sorted sets to rank millions of scores in constant time.
- **Rate limiting**, using atomic counters with expiry per caller.
- **Distributed locks** and job queues, using SETNX and list operations.
- **Pub/sub** for lightweight real-time fan-out between application components.

> ON THE EXAM
>
> Any scenario mentioning sorted sets, leaderboards, rankings or sliding-window rate limits is pointing at Redis, not at a cache in front of a database.

The other purpose-built stores

SERVICE	RECOGNISE IT WHEN THE SCENARIO SAYS
Amazon DocumentDB	MongoDB compatible, existing MongoDB application
Amazon Neptune	Graph, relationships, social network, fraud rings
Amazon Keyspaces	Apache Cassandra compatible, CQL
Amazon Timestream	Time series, IoT metrics at scale
Amazon MemoryDB	Redis compatible, but as a durable primary database
Amazon OpenSearch	Full-text search, log analytics, fuzzy matching

ON THE EXAM

You will not be asked to configure these. You will be asked to recognise the one word in a scenario that names one of them.

Choosing a store

THE SCENARIO SAYS	THE INTENDED ANSWER
Known access patterns, any scale, single-digit ms	DynamoDB
Ad hoc queries, joins, reporting, existing SQL skills	RDS or Aurora
Sub-millisecond reads of DynamoDB, no code change	DAX
Sub-millisecond reads of anything else	ElastiCache
Session state for a stateless tier	ElastiCache
Large objects, files, media, backups	Amazon S3
Full-text or fuzzy search	OpenSearch, fed by Streams

ON THE EXAM

Worth screenshotting. Most store-selection questions on this exam reduce to one row of this table.

Six scenarios, because selection needs practice. For each, name the **disqualifying clause** before you pick.

A new serverless application uses Lambda functions that need to run SQL against an Aurora PostgreSQL cluster. The functions also call a third-party payment API over the internet on every request.

The team is writing the data access layer from scratch. They want the simplest architecture, no standing infrastructure cost while the application is idle, and cold starts kept short.

How should the Lambda functions reach the database?

A Attach the functions to the VPC and connect directly to the cluster endpoint

B Use the Aurora Data API over HTTPS, authorised with IAM

C Attach the functions to the VPC and connect through RDS Proxy

D Attach the functions to the VPC, add a NAT gateway, and connect through RDS Proxy

◆ PAUSE AND DECIDE

How should the Lambda functions reach the database?

A Direct VPC connection

Breaks the payment API call, because a VPC-attached function has no internet route. It also exhausts connections at concurrency.

B Aurora Data API

No VPC, so internet access survives and cold starts stay short. No standing cost, and the code is being written from scratch anyway.

C RDS Proxy in a VPC

Solves connections, still breaks the payment API, and adds a per-hour charge the team explicitly does not want while idle.

D RDS Proxy plus NAT gateway

Works fully, and adds two always-on billed components. It is the correct answer to a question with different constraints.

THE SIGNAL

Three clauses: an internet call, no idle cost, and code written from scratch. Each one favours the Data API, and together they eliminate everything else.

An existing PHP application runs on EC2 behind an Application Load Balancer and stores user session state in local files on each instance. Auto scaling is being introduced, and users are being logged out whenever an instance is replaced or traffic moves.

The team wants the smallest possible change to the application, and sessions must survive an instance failure.

Where should session state be stored?

A Amazon S3, writing a session file per user

B Sticky sessions on the Application Load Balancer, keeping local files

C An Amazon RDS table, with a row per session

D Amazon ElastiCache, using the engine's session handler

◆ PAUSE AND DECIDE

Where should session state be stored?

A S3 per-session file

S3 is object storage, not a session store. Latency is far too high for a read on every page request, and there is no expiry without lifecycle rules.

B Sticky sessions

The tempting one. Sticky sessions keep a user on one instance, but the session still dies when that instance is replaced - which is the stated failure.

C RDS session table

It works, and it puts high-frequency ephemeral writes on a relational database, adding load and cost for data that is discarded hourly.

D ElastiCache session handler

Purpose-built: sub-millisecond reads, native TTL for expiry, survives instance loss, and the session handler is a configuration change rather than a rewrite.

THE SIGNAL

Sticky sessions solve routing, not durability. The requirement is that sessions survive instance failure, and stickiness does nothing for that.

> A mobile game must show a global leaderboard of the top 100 players out of 40 million, updated the moment a score changes, plus each player's own rank. Scores change thousands of times per second.
>
> The current implementation stores scores in DynamoDB and runs a Scan with sorting in the application, which takes minutes and is always out of date.

How should the leaderboard be implemented?

A Amazon ElastiCache for Redis, using a sorted set keyed by score

B A DynamoDB global secondary index with score as the partition key

C DynamoDB Accelerator in front of the existing table

D Export the table to Amazon S3 and query it with Amazon Athena

◆ PAUSE AND DECIDE

How should the leaderboard be implemented?

A Redis sorted set

Sorted sets maintain ranking as an intrinsic property. Top-N and a single player's rank are both constant-time, and updates are immediate.

B GSI on score

Score is a terrible partition key - low cardinality and hot. A GSI also cannot return a global ranking without reading every partition.

C DAX

Caches reads of a query that is itself wrong. A cached Scan is a faster wrong answer, and the data would be stale the moment a score changed.

D Athena over S3

An analytics tool answering a real-time question. Export latency alone breaks the requirement for immediate updates.

THE SIGNAL

Ranking is a data structure problem, not a query problem. DynamoDB has no concept of global order across partitions; Redis has ranking built in.

> A finance team needs a reporting database for settled transactions. Analysts write their own SQL, joining transactions to accounts, customers and currency tables, and the queries are different every week. Volume is about 200 GB and grows slowly.
>
> The engineering team's instinct is DynamoDB, because the rest of the platform uses it.

Which data store fits the requirement?

A DynamoDB with a GSI for each known report

B DynamoDB with PartiQL, so analysts can write SQL-like queries

C Amazon Aurora PostgreSQL

D DynamoDB with DAX to speed up the reporting queries

◆ PAUSE AND DECIDE

Which data store fits the requirement?

A A GSI per report

Requires knowing every query in advance. The scenario says the queries differ every week, so this fails on the first new question asked.

B PartiQL

The best-built distractor. PartiQL gives SQL-like syntax but no joins, and a query without a key condition still becomes a full Scan.

C Aurora PostgreSQL

Ad hoc queries, multi-table joins and existing SQL skills are the definition of a relational workload. 200 GB is unremarkable for Aurora.

D DAX

Caches repeated reads. Queries that are different every week have no repetition to cache.

THE SIGNAL

Two phrases decide it: 'analysts write their own SQL' and 'different every week'. Both mean the access patterns cannot be known in advance, which is DynamoDB's one requirement.

> A product catalogue service caches product details in ElastiCache using lazy loading with a 24-hour TTL. Merchandisers report that price changes take up to a day to appear on the site, and complaints are increasing.
>
> The catalogue is read constantly but written rarely - a few hundred product updates per day against millions of reads. The team wants price changes visible within seconds without losing the cache's benefit.

What should the team change?

A Reduce the TTL to 30 seconds

B Add write-through: update the cache in the same code path that updates the product

C Remove the cache and read from the database on every request

D Increase the ElastiCache node size so more products stay cached

◆ PAUSE AND DECIDE

What should the team change?

A 30-second TTL

Fixes staleness by destroying the cache hit rate. Millions of reads would now miss every 30 seconds, pushing the load back onto the database.

B Write-through on update

A few hundred writes a day is a trivial amount of extra work, and it makes every price change visible immediately while the hit rate stays high.

C Remove the cache

Solves staleness by removing the thing that made the site fast. The scenario explicitly asks to keep the cache's benefit.

D Bigger nodes

Capacity was never the problem. Stale entries stay stale in a larger cache.

THE SIGNAL

The write-to-read ratio decides it. Write-through costs work proportional to writes, and here writes are rare while reads are enormous.

> A service caches rendered HTML fragments, keyed by URL. Fragments are regenerated cheaply if missing, are never read after the page changes, and no fragment matters individually. The cache must scale horizontally to tens of nodes as traffic grows, and the team wants maximum cache memory per pound spent.
>
> There is no requirement for persistence, replication, failover, or any data structure beyond a key and a blob.

Which ElastiCache engine fits best?

A. Redis with cluster mode enabled and one replica per shard

B. Redis with cluster mode disabled and five read replicas

C. MemoryDB for Redis

D. Memcached with nodes added as traffic grows

◆ PAUSE AND DECIDE

Which ElastiCache engine fits best?

A Redis, cluster mode enabled with replicas

Every replica is memory you pay for and never read from. The scenario says no fragment matters, so replication buys nothing.

B Redis, cluster mode disabled

Caps total cache size at one primary's memory, which directly contradicts scaling horizontally to tens of nodes.

C MemoryDB

A durable primary database with Multi-AZ transaction logs. Paying for durability on disposable HTML fragments is the opposite of the requirement.

D Memcached

Multi-threaded, scales out purely by adding nodes, and every gigabyte of node memory is cache rather than replica. Exactly the narrow band where Memcached wins.

THE SIGNAL

This is the deliberate mirror of the usual advice. The scenario removes every reason to choose Redis, one clause at a time - which is what makes Memcached correct.

What to carry forward

- Lambda plus a relational database has three answers: **RDS Proxy**, the **Data API**, or a different store.
- **Minimal code change** selects RDS Proxy. **No VPC or no idle cost** selects the Data API.
- Anything beyond a plain key-value cache selects **Redis**. Leaderboards and sessions are Redis questions.
- Write-through when writes are rare and reads are many. Always add a TTL.
- **Unknown or ad hoc access patterns rule DynamoDB out.** That is its one hard requirement.

> ON THE EXAM
>
> That closes the data stores. Next: Amazon API Gateway - the front door for most of what we have built so far.

MODULE 11 · DOMAIN 1 - DEVELOPMENT WITH AWS SERVICES

Amazon API Gateway

Three API types, four integration styles, five ways to authorise - and the one step everyone forgets.

DOMAIN 1 of 4 EXAM WEIGHT 32% MODULE 11 of 34

admin@sthithapragna.com

API Gateway is a **configurable proxy**. Every question is really asking which piece of configuration handles the requirement.

The three API types

TYPE	WHAT IT IS FOR
REST API	The full-featured one. API keys, usage plans, caching, request validation, WAF
HTTP API	Cheaper and faster, roughly 71% less per request. Fewer features
WebSocket API	Persistent two-way connections. The server can push to the client

ON THE EXAM

Confusingly, the REST API is the older and more capable one. HTTP APIs are newer and deliberately simpler, not more advanced.

REST API or HTTP API?

REST API

When you need the features

- API keys and usage plans
- Response caching
- Request and response validation
- Mapping templates and transformations
- WAF attaches directly

HTTP API

When you need cheap and fast

- About 71% cheaper per request
- Lower latency
- Native JWT authoriser
- Simple proxying to Lambda or HTTP
- No caching, no usage plans, no WAF

THE TIEBREAKER

Any mention of API keys, tiered quotas, caching, request validation or WAF selects the REST API. Otherwise HTTP APIs are the cheaper default.

The request pipeline

01

Method request

Authorisation, API keys, request validation.

›

02

Integration request

Transform and forward to the backend.

›

03

Integration response

Transform what the backend returned.

›

04

Method response

Status codes and headers returned to the caller.

ON THE EXAM

Four stages, and questions locate a requirement in one of them. Validation belongs to the method request; transformation to the integration stages.

Integration types

- **Lambda proxy** passes the whole request through and expects a shaped response back.
- **Lambda custom** uses mapping templates to build the request and the response.
- **HTTP** and **HTTP proxy** do the same for any HTTP backend.
- **AWS service** integration calls another AWS API directly, with no Lambda in between.
- **Mock** returns a canned response from the gateway itself. Useful for stubs and CORS.

> ON THE EXAM
>
> The proxy integrations are the modern default. The custom ones exist for when the payload must be reshaped without touching the backend.

The Lambda proxy contract

- The event carries **path, headers, queryStringParameters, pathParameters and body**.
- The body arrives as a **string**, not a parsed object. You parse it yourself.
- The response must contain **statusCode**, and **body** must also be a string.
- Optional: **headers**, **multiValueHeaders** and **isBase64Encoded** for binary payloads.
- Return an object instead of a string body and API Gateway answers **502 Bad Gateway**.

> ON THE EXAM
>
> A 502 from a Lambda-backed API almost always means the function returned the wrong shape. The function succeeded; the gateway could not use the result.

Proxy or custom integration?

PROXY INTEGRATION

The default

- Whole request passed straight through
- No mapping templates to maintain
- Backend controls status and headers
- Add a route without touching the gateway
- Backend must know the event shape

CUSTOM INTEGRATION

When the shape must change

- Mapping templates reshape both ways
- Backend stays unaware of API Gateway
- Suits legacy or third-party backends
- VTL is fiddly and hard to test
- Every change means a gateway change

THE TIEBREAKER

Custom integration earns its place when you cannot change the backend. If you own the backend, proxy is simpler and easier to evolve.

Mapping templates

- Written in **Velocity Template Language**, applied on the integration request or response.
- Reshape JSON, rename fields, inject values, or convert XML to JSON.
- Access request data through **$input** and context through **$context**.
- **$context.authorizer** carries values an authoriser returned, so the backend can use them.
- REST APIs only. HTTP APIs support only simple parameter mappings.

> ON THE EXAM
>
> The usual exam use: a client sends one format, the backend expects another, and neither can change. That is a mapping template.

Request validation

- Validate required **query string parameters and headers**, and the **body against a JSON Schema model**.
- Invalid requests are rejected with **400** at the gateway. The backend is never invoked.
- That saves Lambda invocations, and removes validation code from every function.
- Models are defined per API and reused across methods.
- REST APIs only. It is a common reason to choose REST over HTTP.

> ON THE EXAM
>
> When a scenario wants malformed requests stopped without invoking the backend, and without writing validation code, this is the answer.

Skipping the backend entirely

- **Mock integration** returns a response the gateway generates. No backend is called.
- Used for stub endpoints during development, and for answering CORS preflight.
- **AWS service integration** calls another AWS API directly from the gateway.
- API Gateway straight to SQS, SNS, Step Functions, Kinesis or DynamoDB, with no Lambda.
- Removes a function that existed only to forward a request. Less cost, less latency, less code.

> ON THE EXAM
>
> When a scenario describes a Lambda function that only writes the request to a queue, a direct service integration is very often the intended answer.

OpenAPI import and export

- An entire API can be defined in an **OpenAPI** document and imported.
- AWS-specific behaviour lives in **x-amazon-apigateway-** extensions.
- Export a working API to OpenAPI to hand a contract to client teams.
- SAM and CloudFormation can define an API this way rather than resource by resource.
- Importing replaces the definition. It still does not deploy it to any stage.

> ON THE EXAM
>
> Note the last line. Even an import leaves the stage serving the old deployment until you create a new one.

Stages and deployments

- **Configuration changes do nothing until you deploy.** This catches almost everyone once.
- A **deployment** is a snapshot of the API. A **stage** points at one deployment.
- Each stage has its own URL, its own settings and its own throttling and caching.
- Typical stages: dev, test and prod, all from the same API definition.
- Rolling back is repointing the stage at an earlier deployment. It is instant.

> ON THE EXAM
>
> 'I changed the configuration and nothing happened' is a deployment question, not a bug. It is a favourite exam scenario precisely because it is so easy to do.

Stage variables

- Name and value pairs attached to a stage, referenced as **${stageVariables.name}**.
- Point each stage at a different Lambda **alias**, so dev calls dev and prod calls prod.
- Or at a different HTTP endpoint, or a different HTTP connection ID.
- Available in mapping templates and passed to Lambda in the context object.
- They are configuration, not secrets. They are visible to anyone who can read the stage.

> ON THE EXAM
>
> The classic pairing is stage variables with Lambda aliases: one API definition, one function, and the stage decides which version runs.

Canary release deployments

- A stage can send a **percentage of traffic** to a canary deployment and the rest to production.
- The canary can have its own stage variables, so it can call a different backend version.
- CloudWatch metrics are reported **separately** for the canary.
- Promote when it looks healthy, or delete the canary to roll back instantly.
- REST APIs only. It also does not shift traffic between Lambda versions - aliases do that.

> ON THE EXAM
>
> Two independent traffic-shifting mechanisms exist: canary deployments at the gateway, and weighted aliases at Lambda. Module 24 covers the second.

Five ways to authorise

MECHANISM	USE IT WHEN
IAM authorisation	Callers are AWS principals signing with SigV4
Cognito user pool authoriser	End users sign in and present a Cognito JWT
Lambda authoriser	Any custom logic: a third-party IdP, or per-tenant rules
JWT authoriser	HTTP APIs only. Any OIDC provider, natively
Resource policy	Restrict by source VPC, IP range or AWS account

ON THE EXAM

API keys are not on this list. They identify a caller for metering, and are explicitly not an authentication mechanism.

Lambda authorisers

- **TOKEN** type receives a single header, typically the bearer token.
- **REQUEST** type receives headers, query strings, paths and context. Far more flexible.
- It returns an **IAM policy** allowing or denying the method, plus an optional context object.
- **Results are cached**, keyed by the token or the identity sources, for up to 3600 seconds.
- Caching is why a revoked token can keep working. Reduce the TTL if that matters.

> ON THE EXAM
>
> The returned context reaches the backend as $context.authorizer, which is how a tenant ID or user role gets passed on without a second lookup.

Cognito user pool authorisers

- The gateway validates a Cognito **ID token** or access token itself. No Lambda involved.
- Signature, expiry and issuer are all checked for you.
- Token claims reach the backend through **$context.authorizer.claims**.
- It authenticates only. It does not do fine-grained per-resource authorisation.
- For rules beyond 'is this a valid user', use a Lambda authoriser instead.

> ON THE EXAM
>
> The dividing line is simple. Cognito authoriser answers 'is this a valid signed-in user'. Anything more nuanced needs a Lambda authoriser.

API keys and usage plans

- **An API key is not authentication.** It identifies a caller so you can meter and throttle them.
- A **usage plan** ties API keys to stages, with a **rate**, a **burst** and a **quota**.
- Rate and burst are per second. Quota is per day, week or month.
- This is how tiered access is sold: bronze, silver and gold plans on one API.
- REST APIs only. HTTP APIs have no API keys and no usage plans.

> ON THE EXAM
>
> Any scenario about per-customer quotas or selling tiers of API access is a usage plan question, and therefore a REST API question.

Throttling, layer by layer

- **Account level:** 10,000 requests per second steady, with a 5,000 request burst bucket, per Region.
- **Stage and method level:** limits you set, applying to everyone.
- **Usage plan level:** per API key, so different customers get different limits.
- The **most restrictive** applicable limit wins, and exceeding any of them returns **429**.
- The account burst quota is **fixed** and cannot be increased. The steady rate can.

> ON THE EXAM
>
> Throttling here protects your backend as well as the platform. A 429 at the gateway is far cheaper than a database falling over behind it.

Response caching

- Enabled **per stage**, sized from **0.5 GB to 237 GB**, and billed hourly while it exists.
- Default TTL is **300 seconds**. Setting it to 0 disables caching for that method.
- The cache key is built from method request parameters you nominate.
- Clients can bypass it with **Cache-Control: max-age=0**, if you grant that permission.
- REST APIs only. Cache invalidation requires an IAM permission, so it can be restricted.

> ON THE EXAM
>
> Caching reduces backend load and cost, and it is billed hourly whether or not it gets hits. It is not free the way a usage plan is.

CORS at the gateway

- The browser sends an **OPTIONS** preflight, which your API must answer.
- The gateway can answer it with a **mock integration**, generated by Enable CORS.
- With **proxy integration**, your backend must also return the CORS headers on the real response.
- Enabling CORS in the console handles OPTIONS only. That is the usual half-fix.
- HTTP APIs configure CORS declaratively and handle both sides for you.

> ON THE EXAM
>
> The classic symptom: preflight succeeds, the real request is blocked. The OPTIONS method is configured; the proxy backend is not returning the headers.

Endpoint types and custom domains

- **Edge-optimised:** requests enter at a CloudFront edge location. The default for REST APIs.
- **Regional:** served from the Region. Better when callers are nearby, or you want your own CloudFront.
- **Private:** reachable only from inside a VPC, through an interface VPC endpoint.
- **Custom domain names** replace the execute-api URL, and need an ACM certificate.
- For an edge-optimised custom domain, the certificate must be in **us-east-1**.

> ON THE EXAM
>
> That last line is a classic gotcha. Edge-optimised means CloudFront, and CloudFront only reads certificates from us-east-1.

Reading the error codes

CODE	WHAT IT MEANS HERE
400	Bad request. Often request validation rejecting the payload
403	Access denied, WAF block, or a resource policy refusing the caller
429	Throttled. Account, stage, method or usage plan limit exceeded
502	Backend returned something unusable. Usually a malformed Lambda proxy response
504	Integration timed out. The backend took longer than the integration timeout

ON THE EXAM

502 against 504 is the pairing that matters. 502 means the backend answered wrongly; 504 means it did not answer in time.

Currency note: the 29-second limit moved twice

- **The examinable answer:** the integration timeout is **29 seconds**, and 504 is what you get past it.
- **June 2024:** raisable via Service Quotas up to **300 seconds**, on Regional and private REST APIs.
- That increase may require **reducing your account throttle quota**. It is a trade, not a free upgrade.
- **November 2025:** response streaming on REST APIs extends effective timeouts to 15 minutes.
- Edge-optimised REST APIs and HTTP APIs are eligible for **neither** increase.

> ON THE EXAM
>
> Two separate changes to the same number, both after exam guide v2.1. Answer 29 seconds, and know that a real system has two ways past it.

Observability

- **Execution logs** record what the gateway did internally. Verbose, and off by default.
- **Access logs** record one line per request, in a format you define with $context variables.
- **X-Ray** traces the request through the gateway and into the backend.
- Key metrics: **Count**, **4XXError**, **5XXError**, **Latency** and **IntegrationLatency**.
- Latency minus IntegrationLatency is the gateway's own overhead. Usually a few milliseconds.

> ON THE EXAM
>
> That last line is a real diagnostic. If Latency is high but IntegrationLatency is low, the delay is in the gateway - often an authoriser.

Two of these hinge on a feature existing only on REST APIs. Check the API type before you check anything else.

> A collaborative document editor must push changes to every connected client the moment another user makes an edit. Clients currently poll a REST endpoint every two seconds, which is expensive and still feels laggy.
>
> Sessions last hours, several hundred clients may be connected to one document, and the server must be able to initiate messages rather than only respond to them.

What should the team build?

A. A REST API with response caching and a shorter polling interval

B. A REST API with response streaming to each client

C. An API Gateway WebSocket API, storing connection IDs in DynamoDB

D. An HTTP API with a JWT authoriser and server-sent events

◆ PAUSE AND DECIDE

What should the team build?

A Cache plus faster polling

Still polling. More frequent polls means more cost, and caching makes the data staler rather than fresher.

B REST with response streaming

Streaming sends one response progressively to one caller. It cannot push an unrelated later event, and it cannot fan out to other clients.

C WebSocket API with connection IDs

A persistent bidirectional connection is exactly the requirement. Storing connection IDs lets the backend push to every client on a document.

D HTTP API with SSE

Server-sent events are one-directional and API Gateway HTTP APIs do not support streaming, so this cannot be built as described.

THE SIGNAL

'The server must be able to initiate messages' is the deciding clause. Only a WebSocket API gives the backend a way to reach a client unprompted.

A developer updates a REST API to add request validation on the POST method and changes the integration to point at a new Lambda alias. They save both changes in the console and confirm the settings show correctly when they reopen the method.

Calls to the production URL still reach the old alias and still accept invalid payloads. Nothing appears in CloudWatch to suggest an error.

What is the most likely explanation?

A The changes were saved but never deployed to the stage, so the stage still serves the previous deployment

B The API Gateway cache is serving stale responses and needs invalidating

C Request validation only applies to GET methods

D The Lambda alias needs a resource-based policy before API Gateway can invoke it

◆ PAUSE AND DECIDE

What is the most likely explanation?

A Never deployed

Configuration is saved against the API, and a stage serves a deployment snapshot. Until a new deployment is created, the stage keeps serving the old one.

B Stale cache

Caching would affect responses, not which alias is invoked, and it would not make invalid payloads pass validation.

C Validation is GET-only

Simply false. Body validation is most useful on POST and PUT, which are the methods that carry bodies.

D Missing resource policy

That would produce visible 500 errors and permission failures in CloudWatch. The scenario says calls succeed against the old alias.

THE SIGNAL

'Settings look correct but behaviour has not changed, and no errors anywhere' is the fingerprint. Nothing is broken - the change simply is not live.

A company sells access to a weather data API. It needs three tiers: free at 10 requests per second and 10,000 per month, standard at 100 per second and 1 million per month, and enterprise at 1,000 per second with no monthly cap.

Excess requests must be rejected at the gateway without invoking the backend, and each customer's usage must be reportable. Customers already authenticate with Cognito.

How should this be implemented?

A An HTTP API with a JWT authoriser and per-route throttling

B A Lambda authoriser that counts requests per customer in DynamoDB and denies over-limit calls

C AWS WAF rate-based rules, one rule per customer tier

D A REST API with usage plans and API keys, one plan per tier, alongside the Cognito authoriser

◆ PAUSE AND DECIDE

How should this be implemented?

A HTTP API with per-route throttling

HTTP APIs have no API keys and no usage plans, so per-customer quotas are impossible. Route throttling applies to everyone equally.

B Lambda authoriser counting in DynamoDB

Rebuilds usage plans by hand, invokes a function on every request including rejected ones, and adds a DynamoDB write to the request path.

C WAF rate-based rules

WAF rate rules work on IP addresses over five-minute windows. They cannot express per-customer monthly quotas.

D REST API with usage plans and API keys

Usage plans express rate, burst and quota per key exactly as described, reject at the gateway, and produce per-key usage reports.

THE SIGNAL

The Cognito detail is a distractor. Authentication and metering are separate jobs, and this API needs both: Cognito for identity, API keys for the tier.

> A Node.js Lambda function behind a REST API with proxy integration returns 502 Bad Gateway for every request. CloudWatch shows the function completing successfully in about 40 ms, with no errors logged and no timeouts.
>
> The handler ends by returning an object containing a status field and a data object.

What is the cause?

A The function timeout is too low for API Gateway's integration timeout

B The response does not match the proxy integration format: it needs statusCode, and body must be a string

C API Gateway lacks permission to invoke the function

D The API has not been deployed since the integration was configured

◆ PAUSE AND DECIDE

What is the cause?

A Function timeout too low

A timeout produces 504, not 502, and the function completes in 40 ms. Both facts rule this out.

B Wrong proxy response shape

Proxy integration requires statusCode and a string body. Returning a bare object means the gateway cannot build a response, so it answers 502.

C Missing invoke permission

That produces 500 with a clear permission error, and the function would never run. Here it runs successfully.

D Not deployed

Scenario 2's answer, and wrong here. An undeployed change means old behaviour, not a 502 on every request.

THE SIGNAL

'The function succeeded and the API still failed' is the whole diagnosis. The failure is between Lambda and the gateway, which is what 502 means.

What to carry forward

- **Six features are REST-only:** API keys, usage plans, caching, request validation, mapping templates, WAF.
- **Nothing takes effect until you deploy to a stage.** The most common mistake there is.
- **502** means a bad backend response. **504** means the backend was too slow.
- API keys meter, they do not authenticate. Pair them with a real authoriser.
- Direct AWS service integrations remove Lambda functions that only forward requests.

> ON THE EXAM
>
> Next: decoupling with SQS, SNS and EventBridge - and the direct integrations from this module become far more useful once those exist.

MODULE 12 · DOMAIN 1 - DEVELOPMENT WITH AWS SERVICES

Decoupling with SQS, SNS and EventBridge

Queue, topic or bus - three shapes of message passing, and the clause in each scenario that picks one.

DOMAIN 1 of 4 EXAM WEIGHT 32% MODULE 12 of 34

admin@sthithapragna.com

A queue is **work to be done**. A topic is **news to be broadcast**. A bus is **news that gets routed**.

The three at a glance

	SHAPE AND DELIVERY
Amazon SQS	Queue. Consumers pull. One consumer per message. Messages persist until deleted
Amazon SNS	Topic. Push to all subscribers at once. Not persisted beyond retries
Amazon EventBridge	Bus. Rules match events to targets. Many AWS and SaaS targets, plus replay

ON THE EXAM

Persistence is the first real difference. SQS holds a message until somebody deletes it; SNS and EventBridge push and are done.

Amazon SQS fundamentals

- Producers send, consumers **poll**. Nothing is pushed to a consumer.
- A message is **hidden** during the visibility timeout, then reappears if not deleted.
- **Deleting is the consumer's job.** Processing a message does not remove it.
- Retention is **4 days by default**, configurable from 1 minute to 14 days.
- Standard queues buffer effectively unlimited throughput and scale with no configuration.

> ON THE EXAM
>
> The delete step is where bugs live. A consumer that processes successfully but never calls DeleteMessage will reprocess the same message forever.

SQS and Lambda together

- An **event source mapping** polls the queue and invokes your function with a batch.
- Lambda **deletes messages for you** when the invocation succeeds. You do not call DeleteMessage.
- On failure, nothing is deleted and the whole batch returns after the visibility timeout.
- **ReportBatchItemFailures** narrows that to only the records that actually failed.
- **Maximum concurrency** on the mapping caps consumption without throttling and re-queuing.

> ON THE EXAM
>
> This is the one case where you do not delete messages yourself. Manual DeleteMessage calls inside a Lambda SQS consumer are a sign something is misunderstood.

The SQS numbers

SETTING	VALUE
Message size	256 KB on the exam. See the currency note shortly
Retention	1 minute to 14 days, default 4 days
Visibility timeout	0 seconds to 12 hours, default 30 seconds
Long polling wait	Up to 20 seconds
Delay queue and message timer	Up to 15 minutes
FIFO throughput	300 TPS per API action, 3,000 with batching
FIFO deduplication window	5 minutes

ON THE EXAM

The 30-second default visibility timeout is shorter than many Lambda timeouts, which is exactly how duplicate processing starts. Module 5 covered the 6x rule.

Standard or FIFO queue?

STANDARD

Throughput

- Nearly unlimited throughput
- Best-effort ordering only
- At-least-once delivery
- Per-message delay supported
- The default choice

FIFO

Order and exactly-once

- 300 TPS, or 3,000 with batching
- Strict order within a message group
- Exactly-once within 5 minutes
- Queue-level delay only
- Name must end in .fifo

THE TIEBREAKER

FIFO costs you two orders of magnitude of throughput. Only choose it when ordering or duplicate suppression is a stated requirement.

FIFO mechanics

- **MessageGroupId** defines the ordering scope. Order is guaranteed **within** a group, not across groups.
- Different groups are processed in **parallel**, so group count is your concurrency.
- One group for everything means a concurrency of exactly one. A classic design mistake.
- **MessageDeduplicationId** suppresses duplicates for 5 minutes.
- **Content-based deduplication** derives that ID from a SHA-256 hash of the body instead.

> ON THE EXAM
>
> Message group ID is the single most important FIFO decision. Use the natural unit of ordering - a customer, an account, a document - not a constant.

Long polling

- **Short polling** returns immediately, often empty. You pay for every empty receive.
- **Long polling** waits up to **20 seconds** for a message to arrive before returning.
- Fewer empty responses means lower cost **and** lower latency, which is unusual.
- Set **ReceiveMessageWaitTimeSeconds** on the queue, or WaitTimeSeconds per request.
- Lambda event source mappings use long polling automatically.

> ON THE EXAM
>
> AWS recommends long polling in nearly every case. When a scenario complains about SQS API costs from an idle queue, this is the answer.

Delays and dead-letter queues

- A **delay queue** hides every new message for up to 15 minutes.
- A **message timer** does the same for one message. Standard queues only.
- A **DLQ** is an ordinary queue named in the source queue's **redrive policy**.
- **maxReceiveCount** decides how many failed receives before a message moves there.
- **Redrive to source** moves messages back once the bug is fixed, without custom code.

> ON THE EXAM
>
> A DLQ must be the same type as its source: a FIFO queue needs a FIFO dead-letter queue. Give the DLQ a longer retention than the source, or you lose evidence.

Currency note: payloads grew to 1 MB

- **The examinable answers:** SQS and SNS messages cap at **256 KB**; Lambda async payloads at **256 KB**.
- **4 August 2025:** SQS raised the maximum message payload to **1 MiB**, standard and FIFO.
- **January 2026:** Lambda asynchronous invocation payloads rose to **1 MB**, including from SQS and EventBridge.
- That removes the usual workaround of storing the body in S3 and sending a pointer.
- Billing still meters per **64 KB**, so a 1 MiB message bills as roughly 16 requests.

> READ THIS TWICE
>
> Note the last line. The limit moved, the pricing model did not, so a large message is not one cheap request. Answer 256 KB on the exam.

Amazon SNS

- Publishers send to a **topic**. Every subscription receives a copy, **pushed**.
- Subscribers: SQS, Lambda, HTTP/S, email, SMS, mobile push, Kinesis Data Firehose.
- Messages are **not stored**. A subscriber that is unavailable relies on the retry policy.
- A **subscription DLQ** catches messages that fail delivery to one subscriber.
- **FIFO topics** exist, and they can only deliver to **FIFO SQS queues**.

> ON THE EXAM
>
> Because SNS does not store messages, a subscriber that is down for an hour loses them. That single fact is why SNS is so often paired with SQS.

The fan-out pattern

- Publish once to a topic, with an **SQS queue subscribed for each consumer**.
- Each consumer gets its own queue, its own backlog and its own retry behaviour.
- A slow or failed consumer cannot affect any of the others.
- Messages persist in each queue, so a consumer can be down and catch up later.
- Adding a consumer means adding a queue. The publisher never changes.

> ON THE EXAM
>
> This is the single most examinable pattern in the module. SNS provides the fan-out; SQS provides the durability that SNS lacks.

Access control on queues and topics

- Queues and topics each have a **resource policy** controlling who may send or subscribe.
- That is how an S3 bucket, an SNS topic or another account is permitted to publish.
- Subscribing a queue to a topic needs a queue policy allowing that topic to send. It is a common omission.
- Encryption at rest uses SQS-managed keys by default, or a KMS key you choose.
- With a customer managed key, every publisher and consumer also needs **kms:Decrypt**.

> ON THE EXAM
>
> A fan-out where one queue silently receives nothing is nearly always a missing queue policy, not a broken subscription.

SNS message filtering

- A **filter policy** on a subscription decides which messages that subscriber receives.
- Filters can match **message attributes**, or the **message body** itself.
- Non-matching messages are simply not delivered, so the subscriber is never invoked.
- One topic can then serve many consumers with different interests.
- Without filtering, every subscriber receives everything and discards what it does not want.

> ON THE EXAM
>
> The same economics as event filtering on a Lambda event source mapping. Filtering at the topic means the consumer is never invoked, so you never pay for it.

Amazon EventBridge

- Events land on a **bus**. The **default bus** already receives events from most AWS services.
- **Custom buses** carry your own application events. **Partner buses** carry SaaS events.
- **Rules** match an **event pattern** against the event and send it to up to 5 targets.
- Patterns match on any field, with prefix, numeric, anything-but and exists matching.
- **Input transformers** reshape an event before it reaches the target.

> ON THE EXAM
>
> The default bus is what makes EventBridge distinctive: AWS services already publish to it, so you can react to service events without instrumenting anything.

Writing an event pattern

- A pattern is JSON that mirrors the event's shape. Fields you omit are ignored.
- Values are given as **arrays**, and any element matching counts as a match.
- Content filters: **prefix**, **suffix**, **numeric**, **cidr**, **exists** and **anything-but**.
- AWS events all carry **source**, **detail-type** and **detail**, so most rules start with source.
- A pattern that matches nothing fails silently. There is no error, just no invocations.

> ON THE EXAM
>
> The silent failure is the practical trap. Use the console's test-pattern feature against a real event rather than assuming a rule matches.

What EventBridge adds

- **Over 20 target types**, including Lambda, SQS, SNS, Step Functions, ECS and API destinations.
- **API destinations** call external HTTP endpoints, with managed auth and rate limiting.
- **Archive and replay** stores matching events and replays them later. SNS cannot do this.
- **Schema registry** discovers event structures and generates code bindings.
- At-least-once delivery, retried for up to 24 hours. A target DLQ catches what fails.

> ON THE EXAM
>
> Archive and replay is the clearest single differentiator against SNS. If a scenario needs to reprocess past events, that is an EventBridge answer.

Scheduling with EventBridge

- **Scheduled rules** on a bus fire on a rate or cron expression. The original approach.
- **EventBridge Scheduler** is a separate, purpose-built service for schedules.
- It supports **one-time** schedules, time zones, daylight saving, and flexible time windows.
- It scales to millions of schedules, and does not consume bus rule quota.
- AWS recommends Scheduler for new work. Scheduled rules remain fully supported.

> ON THE EXAM
>
> 'Run this once at a specific future time, per customer' is Scheduler. A scheduled rule can only do recurring cron on a shared bus.

SNS or EventBridge?

CHOOSE SNS WHEN

Simple fan-out, low latency

- You control both ends
- Fan-out to a known set of subscribers
- Lowest latency of the two
- Email, SMS or mobile push targets
- Very high throughput per topic

CHOOSE EVENTBRIDGE WHEN

Routing, integration, replay

- Reacting to AWS service events
- Complex content-based routing rules
- Many target types, or SaaS partners
- Archive and replay of past events
- Schema discovery and registry

THE TIEBREAKER

Email, SMS and mobile push are SNS-only. Archive, replay, AWS service events and API destinations are EventBridge-only. Those four decide most questions.

Which one does the scenario want?

THE SCENARIO SAYS	THE INTENDED ANSWER
Buffer work, smooth a spike, one worker per item	SQS
Several independent consumers of the same event	SNS fan-out to SQS queues
Order matters, or duplicates must be suppressed	SQS FIFO
React to an AWS service event	EventBridge
Route on event content to different targets	EventBridge
Replay past events after fixing a bug	EventBridge archive and replay
Send an email, an SMS or a push notification	SNS

ON THE EXAM

Worth screenshotting. Most messaging questions on this exam reduce to one row here.

When none of the three is right

- **Multiple consumers must each read every record, and replay it:** that is Kinesis, not SQS.
- **Existing RabbitMQ or ActiveMQ code you cannot change:** that is Amazon MQ.
- **Managed Kafka with existing Kafka tooling:** that is Amazon MSK.
- **Multi-step workflow with branching, retries and human approval:** that is Step Functions.
- **One service reads a queue and writes to another service:** that is EventBridge Pipes.

> ON THE EXAM
>
> Amazon MQ exists almost entirely for lift-and-shift. If a scenario mentions an existing message broker and open protocols like AMQP or MQTT, that is the signal.

Ordering and duplicates, across all three

- **SQS standard:** at-least-once, best-effort ordering. Duplicates are expected.
- **SQS FIFO:** ordered within a group, exactly-once inside a 5-minute window.
- **SNS standard:** no ordering guarantee. FIFO topics deliver only to FIFO queues.
- **EventBridge:** at-least-once, no ordering guarantee at all.
- So idempotent consumers are required almost everywhere. Module 5's pattern applies here too.

> ON THE EXAM
>
> Only SQS FIFO offers ordering, and only within a message group. Any scenario needing global ordering across everything is describing a design problem.

Monitoring a queue

- **ApproximateNumberOfMessagesVisible** is the backlog. Rising means consumers cannot keep up.
- **ApproximateAgeOfOldestMessage** is the best single health signal for a queue.
- **ApproximateNumberOfMessagesNotVisible** is work in flight.
- **NumberOfMessagesSent** against **NumberOfMessagesDeleted** shows whether you are draining.
- Alarm on the **DLQ having any messages at all**. A non-empty DLQ is always worth knowing about.

> ON THE EXAM
>
> Age of oldest message beats queue depth as an alarm. A depth of 10,000 draining quickly is fine; a depth of 5 stuck for an hour is not.

Ask the same question each time: is this **work to be done**, **news to broadcast**, or **news to route**?

> When an order is placed, four independent systems must react: inventory, billing, the shipping partner integration, and an analytics pipeline. Each processes at a different speed, and the analytics consumer is frequently down for maintenance for an hour at a time.
>
> Losing an event for any consumer is unacceptable. The order service must not need changing when a fifth consumer is added later.

What architecture should the team use?

A. Publish to an SNS topic, with a separate SQS queue subscribed for each consumer

B. Publish to an SNS topic, with each consumer's Lambda function subscribed directly

C. Write to one SQS queue that all four consumers poll

D. Have the order service call each of the four systems in turn

◆ PAUSE AND DECIDE

What architecture should the team use?

A SNS fan-out to per-consumer SQS queues

Each consumer gets its own durable backlog, so the analytics outage is survivable. Adding a fifth consumer means adding a queue, not changing the publisher.

B SNS direct to Lambda

The strongest wrong answer. SNS does not store messages, so an hour of analytics downtime loses those events once retries are exhausted.

C One shared queue

A queue message is consumed by one consumer. Four consumers polling one queue means each event is processed once, by whichever gets there first.

D Direct calls

Recouples everything. The order service now knows all four systems, and a slow or failed one blocks order placement.

THE SIGNAL

'Down for an hour' and 'losing an event is unacceptable' together demand durability. SNS provides fan-out; only the queues provide the backlog.

A payments service must apply transactions to each account in the exact order they were submitted. Transactions for different accounts are entirely independent. The system handles about 800 transactions per second across roughly 50,000 accounts.

A first attempt used a FIFO queue with a single message group ID, and throughput collapsed to a few messages per second.

How should the queue be configured?

A Switch to a standard queue and sort transactions by timestamp in the consumer

B Keep one message group but increase the consumer's reserved concurrency

C Use the account ID as the MessageGroupId, so ordering is per account and groups run in parallel

D Split transactions across 50,000 separate FIFO queues, one per account

◆ PAUSE AND DECIDE

How should the queue be configured?

A Standard queue plus sorting

A consumer cannot sort messages it has not received yet. Sorting within a batch does not give ordering across batches.

B More concurrency, one group

The exact misunderstanding that caused the problem. One message group is processed strictly one message at a time, whatever concurrency you configure.

C Account ID as MessageGroupId

Ordering is guaranteed within a group and groups run in parallel. 50,000 accounts gives 50,000 independent ordering scopes, easily covering 800 TPS.

D 50,000 queues

It would preserve ordering, and it means 50,000 queues to create, monitor and consume. Message groups exist precisely to avoid this.

THE SIGNAL

The requirement is order per account, not global order. Message group ID is how you express exactly that scope, and it is the whole reason the feature exists.

A platform team wants a central place for domain events from twelve microservices. Different teams subscribe to different event types, and rules must route on fields inside the event body, not just a type name. Some events must also reach a third-party webhook.

After a recent consumer bug, the team also needs to reprocess three days of past events once the fix ships.

Which service should carry these events?

A One SNS topic per event type, with consumers subscribing to the topics they need

B An EventBridge custom bus, with rules per consumer, API destinations, and an archive

C One SQS queue per consumer, with each service writing to every relevant queue

D An SNS topic with filter policies on each subscription

◆ PAUSE AND DECIDE

Which service should carry these events?

A A topic per event type

Becomes unmanageable quickly, and it still cannot replay past events or call an external webhook with managed auth.

B EventBridge custom bus

Content-based rules, API destinations for the webhook, and archive and replay for the three-day reprocess. All three requirements, one service.

C A queue per consumer

Recouples the producers, which now must know every consumer. Adding a consumer means changing twelve services.

D SNS with filter policies

The closest wrong answer. Filtering works, but SNS has no archive and no replay, so the three-day reprocess is impossible.

THE SIGNAL

'Reprocess three days of past events' is the disqualifying clause. Only EventBridge archives events, so only EventBridge can replay them.

A Lambda function consumes an SQS queue. Its own timeout is 60 seconds and processing usually takes about 45 seconds. The queue's visibility timeout is 30 seconds and maxReceiveCount is 5.

Downstream systems report that some records are created five times over. The configured dead-letter queue is receiving messages steadily, even though CloudWatch shows almost no Lambda errors.

What is the cause?

A maxReceiveCount is too low and should be increased to 10

B The Lambda function is throwing errors that are not being logged

C The DLQ is misconfigured and is receiving messages that succeeded

D The visibility timeout is shorter than the processing time, so messages reappear and are redelivered while still being processed

◆ PAUSE AND DECIDE

What is the cause?

A Raise maxReceiveCount

Would produce ten duplicates instead of five. It raises the threshold without addressing why messages are being received repeatedly.

B Unlogged errors

An unhandled Lambda error is always recorded in the Errors metric. The scenario says there are almost none.

C Misconfigured DLQ

A DLQ receives what the redrive policy sends it. It has no mechanism for capturing successful messages.

D Visibility timeout too short

45 seconds of processing under a 30-second visibility timeout means the message reappears mid-flight and is redelivered, five times, then moves to the DLQ.

THE SIGNAL

Five duplicates and maxReceiveCount of 5 is not a coincidence - it is the tell. AWS recommends visibility timeout of 6x the function timeout, so 360 seconds here.

What to carry forward

- **Work to be done is SQS. News to broadcast is SNS. News to route is EventBridge.**
- **SNS fan-out into SQS queues** is the most examinable pattern here. SNS does not store.
- **MessageGroupId is your FIFO concurrency.** Use the natural unit of ordering.
- Archive and replay, AWS service events and API destinations are EventBridge-only.
- Only SQS FIFO guarantees ordering, and only within a group. Assume duplicates elsewhere.

> ON THE EXAM
>
> Next: Step Functions, which orchestrates the components these three services connect.

MODULE 13 · DOMAIN 1 - DEVELOPMENT WITH AWS SERVICES

Step Functions and Orchestration

Standard against Express, three integration patterns, and error handling that lives outside your code.

DOMAIN 1 of 4 EXAM WEIGHT 32% MODULE 13 of 34

admin@sthithapragna.com

Step Functions moves **coordination** out of your code. Retries, branching, waiting and error handling become configuration you can see.

Amazon States Language

- A state machine is a **JSON document** in Amazon States Language, or ASL.
- It names a **StartAt** state and a map of states, each naming its **Next** or setting **End**.
- Each state receives JSON input and produces JSON output, which becomes the next state's input.
- Workflow Studio builds this visually, and the console renders any definition as a diagram.
- Payloads between states are capped at **256 KB**. Large data goes to S3 with a pointer passed.

> ON THE EXAM
>
> That 256 KB ceiling shapes real designs. Passing an S3 key rather than a document is the standard workaround, and it appears in scenarios.

The eight state types

STATE	WHAT IT DOES
Task	Does the work. Calls a Lambda function or another AWS service
Choice	Branches on a condition. The if statement
Parallel	Runs several branches at once and waits for all of them
Map	Runs the same steps once per item in an array
Wait	Pauses for a duration, or until a timestamp
Pass	Passes input through, optionally injecting fixed data. Useful for testing
Succeed	Ends the execution successfully
Fail	Ends the execution with an error name and cause

ON THE EXAM

Parallel runs different branches concurrently; Map runs the same branch over many items. Confusing those two is a common exam mistake.

Standard or Express workflow?

STANDARD

Long, auditable, exactly-once

- Runs up to one year
- Exactly-once execution
- Full visual execution history
- Priced per state transition
- Supports the callback pattern

EXPRESS

Short, high-volume, cheap

- Runs up to 5 minutes
- At-least-once execution
- History only in CloudWatch Logs
- Priced by duration and memory
- No callback pattern

THE TIEBREAKER

Duration and volume decide it. Over 5 minutes or needing exactly-once means Standard. High-volume short-lived event processing means Express, at a fraction of the cost.

Express workflows in detail

- **Synchronous Express** returns the result to the caller. Good behind API Gateway.
- **Asynchronous Express** returns immediately and the result goes to CloudWatch Logs.
- **At-least-once** means a state can run twice, so tasks must be idempotent.
- No execution history in the console. You query CloudWatch Logs instead.
- Typical use: high-volume event ingestion, IoT data, streaming transformations.

> ON THE EXAM
>
> The idempotency requirement is the real cost of Express. Standard gives exactly-once precisely so you do not have to think about it.

Service integrations

- **Optimised integrations** for common services, with extra behaviour built in.
- **AWS SDK integrations** reach over 14,000 API actions across more than 220 services.
- So a Task state can write to DynamoDB or publish to SNS with no function at all.
- Fewer Lambda functions means less code, lower cost and fewer cold starts.
- The state machine's **IAM role** needs permission for whatever it calls.

> ON THE EXAM
>
> When a scenario describes a Lambda function that only makes one AWS API call inside a workflow, a direct service integration removes it entirely.

The three integration patterns

PATTERN	BEHAVIOUR
`Request Response`	Call and continue immediately. The default
`.sync`	Call and wait for the job to finish. Run a Job
`.waitForTaskToken`	Call, then pause until something calls SendTaskSuccess

ON THE EXAM

The suffix in the resource ARN selects the pattern. Reading that suffix tells you whether the workflow waits, and for what.

The callback pattern

- Step Functions passes a **task token** to the target and then pauses the execution.
- It stays paused, at no compute cost, until **SendTaskSuccess** or **SendTaskFailure** arrives.
- A Standard workflow can wait like this for up to **one year**.
- The classic use is **human approval**: send an email, wait for someone to click.
- **Standard workflows only.** Express does not support callbacks.

> ON THE EXAM
>
> Any scenario with a manual approval step, or an external system that will answer later, is describing this pattern. Nothing else waits indefinitely for free.

Retry

- A **Retry** block lists error names, and Step Functions retries the state on those errors.
- **IntervalSeconds**, **MaxAttempts** and **BackoffRate** control the schedule.
- **States.ALL** matches everything. **States.TaskFailed** and **States.Timeout** are more specific.
- Retriers are evaluated in order, so put specific error names before States.ALL.
- **JitterStrategy** spreads retries out, which prevents a thundering herd.

> ON THE EXAM
>
> This replaces retry loops inside your function code. A Lambda function that retries internally is doing work the state machine should own.

Catch

- A **Catch** block routes to a different state when an error is not resolved by retries.
- Different error names can route to different handlers: notify, compensate, or fail cleanly.
- **ResultPath** on a Catch decides whether the handler sees the error, the input, or both.
- Retry runs first. Catch applies only once retries are exhausted.
- Without a Catch, an unhandled error terminates the whole execution.

> ON THE EXAM
>
> Retry then Catch is the order, and it mirrors ordinary code: try the operation a few times, then handle the failure. Both are configuration, not code.

Map and Parallel

- **Map** runs the same sub-workflow once per item in an array.
- **MaxConcurrency** limits how many run at once. Set it to protect downstream systems.
- **Parallel** runs several **different** branches simultaneously and waits for all.
- In Parallel, if any branch fails the whole state fails, unless that branch catches it.
- Parallel output is an array, one element per branch, in branch order.

> ON THE EXAM
>
> Map is for a list of similar work. Parallel is for a fixed set of different work. If the number of items varies at runtime, it is Map.

Distributed Map

- A Map state in **Distributed mode**, running child workflow executions rather than inline iterations.
- Reads its items directly from **Amazon S3**: a JSON array, a CSV file, or a list of objects.
- Scales to **10,000 parallel child executions**, far past inline Map.
- **ItemBatcher** groups items so one child handles many, cutting cost.
- A **tolerated failure threshold** lets the job continue past a percentage of failures.

> ON THE EXAM
>
> Inline Map is bounded by the 256 KB payload, so the array must fit in state. Distributed Map reads from S3 and has no such ceiling.

Moving data between states

- **InputPath** selects part of the input before the state sees it.
- **Parameters** builds the payload actually sent to the integration.
- **ResultSelector** reshapes what the integration returned.
- **ResultPath** decides where that result is placed relative to the original input.
- **OutputPath** selects what is finally passed on to the next state.

> ON THE EXAM
>
> ResultPath is the one that matters most. Setting it to $ replaces the input entirely; setting it to $.result keeps the input and nests the result inside it.

Currency note: JSONata and Variables

- **The examinable model:** JSONPath, with the five fields on the previous slide.
- **Variables** let one state assign a value that any later state can read directly.
- That removes the need to thread data through every intermediate state.
- **JSONata** is an alternative query language, set with the **QueryLanguage** field.
- Variables total **10 MB per execution**, which is far beyond the 256 KB state payload.

> ON THE EXAM
>
> Genuinely useful in practice, because the 256 KB payload limit was a real constraint. QueryLanguage defaults to JSONPath, so the exam's model is still the default.

Intrinsic functions

- Built-in functions usable inside Parameters, avoiding a Lambda function for trivial work.
- **States.Format** builds a string from a template and values.
- **States.Array**, **States.ArrayLength** and **States.ArrayPartition** manipulate arrays.
- **States.JsonToString** and **States.StringToJson** convert between the two.
- **States.UUID** and **States.MathRandom** generate values inside the workflow.

> ON THE EXAM
>
> The pattern to recognise: a Lambda function that only formats a string or counts an array can be replaced by an intrinsic function.

The limits worth knowing

LIMIT	VALUE
Standard execution duration	1 year
Express execution duration	5 minutes
Payload between states	256 KB
Distributed Map child executions	10,000 in parallel
Execution history entries	25,000 for Standard, then the execution fails
Standard pricing	Per state transition
Express pricing	Per request, plus duration and memory

ON THE EXAM

The 25,000 history entry limit catches long-running loops. A workflow that polls in a Wait loop for weeks will eventually hit it.

Step Functions or just code?

USE STEP FUNCTIONS WHEN

Coordination is the problem

- Several steps with branching
- Retries differ per step
- Something must wait, or wait for a human
- You need to see where it failed
- Steps span multiple services

KEEP IT IN CODE WHEN

The logic is one unit

- A single logical operation
- Sub-second latency matters
- Very high volume, cost-sensitive
- The steps never fail independently
- State transition cost would dominate

THE TIEBREAKER

Step Functions charges per state transition on Standard, so a chatty workflow of trivial steps can cost more than the work it coordinates.

Currency note: Durable Functions, revisited

- **The examinable rule:** multi-step or long-running workflows use Step Functions.
- **Lambda Durable Functions**, announced December 2025, keep the workflow in ordinary code.
- They checkpoint automatically and can wait up to a year without billing idle time.
- **Not in exam guide v2.1.** Answer Step Functions.
- Step Functions still wins on visual history, service integrations without code, and Distributed Map.

> ON THE EXAM
>
> Module 6 flagged this from the Lambda side. Repeated here because this is the module where the wrong instinct would actually cost you a mark.

Two of these are decided by **how long something waits**. Find the duration before you look at the options.

> An expense approval workflow validates a claim, then requires a manager to approve it before payment. Managers are emailed a link and typically respond within a day, but occasionally take up to three days.
>
> The team wants no compute running while waiting, wants to see which claims are stuck awaiting approval, and wants the workflow to continue automatically once a manager clicks.

How should the approval step be implemented?

A. A Wait state of 72 hours, then a Task state that checks an approvals table

B. An Express workflow that polls an approvals table every 30 seconds

C. A Lambda function that loops, sleeping and re-checking until approval arrives

D. A Task state using .waitForTaskToken, with the token sent in the approval email

◆ PAUSE AND DECIDE

How should the approval step be implemented?

A Wait 72 hours then check

Every claim waits the full three days even when approved in ten minutes. It also fails if a manager takes slightly longer.

B Express workflow polling

Express workflows stop at five minutes, so this cannot span three days. Polling also burns invocations to discover nothing has changed.

C Lambda sleep loop

Lambda stops at 15 minutes, and you pay for every second of sleeping. Paying a function to wait is the anti-pattern this pattern removes.

D .waitForTaskToken

The execution pauses with no compute running, appears in the console as awaiting callback, and resumes the moment SendTaskSuccess arrives.

THE SIGNAL

Three requirements: no compute while waiting, visible status, and immediate resumption. The callback pattern is the only option that delivers all three.

A telemetry pipeline receives about 8,000 events per second. Each event runs through the same four steps: validate, enrich from DynamoDB, transform, and write to Kinesis Data Firehose. The whole thing takes under two seconds per event.

Occasional duplicate processing is acceptable because the downstream store is idempotent. Cost at this volume is the primary concern.

Which workflow type should the team use?

A. A Standard workflow, one execution per event

B. An asynchronous Express workflow, one execution per event

C. A Standard workflow with a Distributed Map over batched events

D. A single Lambda function performing all four steps in sequence

◆ PAUSE AND DECIDE

Which workflow type should the team use?

A Standard per event

Standard is priced per state transition. Four transitions x 8,000 per second is an enormous bill for work that finishes in two seconds.

B Express per event

Under five minutes, at-least-once is acceptable, and duration-based pricing is far cheaper at this volume. Exactly what Express was built for.

C Distributed Map over batches

Distributed Map suits a large finite dataset, not a continuous stream. It adds orchestration overhead to a steady flow.

D One Lambda function

The reasonable-looking answer, and it puts branching and retry logic back inside code. Defensible, but it abandons the visibility the workflow provides.

THE SIGNAL

Every clause points at Express: under 5 minutes, duplicates acceptable, cost primary. The scenario grants at-least-once explicitly, which is the permission slip.

A media company must re-encode about 5 million video files listed in an S3 bucket. Each file takes two to ten minutes in an ECS task. The job should run as fast as the encoding fleet allows, and a small percentage of corrupt files should not stop the run.

The team needs a report of which files failed. The list of objects is far too large to pass as workflow input.

How should the job be orchestrated?

A. A Distributed Map state reading the object list from S3, with a tolerated failure threshold

B. An inline Map state over an array of all 5 million object keys

C. A Parallel state with one branch per file

D. An Express workflow triggered once per object by S3 event notifications

◆ PAUSE AND DECIDE

How should the job be orchestrated?

A Distributed Map from S3

Reads items directly from S3 so nothing passes through state, scales to 10,000 child executions, and the tolerated failure threshold handles corrupt files.

B Inline Map

The array would have to fit in the 256 KB state payload. Five million keys is orders of magnitude past that.

C Parallel with one branch per file

Parallel branches are defined statically in the definition. You cannot write five million of them, and any failure fails the state.

D Express per object

Encoding takes up to ten minutes, which exceeds the five-minute Express limit. It also gives no aggregate view of the run.

THE SIGNAL

'Far too large to pass as workflow input' names the 256 KB limit without saying it, and only Distributed Map reads its items from S3 instead.

> A payment workflow calls a third-party gateway. The gateway occasionally returns a 503 that succeeds on retry, and occasionally returns a permanent CardDeclined error. Currently a Lambda function implements its own retry loop, and declined cards surface as generic failures.
>
> The team wants transient errors retried with increasing delays, and declined cards routed to a separate notification step, without adding code to the function.

How should this be configured?

A Increase the Lambda function timeout and retry attempts in the function code

B Wrap the Task in a Parallel state with two branches, one for each error type

C Add a Retry block for the transient error with BackoffRate, and a Catch block routing CardDeclined to the notification state

D Configure a Lambda destination on failure pointing at the notification function

◆ PAUSE AND DECIDE

How should this be configured?

A Retry inside the function

Keeps coordination logic in code, which the scenario explicitly wants removed. A retrying function also holds compute open while it waits.

B Parallel with two branches

Parallel runs branches concurrently, it does not select between them. It cannot express 'if this error, go here'.

C Retry plus Catch

Retry with BackoffRate handles the transient 503 with increasing delays; Catch routes CardDeclined to its own state. Both are configuration, not code.

D Lambda destination on failure

Destinations apply to asynchronous invocation. Step Functions invokes Lambda synchronously, so the destination never fires.

THE SIGNAL

Retry then Catch is the pairing: retry the transient, route the permanent. Option D is the module 5 fact reappearing - destinations need async invocation.

What to carry forward

- **Over 5 minutes or exactly-once means Standard.** High volume and short means Express.
- **.waitForTaskToken** pauses for up to a year at no compute cost. Human approval lives here.
- **Retry then Catch.** Transient errors retried, permanent errors routed.
- Inline Map is bounded by the 256 KB payload. Distributed Map reads items from S3.
- Standard is priced per state transition, so chatty workflows get expensive.

> ON THE EXAM
>
> Next: containers, Elastic Beanstalk and the rest of compute - the last Domain 1 module, and another selection-heavy one.

MODULE 14 · DOMAIN 1 - DEVELOPMENT WITH AWS SERVICES

Containers, Beanstalk and the Rest of Compute

ECS, Fargate, App Runner, Elastic Beanstalk and EC2 – and the clause that picks one when several would work.

DOMAIN 1 of 4 EXAM WEIGHT 32% MODULE 14 of 34

admin@sthithapragna.com

AWS compute is a spectrum of **how much you manage**. Every question is asking where on that spectrum the scenario belongs.

The compute spectrum

SERVICE	YOU PROVIDE, AND YOU MANAGE
AWS Lambda	A function. Nothing else. 15-minute ceiling
AWS App Runner	A container image or source repo, and a port
AWS Fargate	A container and a task definition. No instances
Amazon ECS on EC2	Containers plus the instances they run on
Amazon EKS	The same, but Kubernetes. Choose it when Kubernetes is required
Elastic Beanstalk	Application code. It provisions and manages EC2 for you
Amazon EC2	Everything. Full control, full responsibility

ON THE EXAM

Move down the table only when a stated requirement forces you to. 'Least operational overhead' means move up it.

Amazon ECR

- A private container registry. Authentication is **IAM**, via a token from GetAuthorizationToken.
- **Image scanning**: basic on push, or enhanced scanning powered by Amazon Inspector.
- **Lifecycle policies** expire old images automatically, which is how you control storage cost.
- **Tag immutability** stops a tag being reused, so **v1.2** always means the same image.
- Cross-account access needs a repository policy, which is a resource-based policy.

> ON THE EXAM
>
> Tag immutability is worth knowing. Without it, redeploying 'latest' gives you a different image than the one you tested, and nothing records that.

The four ECS objects

- A **task definition** is the blueprint: image, CPU, memory, ports, environment, roles.
- A **task** is one running instantiation of that blueprint.
- A **service** keeps a desired number of tasks running and registers them with a load balancer.
- A **cluster** is the logical grouping the tasks run in.
- Task definitions are **versioned**. Each change creates a new revision, and you deploy a revision.

> ON THE EXAM
>
> Run a task for a one-off job. Create a service for something that must stay running. That distinction appears directly in scenarios.

Fargate or EC2 launch type?

FARGATE

No instances at all

- Pay per task, per second
- No patching or capacity planning
- Each task gets its own kernel
- No GPU, no privileged containers
- Dearer per unit of raw compute

EC2 LAUNCH TYPE

You run the instances

- Pay for instances, used or not
- You patch and scale the fleet
- Bin-packing gives better density
- GPUs, Spot, custom AMIs available
- Cheaper at steady high utilisation

THE TIEBREAKER

Fargate for variable load and least operational overhead. EC2 launch type for steady high utilisation, GPUs, or anything needing host-level control.

Task role or task execution role?

TASK ROLE

What your code may do

- Used by the application in the container
- Reads S3, writes DynamoDB, calls SQS
- Credentials reach the container at runtime
- Analogous to a Lambda execution role
- Optional, if your code calls nothing

TASK EXECUTION ROLE

What ECS may do to start it

- Used by the ECS and Fargate agents
- Pulls the image from ECR
- Sends container logs to CloudWatch
- Fetches secrets named in the task definition
- Not accessible to your code

THE TIEBREAKER

The clean test: a failure before the container starts is the execution role. A failure inside your application code is the task role.

Networking and secrets

- **awsvpc** network mode gives each task its own elastic network interface and private IP.
- That means security groups apply **per task**, which is why Fargate requires this mode.
- A service registers tasks with an **ALB target group**, which handles dynamic ports and health checks.
- The **secrets** field injects Secrets Manager or Parameter Store values as environment variables.
- Those secrets are fetched by the **task execution role**, never by your application.

> ON THE EXAM
>
> The secrets field is the right answer for credentials in containers. Putting a password in the environment block of a task definition exposes it to anyone who can describe the task.

ECS deployment types

- **Rolling update** is the default. ECS replaces tasks gradually within the service.
- **minimumHealthyPercent** and **maximumPercent** control how far capacity may dip or grow.
- **Blue/green via CodeDeploy** stands up a whole new task set behind a second target group.
- That supports canary and linear traffic shifting, plus automatic rollback on a CloudWatch alarm.
- **External** hands control to a third-party deployment controller.

> ON THE EXAM
>
> Rolling update cannot roll back instantly, because the old tasks are gone. Blue/green keeps them until you shift traffic, which is what makes rollback immediate.

EKS and App Runner

- **Amazon EKS** is managed Kubernetes. Choose it when **Kubernetes specifically** is required.
- It carries real operational weight. If the scenario does not name Kubernetes, it is probably not EKS.
- **AWS App Runner** takes a container image or a source repository and runs it as a web service.
- It handles load balancing, TLS, autoscaling and deployments with no configuration.
- HTTP request-response services only. No batch jobs, no queue workers.

> ON THE EXAM
>
> App Runner is the answer when a small team wants a containerised web service running with the least possible effort and does not want to learn ECS.

Elastic Beanstalk

- You upload application code. Beanstalk provisions EC2, an ASG, a load balancer and CloudWatch.
- **The resources are yours.** You pay for the EC2 instances. Beanstalk itself is free.
- An **application** holds **versions**, deployed into **environments** such as dev and prod.
- **.ebextensions** YAML files in the source bundle customise the environment.
- Blue/green is done by deploying to a second environment and **swapping the CNAMEs**.

> ON THE EXAM
>
> Beanstalk is a convenience layer over EC2, not a serverless service. If a scenario stresses no servers or no idle cost, Beanstalk is the wrong answer.

Beanstalk deployment policies

POLICY	BEHAVIOUR
All at once	Every instance at once. Fastest, with downtime
Rolling	In batches. No downtime, reduced capacity
Rolling + additional batch	Extra batch first, so capacity is maintained
Immutable	New instances, then swapped. Safest, doubles the fleet
Traffic splitting	Immutable, then a percentage of traffic. Canary

ON THE EXAM

Rollback speed is the discriminator. Immutable and traffic splitting leave the old instances untouched.

Currency note: Amazon Linux 2 has retired

- **Amazon Linux 2 reached end of life on 30 June 2026.** All AL2-based Beanstalk platform branches retired with it.
- ECS-optimised Amazon Linux 2 AMIs ended support on the same date.
- From **30 September 2026**, retired branches cannot be used for new environments.
- After **15 January 2027**, their default AMIs become inaccessible, so autoscaling and instance replacement fail.
- The migration target is **Amazon Linux 2023**. AL2023 enforces **IMDSv2** by default.

> ON THE EXAM
>
> This does not change an exam answer, but it is the kind of dated fact a scenario can rest on. The IMDSv2 default is the part that breaks older bootstrap scripts.

EC2 for developers

- **User data** is a script run once at first boot, as root. It bootstraps the instance.
- An **instance profile** attaches an IAM role, so the SDK gets credentials automatically.
- **Never put access keys on an instance.** An instance profile is always the correct answer.
- **Instance metadata** at 169.254.169.254 exposes instance details and role credentials.
- **IMDSv2** requires a session token first, which defeats the SSRF attacks IMDSv1 allowed.

> ON THE EXAM
>
> The credential chain from module 2 ends here: environment variables, then shared config, then the instance profile via metadata. That last hop is what IMDSv2 protects.

Choosing compute

THE SCENARIO SAYS	THE INTENDED ANSWER
Event-driven, short, spiky, no idle cost	AWS Lambda
Longer than 15 minutes, or always running	Fargate or ECS
Containerised web service, smallest possible effort	AWS App Runner
Containers, no instances to manage	ECS on Fargate
GPUs, custom AMIs, or steady high utilisation	ECS on EC2
Existing Kubernetes manifests or tooling	Amazon EKS
Just deploy my code, do not make me choose	Elastic Beanstalk

ON THE EXAM

The 15-minute Lambda ceiling is the single most common discriminator on this table, and it decides scenario 2 shortly.

Six scenarios. For each, decide **where on the spectrum** before you read the options.

A new ECS service on Fargate fails to start. The task status goes to STOPPED almost immediately with the message CannotPullContainerError, and no application logs ever appear in CloudWatch.

The container image is in a private ECR repository in the same account. The task definition has a task role granting DynamoDB access, which the application needs once running.

What is the most likely cause?

A The task role is missing DynamoDB permissions

B The ECR repository policy is blocking the account

C The task execution role is missing or lacks ECR pull and CloudWatch Logs permissions

D The container is exiting because of an application error

◆ PAUSE AND DECIDE

What is the most likely cause?

A Task role missing DynamoDB

The task role is used by application code, which never ran. It also could not cause a pull failure.

B Repository policy blocking

Possible in principle, but same-account access is granted by the IAM identity. A repository policy is for cross-account, which this is not.

C Task execution role

Pulling from ECR and writing to CloudWatch Logs are both execution role duties. Missing permissions explain the pull failure and the absent logs together.

D Application error

The image was never pulled, so the application never started. An application error would produce logs and a different stop reason.

THE SIGNAL

Two symptoms, one cause: the image could not be pulled AND no logs appeared. Both are execution role responsibilities, which is what makes it a single explanation.

A nightly job ingests a 40 GB data extract, validates it, and loads it into a database. The process takes between 40 and 90 minutes depending on file size. It currently runs on a Lambda function that fails partway through most nights.

The team wants the least operational overhead possible and does not want to manage servers or patch anything.

What should the job run on?

A An ECS task on Fargate, triggered by an EventBridge schedule

B A Lambda function with memory raised to 10,240 MB

C A Step Functions workflow that chains several Lambda functions

D An EC2 instance started and stopped by a scheduled script

◆ PAUSE AND DECIDE

What should the job run on?

A Fargate task on a schedule

No 15-minute ceiling, no servers to manage, and it runs only when scheduled. It satisfies both the duration and the operational overhead requirement.

B Maximum memory Lambda

More memory makes it faster, not longer. 15 minutes is a hard limit that no configuration changes.

C Step Functions chaining Lambdas

Genuinely viable if the work splits cleanly, and it requires rewriting a single sequential process into coordinated chunks. Far more effort than the scenario allows.

D Scheduled EC2 instance

Works, and hands back patching, AMI management and the scheduling script. That is the operational overhead the team said it did not want.

THE SIGNAL

'40 to 90 minutes' is the whole question. The 15-minute Lambda ceiling is hard, and Fargate is the least-managed thing on the other side of it.

A Beanstalk environment runs a customer-facing API on eight instances. Deployments must never reduce capacity, must never serve a mix of old and new versions to different users, and must be reversible within seconds if error rates rise.

The team accepts a longer deployment time and a temporarily larger fleet in exchange for that safety.

Which deployment policy should be configured?

A All at once

B Rolling

C Rolling with additional batch

D Immutable

◆ PAUSE AND DECIDE

Which deployment policy should be configured?

A All at once

Takes the whole environment down during the deploy. It fails the capacity requirement outright.

B Rolling

Reduces capacity while batches update, and serves a mix of versions during the deploy. Fails two requirements.

C Rolling with additional batch

The closest wrong answer. It maintains capacity, but still serves a mix of versions mid-deploy, and rollback means another rolling deploy backwards.

D Immutable

New instances in a separate ASG, swapped only when healthy. Capacity is maintained, no mixing, and rollback is terminating the new instances - which is instant.

THE SIGNAL

Three requirements, and 'reversible within seconds' is the one that eliminates C. Rolling deployments have already replaced the old instances; immutable has not.

A three-person startup has a containerised Node.js web API in a Dockerfile. They need it publicly reachable over HTTPS with a certificate, scaling with traffic, and redeploying whenever they push to their main branch.

Nobody on the team knows ECS or Kubernetes, and they explicitly do not want to learn either. There are no batch jobs or background workers.

What should they deploy to?

A. An ECS service on Fargate behind an Application Load Balancer

B. AWS App Runner, pointed at the source repository

C. An EKS cluster with a Kubernetes Deployment and Ingress

D. An Elastic Beanstalk multi-container Docker environment

◆ PAUSE AND DECIDE

What should they deploy to?

A ECS on Fargate with an ALB

Entirely correct, and it means learning task definitions, services, target groups and listeners. The scenario rules that learning out.

B App Runner from the source repo

Builds from the repository, provides HTTPS and a certificate, autoscales, and redeploys on push. Every requirement, with essentially no configuration.

C EKS

The most operational overhead of any option, for a team that named Kubernetes as something they will not learn.

D Beanstalk Docker environment

Plausible, and it provisions EC2 instances the team then owns and patches, which is more than App Runner asks of them.

THE SIGNAL

This is an HTTP request-response service with no background work, which is exactly App Runner's boundary. Change 'no batch jobs' to 'plus a queue worker' and the answer becomes ECS.

> An ECS service on Fargate serves a payment API. Deployments currently use a rolling update. After a recent bad release, error rates rose and the team took eleven minutes to restore service by redeploying the previous task definition.
>
> They want a new version validated against real traffic gradually, and want reverting to be immediate rather than another deployment.

How should deployments be changed?

A Keep rolling updates but set minimumHealthyPercent to 100

B Deploy to a second ECS cluster and switch Route 53 records

C Use the blue/green deployment type with CodeDeploy, shifting traffic in canary steps with an alarm-triggered rollback

D Add a Lambda function that monitors errors and redeploys the old task definition

◆ PAUSE AND DECIDE

How should deployments be changed?

A minimumHealthyPercent 100

Maintains capacity during the deploy and does nothing for rollback. The old tasks are still replaced, so reverting is still a redeployment.

B Second cluster plus Route 53

Achieves blue/green by hand, at the cost of a duplicate cluster and DNS TTL delays that make switchover far from immediate.

C Blue/green with CodeDeploy

The new task set runs alongside the old one, traffic shifts in canary steps, and rollback is shifting the listener back - seconds, with no redeployment.

D Lambda redeploy on error

Automates the eleven-minute recovery rather than shortening it. A redeployment is still a redeployment.

THE SIGNAL

'Immediate rather than another deployment' rules out every option that has already replaced the old tasks. Only blue/green keeps them running.

An image thumbnailing service runs as an always-on ECS service on Fargate with two tasks, costing about 700 dollars a month. Traffic analysis shows it is idle 94% of the time, handling short bursts when users upload photos.

Each thumbnail takes under three seconds. The code is a small Node.js function in a container, and the team is willing to change how it is packaged.

What change would most reduce cost?

A Move the code to a Lambda function triggered by S3 event notifications

B Reduce the ECS service to one task

C Move the service to ECS on EC2 using Spot Instances

D Add ECS service auto scaling based on CPU utilisation

◆ PAUSE AND DECIDE

What change would most reduce cost?

A Lambda on S3 events

Three-second bursts with 94% idle is precisely Lambda's shape. You pay only per invocation, so the idle 94% costs nothing at all.

B One task

Halves the bill and keeps paying for 94% idle. It also removes the redundancy of a second task.

C ECS on EC2 with Spot

Cheaper per hour, and still billed by the hour for an idle service. It also adds instances to manage, moving down the spectrum.

D Service auto scaling

Fargate services scale to a minimum task count, not to zero. There is always at least one task running and billed.

THE SIGNAL

This is the deliberate reverse of scenario 2. There, duration pushed the work off Lambda; here, idle time pulls it back on. Both turn on the same spectrum.

What to carry forward

- **Task role is your code. Task execution role is ECS starting the container.** Failure before start is the execution role.
- **The 15-minute Lambda ceiling** is the most common compute discriminator on this exam.
- Rolling replaces old instances, so rollback is another deploy. **Immutable and blue/green keep them.**
- App Runner for an HTTP service with least effort. EKS only when Kubernetes is named.
- ECS services scale to a minimum task count. **Only Lambda scales to zero.**

> ON THE EXAM
>
> That completes the Domain 1 services. Next: module 15 opens Domain 2 with IAM policies and how access is actually decided.

MODULE 15 · DOMAIN 2 - SECURITY

IAM Policies and How Access Is Decided

Six policy types, one evaluation algorithm, and the two exceptions that catch everybody.

DOMAIN 2 of 4 EXAM WEIGHT 26% MODULE 15 of 34

admin@sthithapragna.com

Every AWS API call is **denied by default**. Something must explicitly allow it, and nothing may explicitly deny it.

Users, groups and roles

- A **user** is a long-lived identity with permanent credentials. A person or, historically, an application.
- A **group** is a container for users. It holds policies. **A group is not an identity** and cannot be a principal.
- A **role** has no permanent credentials. It is assumed, yielding temporary ones.
- Roles are assumed by AWS services, by other accounts, by federated users and by EC2 instances.
- For applications, the answer is **always a role**. Never an access key stored in code or on a server.

> ON THE EXAM
>
> That last line is close to an absolute on this exam. Any option involving embedded access keys is essentially always wrong.

The root user

- The root user has unrestricted access and **cannot be limited by an identity policy**.
- Enable **MFA** on it, remove any access keys, and then stop using it.
- A handful of tasks genuinely require it: closing the account, changing support plans, some billing settings.
- An **SCP** is the only thing that can constrain root, and only in a member account.
- Everything else should use an IAM identity or, better, a federated role.

> ON THE EXAM
>
> Any option that solves a problem by using root credentials is wrong. The root user is for a short list of account-level tasks and nothing else.

The six policy types

TYPE	WHAT IT DOES
Identity-based	Attached to a user, group or role. Grants permissions
Resource-based	Attached to a resource. Grants permissions, and names a Principal
Permissions boundary	Caps what an identity policy can grant. Never grants
Service control policy	Caps what an account may do, across an Organization. Never grants
Session policy	Caps a specific assumed-role session. Never grants
Access control list	Legacy, cross-account, on S3 and a few others. Disabled by default now

ON THE EXAM

Only two of these grant anything. The middle three are ceilings, and confusing a ceiling with a grant is the most common IAM misunderstanding.

Anatomy of a policy

- **Version** is a date, and it should be 2012-10-17. Older versions lack policy variables.
- **Effect** is Allow or Deny. **Action** names the API operations. **Resource** names the ARNs.
- **Principal** appears only in resource-based policies. It names who the policy is about.
- **Condition** makes the statement apply only when the request context matches.
- **Sid** is an optional label. **NotAction** and **NotResource** invert the match, and are easy to get wrong.

> ON THE EXAM
>
> The presence or absence of Principal is the quickest way to tell the two granting policy types apart when you are shown a policy document.

Reading a policy quickly

- Find **Effect** first. A Deny statement matters more than every Allow around it.
- Then **Resource**. A wildcard here is usually where the over-permission lives.
- Then **Condition**. It is where a policy that looks permissive becomes narrow.
- Check for **Principal** to identify whether it is a resource-based policy.
- Watch for **NotAction**, which means everything **except** what is listed. It is easy to misread as a whitelist.

> ON THE EXAM
>
> Exam questions show policies you must read under time pressure. Effect, Resource, Condition, in that order, extracts the meaning fastest.

Actions, resources and wildcards

- Actions are **service:Operation**, such as s3:GetObject or dynamodb:PutItem.
- Wildcards work: **s3:Get*** matches every S3 read operation.
- **Action: "*"** with **Resource: "*"** is administrator access. It is almost never a correct answer.
- An ARN is **arn:partition:service:region:account-id:resource**.
- S3 and IAM ARNs omit region and account, which is why S3 bucket ARNs look shorter.

> ON THE EXAM
>
> For S3, the bucket and the objects are two different ARNs. Listing needs the bucket ARN; reading objects needs the bucket ARN with a slash-star suffix.

How a request is decided

01

Deny by default

Every request starts as an implicit deny.

02

Any explicit Deny?

Check every applicable policy. One Deny ends it.

03

Within the ceilings?

SCPs, permissions boundaries and session policies must all permit it.

04

Any explicit Allow?

An identity or resource policy must grant it. Otherwise the implicit deny stands.

ON THE EXAM

Four steps, in this order. Explicit deny is the only universal short-circuit in IAM: nothing anywhere can override it.

Implicit deny against explicit deny

- **Implicit deny** is the default. Nothing granted the permission.
- **Explicit deny** is an actual Deny statement that matched the request.
- Both surface as **AccessDenied**, so the error message does not tell you which.
- The fix differs entirely: implicit means add a grant, explicit means find and remove the block.
- Adding permissions can never resolve an explicit deny. That is the diagnostic tell.

> ON THE EXAM
>
> 'We gave them administrator access and it still fails' is the signature of an explicit deny, almost always from an SCP or a permissions boundary.

Same-account evaluation

- Within one account, the identity policy and the resource policy are a **union**.
- If **either** allows the action, and neither denies it, the request succeeds.
- So a bucket policy alone can grant access to a principal in the same account.
- And an identity policy alone can grant access to a bucket with no bucket policy.
- An explicit deny in either one still overrides the other's allow.

> ON THE EXAM
>
> This is why removing a bucket policy sometimes changes nothing: the identity policy was already granting the access independently.

Cross-account evaluation

- Across accounts, AWS runs **two separate evaluations**, one in each account.
- **Both must return Allow.** The union rule does not apply.
- The **trusted** account's identity policy must allow the action on that resource.
- The **trusting** account's resource policy must allow that principal.
- Granting only one side is the most common cross-account mistake there is.

> READ THIS TWICE
>
> Same account is OR. Cross account is AND. That single sentence resolves a large share of IAM questions on this exam.

Two ways to share across accounts

- **Assume a role** in the other account. The caller becomes a principal there.
- Permissions then come from the role, and CloudTrail in that account records the activity.
- Or use a **resource-based policy** naming the external principal directly.
- No role switch is needed, and the caller keeps its own identity throughout.
- Not every service supports resource-based policies. S3, SQS, SNS, KMS and Lambda do.

ON THE EXAM

Role assumption is the more general answer, because it works for every service. Resource policies are simpler when the service supports them.

Two exceptions to the OR rule

- **IAM role trust policies** must explicitly allow the principal. An identity policy alone is never enough.
- **KMS key policies** must explicitly allow access, even within the same account.
- So a KMS key with an empty key policy is unusable by anyone except the root account.
- Granting kms:Decrypt in an identity policy achieves nothing if the key policy does not permit it.
- Other services may have similar requirements. Check the service documentation.

> ON THE EXAM
>
> This is why KMS troubleshooting is its own skill. Module 18 returns to it, but the reason lives here in the evaluation logic.

Managed or inline policy?

MANAGED POLICY

A standalone object

- Attach to many identities at once
- Versioned, with rollback to a prior version
- AWS managed, or customer managed
- Reusable and centrally maintained
- The recommended default

INLINE POLICY

Embedded in one identity

- Belongs to exactly one identity
- Deleted when the identity is deleted
- No versioning at all
- Guarantees a strict one-to-one binding
- Useful when the policy must never be reused

THE TIEBREAKER

Prefer customer managed policies. AWS managed policies are convenient and usually far broader than least privilege requires.

Permissions boundaries

- A managed policy attached to a user or role that **caps** its maximum permissions.
- **It grants nothing.** Effective permissions are the intersection of the identity policy and the boundary.
- The classic use is **safe delegation**: let developers create roles, but not privileged ones.
- You require, via a condition, that any role they create carries the boundary.
- An action allowed by the identity policy but absent from the boundary is denied.

> ON THE EXAM
>
> This is how you delegate IAM without enabling privilege escalation. Without a boundary, anyone who can create roles can create an administrator.

Service control policies

- Applied through AWS Organizations to an account or an organizational unit.
- **Grants nothing.** It defines the maximum any principal in that account may do.
- **It applies to the root user too**, which is what makes it a genuine guardrail.
- Typical uses: deny Regions outside approved ones, deny disabling CloudTrail, deny deleting logs.
- It does not apply to the management account, which is why workloads should not live there.

> ON THE EXAM
>
> An SCP is the usual answer to 'the administrator cannot do it either'. Nothing inside the account can override it.

Permissions boundary or SCP?

PERMISSIONS BOUNDARY

Scope: one identity

- Attached to a user or role
- Set by whoever creates the identity
- Used to delegate IAM safely
- Does not affect other identities
- Works without Organizations

SERVICE CONTROL POLICY

Scope: a whole account

- Applied to an account or an OU
- Set centrally by the organization
- Used for organisation-wide guardrails
- Applies to every principal, root included
- Requires AWS Organizations

THE TIEBREAKER

One identity or one account. If the scenario says 'nobody in the account, including administrators', it is an SCP.

The Condition element

- A statement applies only when every condition in its Condition block evaluates true.
- Structure is **operator, then condition key, then value**.
- Multiple keys in one block are combined with **AND**. Multiple values for one key are **OR**.
- If the request context **lacks** the key, the condition fails and the statement does not apply.
- That last point makes **Deny with a condition** subtle, and it is a favourite exam trick.

> ON THE EXAM
>
> A Deny statement conditioned on a key that is absent simply does not apply, so the deny does not happen. This is how people accidentally write policies that do nothing.

Condition operators

OPERATOR	USE
`StringEquals, StringLike`	Exact match, or match with wildcards
`ArnEquals, ArnLike`	Compare ARNs, with wildcard support
`NumericLessThan, NumericGreaterThan`	Numeric comparison
`DateGreaterThan, DateLessThan`	Time-bounded access
`Bool`	True or false keys, such as aws:SecureTransport
`IpAddress, NotIpAddress`	CIDR range matching on the source IP
`Null`	Whether a key is present in the request at all
`...IfExists suffix`	Apply the check only when the key is present

ON THE EXAM

The IfExists suffix is the fix for the absent-key problem: the condition passes when the key is missing, rather than failing.

Global condition keys worth knowing

KEY	WHAT IT CONSTRAINS
aws:SecureTransport	Whether the request used HTTPS. Deny when false
aws:SourceIp	The caller's IP address. Not for AWS service calls
aws:SourceArn / aws:SourceAccount	Which resource or account triggered a service to call on your behalf
aws:PrincipalTag / aws:ResourceTag	Tags on the caller and on the resource. The basis of ABAC
aws:RequestTag	Tags being applied by this request
aws:MultiFactorAuthPresent	Whether MFA was used for these credentials
aws:PrincipalOrgID	Whether the caller belongs to your AWS Organization

ON THE EXAM

aws:PrincipalOrgID is the clean way to allow an entire organisation without listing account IDs. aws:SecureTransport false is the standard HTTPS-only deny.

Policy variables

- Substitute request context values into a policy at evaluation time.
- **${aws:username}** lets one policy give every user their own S3 prefix.
- **${aws:PrincipalTag/team}** substitutes a tag value from the caller.
- They require **Version 2012-10-17**. Older policy versions silently treat them as literal text.
- They work in Resource and in Condition values, not in Action.

> ON THE EXAM
>
> This is how you avoid writing one policy per user. A home-directory pattern in S3 is the standard example, and it appears in scenarios.

RBAC or ABAC?

ROLE-BASED, RBAC

A policy per job function

- Separate policies for separate roles
- Explicit and easy to audit
- Policy count grows with teams
- Every new team needs a change
- The traditional approach

ATTRIBUTE-BASED, ABAC

One policy, driven by tags

- Compares principal tags to resource tags
- One policy can serve every team
- New teams need no policy change
- Depends on disciplined tagging
- Scales to many teams cleanly

THE TIEBREAKER

The signal is scale. 'Hundreds of teams' or 'without creating a new policy each time' means ABAC. It also means tagging must be enforced.

Trust policies

- Every role has a **trust policy**, which is a resource-based policy naming who may assume it.
- The principal can be a service, an account, a specific role, or a federated provider.
- **Two things are needed:** the trust policy must allow it, and the caller must be allowed sts:AssumeRole.
- **ExternalId** in a third-party trust policy prevents the confused deputy problem.
- The role's own permissions come from a separate identity policy attached to it.

> ON THE EXAM
>
> Trust policy and permissions policy are two different documents on the same role. 'Who can wear this hat' against 'what the hat lets you do'.

Writing least privilege

- Start from an AWS managed policy to get working, then narrow it. Do not ship it.
- Use **IAM Access Analyzer policy generation** to build a policy from actual CloudTrail activity.
- Check **last accessed data** to find granted services that have never been used.
- Scope Resource to specific ARNs. A wildcard resource is where over-permission usually hides.
- Add conditions rather than removing actions, where the constraint is contextual.

> ON THE EXAM
>
> Policy generation from CloudTrail is the answer when a scenario asks how to build a least-privilege policy for an existing application.

IAM Access Analyzer

- **External access findings** identify resources shared outside your account or organisation.
- **Unused access findings** identify roles, users and permissions nobody has exercised.
- **Policy validation** checks a policy against grammar and best practice before you save it.
- **Custom policy checks** can fail a CI/CD pipeline when a change grants new access.
- Findings are produced by automated reasoning, not by sampling traffic.

> ON THE EXAM
>
> The custom policy check in a pipeline is worth knowing: it turns 'did this pull request widen permissions' into an automated test.

Diagnosing AccessDenied

- Read the message. It usually names the **principal**, the **action** and the **resource**.
- Is the resource in **another account**? Then both sides must allow it.
- Is there an **SCP or permissions boundary**? Broad permissions failing points there.
- Is a **condition** failing? Check for a missing key, or an IP or MFA requirement.
- Is it **KMS**? The key policy must allow it independently of the identity policy.

> ON THE EXAM
>
> Use the IAM policy simulator to test before deploying, and CloudTrail to see the exact denied call afterwards. The simulator does not cover SCPs.

Credential report and MFA

- The **credential report** is a CSV of every user, listing key age, password age and MFA status.
- It is the standard answer for auditing which users lack MFA or hold stale access keys.
- **aws:MultiFactorAuthPresent** lets a policy require MFA for sensitive actions.
- Use it with **BoolIfExists**, or calls made by AWS services will fail the condition.
- Access keys should be rotated regularly, and the report is how you find the ones that were not.

ON THE EXAM

The BoolIfExists detail matters: a plain Bool check on MFA denies service-made calls, because those requests carry no MFA key at all.

For each one, decide first: is this an **implicit** deny or an **explicit** deny?

A developer's IAM role has the AdministratorAccess managed policy attached. They can work normally in eu-west-1, but every attempt to create resources in us-east-1 fails with AccessDenied.

The account is a member of an AWS Organization. Adding further permissions to the role changes nothing.

What is the cause?

A. A service control policy on the account or its OU denies actions outside approved Regions

B. The AdministratorAccess policy does not cover us-east-1 and needs a Region condition added

C. The role is missing a trust policy for the us-east-1 endpoint

D. us-east-1 must be enabled for the account before it can be used

◆ PAUSE AND DECIDE

What is the cause?

A An SCP denying non-approved Regions

Administrator access failing, and more permissions not helping, is the signature of an explicit deny. An SCP is the only thing that can override AdministratorAccess.

B AdministratorAccess is Region-limited

It is Action star on Resource star with no Region condition. It covers every Region by definition.

C Missing trust policy

Trust policies govern who may assume a role, not which Regions it may act in. The role is clearly already assumed.

D Region not enabled

us-east-1 is enabled by default in every account. Some newer Regions are opt-in, but not this one.

THE SIGNAL

Two clues together: administrator permissions are failing, and adding more changes nothing. Both point at explicit deny, and in an Organization that means an SCP.

> Account A runs a Lambda function that must read objects from a bucket in Account B. The bucket policy in Account B has been updated to allow s3:GetObject with the Lambda execution role's ARN as the Principal. Block Public Access is enabled, and there are no Deny statements anywhere.
>
> The function still receives AccessDenied. The same function reads buckets in Account A without any problem.

What is missing?

A. Block Public Access must be disabled on the bucket in Account B

B. The bucket policy must use the Account B root as the Principal instead

C. The Lambda function must be moved into Account B

D. The Lambda execution role in Account A needs an identity policy allowing s3:GetObject on that bucket

◆ PAUSE AND DECIDE

What is missing?

A Disable Block Public Access

Block Public Access blocks anonymous and public access. Granting a named role in a named account is neither, so it is unaffected.

B Use the Account B root as Principal

That would grant access to Account B's own principals. The caller is in Account A, so this grants the wrong account.

C Move the function

It would work by turning this into a same-account request, and it is a large architectural change to avoid adding one policy statement.

D Add the identity policy in Account A

Cross-account access requires both sides. The resource policy allows it; the identity policy in the trusted account does not yet.

THE SIGNAL

The last sentence is the clue. Same-account reads work because identity OR resource is enough. Cross-account needs AND, and only one side is configured.

> A platform team wants application developers to create their own IAM roles for Lambda functions, so the platform team is not a bottleneck. Security requires that no role a developer creates can ever grant more than read access to S3 and write access to one DynamoDB table.
>
> Developers must not be able to grant themselves or their roles administrator access, directly or indirectly.

How should this be implemented?

A Give developers iam:CreateRole and review every role they create manually

B Apply a service control policy to the account limiting all principals to those permissions

C Grant iam:CreateRole with a condition requiring a permissions boundary, and attach that boundary to every role they create

D Create the roles in advance and let developers choose from a fixed list

◆ PAUSE AND DECIDE

How should this be implemented?

A CreateRole plus manual review

Manual review is not a control, it is a hope. A developer can create and use a privileged role before anyone looks at it.

B SCP on the account

An SCP would cap everyone in the account, including the platform team and any production workload. Far too blunt for delegation.

C CreateRole conditioned on a permissions boundary

Exactly what boundaries exist for. Developers create roles freely, and every role is capped, so privilege escalation is structurally impossible.

D Fixed list of pre-made roles

Removes the escalation risk by removing the delegation. The platform team is a bottleneck again, which is the problem being solved.

THE SIGNAL

The requirement is delegation plus a ceiling, on identities developers create themselves. That combination is the textbook permissions boundary use case.

> A company has around 200 project teams, each with its own EC2 instances, S3 buckets and DynamoDB tables. Each team must access only its own resources. New teams are created several times a week.
>
> The current approach is one IAM policy per team, and maintaining 200 near-identical policies has become unmanageable. All resources are already tagged with a Project key.

What should the company implement?

A. One IAM group per team, with the existing policies attached to groups instead of users

B. An ABAC policy comparing aws:PrincipalTag/Project to aws:ResourceTag/Project

C. A permissions boundary per team, applied to that team's roles

D. A separate AWS account per team, with cross-account roles

◆ PAUSE AND DECIDE

What should the company implement?

A Groups with the same policies

Improves how policies are attached, not how many exist. You still maintain 200 near-identical documents.

B ABAC comparing principal and resource tags

One policy serves every team, and a new team needs only correct tags rather than a new policy. Exactly the problem described.

C A boundary per team

Boundaries cap, they do not grant, so you would still need the 200 granting policies. It adds 200 more documents.

D An account per team

Genuinely used at scale, and an enormous change that adds 200 accounts to govern. Far beyond what the problem requires.

THE SIGNAL

'Already tagged with a Project key' is the planted detail. ABAC needs disciplined tagging, and the scenario tells you it exists.

What to carry forward

- **Explicit deny always wins.** If more permissions do not help, look for a deny.
- **Same account is OR. Cross account is AND.** Both sides must allow a cross-account call.
- **Trust policies and KMS key policies** must allow explicitly, even in the same account.
- Boundaries cap one identity. SCPs cap a whole account, including root. Neither grants.
- ABAC when the scenario says many teams, frequent change, or resources already tagged.

ON THE EXAM

Next: roles, temporary credentials and the credential chain - how an application actually obtains the permissions this module described.

MODULE 16 · DOMAIN 2 - SECURITY

Roles, Temporary Credentials and the Credential Chain

How code actually obtains the permissions module 15 described - and why it should never hold a password.

DOMAIN 2 of 4 EXAM WEIGHT 26% MODULE 16 of 34

admin@sthithapragna.com

An application should never **hold** credentials. It should be **given** credentials, automatically, that expire.

What temporary credentials are

- An **access key ID**, a **secret access key**, and crucially a **session token**.
- They carry an **expiration**. After it, every call fails until they are refreshed.
- The session token is what distinguishes them. Long-lived user keys have only two parts.
- They are issued by **AWS STS**, and they are not stored anywhere in IAM.
- Nothing needs revoking when they leak, because they expire on their own.

> ON THE EXAM
>
> The security token included in the request is invalid usually means the session token was dropped. Copying only the two keys from a set of three is a classic mistake.

Long-lived keys or a role?

IAM USER ACCESS KEY

Two parts, no expiry

- Valid until you delete it
- Must be stored somewhere
- Rotation is a manual process
- Leaks are permanent until noticed
- Appropriate for almost nothing now

ASSUMED ROLE

Three parts, expiring

- Expires in an hour by default
- Delivered at runtime, never stored
- Rotates with no action from you
- A leak expires by itself
- The answer for every workload

THE TIEBREAKER

On this exam, any option that creates or stores an IAM user access key is essentially always wrong. The only real exception is a system that cannot federate at all.

The STS operations

OPERATION	USE IT WHEN
AssumeRole	An AWS identity assumes a role. Same account or cross-account
AssumeRoleWithWebIdentity	A user has signed in with an OIDC provider such as Google or Cognito
AssumeRoleWithSAML	A user has authenticated with a SAML 2.0 identity provider
GetSessionToken	An IAM user needs temporary credentials, typically with MFA
GetFederationToken	A proxy application issues credentials to non-AWS users
GetCallerIdentity	Ask AWS who you currently are. Needs no permissions at all

ON THE EXAM

GetCallerIdentity is the fastest debugging tool in AWS. It answers 'which identity is this code actually running as', which is often the real question.

AssumeRole in detail

- The caller needs **sts:AssumeRole**, and the role's **trust policy** must allow the caller.
- STS returns temporary credentials with the permissions of the **role**, not the caller.
- The caller's own permissions are **replaced**, not added to. Assuming a role can reduce your access.
- **RoleSessionName** appears in CloudTrail, so it is how you attribute activity to a person.
- A **session policy** passed at assume time further restricts the session. It cannot expand it.

> ON THE EXAM
>
> The replacement behaviour surprises people. Assuming a role is switching identity, not gaining an extra one, and you keep nothing from before.

Session durations

OPERATION	DURATION
AssumeRole	15 minutes to 12 hours. Default 1 hour
Role maximum session duration	Set on the role itself, up to 12 hours. It caps the request
Role chaining	1 hour maximum. Requesting more fails outright
GetSessionToken, IAM user	15 minutes to 36 hours. Default 12 hours
GetSessionToken, root user	15 minutes to 1 hour
AssumeRoleWithWebIdentity	15 minutes to 12 hours. Default 1 hour

ON THE EXAM

Two caps apply to AssumeRole: what you request, and the role's own maximum session duration. Requesting more than the role permits fails.

Role chaining

- Using credentials from one assumed role to assume a second role is **role chaining**.
- **The second session is capped at 1 hour**, whatever the roles allow.
- Requesting a longer duration returns an error rather than a shorter session.
- The usual fix is to grant the first role the permissions directly, avoiding the second hop.
- It happens accidentally: a Lambda or Fargate task assuming a cross-account role is already chaining.

> ON THE EXAM
>
> This is the classic 'works on my laptop, fails in production' bug. Locally you assume from an IAM user, which is not chaining. In Lambda you assume from a role, which is.

admin@sthithapragna.com

Assuming a role in code

- Create an **STS client**, call AssumeRole, and build a second client from the returned credentials.
- Most SDKs offer a **credentials provider** that does this and refreshes automatically.
- Calling AssumeRole on every request is wasteful. Cache the credentials until near expiry.
- In the CLI and SDKs, a profile with **role_arn** and **source_profile** does it declaratively.
- Never write the returned credentials to disk or to an environment variable that persists.

> ON THE EXAM
>
> Reach for the SDK's assume-role credentials provider rather than calling AssumeRole yourself. It handles caching and refresh, which hand-written code usually gets wrong.

The default credential provider chain

01

Environment

Java system properties, then AWS_ACCESS_KEY_ID and friends.

›

02

Web identity

A token file, used by EKS and other OIDC runtimes.

›

03

Shared files

The profile in ~/.aws/credentials and ~/.aws/config.

›

04

Container

The ECS or EKS container credentials endpoint.

›

05

Instance

The EC2 instance profile, read from instance metadata.

ON THE EXAM

Order matters, because the first provider that returns credentials wins. A stray environment variable silently overrides the instance role.

Why the chain matters

- Correctly written code needs **no credential configuration at all**. It works everywhere.
- Explicitly passing keys to a client constructor **bypasses the chain entirely**. Almost always wrong.
- A leftover environment variable or profile beats the instance role, silently.
- **GetCallerIdentity** tells you which provider actually won.
- The SDK **refreshes** temporary credentials before they expire, without your code doing anything.

> ON THE EXAM
>
> That last point is why long-running processes do not need refresh logic. Writing your own refresh loop is a sign the chain is being bypassed.

Where each service gets its credentials

RUNTIME	DELIVERY MECHANISM
AWS Lambda	Environment variables, set by the service from the execution role
Amazon EC2	Instance metadata at 169.254.169.254, from the instance profile
Amazon ECS	A container credentials endpoint, from the task role
Amazon EKS	A projected web identity token, exchanged via AssumeRoleWithWebIdentity
AWS CodeBuild	The build project's service role
On-premises servers	IAM Roles Anywhere, using an X.509 certificate

ON THE EXAM

Every one of these delivers temporary, auto-rotating credentials. None of them requires a stored access key, which is why storing one is always the wrong answer.

Instance metadata and IMDSv2

- Metadata is served at **169.254.169.254**, a link-local address reachable only from the instance.
- Role credentials sit under **iam/security-credentials/** and rotate automatically.
- **IMDSv1** answered any GET request, so an SSRF bug in your app could read credentials.
- **IMDSv2** requires a PUT to obtain a session token first, which most SSRF attacks cannot do.
- IMDSv2 can be **required** per instance, and Amazon Linux 2023 enforces it by default.

> ON THE EXAM
>
> Requiring IMDSv2 is a standard hardening answer. The attack it prevents is a server-side request forgery in your own application reading the instance role's credentials.

The trust policy, revisited

- A resource-based policy on the role. Its **Principal** names who may assume it.
- For a service, the principal is a **service principal** such as lambda.amazonaws.com.
- For cross-account, it is an account ARN or a specific role ARN in the other account.
- Naming an account root trusts **anything in that account** that also has sts:AssumeRole.
- Naming a specific role ARN is tighter, and is the least-privilege choice.

> ON THE EXAM
>
> Trusting an account root is not the same as trusting one principal in it. The other account's administrators decide who gets to use that trust.

The confused deputy problem

- A third party assumes a role in your account to do work for you. So far so good.
- If they know another customer's role ARN, they could be tricked into acting against that account.
- **ExternalId** is a secret value the trust policy requires, unique to each customer.
- You supply it to the third party, and they must present it when assuming the role.
- For AWS services calling on your behalf, use **aws:SourceArn** and **aws:SourceAccount** instead.

> ON THE EXAM
>
> ExternalId is for third-party SaaS vendors. SourceArn and SourceAccount are the equivalent when an AWS service is the deputy.

SAML or web identity federation?

SAML 2.0

Corporate identity

- Active Directory, Okta, Ping
- AssumeRoleWithSAML
- Employees signing in to AWS
- Trust is a SAML identity provider
- Console access via the SAML endpoint

WEB IDENTITY, OIDC

Consumer or workload identity

- Google, Facebook, Apple, Cognito
- AssumeRoleWithWebIdentity
- Mobile and web app end users
- Trust is an OIDC provider
- Also how EKS and GitHub Actions work

THE TIEBREAKER

Employees means SAML. App users means web identity. Both end in temporary credentials from a role, so neither ever needs an IAM user.

Web identity in practice

- A mobile app cannot hold AWS credentials. Anything shipped to a device is readable.
- The user signs in to an identity provider and receives a token.
- The app exchanges that token for temporary AWS credentials via a role.
- The role's policy can use **policy variables** so each user reaches only their own data.
- **Amazon Cognito identity pools** package this whole exchange. Module 17 covers it.

> ON THE EXAM
>
> Any option that puts AWS access keys into a mobile app, a browser or a public repository is wrong. Web identity federation is the standard alternative.

admin@sthithapragna.com

IAM Identity Center

- Central sign-in for people across every account in an AWS Organization.
- **Permission sets** become roles provisioned automatically into each account.
- Identity comes from its own directory, or from Active Directory, or from an external provider.
- It replaces creating IAM users in each account, which does not scale and is hard to offboard.
- It is for **humans**. Applications still use roles attached to compute.

> ON THE EXAM
>
> The exam's framing: IAM users are a legacy pattern for people. Identity Center or federation is the modern answer, and a role is always the answer for code.

Service-linked roles

- A role predefined by a service, linked to it, that the service assumes to act on your behalf.
- Created automatically when you first use the feature that needs it.
- You **cannot edit its permissions**, and you cannot delete it while the service still needs it.
- Examples: Auto Scaling, Elastic Load Balancing, RDS and many others.
- Distinct from a **service role**, which you create and control yourself.

> ON THE EXAM
>
> The distinction: a service role is yours to define, a service-linked role is AWS's. If a scenario asks to modify a role's permissions and cannot, it is service-linked.

IAM Roles Anywhere

- Gives temporary AWS credentials to workloads running **outside** AWS.
- Authentication is an **X.509 certificate** from a certificate authority you register as a trust anchor.
- The server exchanges its certificate for temporary credentials from STS.
- This removes long-lived IAM user access keys from on-premises servers.
- It requires a PKI. If you have none, that is a real prerequisite rather than a detail.

> ON THE EXAM
>
> When a scenario says on-premises servers need AWS access without stored access keys, this is the intended answer.

Revoking a session

- Temporary credentials **cannot be deleted**. There is nothing stored to remove.
- Instead, attach an inline policy denying everything when **aws:TokenIssueTime** is before now.
- The console offers this as **Revoke active sessions** on the role.
- Existing sessions stop working immediately. New assumptions still succeed.
- For a leaked long-lived access key, the answer is different: deactivate and delete the key.

> ON THE EXAM
>
> This is the practical answer to 'temporary credentials have leaked and expire in eleven hours'. Waiting is not the only option.

Session tags and source identity

- **Session tags** are key-value pairs passed when assuming a role.
- They surface as **aws:PrincipalTag**, which is what makes ABAC work with federated users.
- **SourceIdentity** records who the original human was, and cannot be changed once set.
- It persists through role chaining, so attribution survives multiple hops.
- Both appear in CloudTrail, which is how shared-role activity is traced to a person.

> ON THE EXAM
>
> Module 15's ABAC needs principal tags. For a federated user there is no IAM identity to tag, so session tags are where those tags come from.

How requests are actually signed

- Every AWS API call is signed with **SigV4**, using the secret key and the session token.
- The signature covers the method, URI, headers, payload hash, timestamp and Region.
- **The secret key is never transmitted.** Only the derived signature is.
- Clock skew beyond about 5 minutes causes signature failures. A real cause of mystery 403s.
- The SDKs do all of this. You only sign manually when calling AWS from something without an SDK.

> ON THE EXAM
>
> The clock skew detail is worth remembering: a container or device with a wrong clock fails every AWS call, and the error mentions the signature rather than the time.

Auditing who did what

- CloudTrail records the **assumed-role ARN** and the **role session name** on every call.
- A shared role with a generic session name makes activity **unattributable**.
- Set a meaningful **RoleSessionName**, such as the user's identifier, and it appears in every event.
- **SourceIdentity** is stronger, because it cannot be changed once set.
- The AssumeRole call itself is also logged, so you can trace a session back to its origin.

> ON THE EXAM
>
> Roles solve the credential problem and create an attribution problem. Session names and source identity are how you solve the second one.

Ask the same question each time: **what identity does this thing already have**, and what is it trying to become?

A mobile photo app lets users upload images to Amazon S3. Users sign in with Google. Each user must be able to read and write only objects under their own prefix in a shared bucket.

The current build embeds an IAM user's access keys in the application. Security has flagged this and it must be removed.

How should the app obtain AWS access?

A. Store the access keys in the device keychain, encrypted, rather than in the binary

B. Exchange the Google token for temporary credentials via a role, using web identity federation, with a policy scoped by policy variables

C. Create an IAM user per app user and issue each one its own access keys

D. Route all uploads through an EC2 proxy that holds the access keys

◆ PAUSE AND DECIDE

How should the app obtain AWS access?

A Keys in the keychain

Still ships credentials to a device you do not control. Encryption at rest does not help when the app must decrypt them to use them.

B Web identity federation

No credentials are shipped at all. The user's existing Google sign-in is exchanged for short-lived credentials, and policy variables scope each user to their own prefix.

C An IAM user per app user

IAM users are for a handful of long-lived identities, not for application end users. It also hits the 5,000 user quota quickly.

D EC2 proxy

It does remove keys from the device, at the cost of an always-on server carrying every byte of every upload. Module 9 rejected this shape already.

THE SIGNAL

Every option except B involves credentials that persist. Federation is the only pattern where nothing long-lived exists to steal.

> An ECS task on Fargate assumes a cross-account role to read from a partner's S3 bucket. The role's maximum session duration is set to 12 hours, and the code requests 12 hours. Locally, a developer running the same code with their own IAM user credentials gets a 12-hour session successfully.
>
> In Fargate the AssumeRole call fails outright with an error about the requested duration.

What explains the difference?

A Fargate tasks cannot assume cross-account roles

B The role's trust policy is missing the ECS service principal

C The task already runs as a role, so this is role chaining, which caps sessions at 1 hour and fails if more is requested

D The task execution role lacks sts:AssumeRole permission

◆ PAUSE AND DECIDE

What explains the difference?

A Fargate cannot assume cross-account roles

It can, and does so routinely. Nothing about Fargate prevents cross-account assumption.

B Missing service principal

That would prevent assumption entirely with an access denied error, not an error about duration. The error names the duration.

C Role chaining caps at 1 hour

The task runs as its task role, so assuming another role is chaining. That caps at 1 hour, and requesting more fails rather than truncating.

D Missing sts:AssumeRole

Also produces access denied rather than a duration error. And the call clearly reaches STS, since STS is what rejects the duration.

THE SIGNAL

Two clues: the error is specifically about duration, and it works from an IAM user but not from a role. Only chaining explains both.

A SaaS monitoring vendor needs read access to its customers' AWS accounts. Each customer creates a role that the vendor's account assumes. The vendor holds thousands of such role ARNs.

A security review notes that if an attacker learns another customer's role ARN and can persuade the vendor's platform to use it, the vendor's legitimate credentials would satisfy that trust policy too.

What should the trust policy require?

A. A condition on sts:ExternalId, with a unique secret value issued to the vendor per customer

B. A condition on aws:SourceIp restricting assumption to the vendor's IP ranges

C. A condition on aws:MultiFactorAuthPresent so the vendor must use MFA

D. A permissions boundary attached to the role the vendor assumes

◆ PAUSE AND DECIDE

What should the trust policy require?

A ExternalId, unique per customer

Purpose-built for exactly this. The vendor must present a secret only that customer issued, so knowing the role ARN alone is not enough.

B SourceIp restriction

The attack comes from the vendor's own platform, at the vendor's own IP addresses. An IP condition would pass.

C Require MFA

MFA is for humans. An automated SaaS platform assuming roles thousands of times cannot present an MFA code.

D Permissions boundary

Caps what the role can do once assumed. It does nothing about who may assume it, which is the actual vulnerability.

THE SIGNAL

This is the confused deputy problem described without naming it. ExternalId is the documented answer whenever a third party assumes roles across many customers.

A company runs about 400 servers in its own data centre. Each needs to write backup files to Amazon S3. Today each server holds an IAM user's access keys in a configuration file, and those keys have not been rotated in three years.

Security requires temporary, automatically rotating credentials. The servers cannot be migrated to EC2, and the company already operates its own certificate authority.

What should the company implement?

A Rotate the access keys automatically with a scheduled Lambda function

B Store the access keys in AWS Secrets Manager and have each server fetch them at startup

C Run an EC2 proxy that the on-premises servers call, using the instance profile

D Use IAM Roles Anywhere, with the existing certificate authority registered as a trust anchor

◆ PAUSE AND DECIDE

What should the company implement?

A Automated key rotation

Improves a bad pattern rather than replacing it. The keys are still long-lived between rotations and still sit in a config file.

B Keys in Secrets Manager

Circular. The server needs AWS credentials to read Secrets Manager, so it still needs a stored credential to start.

C EC2 proxy

Every backup byte now crosses through a server you must run, scale and secure, purely to avoid a credential problem with a direct solution.

D IAM Roles Anywhere

Exchanges an X.509 certificate for temporary credentials from STS. No stored keys, automatic rotation, and the CA already exists.

THE SIGNAL

Option B's circularity is the useful lesson: you cannot use an AWS service to bootstrap AWS credentials, because reaching it requires credentials already.

What to carry forward

- **Applications are given credentials, never hold them.** Every runtime has a delivery mechanism.
- **Role chaining caps at 1 hour** and fails if you ask for more. Lambda and Fargate chain by default.
- The **credential chain** order matters: the first provider to answer wins, silently.
- **ExternalId** for third-party vendors. **SourceArn** and **SourceAccount** for AWS services.
- Revoke leaked temporary credentials with a **TokenIssueTime** deny, not by deleting anything.

> ON THE EXAM
>
> Next: Amazon Cognito - user pools, identity pools, and the token exchange this module has been pointing at.

MODULE 17 · DOMAIN 2 - SECURITY

Amazon Cognito

User pools sign people in. Identity pools hand out AWS credentials. Almost every question turns on that distinction.

DOMAIN 2 of 4 EXAM WEIGHT 26% MODULE 17 of 34

admin@sthithapragna.com

A user pool answers **who is this person**. An identity pool answers **what may they do in AWS**.

User pool or identity pool?

USER POOL

Authentication

- A directory of users you own
- Sign-up, sign-in, password reset, MFA
- Returns JWTs, not AWS credentials
- Authorises API Gateway and ALB directly
- Federates SAML and social providers in

IDENTITY POOL

AWS credentials

- Holds no users of its own
- Exchanges a token for STS credentials
- Returns access key, secret and session token
- Lets a client call S3 or DynamoDB directly
- Supports unauthenticated guest access

THE TIEBREAKER

If the app calls an API you built, a user pool alone is enough. If it calls an AWS service directly from the client, you need an identity pool too.

User pools

- Sign-up, sign-in, email and phone verification, password policy and account recovery.
- Standard attributes such as email and name, plus **custom attributes** prefixed custom:.
- Custom attributes are fixed at creation. You cannot rename or remove one afterwards.
- **Groups** carry an IAM role and a **precedence** number, and appear in the token.
- External providers can federate in, so one pool serves local and corporate users alike.

> ON THE EXAM
>
> The custom attribute constraint is a real design decision. Adding one is easy; changing your mind means a new pool and a migration.

The three tokens

TOKEN	WHAT IT IS FOR
ID token	Proves identity. Carries user attributes as claims. For your application
Access token	Proves authorisation. Carries scopes and groups. For APIs and resource servers
Refresh token	Obtains new ID and access tokens. Opaque, not a JWT, cannot be decoded

ON THE EXAM

ID and access tokens are both JWTs you can decode and inspect. The refresh token is opaque, which is why you cannot read its expiry from the token itself.

Token lifetimes

- **ID and access tokens:** 5 minutes to 1 day. Default **1 hour**.
- **Refresh token:** 60 minutes to **10 years**. Default **30 days**.
- An access token cannot outlive its refresh token. The console enforces that.
- Configured **per app client**, so a mobile app and a web app can differ.
- **RevokeToken** invalidates a refresh token and every access token issued from it.

> ON THE EXAM
>
> Long refresh tokens mean a signed-out or deactivated user can keep working. RevokeToken is how you stop that without waiting for expiry.

ID token or access token?

ID TOKEN

Who the user is

- Contains email, name, custom attributes
- Audience is your application
- Used to personalise the interface
- Should not be sent to third-party APIs
- Analogous to an identity card

ACCESS TOKEN

What they may do

- Contains scopes and cognito:groups
- Audience is a resource server
- Used to authorise API calls
- The correct token for an API to validate
- Analogous to a ticket

THE TIEBREAKER

Sending an ID token to an API is the classic mistake: it leaks user attributes and was never intended as an authorisation credential.

Validating a token

- Verify the **signature** against the pool's public keys, published at the **jwks_uri** endpoint.
- Check **exp** for expiry, **iss** for the issuing pool, and **aud** or client_id for the app client.
- Check **token_use**: it should be **access** for an API, and **id** for your application.
- Cache the JWKS. Fetching it on every request is a needless round trip.
- API Gateway's Cognito authoriser and ALB authentication do all of this for you.

> ON THE EXAM
>
> Decoding a JWT without verifying the signature is not validation. Anyone can craft a token whose claims say anything at all.

App clients and OAuth flows

- An **app client** represents one application. It sets callback URLs, scopes and token lifetimes.
- Public clients, such as mobile and browser apps, must have **no client secret**.
- **Authorization code grant** with PKCE is the correct flow for user-facing apps.
- **Implicit grant** returns tokens in the URL fragment and is deprecated. Do not choose it.
- **Client credentials** is for machine-to-machine. It returns an access token and no user.

> ON THE EXAM
>
> Machine-to-machine means client credentials, and there is no ID token because there is no user. That absence is itself an exam signal.

Server-side and admin APIs

- **InitiateAuth** and friends are called by the client, with no AWS credentials needed.
- **AdminInitiateAuth** and the other Admin operations are called by your backend, with IAM credentials.
- Admin operations can create, disable, delete and reset users without their involvement.
- **ADMIN_USER_PASSWORD_AUTH** sends the password to your server first. Only use it when the client cannot use SRP.
- **SRP** is the default flow, and never transmits the password at all.

> ON THE EXAM
>
> The Admin prefix is the tell: those calls require AWS credentials, so they can only come from a backend, never from a browser or a device.

Currency note: Managed Login and tiers

- **The examinable term:** the **hosted UI**, a Cognito-provided sign-in page.
- **Managed Login** replaced it, adding a no-code branding editor.
- **Passwordless sign-in** arrived at the same time: passkeys, email OTP and SMS OTP.
- User pools now have tiers: **Lite**, **Essentials** and **Plus**. Essentials is the default.
- **Advanced security features** were renamed **threat protection** and moved into the Plus tier.

> READ THIS TWICE
>
> The rename matters most. Exam material says 'advanced security features'; the console says 'threat protection'. They are the same capability.

Federating into a user pool

- A user pool can trust **social providers**, **SAML 2.0** providers and generic **OIDC** providers.
- Federated users appear in the pool as normal users, with an **identities** attribute.
- Your application receives Cognito tokens either way, so it never handles multiple formats.
- **Attribute mapping** copies claims from the provider into pool attributes.
- Provider profile changes do not propagate automatically. They map at sign-in.

> ON THE EXAM
>
> This is the real value of a user pool as a front door: your code deals with one token format regardless of how the user authenticated.

User pool domains

- Managed Login needs a domain: either a Cognito prefix domain or one of your own.
- A Cognito domain looks like **prefix.auth.region.amazoncognito.com** and needs no setup.
- A **custom domain** such as auth.example.com looks like your brand and needs an ACM certificate.
- That certificate must be in **us-east-1**, because a CloudFront distribution fronts it.
- Callback and sign-out URLs are configured per app client, and must match exactly.

> ON THE EXAM
>
> The us-east-1 certificate requirement is the same gotcha as an edge-optimised API Gateway custom domain, and for the same reason: CloudFront.

MFA and threat protection

- MFA can be **off**, **optional** per user, or **required** for everyone.
- Factors: SMS, **TOTP** authenticator apps, and email. Passkeys cover the passwordless case.
- **Adaptive authentication** scores each sign-in for risk and can step up to MFA.
- **Compromised credentials detection** checks credentials against known breach data.
- Both of those sit in the **Plus** tier, formerly advanced security features.

> ON THE EXAM
>
> The exam framing: risk-based step-up authentication and breach checking are a paid tier, not a default. Basic MFA is available everywhere.

Lambda triggers

TRIGGER	USE IT TO
Pre sign-up	Auto-confirm users, or reject sign-ups from unapproved domains
Post confirmation	Create the user's record in your own database
Pre authentication	Block a sign-in based on your own rules
Post authentication	Log the sign-in, or update a last-seen timestamp
Pre token generation	Add or modify claims in the token
Migrate user	Authenticate against a legacy system and import the user
Custom message	Rewrite verification and invitation emails and SMS
Define, Create and Verify auth challenge	Build a custom authentication flow

ON THE EXAM

These are the most examinable part of user pools. Each answers a specific 'how do I hook into sign-in' question, and the names are self-describing.

The pre token generation trigger

- Runs just before tokens are issued, and can **add, modify or suppress claims**.
- The standard way to inject a **tenant ID**, a role, or entitlements into a token.
- Those claims then reach your backend, so it needs no extra database lookup per request.
- It can also modify group membership for that session.
- Adding claims grows the token. Very large tokens can exceed header size limits.

> ON THE EXAM
>
> This is how multi-tenant applications avoid a lookup on every request: the tenant is already in the verified token, and the signature makes it trustworthy.

The user migration trigger

- The problem: you have an existing user database, but you only store password **hashes**.
- Hashes cannot be imported, so a bulk import would force everyone to reset their password.
- The trigger fires when an unknown user attempts to sign in.
- Your function authenticates them against the old system and returns their attributes.
- Cognito then creates the user, using the password they just typed. Migration is invisible.

> ON THE EXAM
>
> This is the answer whenever a scenario says users must not be forced to reset their passwords during a migration to Cognito.

Custom authentication flows

- **Define auth challenge** decides what challenge comes next, or that authentication is complete.
- **Create auth challenge** generates the challenge, such as sending a one-time code.
- **Verify auth challenge response** checks the answer the user gave.
- Together these build passwordless flows, CAPTCHA steps, or bespoke multi-step sign-in.
- Since November 2024, native passwordless options exist, so custom flows are needed less often.

ON THE EXAM

The three run in a loop: define decides, create issues, verify checks, then define decides again. That loop is the whole mechanism.

Groups and precedence

- A group can carry an **IAM role** and a numeric **precedence**.
- Membership appears in the token as the **cognito:groups** claim.
- Your API can authorise on that claim without any additional lookup.
- **Lower precedence wins.** Zero outranks one, which is the opposite of most intuitions.
- With an identity pool, the role of the highest-priority group is the one selected.

> ON THE EXAM
>
> Groups give you role-based access control for free. Anything finer than that needs claims, an authoriser, or Verified Permissions.

Identity pools

- Takes a token from a trusted provider and returns **temporary AWS credentials**.
- Providers: a Cognito user pool, social providers, SAML, OIDC, or a developer-authenticated identity.
- Two roles are configured: one for **authenticated** users, one for **unauthenticated** guests.
- Guest access is genuinely useful: browse before signing in, with a deliberately narrow role.
- The credentials are ordinary STS credentials, so any AWS SDK uses them normally.

> ON THE EXAM
>
> An identity pool is a managed wrapper around AssumeRoleWithWebIdentity from module 16. It is the same exchange, with the plumbing done for you.

Choosing the role

- **Default roles** give every authenticated user the same role. Simple, and often too broad.
- **Rules-based** selection matches token claims and picks a role accordingly.
- **Token-based** selection uses the **cognito:preferred_role** claim from group membership.
- Role policies can use **policy variables** so each user reaches only their own data.
- **${cognito-identity.amazonaws.com:sub}** is the identity ID, and scopes an S3 prefix per user.

> ON THE EXAM
>
> That last variable is the standard per-user isolation pattern: one bucket, one policy, and every user confined to their own prefix.

Basic and enhanced flows

- **Enhanced flow** is the default: one call to **GetCredentialsForIdentity**.
- It always issues **1-hour** credentials. There is no duration parameter.
- **Basic flow** is three calls: GetId, GetOpenIdToken, then AssumeRoleWithWebIdentity.
- Because the last call is yours, you can request up to the role's maximum session duration.
- Basic flow is disabled by default and must be enabled on the identity pool.

> ON THE EXAM
>
> If a scenario needs identity pool credentials to last longer than an hour, the enhanced flow cannot do it. Basic flow plus a longer role session duration is the answer.

The full flow, end to end

01

Sign in

The user authenticates against the user pool and receives three tokens.

02

Exchange

The app passes the ID token to the identity pool.

03

Credentials

The identity pool returns temporary AWS credentials for a role.

04

Call AWS

The app calls S3 or DynamoDB directly, scoped by the role's policy.

ON THE EXAM

Note step 2 uses the ID token, not the access token. The identity pool is establishing who the user is, which is exactly what the ID token proves.

Cognito with API Gateway

- A **Cognito user pool authoriser** on a REST API validates the token for you. No Lambda.
- Claims reach the backend through **$context.authorizer.claims**.
- HTTP APIs use a **JWT authoriser**, which works with Cognito or any OIDC provider.
- Neither does per-resource authorisation. They answer only 'is this a valid user'.
- For finer rules, use a **Lambda authoriser** or **Amazon Verified Permissions**.

> ON THE EXAM
>
> Module 11 made the same point from the API Gateway side. The Cognito authoriser is authentication; anything conditional on the resource needs more.

Cognito beyond API Gateway

- An **Application Load Balancer** can authenticate users against a user pool before forwarding.
- That gives sign-in to an existing application with no code change at all.
- **AWS AppSync** accepts Cognito user pools as an authorisation mode for GraphQL.
- **Amazon S3** and other services are reached through identity pool credentials, not tokens.
- The ALB passes user claims to the backend in **x-amzn-oidc** headers.

> ON THE EXAM
>
> ALB authentication is the answer when a scenario wants sign-in in front of a legacy application that cannot be modified.

Amazon Verified Permissions

- A managed authorisation service using the **Cedar** policy language.
- Answers questions like: may **this user** perform **this action** on **this document**?
- Integrates with Cognito, so token claims become attributes in the decision.
- Policies live outside application code, so authorisation rules can change without a deploy.
- Distinct from IAM: IAM governs AWS API calls, Verified Permissions governs your application's own resources.

> ON THE EXAM
>
> The dividing line: IAM decides whether you may call s3:GetObject. Verified Permissions decides whether Alice may edit document 42 in your app.

Which identity service?

THE SCENARIO SAYS	THE INTENDED ANSWER
Application end users signing up and signing in	Cognito user pool
A client calling S3 or DynamoDB directly	Cognito identity pool
Employees signing in to the AWS console across accounts	IAM Identity Center
An application or AWS service needing AWS permissions	An IAM role
Per-resource rules inside your own application	Amazon Verified Permissions
Guest access before signing in	Identity pool unauthenticated role

ON THE EXAM

Six rows, and most identity questions on this exam reduce to one of them. The first two are the pair candidates confuse most.

Quotas and practical limits

- Request rates are quota-limited per API category, and heavy sign-in traffic can be throttled.
- Cognito is **Regional**. A user pool does not replicate across Regions.
- There is no built-in export of users **with passwords**, which makes leaving Cognito awkward.
- Caching tokens for their full lifetime is the single biggest cost and throttling saving.
- **Cognito Sync is legacy.** AWS AppSync is the successor for syncing application data.

> ON THE EXAM
>
> The Regional point matters for multi-Region designs: a user pool in one Region is a dependency, and there is no managed replication.

Ask first: does this application call your API, or does it call an AWS service directly?

> A photo-sharing mobile app lets users sign up with email or with Google. Once signed in, the app uploads photos directly from the device to Amazon S3, and each user must be able to access only their own prefix.
>
> The team does not want to build a user directory, and does not want uploads passing through their own servers.

What should the team use?

A. A Cognito user pool only, with the ID token sent to S3

B. A Cognito identity pool only, with users authenticating against Google

C. A Cognito user pool for sign-in, feeding an identity pool that returns credentials scoped by policy variables

D. An IAM user per app user, with keys delivered after sign-in

◆ PAUSE AND DECIDE

What should the team use?

A User pool only

S3 does not accept Cognito JWTs. It requires SigV4-signed requests with AWS credentials, which a user pool does not issue.

B Identity pool only

It would work for Google sign-in alone, and the app also needs email sign-up, which requires a directory. An identity pool holds no users.

C User pool plus identity pool

The user pool provides the directory and Google federation; the identity pool converts the token into credentials, scoped per user by policy variables.

D IAM user per app user

Module 16 ruled this out. IAM users are not for application end users, and the 5,000 user quota makes it unworkable anyway.

THE SIGNAL

Two requirements force both halves: email sign-up needs a directory, and direct S3 access needs AWS credentials. Neither service does both.

A company is moving 400,000 users from a self-hosted authentication system to a Cognito user pool. The existing database stores only bcrypt password hashes, which Cognito cannot import.

Forcing every user to reset their password has been rejected by the business. Users should notice nothing.

How should the migration be done?

A Bulk import users with the CSV import job and send everyone a password reset email

B Configure a user migration Lambda trigger that authenticates against the legacy system on first sign-in

C Store the bcrypt hashes in a custom attribute and validate them in a pre authentication trigger

D Run both systems in parallel indefinitely and route users based on a flag

◆ PAUSE AND DECIDE

How should the migration be done?

A CSV import plus resets

Exactly what the business rejected. CSV import cannot carry passwords, so every user must reset.

B User migration trigger

Built for this. The user signs in normally, the trigger validates against the old system, and Cognito creates the user with the password they typed.

C Hashes in a custom attribute

Pre authentication runs after Cognito has already tried to verify the password, so it cannot substitute its own check. It also stores hashes in a readable attribute.

D Parallel systems

Avoids the migration rather than performing it, and leaves two authentication systems to maintain forever.

THE SIGNAL

'Only hashes' plus 'no password resets' is the exact shape the migration trigger exists for. It appears whenever a scenario describes moving to Cognito.

A multi-tenant SaaS application uses a Cognito user pool behind an API Gateway REST API with a Cognito authoriser. Each Lambda function currently queries DynamoDB on every request to discover which tenant the caller belongs to, adding latency and cost.

Tenant membership changes rarely. The team wants the tenant available to the backend without a lookup, and wants it to be tamper-proof.

What should the team implement?

A Have the client send the tenant ID in a custom HTTP header

B Cache the tenant lookup in ElastiCache, keyed by user ID

C Replace the Cognito authoriser with a Lambda authoriser that does the lookup once and caches it

D Add a pre token generation trigger that injects the tenant ID as a claim

◆ PAUSE AND DECIDE

What should the team implement?

A Client-supplied header

A client-controlled value is not tamper-proof. Any caller could claim another tenant, which is a cross-tenant data breach.

B ElastiCache lookup

Faster than DynamoDB, and still a lookup on every request, plus a cache to run and invalidate.

C Lambda authoriser with caching

Genuinely works, and adds a function invocation and its own cache TTL to manage. More moving parts than putting the value in the token.

D Pre token generation trigger

The tenant becomes a signed claim. No lookup at all, and the signature makes it tamper-proof. Membership changes rarely, so token lifetime is not an issue.

THE SIGNAL

'Tamper-proof' is the deciding word. A signed token claim cannot be altered by the client; a header can. And 'changes rarely' makes token staleness acceptable.

> A document management application uses a Cognito user pool and an API Gateway Cognito authoriser. Any signed-in user can currently call any endpoint. The business needs rules such as: a user may edit a document if they own it, or if they are in the reviewers list for that document's project.
>
> These rules change frequently, and the security team wants them auditable outside application code.

How should authorisation be implemented?

A. Amazon Verified Permissions, with Cedar policies evaluated per request using token claims

B. Cognito groups, with one group per document

C. An IAM policy per user, using policy variables to match document ownership

D. Hard-code the rules in each Lambda function and deploy when they change

◆ PAUSE AND DECIDE

How should authorisation be implemented?

A Verified Permissions with Cedar

Purpose-built for per-resource application authorisation. Policies live outside code, are auditable, and change without a deployment.

B A group per document

Groups are coarse-grained and capped in number. One group per document does not scale past a few hundred documents.

C An IAM policy per user

IAM governs AWS API calls, not your application's documents. It has no concept of a reviewers list on a project.

D Rules in code

Exactly what the security team ruled out. Frequently changing rules would need a deployment each time, and nothing is auditable.

THE SIGNAL

The dividing line again: IAM decides whether you may call s3:GetObject. Verified Permissions decides whether this user may edit that document.

What to carry forward

- **User pool authenticates. Identity pool issues AWS credentials.** Direct AWS calls need both.
- **Access token for APIs, ID token for your app.** The identity pool exchange uses the ID token.
- **Lambda triggers** are the most examinable part: migrate user, pre token generation above all.
- Tokens: 1 hour by default, refresh 30 days. Enhanced identity pool flow always gives 1 hour.
- The Cognito authoriser answers 'valid user'. Per-resource rules need Verified Permissions.

> ON THE EXAM
>
> Next: AWS KMS and encryption in practice, where module 15's key policy exception finally gets the attention it deserves.

MODULE 18 · DOMAIN 2 - SECURITY

AWS KMS and Encryption in Practice

Envelope encryption, key policies, grants – and why a key policy can deny an administrator.

DOMAIN 2 of 4 EXAM WEIGHT 26% MODULE 18 of 34

admin@sthithapragna.com

KMS never sees your data. It protects the **key** that protects your data, and that indirection is the whole design.

The three kinds of KMS key

TYPE	WHO CONTROLS IT
Customer managed	Yours. You set the key policy, rotation, aliases and deletion
AWS managed	Created by a service, named aws/s3 or similar. You cannot edit its policy
AWS owned	Owned by AWS and shared across accounts. You never see it at all

ON THE EXAM

Only customer managed keys give you audit control, cross-account sharing and configurable rotation. Any scenario needing those rules out the other two.

What a KMS key costs

- A customer managed key costs about **1 dollar a month**, plus a charge per request.
- Rotation adds up to **2 dollars a month**, capped after the second rotation.
- AWS managed keys are **free** to keep. You still pay for the requests.
- A key per tenant sounds tidy and becomes expensive fast at thousands of tenants.
- **Encryption context** often isolates tenants adequately without a key each.

> ON THE EXAM
>
> The per-request charge is what makes high-volume workloads expensive, not the monthly key fee. That is why Bucket Keys and caching matter.

Symmetric and asymmetric keys

- **Symmetric** is the default: one key encrypts and decrypts, and it never leaves KMS.
- **Asymmetric** gives a public and private pair. The public key can be downloaded and used outside AWS.
- Asymmetric supports encryption, or **signing and verification**, but a key does one or the other.
- **HMAC** keys generate and verify message authentication codes.
- **Automatic rotation works only on symmetric keys.** Asymmetric and HMAC keys cannot rotate automatically.

> ON THE EXAM
>
> Choose asymmetric only when the other party cannot call KMS. If both sides can, symmetric is simpler, cheaper and rotatable.

The 4 KB limit

- **Encrypt** and **Decrypt** handle at most **4 KB** of data directly.
- That is fine for a password, a token or another key. It is useless for a file.
- Every call is a network round trip, so encrypting a large file directly would be impossibly slow anyway.
- So KMS encrypts a **data key**, and the data key encrypts your data locally.
- **ReEncrypt** moves ciphertext from one KMS key to another without exposing plaintext.

> ON THE EXAM
>
> 4 KB is the number that forces the design. Any scenario about encrypting a file with KMS is really asking whether you know about envelope encryption.

Envelope encryption

01

Generate

Call GenerateDataKey. KMS returns a plaintext key and an encrypted copy.

02

Encrypt

Encrypt your data locally with the plaintext key.

03

Discard

Wipe the plaintext key from memory. Store the encrypted copy beside the data.

04

Decrypt later

Send the encrypted key to KMS, get the plaintext back, decrypt the data.

ON THE EXAM

Step three is the point. The plaintext key exists only in memory, briefly; what is stored is useless without a call to KMS.

The data key operations

- **GenerateDataKey** returns both plaintext and ciphertext. The normal case.
- **GenerateDataKeyWithoutPlaintext** returns only the ciphertext.
- Use it when a component prepares keys for later use and must never see plaintext itself.
- **Decrypt** takes the encrypted data key and returns the plaintext key.
- The ciphertext blob records which KMS key made it, so Decrypt needs no key ID for symmetric keys.

> ON THE EXAM
>
> That last point matters: any principal with kms:Decrypt on the right key can decrypt, without being told which key it is. The blob says so.

Key policies

- **A key policy must explicitly allow access.** An identity policy alone is never enough.
- That is true even within the same account, unlike every other resource policy.
- So a key with an empty policy is unusable by everyone except the root user.
- Every key has exactly one key policy. It cannot be deleted, only edited.
- Locking yourself out is possible, and recovering requires AWS Support.

> READ THIS TWICE
>
> This is why KMS troubleshooting is its own skill. Granting kms:Decrypt in an IAM policy achieves nothing if the key policy does not also permit it.

The default key policy

- The default policy gives the **account root** full access to the key.
- That single statement enables IAM policies in the account to grant key access.
- Remove it and IAM policies stop working entirely for that key.
- Console key creation also adds explicit **key administrator** and **key user** statements.
- Key administrators manage the key. **Key users use it.** Being an administrator does not let you decrypt.

> ON THE EXAM
>
> The last line surprises people. A key administrator can rotate, tag and delete a key without ever being able to decrypt anything with it. That separation is deliberate.

Grants

- A **grant** gives a principal specific key operations without editing the key policy.
- Created with **CreateGrant**, removed with **RetireGrant** or **RevokeGrant**.
- Scoped to named operations, and can carry **encryption context constraints**.
- AWS services use grants constantly. An encrypted EBS volume creates one behind the scenes.
- Grants are **eventually consistent**. Use the returned grant token for immediate use.

ON THE EXAM

Grants suit temporary or programmatic access; key policies suit stable, long-term permissions. Adding a grant does not touch the key policy at all.

Key policy or grant?

KEY POLICY

Stable permissions

- One document per key
- Long-lived, reviewed access
- Changes require editing the policy
- Visible in one place
- The primary access control

GRANT

Temporary or programmatic

- Many grants per key
- Created and retired at runtime
- No policy edit required
- Scoped to specific operations
- How AWS services delegate to themselves

THE TIEBREAKER

'Temporary access that can be revoked without changing the key policy' is a grant, every time. Anything permanent belongs in the policy.

Encryption context

- Key-value pairs supplied at encrypt time and required, identically, at decrypt time.
- It is **not encrypted**. It is authenticated, so it cannot be changed without breaking decryption.
- It appears in **CloudTrail**, which makes decrypt calls far easier to audit.
- It can be enforced in a key policy or grant using the **kms:EncryptionContext** condition key.
- Typical use: bind ciphertext to a tenant or a document, so it cannot be decrypted in another context.

> ON THE EXAM
>
> Because it is logged rather than encrypted, never put anything sensitive in an encryption context. Put an identifier there, not a secret.

Condition keys for KMS

- **kms:ViaService** allows use only through a named service, such as s3.eu-west-1.amazonaws.com.
- So a principal can decrypt objects through S3 but never call KMS Decrypt directly.
- **kms:EncryptionContext:key** requires a specific context value.
- **kms:CallerAccount** and **kms:GrantIsForAWSResource** constrain who and what.
- These make a key policy far tighter than a bare list of principals.

> ON THE EXAM
>
> ViaService is the most useful of these: it turns 'may use this key' into 'may use this key only for this service', which is usually what you actually meant.

Aliases

- A friendly name, always prefixed **alias/**, pointing at one key.
- Applications reference the alias, so the underlying key can change without a code change.
- That is what makes **manual rotation** workable: create a new key, repoint the alias.
- Aliases are **Regional**. The same alias in another Region is a different key.
- AWS managed keys use reserved aliases such as **alias/aws/s3**.

> ON THE EXAM
>
> Referencing aliases rather than key IDs is what makes key changes possible at all. Hard-coded key ARNs make rotation a code change.

Automatic key rotation

- KMS generates new **backing key material**. The key ID, ARN and policy stay the same.
- **Old key material is retained**, so existing ciphertext still decrypts. Nothing is re-encrypted.
- New encryption uses the newest material automatically. Applications notice nothing.
- Symmetric customer managed keys only. Not asymmetric, not HMAC, not custom key stores.
- **AWS managed keys rotate annually** and the schedule cannot be changed.

> ON THE EXAM
>
> The critical point: rotation does not re-encrypt anything. Old data keeps decrypting under retained material, which is why enabling rotation is safe.

Currency note: rotation became configurable

- **The examinable answer:** automatic rotation happens **once a year**.
- Since **12 April 2024** the period is configurable from **90 days to 2,560 days**, about seven years.
- **RotateKeyOnDemand** triggers an immediate rotation, up to **10 times** per key.
- Rotation history is now visible, which matters for compliance evidence.
- Pricing changed too: rotation charges are capped after the second rotation.

> ON THE EXAM
>
> 365 days is still the default and still the exam answer. The change is that it is a default rather than the only option.

Deleting a key

- You **schedule** deletion with a waiting period of **7 to 30 days**. Default 30.
- During the wait the key is unusable, so failures surface before anything is destroyed.
- Deletion can be cancelled at any point during the waiting period.
- Once deleted, **every ciphertext under that key is permanently unrecoverable**.
- Prefer **disabling** the key. It has the same effect and is instantly reversible.

> ON THE EXAM
>
> Alarm on the ScheduledKeyDeletion CloudTrail event. A key scheduled for deletion is one of the few genuinely irreversible actions in AWS.

Multi-Region keys

- A primary key plus **replicas** in other Regions, sharing the same key material and key ID.
- Ciphertext encrypted in one Region **decrypts in another**, with no re-encryption.
- **Ordinary KMS keys are Regional.** Two separate keys cannot decrypt each other's ciphertext.
- Each replica has its own key policy, grants and tags, managed independently.
- This is not automatic. A key must be created as multi-Region from the start.

> ON THE EXAM
>
> The classic failure: cross-Region replication or disaster recovery fails because the target Region's key is a different key. Multi-Region keys are the specific fix.

Server-side encryption options

OPTION	WHAT IT MEANS FOR KMS
SSE-S3	An AWS owned key. No KMS calls, no KMS cost, no audit trail
SSE-KMS	A KMS key. CloudTrail records every use, and access can be revoked
DSSE-KMS	Two independent layers of KMS encryption. Doubles the KMS calls
SSE-C	You send the key with every request. KMS is not involved
Client-side	You encrypt before upload, typically with the AWS Encryption SDK

ON THE EXAM

The reason to choose SSE-KMS over SSE-S3 is control and audit, not stronger encryption. Both use AES-256.

S3 Bucket Keys

- By default, SSE-KMS makes a KMS call for **every object** written and read.
- At scale that is expensive, and it consumes the account's shared KMS request quota.
- An **S3 Bucket Key** is a short-lived bucket-level key derived once from your KMS key.
- S3 then uses it for many objects, cutting KMS requests by up to **99%**.
- Enabled per bucket, and it works with existing keys with no application change.

> ON THE EXAM
>
> Any scenario about KMS throttling or cost on a high-volume S3 workload is pointing at Bucket Keys. It is a configuration change, not a redesign.

Quotas and throttling

- KMS request quotas are **shared across the account and Region**, not per key.
- So one heavy workload can throttle unrelated services that also use KMS.
- Exceeding it returns **ThrottlingException**, which the SDKs retry with backoff.
- Reduce calls with **S3 Bucket Keys**, with **data key caching**, or by requesting a quota increase.
- Cryptographic operations are the metered ones. Describing a key is a separate, higher quota.

> ON THE EXAM
>
> The shared quota is the surprising part. A batch job encrypting millions of objects can cause throttling in a completely unrelated application.

Data key caching

- The **AWS Encryption SDK** can cache data keys and reuse them across many encryptions.
- That cuts KMS calls dramatically for high-throughput client-side encryption.
- It **weakens isolation**: more data ends up protected by the same data key.
- Limits bound the risk: maximum age, maximum messages and maximum bytes per cached key.
- AWS is explicit that this trades some security for performance. Set the limits deliberately.

> ON THE EXAM
>
> An honest trade-off worth knowing. Caching is correct for high-volume workloads with modest sensitivity, and wrong when each item must be independently protected.

Where key material comes from

- **KMS generated** is the default, and the only one supporting automatic rotation.
- **Imported**, sometimes called bring your own key, when you must control key generation.
- Imported material can **expire**, and if you lose your copy the key is unrecoverable.
- **CloudHSM custom key store** keeps material in a HSM cluster you control.
- **External key store**, or XKS, keeps material in a key manager outside AWS entirely.

> ON THE EXAM
>
> Each step away from KMS-generated adds control and adds operational burden. XKS means AWS cannot decrypt your data even in principle - and neither can you if your key manager is down.

KMS or CloudHSM?

AWS KMS

Managed and integrated

- Multi-tenant, FIPS 140-3 Level 3 validated
- Integrated with most AWS services
- AWS manages availability and patching
- Priced per key and per request
- The default for almost everything

AWS CLOUDHSM

Single-tenant hardware

- Dedicated HSM cluster you control
- You manage users, keys and backups
- Supports PKCS#11 and JCE directly
- Priced per HSM hour, considerably more
- For strict compliance or non-AWS workloads

THE TIEBREAKER

Choose CloudHSM only when a requirement names single-tenant hardware, or when an application needs PKCS#11. Otherwise KMS, every time.

Certificates and encryption in transit

- **ACM** issues and renews public TLS certificates **free**, and renews them automatically.
- ACM public certificates **cannot be exported**. The private key never leaves AWS.
- They attach to CloudFront, ALB, API Gateway and App Runner, not to an EC2 instance.
- For CloudFront or an edge-optimised API, the certificate must be in **us-east-1**.
- **ACM Private CA** issues internal certificates, and those can be exported.

ON THE EXAM

The non-exportable rule is the common trap: you cannot install an ACM public certificate on an EC2 instance or an on-premises server.

Diagnosing a KMS AccessDenied

- Does the **key policy** allow the principal? Nothing works without this.
- Does the **identity policy** allow the kms action? Both are required, unlike other services.
- Is a **grant** needed, or was one retired?
- Does the **encryption context** match what was used at encrypt time?
- Is the key in **another Region**, or scheduled for deletion, or simply disabled?

> ON THE EXAM
>
> CloudTrail records every KMS call with the encryption context and the principal, which usually identifies the gap faster than reading policies.

Monitoring key use

- Every KMS call appears in **CloudTrail**, including the encryption context and the caller.
- That is how you prove which principal decrypted what, and when.
- Alarm on **ScheduledKeyDeletion**. It is the one action you cannot undo after the window.
- Watch for **ThrottlingException** volume, which signals the shared quota is under pressure.
- **DisableKey** and key policy changes are also worth alarming on.

> ON THE EXAM
>
> CloudTrail auditability is the whole reason to choose SSE-KMS over SSE-S3. If nobody watches the trail, that reason evaporates.

Two of these turn on the same fact: **a key policy must allow it, whatever the IAM policy says**.

> A Lambda function must encrypt 40 MB report files before writing them to Amazon S3. The team wants to use a customer managed KMS key so that key use is auditable and access can be revoked centrally.
>
> A first attempt called the KMS Encrypt API with the file contents and failed.

How should the encryption be implemented?

A Split the file into 4 KB chunks and call Encrypt on each one

B Request a quota increase for the KMS Encrypt payload size

C Use an asymmetric KMS key, which has no payload limit

D Call GenerateDataKey, encrypt the file locally with the plaintext key, and store the encrypted data key with the object

◆ PAUSE AND DECIDE

How should the encryption be implemented?

A 4 KB chunks

40 MB is about 10,000 chunks, so 10,000 KMS calls per file. Slow, expensive, and it would throttle the account.

B Quota increase

4 KB is a hard API limit, not an adjustable quota. No request changes it.

C Asymmetric key

Asymmetric keys have a smaller payload limit, not a larger one. They are also far slower for bulk data.

D Envelope encryption

The intended pattern. One KMS call per file, encryption happens locally at full speed, and the stored encrypted data key is useless without KMS.

THE SIGNAL

Every wrong answer tries to make KMS encrypt the data. The whole design is that it never does - it encrypts the key that encrypts the data.

> Account A must read objects from a bucket in Account B that are encrypted with SSE-KMS using a customer managed key in Account B. The bucket policy in Account B allows the role in Account A, and the role's identity policy in Account A allows both s3:GetObject and kms:Decrypt on the key ARN.
>
> Downloads still fail with AccessDenied from KMS, not from S3.

What is missing?

A. The KMS key policy in Account B must explicitly allow the role in Account A to use the key

B. The key must be converted to a multi-Region key

C. Account A must create its own KMS key with the same key material

D. The bucket must be switched from SSE-KMS to SSE-S3

◆ PAUSE AND DECIDE

What is missing?

A Key policy must allow the role

KMS requires an explicit allow in the key policy. The identity policy grants the intent; the key policy grants the access. Both are needed.

B Multi-Region key

Multi-Region solves cross-Region, not cross-account. Nothing here suggests two Regions are involved.

C Duplicate the key material

Key material cannot be copied out of KMS, and a separate key could never decrypt this ciphertext anyway.

D Switch to SSE-S3

It would work by abandoning the audit and revocation control the team chose SSE-KMS for. Solving the problem by removing the requirement.

THE SIGNAL

The error coming from KMS rather than S3 is the clue. S3 access is already correct; the key is refusing separately, which is the behaviour unique to KMS.

> A data pipeline writes about 8 million objects a day to a bucket using SSE-KMS with a customer managed key. Teams across the account report intermittent ThrottlingException errors from unrelated services that also use KMS, and the KMS bill has grown sharply.
>
> The encryption requirement, including CloudTrail auditability of key use, cannot be relaxed.

What should the team do?

A. Switch the bucket to SSE-S3 to remove KMS from the path

B. Create a separate KMS key for the pipeline so it has its own request quota

C. Enable S3 Bucket Keys on the bucket

D. Add exponential backoff to the pipeline and accept the throttling

◆ PAUSE AND DECIDE

What should the team do?

A Switch to SSE-S3

Removes KMS calls and removes the CloudTrail audit trail with them, which the scenario says cannot be relaxed.

B A separate key

The tempting one, and it rests on a misunderstanding. KMS request quotas are per account and Region, not per key, so a second key shares the same quota.

C S3 Bucket Keys

Cuts KMS requests by up to 99% by deriving a bucket-level key, keeping SSE-KMS and its audit trail. A per-bucket setting with no code change.

D Backoff

Manages the symptom for the pipeline while unrelated services keep being throttled. It also does nothing about the bill.

THE SIGNAL

Option B is the instructive wrong answer: quotas are per account and Region. Splitting keys is a natural instinct and simply does not help.

A third-party analytics vendor needs to decrypt a specific set of exported files for a two-week engagement. The files are encrypted with a customer managed key whose policy is reviewed quarterly and change-controlled.

Security wants the access scoped to decryption only, revocable at any moment, and does not want the key policy modified for a temporary arrangement.

How should the access be granted?

A Add a statement to the key policy allowing the vendor, and remove it after two weeks

B Create a KMS grant for the vendor's role, scoped to Decrypt, and retire it when the engagement ends

C Share the plaintext data key with the vendor directly

D Create a second KMS key for the vendor and re-encrypt the files under it

◆ PAUSE AND DECIDE

How should the access be granted?

A Edit the key policy

Directly contradicts the requirement. It also puts a temporary change through a quarterly change-control process twice.

B A KMS grant scoped to Decrypt

Grants exist for exactly this: temporary, operation-scoped access created and retired at runtime, with no key policy change.

C Share the plaintext data key

Hands over a key that cannot be revoked, cannot be audited, and protects every file it ever encrypted.

D Second key and re-encrypt

Works, at the cost of re-encrypting the whole dataset and maintaining a second key. ReEncrypt exists, but this is disproportionate.

THE SIGNAL

Three requirements - temporary, operation-scoped, no policy change - describe a grant precisely. That combination appears whenever grants are the answer.

What to carry forward

- **KMS encrypts keys, not data.** Over 4 KB means envelope encryption with GenerateDataKey.
- **A key policy must explicitly allow access**, even same-account. An identity policy alone never suffices.
- **Grants for temporary access**, key policies for permanent access.
- Rotation changes backing material only. Nothing is re-encrypted, and old ciphertext still decrypts.
- KMS quotas are **per account and Region**. Bucket Keys and data key caching reduce calls.

> ON THE EXAM
>
> Next: Secrets Manager, Parameter Store and application configuration - where the things KMS protects actually live.

MODULE 19 · DOMAIN 2 - SECURITY

Secrets, Parameters and Configuration

Secrets Manager, Parameter Store and AppConfig – three stores, and the clause in each scenario that picks one.

DOMAIN 2 of 4 EXAM WEIGHT 26% MODULE 19 of 34

admin@sthithapragna.com

A secret and a configuration value look identical. What separates them is whether it **rotates**.

Where secrets must never live

- **In source code.** It reaches every clone, every fork and the whole commit history.
- **In a container image.** Anyone who can pull the image can read the layers.
- **In a Lambda environment variable.** Readable by anyone with GetFunctionConfiguration.
- **In a CloudFormation template parameter** without NoEcho, where it appears in the stack events.
- Rotating a leaked secret is the fix. Deleting the commit is not, because the history remains.

> ON THE EXAM
>
> Environment variables are the subtle one. They are fine for a table name or an endpoint, and wrong for anything that grants access.

AWS Secrets Manager

- Stores a string or binary secret up to **64 KB**, encrypted with a KMS key.
- Retrieved with **GetSecretValue**, which returns the current value by default.
- Supports **automatic rotation**, which is the feature that justifies its cost.
- Has a **resource policy**, so it can be shared across accounts.
- Can be **replicated to other Regions**, staying in sync with the primary.

> ON THE EXAM
>
> About 40 cents per secret per month plus API calls. Trivial for fifty database passwords, and significant for five thousand configuration values.

Sharing a secret

- A secret has a **resource policy**, so another account can be named as a principal.
- The other account also needs **kms:Decrypt** on the key. Module 18's rule applies here.
- The default AWS managed key **cannot** be shared, so cross-account sharing needs a customer managed key.
- **Replication** keeps read-only copies in other Regions, updated when the primary changes.
- A replica can be **promoted** to standalone if the primary Region is lost.

> ON THE EXAM
>
> The AWS managed key restriction is the practical trap: cross-account sharing fails until the secret is re-encrypted under a customer managed key.

Staging labels

- **AWSCURRENT** is the live value. This is what GetSecretValue returns by default.
- **AWSPENDING** is a new candidate value during rotation, not yet live.
- **AWSPREVIOUS** is the last known-good value, kept for rollback.
- A label attaches to exactly one version. Moving AWSCURRENT moves AWSPREVIOUS automatically.
- Applications should always read the default label and never pin to a version ID.

> ON THE EXAM
>
> Labels are why an application never sees a half-rotated secret: the new value is staged as AWSPENDING and only becomes AWSCURRENT once it has been tested.

The rotation lifecycle

01

createSecret

Generate a new value and store it as AWSPENDING.

02

setSecret

Update the target system to accept the new credential.

03

testSecret

Verify the new credential actually works.

04

finishSecret

Promote AWSPENDING to AWSCURRENT. The old value becomes AWSPREVIOUS.

ON THE EXAM

Four steps a rotation Lambda must implement. testSecret is the one that matters: promotion only happens after the new credential has been proven to work.

Managed and custom rotation

- **Managed rotation** needs no Lambda at all. AWS runs it for you.
- Covers RDS, Aurora, DocumentDB and Redshift admin credentials, among others.
- **Rotation by Lambda** uses an AWS-provided template or your own function.
- Templates exist for the common databases. Anything else means writing the four steps.
- The rotation function needs network access to the target, so a private database means a VPC.

> ON THE EXAM
>
> When a scenario says 'rotate an RDS password with no code', managed rotation is the answer, and it genuinely involves writing nothing.

Single and alternating users

- **Single user** rotates the password of one account, in place.
- There is a brief window where an application holding the old password fails.
- **Alternating users** keeps two accounts and switches between them each rotation.
- The old credential stays valid until the next rotation, so there is no failure window.
- Alternating needs a superuser credential to manage the second account.

> ON THE EXAM
>
> Alternating users is the answer when a scenario says rotation must cause no downtime and applications may cache credentials.

Systems Manager Parameter Store

- Hierarchical key-value storage. Keys look like **/myapp/prod/db/host**.
- Types: **String**, **StringList** and **SecureString**, which is KMS-encrypted.
- **GetParametersByPath** fetches an entire branch in one call, which suits per-environment config.
- Parameters are **versioned**, and labels can pin a release to a known version.
- **No built-in rotation.** That is the dividing line with Secrets Manager.

> ON THE EXAM
>
> The hierarchy is the practical advantage. One GetParametersByPath call on /myapp/prod/ returns every setting an environment needs.

Standard and advanced tiers

	STANDARD AGAINST ADVANCED
Value size	4 KB against 8 KB
Parameters per Region	10,000 against 100,000
Cost	Free against about 5 cents per parameter per month
Parameter policies	Not available against available
Cross-account sharing	Not available against available

ON THE EXAM

Tier is not a security setting. A parameter is a SecureString, or it is not, entirely independently of which tier it sits in.

Parameter policies

- **Expiration** deletes a parameter automatically at a given time.
- **ExpirationNotification** sends an EventBridge event ahead of that expiry.
- **NoChangeNotification** fires when a parameter has not been updated for a period.
- Together these give a poor-man's rotation reminder for values Parameter Store cannot rotate.
- They notify. They do not rotate anything. That distinction is the exam point.

> ON THE EXAM
>
> A no-change notification tells you a credential has gone stale. Secrets Manager would simply have rotated it. That gap is the reason Secrets Manager exists.

Parameter Store and Secrets Manager together

- Parameter Store can **reference** a Secrets Manager secret, using a /aws/reference/secretsmanager/ path.
- So one GetParametersByPath call can return configuration and a rotating secret together.
- The secret still lives in Secrets Manager, so rotation and sharing still work.
- The caller needs permission on **both** services for the reference to resolve.
- This keeps application code reading from one place, whatever the value actually is.

> ON THE EXAM
>
> A useful pattern: rotating credentials stay in Secrets Manager where they belong, while the application still fetches everything from one hierarchy.

Secrets Manager or Parameter Store?

SECRETS MANAGER

Rotation and sharing

- Automatic rotation, managed or custom
- Cross-account resource policy
- Cross-Region replication
- About 40 cents per secret per month
- Purpose-built for credentials

PARAMETER STORE

Configuration at scale

- No rotation at all
- Standard tier is free
- Hierarchical paths and bulk fetch
- Holds configuration and secrets together
- Advanced tier adds policies and sharing

THE TIEBREAKER

Rotation is the fork. If a value must rotate, or be shared across accounts or Regions, it is Secrets Manager. Everything else belongs in Parameter Store.

Reading secrets efficiently

- Calling GetSecretValue on every request adds latency, cost and throttling risk.
- Fetch at **startup**, outside the Lambda handler, so it happens once per environment.
- The **Parameters and Secrets Lambda extension** caches values behind a localhost endpoint.
- Its cache TTL is configurable, so rotated values are still picked up.
- AWS also publishes caching client libraries for several languages.

> ON THE EXAM
>
> Module 3's scenario 4 was exactly this. Caching with a TTL shorter than the rotation period is the pattern that satisfies both latency and freshness.

Injecting secrets into compute

- **ECS task definitions** have a **secrets** field that injects values as environment variables at start.
- The **task execution role** fetches them, so the value never appears in the task definition.
- **CloudFormation dynamic references** resolve a secret at deploy time without storing it in the template.
- Written as **{{resolve:secretsmanager:...}}** or **{{resolve:ssm-secure:...}}**.
- A resolved value can still leak into logs or drift detection. Reference, do not embed.

> ON THE EXAM
>
> The ECS secrets field is the right answer for containers, and it is a different thing from putting the value in the plain environment block.

AWS AppConfig

- Part of Systems Manager. Built for configuration that changes **while the application runs**.
- **Feature flags** and operational settings, toggled without a deployment.
- **Validators** check a configuration before it is released. A JSON Schema, or a Lambda function.
- Configuration is stored in Parameter Store, S3, or AppConfig's own hosted store.
- It is not a secret store. Secrets still belong in Secrets Manager or a SecureString.

> ON THE EXAM
>
> The distinguishing capability is validation and controlled release. Parameter Store changes a value instantly, everywhere, with nothing checking it first.

AppConfig deployment strategies

- A strategy defines **growth rate**, **growth type**, **bake time** and final duration.
- Linear or exponential growth, so a change reaches hosts gradually rather than all at once.
- **Bake time** is a monitoring window after full deployment before it is declared successful.
- A **CloudWatch alarm** can trigger an **automatic rollback** during deployment or bake time.
- Predefined strategies exist: AllAtOnce, Linear50PercentEvery30Seconds, Canary10Percent20Minutes.

> ON THE EXAM
>
> Automatic rollback on an alarm is the feature to remember. A bad configuration change is reverted without anyone being paged.

AppConfig or Parameter Store?

PARAMETER STORE

Read a value

- Simple key-value retrieval
- Changes apply instantly, everywhere
- No validation before release
- No gradual rollout
- Free at standard tier

APPCONFIG

Release a change

- Validated before deployment
- Rolled out gradually
- Automatic rollback on an alarm
- Built for feature flags
- Charged per configuration request

THE TIEBREAKER

Ask whether a bad value would cause an incident. If yes, the validation and rollback are worth paying for. If not, Parameter Store is simpler.

When secrets leak

- **Rotate first.** A leaked secret is compromised the moment it leaves your control.
- Removing the commit does not help. The value exists in every clone and every fork.
- **Amazon Q Developer** and **CodeGuru** scan for hard-coded credentials in code review.
- **Amazon Macie** discovers credentials and personal data already sitting in S3.
- **GuardDuty** alerts on credentials being used from an unexpected location.

> ON THE EXAM
>
> The ordering matters: rotate, then investigate. Cleaning up the repository is housekeeping, not remediation.

Configuration in the deployment pipeline

- CodeBuild reads secrets through **secrets-manager** and **parameter-store** blocks in the buildspec.
- Those resolve at build time, so the value never sits in the buildspec file.
- CodeBuild **masks** known secret values in logs, but only ones it resolved itself.
- A secret echoed by your own script can still reach the build log in plain text.
- Give each environment its own secret, and the pipeline role access to only what it deploys.

> ON THE EXAM
>
> Build logs are a common leak path and are often more widely readable than the secrets themselves. Never echo a resolved value.

Where does this value belong?

THE SCENARIO SAYS	THE INTENDED ANSWER
A database password that must rotate	Secrets Manager, managed rotation
Thousands of config values, cost-sensitive	Parameter Store standard tier
A static API key, one account, no rotation	Parameter Store SecureString
A secret shared across accounts or Regions	Secrets Manager
A feature flag toggled without deploying	AWS AppConfig
A config change that must roll back on an alarm	AWS AppConfig
An endpoint URL or a table name	Environment variable or Parameter Store

ON THE EXAM

Worth screenshotting. Rotation, sharing and controlled release are the three capabilities that pull a value out of the cheapest option.

Ask first: does this value need to rotate, to be shared, or to be released carefully?

> Compliance requires the master password of an Amazon RDS for PostgreSQL database to change every 30 days. Several applications connect to it, and none may fail during the change.
>
> The team wants to write and maintain as little code as possible, and applications should not need modifying to handle the change.

How should the rotation be implemented?

A. A scheduled Lambda function that changes the password and updates a Parameter Store SecureString

B. Secrets Manager with managed rotation on a 30-day schedule, using the alternating users strategy

C. A Parameter Store advanced parameter with a no-change notification after 30 days

D. Rotate manually every month and redeploy the applications with the new password

◆ PAUSE AND DECIDE

How should the rotation be implemented?

A Scheduled Lambda plus Parameter Store

Rebuilding rotation by hand, without staging labels or a test step. A failed run leaves a changed password and a stale parameter.

B Managed rotation, alternating users

RDS is covered by managed rotation, so no code at all. Alternating users keeps the previous credential valid, so nothing fails mid-rotation.

C No-change notification

Notifies that the password is stale. It does not change it, which is the entire requirement.

D Manual rotation plus redeploy

A monthly deployment purely to change a password, with downtime and a manual step that will eventually be forgotten.

THE SIGNAL

Three clauses map to three features: RDS means managed rotation, 'none may fail' means alternating users, and 'as little code as possible' rules out A.

> A platform team manages about 3,000 configuration values across dev, staging and production: endpoint URLs, timeouts, feature toggles, queue names and a handful of static third-party API keys. None of them rotates.
>
> Applications should fetch all settings for their environment in one call. Cost matters, and the current approach of one Secrets Manager secret per value costs over 1,200 dollars a year.

Where should these values live?

A. Keep Secrets Manager, but consolidate values into fewer JSON secrets

B. Environment variables set at deployment time by the CI/CD pipeline

C. Parameter Store advanced tier, for the higher parameter limit

D. Parameter Store standard tier, using a path hierarchy per environment, with SecureString for the API keys

◆ PAUSE AND DECIDE

Where should these values live?

A Fewer, larger secrets

Reduces cost by making configuration harder to read and change. Updating one timeout means rewriting a large JSON blob.

B Environment variables from CI/CD

Changing any value now requires a deployment, and the API keys end up readable in the console.

C Advanced tier

Advanced costs about 5 cents per parameter per month, so 3,000 parameters is roughly 1,800 dollars a year. Worse than the problem.

D Standard tier with hierarchy

3,000 is within the 10,000 standard limit, standard is free, GetParametersByPath serves the one-call requirement, and SecureString covers the API keys.

THE SIGNAL

Option C is the trap: 'more parameters' sounds like it needs the higher tier, but 3,000 is well inside the standard limit of 10,000.

A team wants to release a new checkout algorithm to 5% of traffic, watch error rates, and expand gradually. If errors rise, the change must revert automatically without a deployment or a human decision.

The toggle is read by a fleet of ECS tasks. A malformed value has previously taken the checkout page down.

What should the team use?

A AWS AppConfig, with a validator, a canary deployment strategy and a CloudWatch alarm for rollback

B A Parameter Store parameter that tasks poll every 30 seconds

C A Secrets Manager secret holding the flag, with rotation disabled

D An environment variable, redeploying the service to change the percentage

◆ PAUSE AND DECIDE

What should the team use?

A AppConfig with validator and alarm

Gradual rollout, validation before release, and automatic rollback on an alarm. All three requirements come from one service.

B Polled Parameter Store value

The closest wrong answer. It changes the value without a deployment, and offers no validation, no gradual rollout and no automatic rollback.

C Secrets Manager

A feature flag is not a secret. It costs more, and adds none of the release-control features the scenario needs.

D Environment variable

Requires a deployment for every percentage change, which is exactly what the team is trying to avoid.

THE SIGNAL

Three requirements, and only AppConfig has all three. The 'malformed value took checkout down' sentence is what makes validation non-optional.

A Lambda function reads a database credential from Secrets Manager at the top of its handler. It is invoked about 600 times per second at peak. The team sees roughly 50 ms of added latency on every request and occasional ThrottlingException errors from Secrets Manager.

The credential rotates every 7 days. A rotated credential must be picked up without redeploying the function.

What is the best change?

A. Move the secret into a Lambda environment variable encrypted with a customer managed KMS key

B. Request a Secrets Manager quota increase and add exponential backoff

C. Add the Parameters and Secrets Lambda extension, reading from its local endpoint with a cache TTL well under 7 days

D. Move the GetSecretValue call outside the handler, into module initialisation

◆ PAUSE AND DECIDE

What is the best change?

A Encrypted environment variable

Readable by anyone with GetFunctionConfiguration, and frozen at deploy time, so a rotated credential would never be picked up.

B Quota increase and backoff

Pays for 600 identical calls per second rather than removing them. Backoff manages the symptom and adds latency of its own.

C Parameters and Secrets extension

Removes the API call from the request path entirely, and the TTL guarantees a rotated credential is collected without a deployment.

D Fetch during initialisation

The strongest wrong answer. It fixes latency and throttling, and a warm environment can live for hours, so a rotated credential may be missed for a long time.

THE SIGNAL

Compare with module 3's scenario 4, which was the same shape. Option D is right for a static secret and wrong here, because rotation is the added constraint.

What to carry forward

- **Rotation is the fork.** Rotating, shared or replicated means Secrets Manager. Everything else, Parameter Store.
- **AWSCURRENT, AWSPENDING, AWSPREVIOUS**, and a rotation function's four steps.
- **Alternating users** when rotation must cause no failure window.
- Parameter Store tier is capacity, not security. SecureString is independent of tier.
- **AppConfig** when a configuration change needs validation, gradual rollout or automatic rollback.

> ON THE EXAM
>
> That completes Domain 2. Next: module 20 opens Domain 3 with CloudFormation, and the deployment domain.

MODULE 20 · DOMAIN 3 - DEPLOYMENT

AWS CloudFormation

Template anatomy, Ref against GetAtt, and the update behaviours that decide whether a change is safe.

DOMAIN 3 of 4 EXAM WEIGHT 24% MODULE 20 of 34

admin@sthithapragna.com

A template describes the **desired state**. CloudFormation works out the actions – and the risk is in what it decides to replace.

Template anatomy

SECTION	WHAT IT HOLDS
AWSTemplateFormatVersion	Almost always 2010-09-09. Optional
Description	A string describing the template. Optional
Metadata	Extra data, including cfn-init configuration
Parameters	Values supplied at deploy time
Mappings	Fixed lookup tables, keyed by Region or environment
Conditions	Boolean expressions controlling whether resources are created
Transform	Macros to apply. SAM uses this
Resources	The resources themselves. The only required section
Outputs	Values returned, and optionally exported to other stacks

ON THE EXAM

Resources is the only mandatory section. Everything else is optional, which is worth knowing because the exam asks it directly.

YAML or JSON?

- Both are fully supported and produce identical stacks.
- YAML allows **comments**, multi-line strings and the short form **!Ref** and **!GetAtt**.
- The short form cannot be nested inside another short form. Use the full **Fn::** syntax there.
- JSON is easier to generate programmatically, which is why tools emit it.
- YAML indentation errors are the most common template syntax failure. Validate before deploying.

> ON THE EXAM
>
> Exam questions use both. Recognise !GetAtt and Fn::GetAtt as the same thing, because a question may show either.

Parameters

- Typed: String, Number, List, CommaDelimitedList, and AWS-specific types.
- AWS-specific types like **AWS::EC2::KeyPair::KeyName** are validated before deployment starts.
- **AllowedValues**, **AllowedPattern**, **MinLength** and **Default** constrain what can be passed.
- **NoEcho** masks a value in the console and in events. Use it for anything sensitive.
- **SSM parameter types** fetch the current value from Parameter Store at deploy time.

> ON THE EXAM
>
> NoEcho is the security answer, and it is imperfect: the value still reaches the resource. A dynamic reference to Secrets Manager is better.

Pseudo parameters

- **AWS::Region** and **AWS::AccountId**, so a template works anywhere without editing.
- **AWS::StackName** and **AWS::StackId**, useful for naming and tagging.
- **AWS::Partition**, which is aws, aws-cn or aws-us-gov. Needed to build correct ARNs.
- **AWS::NoValue** removes a property entirely when used with Fn::If.
- **AWS::URLSuffix** differs in China, which is why hard-coded endpoints break there.

ON THE EXAM

AWS::NoValue is the clever one. Returning it from a condition deletes the property rather than setting it to null, which is how optional properties are done.

Ref

- For a **parameter**, Ref returns the value that was passed in.
- For a **resource**, Ref returns that resource's **physical ID**, whatever that means for its type.
- What Ref returns varies by resource type, and that is where mistakes happen.
- An S3 bucket returns its **name**. An EC2 instance returns its **instance ID**.
- An IAM role returns its **role name**, not its ARN. A common source of failures.

> ON THE EXAM
>
> That last line matters constantly, because most places you use a role need the ARN. Ref gives you the name, and GetAtt gives you the ARN.

Fn::GetAtt

- Returns a **named attribute** of a resource, rather than its physical ID.
- Syntax takes two arguments: the logical resource name and the attribute name.
- **!GetAtt MyRole.Arn**, **!GetAtt MyBucket.DomainName**, **!GetAtt MyDB.Endpoint.Address**.
- Nested attributes use a dot, as in the database endpoint example above.
- Available attributes are per resource type. Only the documented ones exist.

> ON THE EXAM
>
> The database endpoint is the classic case. Ref on an RDS instance gives an identifier; GetAtt Endpoint.Address gives the hostname you actually connect to.

Ref or GetAtt?

REF

The physical ID

- One argument: the logical name
- Parameter value, or resource physical ID
- Bucket name, instance ID, role name
- Queue URL for an SQS queue
- Topic ARN for an SNS topic

FN::GETATT

A named attribute

- Two arguments: resource and attribute
- Anything the resource type publishes
- ARNs, endpoints, DNS names
- Queue ARN for an SQS queue
- Nested attributes use a dot

THE TIEBREAKER

Note SNS and SQS disagree: Ref gives a topic ARN but a queue URL. That inconsistency is exactly the kind of detail the exam tests.

Fn::Sub and Fn::Join

- **Fn::Sub** substitutes variables into a string using **${...}** placeholders.
- **${MyBucket}** is a Ref. **${MyRole.Arn}** is a GetAtt, inline.
- **${!Literal}** escapes, producing a literal dollar-brace in the output.
- **Fn::Join** concatenates a list with a delimiter. Older, and far harder to read.
- Sub can do almost everything Join does, more legibly. Prefer it.

> ON THE EXAM
>
> Sub handles Ref and GetAtt inline, which is why modern templates rarely use Join. Recognise Join, because older templates and exam questions still use it.

The other intrinsic functions

- **Fn::FindInMap** looks a value up in Mappings. Three arguments: map, top key, second key.
- **Fn::ImportValue** reads a value exported by another stack.
- **Fn::If**, **Fn::Equals**, **Fn::Not**, **Fn::And**, **Fn::Or** build and use conditions.
- **Fn::Select** and **Fn::Split** pick from and produce lists. **Fn::GetAZs** lists a Region's zones.
- **Fn::Base64** encodes user data. **Fn::Cidr** carves a CIDR block into subnets.

> ON THE EXAM
>
> Select combined with GetAZs is the standard way to place subnets across availability zones without hard-coding zone names.

Mappings and Conditions

- **Mappings** are static two-level lookup tables, typically keyed by Region or environment.
- They cannot use intrinsic functions. The values must be literal.
- **Conditions** are named boolean expressions declared in their own section.
- A resource with a **Condition** attribute is created only when that condition is true.
- So one template can create a read replica in production and omit it in dev.

> ON THE EXAM
>
> Conditions plus AWS::NoValue is the combination that makes one template serve every environment without duplicating it.

Outputs and exports

- An **Output** returns a value, visible on the stack and readable by the CLI.
- Adding **Export** publishes it under a name unique across the Region.
- Another stack reads it with **Fn::ImportValue**.
- **An exported value cannot be changed or deleted while another stack imports it.**
- That creates hard coupling, and it is the main argument against cross-stack references.

> ON THE EXAM
>
> The locking is the trade-off. Exports give clean separation and make the producing stack difficult to change, which surprises teams later.

Nested stacks or cross-stack references?

NESTED STACKS

One deployment unit

- Child stacks as resources in a parent
- Deployed and deleted together
- Good for reusable components
- Values passed as parameters
- Parent must be updated to change a child

CROSS-STACK REFERENCES

Independent lifecycles

- Separate stacks, separately deployed
- Linked by Export and ImportValue
- Good for shared infrastructure
- Different teams can own each stack
- Exports lock the producing stack

THE TIEBREAKER

Nested when components share a lifecycle and are deployed as one thing. Cross-stack when a networking team owns the VPC and application teams own their own stacks.

Dependencies

- CloudFormation infers order from **Ref** and **GetAtt**. Referencing something creates a dependency.
- Resources with no dependency between them are created **in parallel**.
- **DependsOn** declares an order that cannot be inferred.
- The classic case is an EC2 instance that needs an internet gateway attachment to exist first.
- Overusing DependsOn serialises the deployment and makes it slower for no benefit.

> ON THE EXAM
>
> Most dependency problems are already solved by a Ref. Reach for DependsOn only when the ordering requirement is real but invisible to CloudFormation.

The stack lifecycle

- **CREATE_IN_PROGRESS** to **CREATE_COMPLETE**, or to **ROLLBACK_IN_PROGRESS** on failure.
- **ROLLBACK_COMPLETE** after a failed create means the stack **can only be deleted**, not updated.
- **UPDATE_ROLLBACK_COMPLETE** after a failed update leaves the previous version running.
- **UPDATE_ROLLBACK_FAILED** needs manual intervention: continue the rollback, skipping resources.
- **DELETE_FAILED** usually means a non-empty S3 bucket or a resource with a dependency outside the stack.

> ON THE EXAM
>
> ROLLBACK_COMPLETE trips everyone once. A stack in that state cannot be updated; you must delete it and create it again.

Rollback

- By default a failed operation **rolls back automatically** to the last known good state.
- **You still pay** for resources that existed during the failed attempt.
- Disabling rollback leaves failed resources in place, which is how you debug the cause.
- **Rollback triggers** monitor CloudWatch alarms during and after a deployment.
- If an alarm fires within the monitoring period, the whole stack update rolls back.

> ON THE EXAM
>
> Rollback triggers are the CloudFormation equivalent of AppConfig's alarm rollback: a deployment that looks successful can still be reverted by a metric.

Change sets

- A change set is a **preview** of what an update would do, without doing it.
- It lists each resource as **Add**, **Modify** or **Remove**.
- Crucially it shows **Replacement: True, False or Conditional**.
- You then execute the change set, or delete it and change the template.
- Change set creation also performs **early validation**, catching property errors before provisioning.

> READ THIS TWICE
>
> Replacement: True on a database is the warning a change set exists to give you. Applying a template directly gives you no such warning.

The three update behaviours

BEHAVIOUR	WHAT HAPPENS
Update with no interruption	Changed in place. The resource keeps running and keeps its physical ID
Update with some interruption	Briefly unavailable, but the same resource. An EC2 instance type change
Replacement	A new resource is created and the old one deleted. New physical ID

ON THE EXAM

Replacement is the dangerous one. Data on the old resource is gone, and anything referencing the old physical ID outside the stack now points at nothing.

DeletionPolicy

- **Delete** is the default for almost everything. The resource goes with the stack.
- **Retain** keeps the resource, removing it from CloudFormation's scope. It keeps costing money.
- **Snapshot** takes a final snapshot first, for types that support one.
- **Exception:** RDS DB clusters and standalone DB instances default to **Snapshot**, not Delete.
- **RetainExceptOnCreate** retains on delete but not when a failed create rolls back.

> ON THE EXAM
>
> That RDS exception is worth knowing precisely. Most resources default to Delete; databases default to Snapshot, which is AWS protecting you from yourself.

UpdateReplacePolicy

- Applies when an **update** replaces a resource, rather than when a stack is deleted.
- Same options: Delete, Retain, Snapshot.
- Without it, a replacement **deletes the old resource and its data**, quietly.
- Set both attributes together on anything holding data. They cover different events.
- Neither prevents replacement. They only decide what happens to the old resource.

ON THE EXAM

Setting DeletionPolicy alone is a common half-measure: it protects against stack deletion and does nothing about an update that replaces the resource.

Stack policies and termination protection

- A **stack policy** is a JSON document controlling which resources may be updated.
- Typically: deny Update:Replace and Update:Delete on the database, allow everything else.
- It applies only to **stack updates**. It does not protect against stack deletion.
- **Termination protection** blocks deletion of the whole stack until it is disabled.
- The two are complementary, and neither replaces DeletionPolicy.

ON THE EXAM

Three separate protections, each covering a different event: stack policy for updates, termination protection for deletion, DeletionPolicy for what survives either.

Drift detection

- Compares each resource's actual configuration against the template that created it.
- Reports **IN_SYNC**, **MODIFIED**, **DELETED** or **NOT_CHECKED** per resource.
- **It detects. It does not fix.** Correcting drift is a separate action.
- Drifted resources may be overwritten by the next stack update, without warning.
- Not every resource type supports drift detection, hence NOT_CHECKED.

> ON THE EXAM
>
> 'How do I find changes made outside CloudFormation' is the exam phrasing, and drift detection is the answer every time.

Importing existing resources

- **Resource import** brings an existing resource into a stack without recreating it.
- You supply a template describing it, plus its identifier, and CloudFormation adopts it.
- The template must match the real configuration, or the import fails.
- Also used to **resolve drift**: remove with Retain, then import with a corrected template.
- The **IaC generator** writes that template for you, which is why the two features pair.

> ON THE EXAM
>
> Import is how a hand-built resource becomes managed infrastructure. Without it the only option was to recreate the resource, which usually meant downtime.

StackSets

- Deploys a single template across **many accounts and many Regions** from one operation.
- Two permission models: **self-managed** with IAM roles, or **service-managed** with Organizations.
- Service-managed adds **automatic deployment** to accounts as they join an OU.
- **Operation preferences** control concurrency and a failure tolerance.
- **Stack instances** are the per-account, per-Region deployments of the stack set.

> ON THE EXAM
>
> The signal is 'across all our accounts' or 'in every Region'. A single stack cannot span either, and doing it by hand does not scale.

Custom resources

- A resource backed by a **Lambda function** or an SNS topic that you implement.
- Used for anything CloudFormation has no resource type for, or for setup steps.
- Your function receives **Create**, **Update** and **Delete** events and must handle all three.
- **It must signal back to a pre-signed URL**, or the stack hangs until it times out.
- The classic use is emptying an S3 bucket so the stack can delete it.

> ON THE EXAM
>
> The signalling requirement causes the classic symptom: a stack stuck in CREATE_IN_PROGRESS for an hour because the function threw before responding.

Helper scripts and signals

- **cfn-init** reads AWS::CloudFormation::Init metadata and installs packages, files and services.
- **cfn-signal** reports success or failure back to CloudFormation.
- **cfn-hup** watches for metadata changes and re-runs configuration.
- A **CreationPolicy** makes CloudFormation wait for a signal before marking a resource complete.
- Without a CreationPolicy, an instance is complete when it boots, not when your software is ready.

> ON THE EXAM
>
> That last line is the point. CloudFormation cannot see inside your instance, so without cfn-signal it will happily report success on a half-configured server.

Currency note: the modern tooling

- **IaC generator**, February 2024, produces a template from existing resources. Over 600 types.
- Targeted resource scans followed in March 2025, so you scan a subset rather than the account.
- **Stack refactoring**, February 2025, moves resources between stacks without recreating them.
- **Infrastructure Composer**, renamed from Application Composer in October 2024, builds templates visually.
- **Git sync** deploys a stack directly from a repository via AWS CodeConnections.

> ON THE EXAM
>
> IaC generator is the one to remember: 'we have resources created by hand and want them under CloudFormation' used to mean writing the template yourself.

Transforms

- **AWS::Serverless-2016-10-31** is the SAM transform. Module 21 covers it properly.
- **AWS::Include** pulls in a template snippet from S3.
- **AWS::LanguageExtensions** adds functions the core language lacks.
- It provides **Fn::ForEach** for loops, plus Fn::Length and Fn::ToJsonString.
- Custom **macros** are Lambda functions that rewrite a template before deployment.

> ON THE EXAM
>
> Fn::ForEach removes the classic complaint that CloudFormation has no loops. It needs the LanguageExtensions transform declared explicitly.

Diagnosing a failed stack

SYMPTOM	USUAL CAUSE
Stuck in CREATE_IN_PROGRESS for an hour	A custom resource that never signalled back
ROLLBACK_COMPLETE and cannot update	A failed create. Delete the stack and start again
DELETE_FAILED on a bucket	The bucket is not empty. CloudFormation will not empty it
Export cannot be updated	Another stack is importing it
Resource replaced unexpectedly	A property change that forces replacement. A change set would have shown it
No export named X	The producing stack has not deployed, or the name differs

ON THE EXAM

Read stack events from the bottom. The first failure is the real cause; everything above it is consequence.

Two of these turn on the same question: **will this change replace the resource**?

A team updated a production stack to change an RDS instance's database name. CloudFormation replaced the instance, creating a new empty database and deleting the original. Several hours of transactions were lost.

The team wants to be certain this cannot happen again, and wants to see the impact of a change before applying it.

What should the team adopt?

A Enable termination protection on the stack

B Enable drift detection and run it before each deployment

C Create a change set before every update, and set DeletionPolicy and UpdateReplacePolicy on the database

D Disable automatic rollback so failed updates leave resources in place

◆ PAUSE AND DECIDE

What should the team adopt?

A Termination protection

Protects against deleting the whole stack. This was an update, and the stack was never deleted.

B Drift detection

Finds changes made outside CloudFormation. The change here came through CloudFormation, so there was no drift to find.

C Change set plus both policies

The change set shows Replacement: True in advance, and UpdateReplacePolicy: Snapshot means the old database survives even if it happens.

D Disable rollback

Affects what happens after a failure. This update succeeded exactly as CloudFormation intended, which is the problem.

THE SIGNAL

Note the pairing: the change set gives warning, the policy gives protection. UpdateReplacePolicy is the one that matters, because this was a replacement, not a deletion.

A template creates an IAM role and a Lambda function. The function's Role property is set with !Ref ExecutionRole. Stack creation fails with an error stating the role ARN is invalid.

The role itself is created successfully. Only the function fails.

What is wrong?

A Ref on an IAM role returns the role name. The Role property needs !GetAtt ExecutionRole.Arn

B The role needs a DependsOn attribute so it is created first

C The role's trust policy is missing the Lambda service principal

D IAM roles cannot be referenced from a Lambda resource in the same template

◆ PAUSE AND DECIDE

What is wrong?

A Ref returns the name, use GetAtt Arn

The Role property requires an ARN. Ref on an IAM role returns the role name, so CloudFormation passes a bare name where an ARN is expected.

B Missing DependsOn

The Ref already creates an implicit dependency, and the role was created successfully. Ordering is not the issue.

C Missing trust policy principal

That would fail at invocation or produce a different error. The error explicitly says the ARN is invalid, which is a format problem.

D Cannot reference in the same template

This is the normal pattern and works fine. Templates reference roles from functions constantly.

THE SIGNAL

The error message names the problem: it says ARN. Ref returned a name, and the difference between the two is the single most tested CloudFormation detail.

> A company must deploy the same security baseline stack, containing IAM roles, a CloudTrail trail and Config rules, into 60 AWS accounts across 4 Regions. New accounts are created most weeks and must receive the baseline automatically.
>
> All accounts belong to one AWS Organization. Deploying by hand is no longer viable.

How should this be deployed?

A. A nested stack in the management account, with one child stack per target account

B. A service-managed StackSet targeting the organizational unit, with automatic deployment enabled

C. A pipeline that assumes a role in each account and runs create-stack in a loop

D. One stack in the management account, exporting values the other accounts import

◆ PAUSE AND DECIDE

How should this be deployed?

A **Nested stacks**

Nested stacks are created in the same account and Region as the parent. They cannot span accounts at all.

B **Service-managed StackSet**

Purpose-built: one template across accounts and Regions, and automatic deployment adds new accounts to the OU without any action.

C **Pipeline looping create-stack**

Rebuilds StackSets by hand, with 240 stacks to track, no failure tolerance and its own code to maintain.

D **Exports and imports**

Exports are Regional and same-account. Fn::ImportValue cannot read an export from another account.

THE SIGNAL

'New accounts must receive it automatically' is the clause that picks service-managed over self-managed. Only the Organizations model does that.

A stack creates an S3 bucket populated by an application. Attempts to delete the stack fail with DELETE_FAILED on the bucket. A separate change also needs a one-off DynamoDB table seeded with reference data at creation time.

The team wants both handled inside the template rather than by manual steps.

What should the template use?

A DeletionPolicy: Retain on the bucket, and a Mappings section for the seed data

B A stack policy denying deletion of the bucket, and cfn-init for the seed data

C Termination protection, and a DependsOn between the table and the application

D A Lambda-backed custom resource that empties the bucket on Delete and writes the seed data on Create

◆ PAUSE AND DECIDE

What should the template use?

A **Retain plus Mappings**

Retain avoids the failure by orphaning the bucket, which still exists and still costs money. Mappings hold literal values and cannot write to a table.

B **Stack policy plus cfn-init**

Stack policies govern updates, not deletion. cfn-init runs on EC2 instances, and there are none here.

C **Termination protection plus DependsOn**

Termination protection makes deletion harder rather than possible, and DependsOn only orders resources. Neither writes any data.

D **Lambda-backed custom resource**

The standard answer for both halves: emptying a bucket on Delete, and running one-off setup on Create. Both are things CloudFormation cannot do natively.

THE SIGNAL

Two requirements, one mechanism. 'CloudFormation cannot do this natively' is the custom resource signal, and emptying a bucket is its most cited example.

What to carry forward

- **Ref returns the physical ID, GetAtt returns an attribute.** A role's ARN needs GetAtt.
- **Change sets show Replacement** before it happens. That column is why they exist.
- **DeletionPolicy and UpdateReplacePolicy cover different events.** Set both on anything holding data.
- Drift detection finds out-of-band changes. It reports them; it does not fix them.
- StackSets for many accounts or Regions. Custom resources for what CloudFormation cannot do.

> ON THE EXAM
>
> Next: AWS SAM and the CDK - two higher-level ways to produce the templates this module has been reading.

MODULE 21 · DOMAIN 3 - DEPLOYMENT

AWS SAM and the CDK

Two higher-level ways to produce CloudFormation - and the four lines that give a Lambda function a canary deployment.

DOMAIN 3 of 4 EXAM WEIGHT 24% MODULE 21 of 34

admin@sthithapragna.com

SAM and the CDK are both **CloudFormation generators**. Every stack, change set and rollback rule from module 20 still applies underneath.

What SAM actually is

- A SAM template is a CloudFormation template with **Transform: AWS::Serverless-2016-10-31**.
- That macro expands SAM's short resource types into full CloudFormation resources.
- One **AWS::Serverless::Function** can expand into a function, a role, a log group and permissions.
- Ordinary CloudFormation resources can sit in the same template, unchanged.
- Without that Transform line, none of the SAM syntax is recognised.

> ON THE EXAM
>
> SAM is a dialect, not a different product. Anything CloudFormation can do, a SAM template can also do, by writing plain CloudFormation alongside.

The SAM resource types

TYPE	WHAT IT CREATES
AWS::Serverless::Function	A Lambda function, its role, log group and event source mappings
AWS::Serverless::Api	A REST API, from a template definition or OpenAPI
AWS::Serverless::HttpApi	An HTTP API
AWS::Serverless::SimpleTable	A DynamoDB table with a partition key only
AWS::Serverless::StateMachine	A Step Functions state machine
AWS::Serverless::LayerVersion	A Lambda layer
AWS::Serverless::Application	A nested application, often from the Serverless Application Repository

ON THE EXAM

SimpleTable is genuinely limited: partition key only, no sort key and no indexes. A real table needs AWS::DynamoDB::Table instead.

The Function resource

- **CodeUri**, **Handler** and **Runtime** are the essentials. Everything else has defaults.
- **Events** declares triggers, and SAM creates the plumbing each one needs.
- An **Api** event creates the API Gateway route, method and invoke permission automatically.
- An **SQS** event creates the event source mapping and adds the queue permissions to the role.
- **Policies** attaches IAM policies, including SAM's own policy templates.

> ON THE EXAM
>
> The Events section is where SAM saves the most work. Declaring a trigger generates the resource, the mapping and the permission, which is three things in CloudFormation.

Implicit APIs

- An **Api** event with no RestApiId creates an **implicit API** called ServerlessRestApi.
- Every function with an Api event joins that same implicit API automatically.
- So you get an API you never declared, with a logical ID you did not choose.
- Configure it through the **Globals** section, or declare an explicit AWS::Serverless::Api.
- Reference it as **!Ref ServerlessRestApi** if you need its ID in an output.

> ON THE EXAM
>
> The implicit API is convenient and invisible, which is exactly why it confuses people. Declaring an explicit API is clearer once a template grows.

Globals

- A **Globals** section sets properties inherited by every resource of a type.
- Typically Runtime, Timeout, MemorySize, Tracing and Environment for all functions.
- A property set on an individual resource **overrides** the global.
- Maps such as Environment variables are **merged**, not replaced. Scalars are replaced.
- It applies to Function, Api, HttpApi, SimpleTable and a few others.

ON THE EXAM

The merge behaviour catches people. A function that defines its own environment variables still inherits the global ones, which is usually what you wanted.

Policy templates

- Named, parameterised policies that expand into scoped IAM statements.
- **DynamoDBCrudPolicy**, **S3ReadPolicy**, **SQSSendMessagePolicy** and dozens more.
- Each takes a parameter, so the policy is scoped to one named resource.
- They are convenient rather than minimal. CrudPolicy grants more than a read-only function needs.
- Inline policy documents and managed policy ARNs still work in the same Policies list.

ON THE EXAM

Policy templates are the right default because they scope to a specific resource. Compare with AdministratorAccess, which is what unhurried developers reach for instead.

The SAM workflow

01

sam init

Scaffold a project from a template.

02

sam build

Install dependencies and stage artifacts in .aws-sam.

03

sam deploy

Package, upload to S3, and create a change set to execute.

04

sam logs / sam local

Tail logs, or invoke the function on your machine.

ON THE EXAM

sam deploy --guided runs once and writes samconfig.toml, so later deploys need no arguments. That file is what makes deployment repeatable.

Testing locally

- **sam local invoke** runs one function in a Docker container that mimics the Lambda runtime.
- **sam local start-api** runs an API Gateway emulator on localhost.
- **sam local start-lambda** exposes a local Lambda endpoint the SDK can call.
- **sam local generate-event** produces realistic sample events for dozens of sources.
- It emulates the runtime, **not the platform**. IAM, throttling and cold starts are not reproduced.

> ON THE EXAM
>
> That last line matters. Local testing catches logic errors quickly and will never catch a missing IAM permission, because locally there is no IAM.

sam sync and SAM Accelerate

- A full build and deploy cycle takes a minute or two, which is painful fifty times a day.
- **sam sync** pushes code changes directly to the deployed resources, bypassing CloudFormation.
- **sam sync --watch** monitors your files and syncs automatically on save.
- Code-only changes take seconds. Template changes still trigger a CloudFormation deployment.
- **For development only.** It deliberately bypasses change sets, so the stack can drift.

> ON THE EXAM
>
> This is the answer for 'our feedback loop is too slow' - and never the answer for production, because bypassing CloudFormation is the whole mechanism.

AutoPublishAlias

- Setting **AutoPublishAlias: live** on a function changes what every deployment does.
- SAM detects the code has changed and **publishes a new version**.
- It creates the alias if needed, and **points it at that new version**.
- It repoints **every event source** at the alias rather than at $LATEST.
- It is a prerequisite for DeploymentPreference. Without it, that property errors.

ON THE EXAM

This single line is what makes versioned, alias-based deployment the default rather than something you wire up by hand. Module 24 covers the underlying mechanics.

DeploymentPreference

- Adding **DeploymentPreference** makes SAM create a **CodeDeploy** deployment group automatically.
- **Canary10Percent5Minutes** shifts 10% of traffic, waits, then shifts the rest.
- **Linear10PercentEvery1Minute** shifts in equal increments. **AllAtOnce** does not shift at all.
- **Alarms** lists CloudWatch alarms. If one fires, CodeDeploy **rolls back immediately**.
- **Hooks** run PreTraffic and PostTraffic validation functions around the shift.

> READ THIS TWICE
>
> Four lines of YAML give you gradual traffic shifting, alarm-triggered automatic rollback and validation hooks. Nothing else on this exam buys as much for as little.

What SAM generates underneath

- **sam build** then **sam deploy** produces an ordinary CloudFormation stack.
- Inspect the expanded template in the console, or with **aws cloudformation get-template**.
- A twenty-line SAM template routinely becomes several hundred lines of CloudFormation.
- Stack events reference the **generated** logical IDs, not your SAM ones.
- So a failure names something like MyFunctionRole, which you never wrote.

ON THE EXAM

Knowing SAM expands into CloudFormation is what lets you debug it. The error messages come from CloudFormation and name resources SAM invented.

The AWS CDK

- Define infrastructure in TypeScript, Python, Java, C# or Go. It **synthesises** CloudFormation.
- So you get loops, conditionals, functions, types and an IDE that autocompletes.
- Reusable abstractions are ordinary classes, shared as ordinary packages.
- **CDK v2** ships as one package, **aws-cdk-lib**, plus the **constructs** library.
- The deployed artefact is still a CloudFormation template. Nothing bypasses it.

> ON THE EXAM
>
> The advantage is abstraction and testing. The cost is that a generated template is further from what you wrote, which makes some failures harder to trace.

The three construct levels

LEVEL	WHAT IT GIVES YOU
L1, Cfn prefix	A direct one-to-one mapping of a CloudFormation resource. No defaults
L2	A curated class with sensible defaults, helper methods and IAM grants
L3, patterns	A whole architecture in one construct. Several resources wired together

ON THE EXAM

L2 is where you normally work. grantRead on a bucket writes the IAM policy for you, which is the single biggest practical saving.

App, stack and construct

- An **App** contains one or more **Stacks**, and each stack becomes a CloudFormation stack.
- Stacks contain constructs, which contain other constructs, forming a tree.
- Every construct has a **construct ID**, and its path in the tree generates the logical ID.
- **Renaming or moving a construct changes its logical ID**, which CloudFormation reads as delete and recreate.
- Passing one construct to another creates the reference, and CDK adds the dependency.

> ON THE EXAM
>
> That fourth point causes real outages. A tidy-up refactor that renames a construct can silently replace a database, because the logical ID moved.

The CDK workflow

01

cdk bootstrap

Create the CDKToolkit stack in each account and Region. Once only.

02

cdk synth

Generate the CloudFormation template into cdk.out.

03

cdk diff

Compare the synthesised template against the deployed stack.

04

cdk deploy

Upload assets and deploy the stack.

ON THE EXAM

cdk diff is the CDK's change set: it shows what would change, including resource replacements, before you deploy anything.

Bootstrapping

- Creates a **CDKToolkit** stack holding an S3 bucket, an ECR repository and IAM roles.
- The bucket stores **assets**: function code, Docker images and large templates.
- Required **once per account and Region**. Deploying without it fails immediately.
- The bootstrap version can go stale, and a newer CDK will refuse to deploy until you re-bootstrap.
- Cross-account deployment needs the target account bootstrapped to trust the deploying account.

> ON THE EXAM
>
> 'This stack uses assets, so the toolkit stack must be deployed' is the error, and it means exactly one thing: run cdk bootstrap.

Escape hatches

- An L2 construct may not surface a property CloudFormation supports, especially a new one.
- Reach the underlying L1 with **node.defaultChild**, cast it, and set the property directly.
- **addPropertyOverride** and **addOverride** edit the synthesised template directly.
- **addDeletionOverride** removes a property the construct added.
- **Aspects** visit every construct in the tree, which is how you apply a tag or a check globally.

ON THE EXAM

Escape hatches are why an L2 construct is never a dead end. If the CDK is behind CloudFormation, you can still set the property.

Testing CDK code

- **Fine-grained assertions** check the synthesised template contains what you expect.
- Assert a bucket has encryption enabled, or a function has the right timeout.
- **Snapshot tests** compare the whole template against a stored copy, catching unintended changes.
- These run in your normal unit test framework, in seconds, with nothing deployed.
- They test the template, not the deployment. A valid template can still fail to deploy.

> ON THE EXAM
>
> This is the CDK's clearest advantage over SAM and raw CloudFormation: infrastructure that can be unit tested in an ordinary CI pipeline.

CDK Pipelines and migration

- **CDK Pipelines** defines a CodePipeline in CDK code that deploys the CDK app.
- It is **self-mutating**: adding a stage updates the pipeline on its next run.
- **cdk migrate** generates a CDK app from an existing stack or template.
- It pairs with CloudFormation's IaC generator from module 20.
- Generated code is a starting point. It uses L1 constructs and needs refactoring.

> ON THE EXAM
>
> Self-mutation is the notable idea: the pipeline is infrastructure too, so it deploys its own changes rather than needing manual edits.

SAM or the CDK?

AWS SAM

Serverless, declaratively

- YAML, close to CloudFormation
- Shortest path for Lambda and API Gateway
- Excellent local testing with sam local
- AutoPublishAlias and DeploymentPreference
- Limited outside serverless

AWS CDK

Anything, programmatically

- A real language, with types and loops
- Covers every AWS service
- Unit testable with assertions
- Reusable constructs as packages
- More to learn, and more to go wrong

THE TIEBREAKER

Purely serverless with a small team means SAM. Mixed infrastructure, shared abstractions or a team that wants tests means the CDK. They can be combined.

Which tool does the scenario want?

THE SCENARIO SAYS	THE INTENDED ANSWER
Gradual Lambda rollout with automatic rollback	SAM AutoPublishAlias and DeploymentPreference
Test a Lambda function on a developer machine	sam local invoke
Deployment feedback loop is too slow	sam sync, for development only
Unit test infrastructure definitions	CDK assertions
A property the L2 construct does not expose	A CDK escape hatch to L1
Existing resources, want them as code	IaC generator, or cdk migrate
Preview changes before applying them	cdk diff, or a CloudFormation change set

ON THE EXAM

Worth screenshotting. Most SAM and CDK questions reduce to one row, and most name a capability that only one tool has.

Two of these name a capability only one tool has. Find that capability before you weigh the tools.

A payments Lambda function is deployed by SAM. Deployments currently replace the code for all traffic at once, and a recent bad release caused 20 minutes of failed payments before anyone noticed.

The team wants new versions to take a small share of traffic first, and to revert automatically if the error rate alarm fires. They want the smallest possible change to the template.

What should be added to the template?

A AutoPublishAlias and a DeploymentPreference of Canary10Percent5Minutes, with the error alarm listed

B A second Lambda function and a Route 53 weighted record between them

C Provisioned concurrency on the function, so new code warms before receiving traffic

D An API Gateway canary release deployment on the stage

◆ PAUSE AND DECIDE

What should be added to the template?

A AutoPublishAlias plus DeploymentPreference

Four lines. SAM creates the CodeDeploy deployment group, shifts 10% of traffic for five minutes, and rolls back automatically if the alarm fires.

B Two functions and weighted DNS

Builds traffic shifting by hand, with two functions to keep in step and DNS TTLs that make rollback slow rather than immediate.

C Provisioned concurrency

Addresses cold starts, not release safety. Warm bad code fails just as reliably as cold bad code.

D API Gateway canary

Shifts traffic between API deployments, not between Lambda versions. It also does not roll back on a CloudWatch alarm.

THE SIGNAL

The requirement is traffic shifting at the Lambda layer with alarm rollback. Module 11 flagged the API Gateway canary as a different layer, and this is where that distinction pays.

> A developer is iterating on a Lambda function's business logic. Each change requires sam build and sam deploy, taking about 90 seconds before they can test against real DynamoDB and SQS resources in a development account.
>
> They make dozens of changes an hour. Local emulation is not sufficient because the logic depends on real service behaviour.

What should the developer use?

A. sam local start-api, with the SDK pointed at the development account

B. Reduce the deploy time by shrinking the deployment package

C. sam sync --watch, accepting that it bypasses CloudFormation during development

D. Deploy directly with the Lambda UpdateFunctionCode API from a script

◆ PAUSE AND DECIDE

What should the developer use?

A sam local start-api

The scenario rules this out: local emulation is explicitly insufficient because the logic depends on real service behaviour.

B Smaller package

Most of the 90 seconds is CloudFormation change set creation and execution, not upload. A smaller package barely moves it.

C sam sync --watch

Purpose-built for this: pushes code changes in seconds, watches files automatically, and the CloudFormation bypass is acceptable in a development account.

D UpdateFunctionCode script

Hand-builds what sam sync does, without dependency handling or multi-function support, and with the same drift downside.

THE SIGNAL

Note what makes C acceptable: a development account. The same answer would be wrong for production, because bypassing CloudFormation is the mechanism.

A team uses the CDK in TypeScript. They need to set a newly released property on an S3 bucket that the L2 Bucket construct does not yet expose. The property is fully supported by CloudFormation.

They do not want to abandon the L2 construct, which they use for its grant methods and defaults elsewhere in the stack.

How should they set the property?

A Replace the L2 Bucket with a CfnBucket and set every property manually

B Access the underlying CfnBucket through node.defaultChild and use addPropertyOverride

C Write the bucket in a separate raw CloudFormation template and deploy it alongside

D Wait for the CDK library to add support in a future release

◆ PAUSE AND DECIDE

How should they set the property?

A Drop to CfnBucket entirely

Works, and discards the defaults and grant methods the scenario says they want. An escape hatch keeps both.

B defaultChild plus addPropertyOverride

Exactly what escape hatches are for. The L2 construct is kept, and the property is written straight into the synthesised template.

C Separate raw template

Splits one bucket out of the stack that references it, creating a cross-stack dependency for a single property.

D Wait

Not a solution. Escape hatches exist so that CDK library lag never blocks you.

THE SIGNAL

The clause 'they do not want to abandon the L2 construct' is what separates B from A. Both set the property; only one keeps the grant methods.

> A platform team must define infrastructure spanning Lambda functions, an ECS service, a VPC, RDS and several S3 buckets. They want reusable components shared across four product teams, and they want infrastructure covered by unit tests in the existing CI pipeline.
>
> The team is fluent in TypeScript and already writes tests for application code.

Which tool should they adopt?

A AWS SAM, using nested applications for the reusable components

B Raw CloudFormation YAML with nested stacks

C Elastic Beanstalk, to avoid infrastructure definitions entirely

D The AWS CDK in TypeScript, with shared constructs as an internal package

◆ PAUSE AND DECIDE

Which tool should they adopt?

A SAM with nested applications

SAM is built for serverless. A VPC, RDS and an ECS service mean writing plain CloudFormation for most of it, losing SAM's advantage.

B Raw CloudFormation

Covers everything and offers no unit testing and no real abstraction. Nested stacks share templates, not typed components.

C Elastic Beanstalk

Deploys an application onto managed EC2. It does not define a VPC, RDS or S3 buckets, which is the actual requirement.

D CDK in TypeScript

Covers every service, shared constructs are an ordinary package four teams can depend on, and assertions give unit tests in the existing pipeline.

THE SIGNAL

Three clauses select the CDK: mixed non-serverless infrastructure, reusable typed components, and unit tests. No other option offers all three.

What to carry forward

- **Both generate CloudFormation.** Every rule from module 20 still applies underneath.
- **AutoPublishAlias plus DeploymentPreference** gives canary shifting and alarm rollback in four lines.
- **sam local** tests logic. **sam sync** speeds the loop, for development only.
- CDK constructs come in three levels, and an **escape hatch** reaches L1 when L2 lags.
- Renaming a CDK construct changes its logical ID, which CloudFormation reads as replacement.

ON THE EXAM

Next: CodePipeline, CodeBuild and CodeArtifact – the pipeline that runs these tools rather than a developer running them by hand.

MODULE 22 · DOMAIN 3 - DEPLOYMENT

CI/CD – CodePipeline, CodeBuild and CodeArtifact

Stages and actions, the buildspec phases, and where artifacts actually live between them.

DOMAIN 3 of 4 EXAM WEIGHT 24% MODULE 22 of 34

admin@sthithapragna.com

A pipeline moves **artifacts** between stages. Almost every failure is either a missing artifact or a missing permission.

The pipeline model

- A pipeline has **stages**, executed in order. Each stage contains one or more **actions**.
- Every pipeline needs at least **two stages**, and the first must be a source stage.
- A stage succeeds only if every action in it succeeds. A failure stops the pipeline there.
- Actions declare **input artifacts** and **output artifacts** by name.
- An action can only consume an artifact produced by an **earlier** stage.

> ON THE EXAM
>
> The naming is what links stages together. An action asking for an artifact name that nothing produced fails validation before the pipeline even runs.

The action categories

CATEGORY	TYPICAL PROVIDERS
Source	CodeCommit, S3, ECR, GitHub and others via CodeConnections
Build	CodeBuild, Jenkins
Test	CodeBuild, third-party testing tools
Deploy	CodeDeploy, CloudFormation, ECS, S3, Elastic Beanstalk, Service Catalog
Approval	Manual approval only
Invoke	Lambda, Step Functions

ON THE EXAM

The Invoke category is the useful escape hatch: a Lambda action can do anything the pipeline has no built-in provider for.

Artifacts

- Artifacts are zip files stored in an **S3 artifact bucket** for the pipeline.
- The source action produces the first one. Each later action can add more.
- **CodeBuild only uploads what its buildspec artifacts section lists.** Nothing else survives.
- The bucket is encrypted, and the pipeline role needs read and write access to it.
- Cross-region actions need an artifact bucket in **each** Region involved.

> ON THE EXAM
>
> The third point causes the most common CI/CD failure there is: a build that passes, and a deploy stage that finds nothing to deploy.

Ordering within a stage

- Actions in a stage with the **same runOrder** execute in **parallel**.
- Different runOrder values execute in sequence, lowest first.
- So parallel test suites are one stage with several actions sharing a runOrder.
- Up to 50 actions per stage, and 500 per pipeline.
- A parallel action cannot consume an artifact produced by another action at the same runOrder.

> ON THE EXAM
>
> runOrder is the only concurrency control inside a stage. Three test suites at runOrder 1 run together; a deploy at runOrder 2 waits for all three.

Manual approval

- An approval action **pauses the pipeline** until someone approves or rejects it.
- It can publish to an **SNS topic**, so approvers are notified rather than watching the console.
- A review URL and comments can be attached, giving the approver context.
- It times out after **7 days** and the pipeline fails if nobody responds.
- Approvers need **codepipeline:PutApprovalResult** on that stage.

> ON THE EXAM
>
> Approval actions produce no artifacts and consume none. They exist purely to stop the pipeline, which is why they sit in their own stage.

Watching a pipeline

- CodePipeline emits **EventBridge events** for every stage and action state change.
- So a failed stage can trigger a Lambda function, an SNS topic or a chat notification.
- **Notification rules** are the built-in route, sending to SNS or AWS Chatbot.
- EventBridge is also how one pipeline **starts another**, for chained deployments.
- Nobody watches a console. If a failure has no notification, it is discovered by a user.

> ON THE EXAM
>
> The same EventBridge pattern from module 12 applies here: the pipeline publishes, and rules decide who cares.

Source providers

- **CodeConnections**, formerly CodeStar Connections, links GitHub, GitLab and Bitbucket.
- A connection is authorised once and reused across pipelines. No personal access tokens per repository.
- **CodeCommit** and **S3** sources are native and need no connection.
- **Polling for source changes is legacy.** EventBridge or webhook triggers are faster and cheaper.
- An **ECR** source starts a pipeline when a new container image is pushed.

> ON THE EXAM
>
> Polling checks periodically and adds minutes of delay. Event-driven triggers start the pipeline within seconds, which is why AWS disables polling on new pipelines.

Currency note: pipeline type V2

- **The examinable model** is the V1 pipeline: stages, actions and artifacts.
- **V2** is now the default and adds four things V1 lacks.
- **Triggers with filters** on branches, file paths, pull requests and git tags.
- **Pipeline-level variables**, passed in at execution time.
- **Stage conditions** and **stage rollback**, plus **parallel** and **queued** execution modes.

> ON THE EXAM
>
> V2 is priced differently: V1 is a flat monthly fee per pipeline, V2 is per action-minute. That matters for pipelines that run rarely.

Execution modes and stage conditions

- **SUPERSEDED** is the default. A newer execution overtakes and cancels an older one.
- **QUEUED** runs executions one at a time, in order, with none discarded.
- **PARALLEL** runs executions independently, with no ordering at all.
- **Stage conditions** evaluate rules on entry, on success or on failure.
- A failing condition can **Skip** the stage, **Fail** it, or trigger an automatic **Rollback**.

ON THE EXAM

SUPERSEDED explains a real surprise: push twice quickly and the first execution is cancelled mid-flight. QUEUED is the fix when every commit must deploy.

Variables and namespaces

- An action can declare a **namespace**, exposing its output variables to later actions.
- Referenced as **#{Namespace.VariableName}** in a later action's configuration.
- A source action publishes **CommitId**, **BranchName** and similar.
- CodeBuild exports variables listed in its buildspec **exported-variables** section.
- Variables flow **forward only**, exactly like artifacts.

> ON THE EXAM
>
> This is how a commit ID reaches a deploy action without being written into an artifact. Small values travel as variables; files travel as artifacts.

AWS CodeBuild

- Runs builds in a managed container. No build servers to maintain, billed per build minute.
- A **build project** defines the source, environment, service role and buildspec.
- Environments are AWS-provided images, or your own image from ECR or Docker Hub.
- It can run inside a **VPC** to reach private resources, at the cost of slower start-up.
- The **service role** is CodeBuild's own identity. Missing permissions here are a frequent failure.

> ON THE EXAM
>
> CodeBuild is stateless: each build starts from a fresh container. Anything you want to persist must be an artifact, a cache, or written somewhere yourself.

buildspec anatomy

SECTION	WHAT IT DOES
`version`	0.2 for anything current
`env`	Variables, plus parameter-store and secrets-manager blocks
`phases`	install, pre_build, build, post_build
`artifacts`	Which files are uploaded as output. Nothing else survives
`cache`	Paths preserved between builds
`reports`	Test and coverage report groups

ON THE EXAM

By default it must be named buildspec.yml at the repository root, though the project can point elsewhere. YAML indentation errors are the usual first failure.

The four phases

- **install** for runtime versions and build tooling. **pre_build** for setup such as logging in to ECR.
- **build** for compiling and testing. **post_build** for packaging and pushing images.
- If any command fails, the phase fails and the build fails.
- **post_build runs even if build failed**, which is deliberate.
- So post_build is where you publish test reports that explain the failure.

> ON THE EXAM
>
> That post_build behaviour is examinable and useful. It runs on failure so that diagnostic output still gets published, but the build is still marked failed.

Environment variables in a build

- **env/variables** holds plain values. Visible to anyone who can read the buildspec.
- **env/parameter-store** resolves values from Parameter Store at build time.
- **env/secrets-manager** resolves values from Secrets Manager at build time.
- The project itself can set variables, and a pipeline action can override them.
- CodeBuild masks values it resolved itself in logs. It cannot mask what your script echoes.

> ON THE EXAM
>
> Module 19 made the same point from the secrets side. A resolved secret echoed by your own script reaches the build log in plain text.

Artifacts and caching

- The **artifacts** section lists files to upload, with **files**, **base-directory** and **discard-paths**.
- **secondary-artifacts** produces several named outputs from one build.
- **S3 caching** stores paths in a bucket between builds. Good for dependencies.
- **Local caching** keeps a layer on the build host. Faster, and only helps consecutive builds.
- Caching node_modules or a Maven repository is usually the biggest single speed win.

> ON THE EXAM
>
> A cache is an optimisation, never a dependency. A build that only works with a warm cache will fail the first time it runs on a fresh host.

Test reports

- A **reports** section points at test output files and a report group.
- Formats: JUnit XML, Cucumber JSON, TestNG, Visual Studio TRX and others.
- The console then shows pass and fail counts, durations and trends over time.
- **Code coverage reports** are supported in JaCoCo, Cobertura, SimpleCov and Clover.
- Reports are produced even when the build fails, provided the phase runs.

> ON THE EXAM
>
> Without reports, a failed test is a wall of console output. With them, the console shows which test failed and how the failure rate is trending.

Debugging a build

- Build logs go to **CloudWatch Logs**, and optionally to S3.
- **Session Manager** can attach to a running build, pausing it with codebuild-breakpoint.
- That gives a shell inside the actual build container, which is far faster than guessing.
- **Local builds** run the same image on your machine using the CodeBuild agent.
- **Most build failures are permissions or a missing file**, not the build tool itself.

> ON THE EXAM
>
> Session Manager access to a paused build is the underused feature here. It turns 'why does this only fail in CI' into a question you can answer directly.

Build performance and cost

- Billing is per build minute, by **compute type**. A larger instance costs more per minute.
- A bigger instance is often **cheaper overall**, because the build finishes proportionally faster.
- **Dependency caching** usually saves more than any compute change.
- **Batch builds** run several configurations in parallel, such as a build matrix.
- A custom image with tooling pre-installed removes install-phase time from every build.

> ON THE EXAM
>
> The counterintuitive one is compute size. Doubling the instance rarely doubles the bill, because minutes fall at the same time as the per-minute rate rises.

AWS CodeArtifact

- Managed artifact repository for **npm, PyPI, Maven, NuGet, Ruby, Swift** and generic packages.
- A **domain** contains **repositories**. Storage is deduplicated across the domain.
- Authentication uses an **IAM-derived token** from the login command. No static registry credentials.
- Repositories have resource policies, so they can be shared across accounts.
- Assets are encrypted with KMS, and every fetch is recorded in CloudTrail.

> ON THE EXAM
>
> The IAM integration is the real difference from running your own registry: package access is governed by the same policies as everything else in the account.

Upstreams and external connections

- An **external connection** links a repository to a public registry such as npmjs or PyPI.
- A package requested but not present is fetched from upstream and then **cached permanently**.
- So a later removal from the public registry does not break your builds.
- **Upstream repositories** chain CodeArtifact repositories together.
- The usual shape: a team repository upstream of a shared repository upstream of the public registry.

> ON THE EXAM
>
> The permanent caching is the resilience argument. A package deleted from a public registry stays available to you, because you already have a copy.

Package origin controls

- **The attack:** publish a public package with the same name as an internal one, at a higher version.
- A naive resolver prefers the higher version and installs the attacker's code.
- **Package origin controls** decide whether a package may be published, fetched from upstream, or both.
- Setting upstream to BLOCK for an internal package name means the public version is never pulled.
- CodeArtifact applies this automatically for packages first published internally.

> ON THE EXAM
>
> This is a real supply-chain attack with a specific named defence. If a scenario describes internal package names appearing publicly, this is the answer.

Currency note: CodeCommit reopened

- **The examinable position:** CodeCommit is a source provider like any other.
- AWS **closed it to new customers in June 2024**, and much writing still treats it as dead.
- It was **reopened to general availability in November 2025**, with an active roadmap.
- So it is a live, actively developed service again, not a legacy one.
- Answer normally on the exam. CodeCommit remains a valid source action.

> ON THE EXAM
>
> Worth knowing because a lot of 2024 and 2025 material confidently states CodeCommit is being retired. That was true, and it is not true now.

Cross-account and cross-region

- A pipeline in a tooling account can deploy into **separate dev, staging and production accounts**.
- Each action then needs a **role in the target account** that the pipeline assumes.
- The artifact bucket must use a **customer managed KMS key**, shared with the target accounts.
- **The default AWS managed key cannot be shared**, which is the usual first failure.
- Cross-region actions need an artifact bucket in each Region, which CodePipeline can create.

> ON THE EXAM
>
> Module 18's rule again: cross-account access to an encrypted artifact needs the key policy to allow it, and the default key never will.

Diagnosing a pipeline failure

SYMPTOM	USUAL CAUSE
Deploy stage finds no files	The buildspec artifacts section did not list them
Build fails immediately, no phase output	buildspec.yml missing, misnamed, or invalid YAML
Build cannot reach a private database	The project is not configured for the VPC
Cross-account deploy fails on decrypt	Artifact bucket uses the default KMS key
Pipeline never starts on a push	Polling disabled and no trigger configured
An execution disappears mid-run	SUPERSEDED mode. A newer execution overtook it
Access denied writing artifacts	The CodeBuild service role lacks S3 permissions

ON THE EXAM

Notice how many are configuration rather than code. The pipeline is usually working exactly as configured, which is the problem.

Ask first: is this about **what moved**, or about **who was allowed**?

> A pipeline has a source stage, a CodeBuild stage and a CodeDeploy stage. The build stage reports success, tests pass and the console shows green. The deploy stage then fails, reporting that the input artifact contains no deployable files.
>
> The build writes its compiled output to a dist directory and the appspec file to the repository root.

What is the most likely cause?

A. The CodeDeploy service role lacks permission to read the artifact bucket

B. The build and deploy stages are in different Regions

C. The build stage needs a longer timeout so files finish writing

D. The buildspec artifacts section does not list the dist directory and the appspec file

◆ PAUSE AND DECIDE

What is the most likely cause?

A CodeDeploy role permissions

That would produce an access denied error, not a report that the artifact contains nothing. The artifact was read successfully; it was empty.

B Different Regions

A cross-Region artifact problem fails on retrieval with a bucket error, and the pipeline would have failed validation earlier.

C Build timeout

A timeout fails the build. This build succeeded, and CodeBuild does not upload partially written files.

D Artifacts section incomplete

CodeBuild uploads only what the artifacts section lists. A green build that produces an empty artifact is this, essentially every time.

THE SIGNAL

'Build green, deploy finds nothing' is the single most recognisable CI/CD symptom. The build did its job; it just was not asked to keep anything.

A pipeline deploys to staging automatically. Compliance requires that a named release manager explicitly authorises the production deployment, and that they are notified rather than having to watch the console.

If nobody responds, the deployment must not proceed. The team does not want to build a custom approval application.

How should this be implemented?

A A Lambda invoke action that polls a DynamoDB approvals table until a record appears

B A manual approval action in its own stage, publishing to an SNS topic the release manager subscribes to

C Disable the production stage and enable it manually when a release is authorised

D A stage condition that checks a pipeline variable set by the release manager

◆ PAUSE AND DECIDE

How should this be implemented?

A **Lambda polling DynamoDB**

Rebuilds manual approval by hand, including the notification, the timeout and the audit trail. It is the custom application the team ruled out.

B **Manual approval with SNS**

Purpose-built: the pipeline pauses, SNS notifies the approver, PutApprovalResult permission names who may approve, and it fails after seven days.

C **Disable the stage**

Not an approval, and it leaves no record of who authorised the release, which is what compliance actually asked for.

D **Stage condition on a variable**

Conditions evaluate automatically against rules. There is no human in the loop, and no notification.

THE SIGNAL

The compliance angle is what makes B right rather than merely convenient: the approval is recorded, attributed to a principal, and governed by an IAM permission.

> A CodeBuild project must authenticate to a third-party API during the build. The API key currently sits in the buildspec's env/variables block, and a security review has flagged that anyone with repository access can read it.
>
> The key rotates every 30 days. The build log must not contain it.

How should the key be supplied?

A Store it in Secrets Manager and reference it from the buildspec's env/secrets-manager block

B Set it as a plaintext environment variable on the build project instead of in the buildspec

C Encrypt the key with KMS and paste the ciphertext into env/variables

D Pass it as a pipeline-level variable at execution time

◆ PAUSE AND DECIDE

How should the key be supplied?

A secrets-manager block

The buildspec holds only a reference, the value resolves at build time, rotation is handled by Secrets Manager, and CodeBuild masks values it resolved itself.

B Plaintext project variable

Moves the secret out of the repository and into the project configuration, readable by anyone who can describe the project. Still plaintext.

C Ciphertext in the buildspec

Requires decrypting in the build, so your script handles the plaintext and can echo it. It also breaks on every rotation.

D Pipeline-level variable

Someone must supply the value at execution time, so it ends up in a console field and in the execution history.

THE SIGNAL

Three requirements: not in the repository, rotates, and not in the log. Only the secrets-manager block satisfies all three without extra code.

> A company publishes internal npm packages under names like @acme/auth-client. A security audit warns that an attacker could publish a package with the same name to the public npm registry at a higher version, and builds might resolve to it.
>
> Builds must still be able to install genuine public dependencies from npm.

What should the company implement?

A Pin every dependency to an exact version in package-lock.json

B Remove the external connection to npm and vendor all public packages manually

C CodeArtifact with an external connection to npm, and package origin controls blocking upstream versions of internal package names

D Run npm audit as a build phase and fail the build on any finding

◆ PAUSE AND DECIDE

What should the company implement?

A Pin exact versions

Helps, and breaks the moment anyone adds or upgrades a dependency. It is a discipline rather than a control, and one mistake reopens the hole.

B Vendor everything manually

Eliminates the risk by eliminating the convenience. It also contradicts the requirement to install genuine public dependencies.

C CodeArtifact origin controls

The named defence for dependency confusion. Internal package names can never be fetched from upstream, while genuine public packages still resolve normally.

D npm audit in the build

Reports known vulnerabilities in real packages. A malicious package impersonating an internal name is not a known vulnerability.

THE SIGNAL

This attack has a specific name, dependency confusion, and a specific AWS answer. Recognising the shape is what makes the question quick.

What to carry forward

- **Only what the buildspec artifacts section lists survives the build.** The commonest failure there is.
- **post_build runs even when build fails**, so diagnostics still get published.
- Secrets reach a build through **env/secrets-manager**, never through env/variables.
- Same runOrder means parallel. Manual approval pauses the pipeline and times out at 7 days.
- Cross-account pipelines need a **customer managed KMS key** on the artifact bucket.

> ON THE EXAM
>
> Next: CodeDeploy and deployment strategies - what the deploy stage actually does once it has the artifact.

MODULE 23 · DOMAIN 3 - DEPLOYMENT

CodeDeploy and Deployment Strategies

Three compute platforms, three different appspec files, and lifecycle hooks that are not the same on any of them.

DOMAIN 3 of 4 EXAM WEIGHT 24% MODULE 23 of 34

admin@sthithapragna.com

CodeDeploy's job is **controlling exposure**. It decides how fast new code meets real traffic, and what happens when it goes wrong.

The three compute platforms

PLATFORM	HOW THE DEPLOYMENT WORKS
EC2/On-Premises	The agent runs scripts on each instance. In-place or blue/green
AWS Lambda	Traffic shifts between two versions behind an alias. Always blue/green in effect
Amazon ECS	A replacement task set is created and traffic shifts to it. Blue/green only

ON THE EXAM

Only EC2 supports in-place deployment. Lambda and ECS always create something new and shift traffic to it, which is why rollback on those is immediate.

Applications and deployment groups

- An **application** names the compute platform and contains deployment groups.
- A **deployment group** is a set of targets plus the settings used to deploy to them.
- For EC2, targets are chosen by **tags** or by Auto Scaling group.
- Each group has its own deployment configuration, alarms and rollback settings.
- So dev, staging and production are typically three groups in one application.

ON THE EXAM

Tag-based targeting means an instance joins a deployment group by carrying the right tag. A mistagged instance silently receives deployments it should not.

The appspec file

- **appspec.yml** for EC2, at the root of the application bundle.
- **appspec.yml or appspec.json** for Lambda and ECS, supplied to the deployment.
- **The structure is not the same across platforms.** An EC2 appspec is invalid for Lambda.
- EC2 has **files** and **permissions** sections. Lambda and ECS have a **Resources** section instead.
- All three have a **hooks** section, with different hooks available.

> ON THE EXAM
>
> This is the detail people miss. It is one filename and three different formats, and the exam shows you a fragment and asks which platform it is for.

The EC2 appspec sections

- **version** is 0.0, and **os** is linux or windows.
- **files** maps a source path in the bundle to a destination on the instance.
- **permissions** sets owner, group and mode on the copied files.
- **hooks** maps lifecycle events to scripts, each with an optional timeout and runas.
- A hook script that exceeds its timeout fails the deployment. The default is 3600 seconds.

> ON THE EXAM
>
> The files section is why an EC2 bundle needs a defined structure. CodeDeploy copies exactly what you list, to exactly where you say.

In-place deployment on EC2

- The application is stopped on each instance, updated, and started again.
- Instances are deregistered from the load balancer while being updated.
- **Capacity drops during the deployment**, by however much the configuration allows.
- **Rollback means deploying the previous version again.** It is not instant.
- Cheap, because no additional instances are launched.

> ON THE EXAM
>
> The same trade-off as Beanstalk's rolling policy in module 14: no extra cost, reduced capacity, and rollback is another full deployment.

EC2 deployment configurations

CONFIGURATION	BEHAVIOUR
`CodeDeployDefault.AllAtOnce`	Every instance at once. Fastest, and there is downtime
`CodeDeployDefault.HalfAtATime`	Half the fleet, then the other half. 50% capacity maintained
`CodeDeployDefault.OneAtATime`	One instance at a time. Slowest and safest
`Custom`	You define minimum healthy hosts, as a count or a percentage

ON THE EXAM

Minimum healthy hosts is the underlying setting. The named configurations are just presets over it, and a custom one lets you pick any point on the scale.

Blue/green on EC2

- New instances are launched, the application is deployed to them, then traffic is switched.
- The load balancer is repointed at the replacement instances all at once.
- **Rollback is repointing back**, so it takes seconds rather than a full deployment.
- Original instances are terminated after a configurable **wait time**, or kept.
- It briefly doubles your instance count, and therefore your cost.

> ON THE EXAM
>
> The termination wait time is the useful control: keep the old fleet for an hour, and rollback stays instant for that whole window.

EC2 lifecycle hooks, in order

HOOK	WHEN IT RUNS
ApplicationStop	Before anything. Runs the PREVIOUS deployment's script
DownloadBundle	CodeDeploy copies the bundle. You cannot script this
BeforeInstall	Before files are copied. Backups, decryption
Install	CodeDeploy copies the files. You cannot script this
AfterInstall	After files are copied. Configuration, permissions
ApplicationStart	Start the application
ValidateService	Verify the deployment worked. The last chance to fail it

ON THE EXAM

ApplicationStop is the one that surprises people: it runs the script from the previously deployed revision, because the new one is not on the instance yet.

The blue/green traffic hooks

- **BeforeBlockTraffic**, **BlockTraffic**, **AfterBlockTraffic** on the original instances.
- **BeforeAllowTraffic**, **AllowTraffic**, **AfterAllowTraffic** on the replacement instances.
- **BlockTraffic and AllowTraffic cannot be scripted.** CodeDeploy performs them.
- So four of the six are yours: the Before and After pairs.
- Use BeforeAllowTraffic to warm a cache, and AfterAllowTraffic to verify live traffic.

ON THE EXAM

The pattern across all platforms: the verbs themselves are CodeDeploy's, and the Before and After variants are yours. Install, DownloadBundle, BlockTraffic and AllowTraffic are never scriptable.

The CodeDeploy agent

- **EC2 and on-premises deployments require the agent.** Lambda and ECS do not.
- It polls CodeDeploy for work, downloads the bundle and runs the appspec hooks.
- Installable through user data, a custom AMI, or Systems Manager State Manager.
- If the agent is not running, the deployment simply **times out** rather than failing fast.
- The instance profile needs permission to read the artifact bucket.

> ON THE EXAM
>
> 'The deployment hung and eventually timed out' on EC2 is nearly always the agent: not installed, not running, or lacking permission to fetch the bundle.

Auto Scaling and deployments

- When a deployment group targets an Auto Scaling group, **new instances deploy automatically**.
- A scale-out event launches an instance, and CodeDeploy deploys the latest revision to it.
- That means an instance launched mid-deployment can receive the **wrong** revision.
- Scaling during a deployment is best suspended, which CodeDeploy can do for you.
- A failed deployment to a new instance causes it to be terminated and relaunched.

> ON THE EXAM
>
> The mid-deployment race is the subtle part: two sources of truth about what should be running, briefly disagreeing with each other.

Lambda deployments

- CodeDeploy shifts traffic between two **versions** of a function, using an **alias**.
- The alias is the stable ARN clients call. Weights on it determine the split.
- There are no instances, no agent and no files to copy.
- The appspec names the function, the alias, and the target and current versions.
- This is exactly what SAM's **DeploymentPreference** configures for you.

> ON THE EXAM
>
> Module 21 wrote four lines of YAML; this is what those four lines create. Module 24 covers the version and alias mechanics underneath.

Lambda deployment configurations

CONFIGURATION	BEHAVIOUR
`Canary10Percent5Minutes`	10% for 5 minutes, then the remaining 90%
`Canary10Percent30Minutes`	10% for 30 minutes, then the rest. Also 10 and 15 minute variants
`Linear10PercentEvery1Minute`	10% more every minute, over 10 minutes
`Linear10PercentEvery10Minutes`	10% more every 10 minutes, over 100 minutes
`AllAtOnce`	All traffic immediately. No gradual shift

ON THE EXAM

Canary shifts in two steps: a small slice, a pause, then everything. Linear shifts in equal increments throughout. The names state the numbers exactly.

Lambda hooks

- **BeforeAllowTraffic** runs before any traffic reaches the new version.
- **AfterAllowTraffic** runs after the shift has completed.
- That is the complete list. There is no install phase and no ValidateService.
- Each hook is a **Lambda function** you write, not a shell script.
- It must call **PutLifecycleEventHookExecutionStatus** with Succeeded or Failed.

> ON THE EXAM
>
> That last line matters: a hook that never reports its status leaves the deployment waiting. Returning normally from the function is not enough.

What a hook function receives

- The hook function receives a **DeploymentId** and a **LifecycleEventHookExecutionId**.
- It does **not** receive the new function version. You pass that in through environment variables.
- It must call **PutLifecycleEventHookExecutionStatus** with that execution ID.
- Its execution role needs **codedeploy:PutLifecycleEventHookExecutionStatus**.
- Returning Failed rolls the deployment back. Timing out does too, eventually.

> ON THE EXAM
>
> Missing that IAM permission is the classic first failure: the test passes, the status call is denied, and the deployment stalls until it times out.

ECS deployments

- CodeDeploy creates a **replacement task set** running the new task definition.
- The load balancer needs **two target groups**: one live, one for the replacement.
- An optional **test listener** on a separate port reaches the new tasks before real traffic does.
- That is what makes running integration tests against the new version possible.
- Traffic shifts by canary, linear or all at once, exactly as for Lambda.

> ON THE EXAM
>
> The test listener is the distinctive ECS capability. You can exercise the replacement task set with real requests on a separate port while production traffic is untouched.

Lifecycle hooks by platform

PLATFORM	HOOKS AVAILABLE TO YOU
EC2, in-place	ApplicationStop, BeforeInstall, AfterInstall, ApplicationStart, ValidateService
EC2, blue/green	The above, plus the Before and After variants of BlockTraffic and AllowTraffic
AWS Lambda	BeforeAllowTraffic, AfterAllowTraffic
Amazon ECS	BeforeInstall, AfterInstall, AfterAllowTestTraffic, BeforeAllowTraffic, AfterAllowTraffic

READ THIS TWICE

Worth screenshotting and memorising. AfterAllowTestTraffic exists only on ECS, and ValidateService exists only on EC2. Those two facts alone answer several questions.

Automatic rollback

- Roll back **on deployment failure**, or when a **CloudWatch alarm** enters ALARM state.
- A rollback is a **new deployment** of the previous revision, with its own deployment ID.
- For blue/green and Lambda that is near-instant, because the old version still exists.
- For in-place it is a full redeployment, and takes as long as one.
- A failing lifecycle hook triggers rollback too, which is why validation hooks matter.

> ON THE EXAM
>
> Alarm-based rollback is what makes a canary meaningful. Without an alarm, shifting 10% of traffic just means 10% of users see the failure for five minutes.

Deployment history and redeploying

- Every deployment keeps its **revision**, so a previous one can be redeployed directly.
- That is the manual equivalent of a rollback, and it is how you recover after the fact.
- **CreateDeployment** can target a specific revision rather than the latest.
- Deployment history shows every attempt, its configuration and which instances failed.
- A rolled-back deployment appears as **two** entries: the failure and the rollback.

ON THE EXAM

Worth knowing for troubleshooting questions: the history is the audit trail, and the rollback is a separate deployment with its own ID and its own outcome.

Alarms, triggers and stopping

- **Alarms** on a deployment group stop and roll back a deployment automatically.
- **Triggers** publish deployment and instance events to an **SNS topic**.
- CodeDeploy also emits **EventBridge events**, for anything more elaborate.
- **StopDeployment** halts a running deployment, optionally rolling back.
- A deployment can be configured to fail if the alarm state cannot be determined.

ON THE EXAM

Choose alarms that reflect user experience: error rate and latency. An alarm on CPU will not notice that every request is returning a 500.

Currency note: ECS built-in blue/green

- **The examinable answer:** blue/green on ECS uses **CodeDeploy**.
- Amazon ECS now offers **native blue/green deployments** without CodeDeploy.
- It has its own lifecycle hooks, invoked as Lambda functions, covering more stages.
- Hooks return SUCCEEDED, IN_PROGRESS or FAILED, and a FAILED result rolls back.
- **Answer CodeDeploy on the exam.** Exam guide v2.1 predates this entirely.

> ON THE EXAM
>
> The CodeDeploy path still works and is still supported. This is an additional option, not a replacement, but new ECS work increasingly uses the native route.

Every deployment mechanism in this course

WHERE	MECHANISM
Lambda function versions	CodeDeploy canary or linear, via an alias
ECS service	CodeDeploy blue/green with two target groups
EC2 fleet	CodeDeploy in-place or blue/green
Elastic Beanstalk	Deployment policies: rolling, immutable, traffic splitting
API Gateway stage	Canary release deployment on the stage
CloudFormation stack	Change sets, and rollback triggers on alarms
AppConfig	Deployment strategy with bake time and alarm rollback

ON THE EXAM

Seven mechanisms across seven services, and they are not interchangeable. Each operates on a different layer, which is exactly what questions test.

Choosing a strategy

- **Rollback must be immediate** means blue/green or traffic shifting. In-place cannot do it.
- **Validate before real users** means a test listener, or a BeforeAllowTraffic hook.
- **Cannot afford double capacity** rules out blue/green and immutable.
- **A percentage of traffic first** means canary or linear.
- **No downtime and no extra cost** is the one combination nothing gives you.

> ON THE EXAM
>
> That last line is worth stating plainly. Zero downtime is bought either with extra capacity or with reduced capacity. There is no third option.

Diagnosing a failed deployment

- The console shows **which lifecycle event failed** on which instance. Start there.
- On EC2, script output goes to the agent log on the instance.
- **A hang that times out** is nearly always the agent not running, or a hook not reporting status.
- **Failure at ApplicationStop** on a first deployment means the previous revision's script is broken or missing.
- A failing script must exit non-zero. A script that fails silently lets the deployment continue.

> ON THE EXAM
>
> The ApplicationStop case is a genuine trap: the failure is in code from the previous deployment, not the one you are trying to deploy.

Ask first: **which platform**? The hooks, the appspec and the rollback speed all change with it.

A Lambda function is deployed with a Canary10Percent10Minutes configuration. The team wants an automated smoke test to run against the new version and abort the deployment before any production traffic reaches it.

The smoke test calls the new version directly and checks several responses.

How should the smoke test be wired in?

A. A ValidateService hook in the appspec, running the smoke test script

B. A BeforeAllowTraffic hook, as a Lambda function reporting Succeeded or Failed

C. An AfterAllowTestTraffic hook, invoked once the test listener serves the new version

D. A CloudWatch alarm on the function's error metric, with automatic rollback enabled

◆ PAUSE AND DECIDE

How should the smoke test be wired in?

A ValidateService hook

ValidateService exists only on the EC2 platform. It is not available for a Lambda deployment at all.

B BeforeAllowTraffic hook

One of Lambda's two hooks, and it runs before any traffic shifts. Reporting Failed aborts the deployment before a single user is affected.

C AfterAllowTestTraffic hook

An ECS-only hook, tied to the test listener. Lambda deployments have no test listener and no such hook.

D Alarm with rollback

A useful safety net, and it reacts to errors after traffic has already reached the new version. The requirement is to test beforehand.

THE SIGNAL

Two distractors are hooks from other platforms. Identifying the platform first eliminates both immediately, which is why that habit is worth building.

A new EC2 deployment group has been created. The first deployment to it fails almost immediately, with the console reporting that the ApplicationStop lifecycle event failed on every instance.

The instances were previously used for a different application deployed by CodeDeploy some months ago. The new revision's scripts have been tested and work correctly.

What is the cause?

A The CodeDeploy agent is not installed on the instances

B The new revision's appspec.yml has invalid YAML

C The instance profile lacks permission to read the artifact bucket

D ApplicationStop runs the previous revision's script, which is broken or missing

◆ PAUSE AND DECIDE

What is the cause?

A Agent not installed

Without the agent the deployment would hang and time out, not report a specific lifecycle event failure. Reporting ApplicationStop proves the agent is running.

B Invalid appspec YAML

That fails before any lifecycle event runs. It also contradicts the statement that the new scripts work.

C Bucket permissions

That fails at DownloadBundle, which comes after ApplicationStop. The failure would name a different event.

D Previous revision's script

ApplicationStop runs from the last successfully deployed revision, because the new bundle is not on the instance yet. Old, stale scripts are the cause.

THE SIGNAL

The clue is 'previously used for a different application'. Those instances still hold that application's CodeDeploy state, and ApplicationStop runs from it.

> An ECS service must be deployed such that a full integration test suite runs against the new tasks, using real HTTP requests, before any production traffic reaches them. The tests take about four minutes.
>
> Production traffic must be entirely unaffected while the tests run, and a failing test must abort the deployment.

How should this be configured?

A. Run the tests in a CodeBuild stage before the deploy stage

B. Use a Linear10PercentEvery1Minute configuration so failures affect few users

C. Configure a test listener on a second port, with an AfterAllowTestTraffic hook running the tests

D. Add a ValidateService hook that runs the integration suite

◆ PAUSE AND DECIDE

How should this be configured?

A. CodeBuild stage before deploy

Tests a build artifact, not the deployed service. It cannot make real HTTP requests to tasks that do not exist yet.

B. Linear traffic shifting

Limits the blast radius rather than preventing it. Production traffic reaches the new tasks immediately, which the scenario forbids.

C. Test listener plus AfterAllowTestTraffic

The test listener serves the replacement task set on a separate port, so real requests reach it while production traffic does not. A failed hook aborts the deployment.

D. ValidateService hook

An EC2-only hook. It does not exist for ECS deployments.

THE SIGNAL

'Real HTTP requests' plus 'production traffic entirely unaffected' describes the test listener precisely. It is the one ECS capability nothing else replicates.

> A trading application runs on a fleet of 20 EC2 instances. Deployments must cause no reduction in capacity, and a bad release must be reversible within seconds. The business will fund additional instances during a deployment.
>
> The application cannot tolerate running two versions simultaneously against the same database.

Which deployment approach fits?

A. CodeDeploy blue/green, with the load balancer switched all at once and a termination wait time

B. CodeDeploy in-place with OneAtATime

C. CodeDeploy in-place with a custom configuration requiring 100% minimum healthy hosts

D. CodeDeploy blue/green with Canary10Percent5Minutes traffic shifting

◆ PAUSE AND DECIDE

Which deployment approach fits?

A Blue/green, switch all at once

Full capacity throughout, no period where both versions serve traffic, and rollback is repointing the load balancer at instances still running.

B In-place OneAtATime

Reduces capacity by one instance at a time, and rollback is a full redeployment taking twenty instance updates. Both requirements fail.

C In-place at 100% healthy

Requiring every instance to stay healthy means no instance can be taken out to update. The deployment cannot proceed at all.

D Blue/green with canary

The closest wrong answer. Canary shifting means both versions serve traffic during the shift, which the last sentence explicitly forbids.

THE SIGNAL

The final sentence is doing the work. It eliminates every gradual strategy, including the canary that would otherwise be the safest choice.

What to carry forward

- **Hooks differ by platform.** ValidateService is EC2 only. AfterAllowTestTraffic is ECS only.
- **ApplicationStop runs the previous revision's script**, which is why it fails on stale instances.
- **Install, DownloadBundle, BlockTraffic and AllowTraffic cannot be scripted.**
- Lambda and ECS roll back instantly. In-place EC2 rollback is a full redeployment.
- Zero downtime costs either extra capacity or reduced capacity. There is no third option.

ON THE EXAM

Next: Lambda versions, aliases and safe releases - the mechanics underneath the traffic shifting this module has been configuring.

MODULE 24 · DOMAIN 3 - DEPLOYMENT

Lambda Versions, Aliases and Safe Releases

$LATEST, published versions and the alias that stands between your clients and your deployments.

DOMAIN 3 of 4 EXAM WEIGHT 24% MODULE 24 of 34

admin@sthithapragna.com

A version is **immutable**. An alias is a **movable pointer**. Clients should only ever hold the pointer.

$LATEST and published versions

- **$LATEST** is the mutable working copy. Every code or configuration update changes it.
- **PublishVersion** snapshots $LATEST into a numbered, immutable version.
- Version numbers increment from 1 and are never reused, even after deletion.
- **A published version can never be modified.** To change it, publish a new one.
- Invoking the unqualified function name invokes **$LATEST**, which is almost never what production wants.

> ON THE EXAM
>
> That last line is the root of most problems in this module. An unqualified ARN means every deployment reaches production the instant the code updates.

What a version actually captures

- The **code**, the **runtime**, the **handler** and the **memory and timeout settings**.
- The **environment variables** as they were at publish time.
- The **layers** attached, at their specific layer versions.
- So changing an environment variable requires **publishing a new version** to take effect.
- Not captured: the execution role's **policy**, which is looked up at invocation time.

> ON THE EXAM
>
> The environment variable point catches people constantly. Updating a variable changes $LATEST only; the version your alias points at still holds the old value.

Qualified and unqualified ARNs

- **Unqualified:** ...function:my-function. This resolves to **$LATEST**.
- **Qualified by version:** ...function:my-function**:3**. Pinned to version 3 forever.
- **Qualified by alias:** ...function:my-function**:live**. Follows wherever live points.
- The alias form is what almost everything should use.
- A version-qualified ARN never moves, so a client holding one never receives updates.

ON THE EXAM

Three ARN shapes, three completely different behaviours. Reading the suffix on an ARN in an exam question tells you what will happen when the function is deployed.

Aliases

- A named pointer to one published version. Typical names: **live**, **prod**, **staging**.
- Deploying is **UpdateAlias** to point at a new version. Rolling back is pointing it back.
- An alias has its own ARN, its own permissions and its own CloudWatch metrics.
- **An alias cannot point at $LATEST**, and cannot point at another alias.
- Several aliases can point at the same version, which is how staging and prod share code.

> ON THE EXAM
>
> Rollback being a pointer move is the whole value. The previous version still exists, unchanged, so reverting takes one API call and no redeployment.

Point at a version or an alias?

A VERSION ARN

Pinned forever

- Always runs exactly that code
- Never changes underneath a client
- Updating means editing the caller
- Useful for a deliberate long-term pin
- Wrong for anything you deploy to

AN ALIAS ARN

Stable, and movable

- Stable identifier for the client
- You move it to deploy
- Rollback is one API call
- Carries its own permissions
- The right default for everything

THE TIEBREAKER

Both are qualified ARNs, and only one lets you deploy. A version ARN is stable in the wrong way: nothing changes, including the things you want to change.

Weighted aliases

- A **routing configuration** on an alias sends a percentage of invocations to a second version.
- Specify 10% to the additional version, and the primary automatically receives 90%.
- This is the mechanism CodeDeploy drives during a canary or linear deployment.
- Distribution is **probabilistic**, so at low traffic the actual split can vary noticeably.
- It works for synchronous and asynchronous invocation, and for event source mappings.

> ON THE EXAM
>
> This is what module 23's Canary10Percent5Minutes actually does: set a weight of 10%, wait, then move the alias fully. CodeDeploy just automates the two calls.

Weighted alias constraints

- **Maximum two versions.** You cannot split traffic three ways.
- **Both must be published versions.** Neither can be $LATEST.
- Both must have the **same execution role**.
- Both must have the **same dead-letter queue configuration**, or none.
- **A weighted alias cannot have provisioned concurrency.** The two are mutually exclusive.

> READ THIS TWICE
>
> The last one is the least known and the most disruptive: enabling a canary on a function with provisioned concurrency fails outright.

Async invocation and weights

- Weighted routing applies to **asynchronous** invocation and event source mappings too.
- But a **retry** of an async invocation may land on the other version.
- So an event can be processed by version 7 first and version 8 on retry.
- Two versions therefore run against the same data during a canary.
- This is why **backward-compatible changes** matter during a gradual rollout.

> ON THE EXAM
>
> A canary means both versions are live simultaneously. If version 8 writes a field version 7 cannot read, the canary itself causes the failure.

Knowing which version ran

- The **START log line** in CloudWatch includes the version that handled each invocation.
- The **ExecutedVersion** metric dimension splits CloudWatch metrics by version.
- Synchronous responses carry an **x-amz-executed-version** header.
- So error rate can be compared between the canary and the primary directly.
- That comparison is what an alarm-based rollback needs in order to be meaningful.

> ON THE EXAM
>
> Without ExecutedVersion, a canary alarm measures the blend of both versions. Splitting by version is what lets you detect that only the new one is failing.

What should point at an alias

- **API Gateway integrations**, using the alias-qualified ARN or a stage variable.
- **Event source mappings** for SQS, Kinesis and DynamoDB Streams.
- **Function URLs**, which take a qualifier.
- **EventBridge targets** and S3 event notifications.
- Anything pointing at the unqualified function is effectively deployed to instantly.

> ON THE EXAM
>
> Module 11's stage variables pair with this: the dev stage points at the dev alias, prod at the prod alias, from one API definition and one function.

Provisioned concurrency and versions

- **Provisioned concurrency cannot be set on $LATEST.** It requires a version or an alias.
- That is itself an argument for versioning: without it you cannot use the feature at all.
- Configured on an alias, it moves with the alias when you update it.
- New environments must warm up, so a deployment briefly loses the benefit.
- Module 4 covered the cold start side. This is the deployment side of the same feature.

> ON THE EXAM
>
> The warm-up gap is real: immediately after moving the alias, the new version's provisioned environments are still initialising, so early invocations can still cold start.

Permissions per qualifier

- A function's **resource policy** can be scoped to a specific version or alias.
- So API Gateway can be permitted to invoke **live** but not **staging**.
- Permission granted on the unqualified function does **not** automatically cover an alias.
- That is a common cause of a 502 after switching an integration to an alias ARN.
- SAM and CDK add the correct qualified permission for you.

> ON THE EXAM
>
> Switching an integration from the function to an alias without adding the matching permission is a classic self-inflicted outage. The ARN changed, so the policy no longer matches.

Layers and code storage

- Layers are versioned too, and a function version pins **specific layer versions**.
- So updating a layer does not change existing function versions. Nothing shifts under you.
- Each account gets **300 GB of code storage per Region**, covering functions and layers.
- Exceeding it returns **CodeStorageExceededException** and new deployments fail.
- The fix is deleting old versions and unused layer versions. Nothing expires on its own.

ON THE EXAM

Publishing a version on every deployment is correct practice and accumulates storage indefinitely. Nothing cleans it up automatically, so eventually deployments simply stop working.

The safe release pattern

01

Deploy code

Update the function. This changes $LATEST only.

02

Publish

PublishVersion freezes it as an immutable numbered version.

03

Shift

Weight the alias so a small share of traffic reaches the new version.

04

Promote or revert

Move the alias fully, or drop the weight back to zero.

ON THE EXAM

Every tool in this course automates these four steps. SAM's AutoPublishAlias does 1 and 2; DeploymentPreference does 3 and 4 through CodeDeploy.

How each tool does this

TOOL	WHAT IT HANDLES
Console or CLI	PublishVersion and UpdateAlias, called by hand
AWS SAM	AutoPublishAlias publishes and repoints. DeploymentPreference shifts traffic
AWS CDK	currentVersion and an Alias construct, with CodeDeploy for shifting
CloudFormation	AWS::Lambda::Version and AWS::Lambda::Alias with a RoutingConfig
CodeDeploy	Drives the alias weights on a canary or linear schedule, with rollback

ON THE EXAM

They all end at the same two API calls. Knowing that makes the tools easier to reason about, and makes a failure in any of them easier to place.

Cleaning up

- Every deployment publishes a version, and versions accumulate forever.
- **A version referenced by an alias cannot be deleted.** Move the alias first.
- Keep the last few versions so rollback targets remain available.
- A scheduled Lambda function is the usual cleanup mechanism. There is no lifecycle rule.
- Deleting a version does not free its number for reuse.

> ON THE EXAM
>
> Keep enough versions to roll back to, and delete the rest. 'How far back might we need to revert' is the question that sets the number.

Versions across environments

- Several aliases on one function can point at different versions.
- So **staging** points at version 9 while **prod** still points at version 7.
- Promotion is then moving prod to the version staging already validated.
- The **same code** runs in both, so environment differences must come from elsewhere.
- Use API Gateway stage variables, or separate accounts, rather than per-environment code.

> ON THE EXAM
>
> Promoting the exact version that passed staging is the value here. Rebuilding for production reintroduces the risk that testing was meant to remove.

The common mistakes

- Production pointing at the **unqualified function**, so every deployment is immediate and irreversible.
- Changing an **environment variable** and expecting the alias to see it without publishing.
- Switching an integration to an alias without adding the **qualified invoke permission**.
- Trying to weight an alias that has **provisioned concurrency**.
- Never deleting old versions, until **CodeStorageExceededException** stops all deployments.

> ON THE EXAM
>
> Five mistakes, and four of them produce a failure that looks like something else entirely. That is why this small topic carries so much exam weight.

For each one, ask: what exactly is the client pointing at?

> An API Gateway REST API integrates with a Lambda function using the function's ARN with no qualifier. A developer updates the function code to fix a bug in a development account, but the same deployment process runs against production.
>
> Production begins returning errors within seconds of the deployment. Reverting means redeploying the previous code, which takes several minutes.

What change would prevent this?

A Add reserved concurrency to the production function to isolate it

B Pin the API Gateway integration to a specific version number

C Publish a version on each deployment and point the integration at an alias, updated separately

D Enable provisioned concurrency so the old code stays warm

◆ PAUSE AND DECIDE

What change would prevent this?

A Reserved concurrency

Caps how many invocations run at once. It has nothing to do with which code version they run.

B Pin to a version number

Fixes the immediacy and creates a new problem: the integration never moves, so every release means editing and redeploying the API.

C Publish a version, point at an alias

Code updates change $LATEST only, so nothing reaches production until the alias moves. Reverting is moving it back, which is one API call.

D Provisioned concurrency

Addresses cold starts. Warm environments running the broken code fail exactly as fast as cold ones.

THE SIGNAL

Option B is the instructive near miss. It solves the stated problem by removing the ability to deploy at all, which is why the alias exists in between.

A team runs a latency-sensitive function with provisioned concurrency configured on its live alias. They want to add canary deployments, shifting 10% of traffic to each new version for five minutes.

Configuring the routing weight fails with InvalidParameterValueException, stating that an alias with weights cannot be used with provisioned concurrency.

What is the correct interpretation?

A The two features are mutually exclusive on the same alias, so the design must choose one or restructure

B Provisioned concurrency must be increased before weights can be applied

C The alias must point at $LATEST for weighted routing to work

D Weighted routing requires the function to have reserved concurrency configured first

◆ PAUSE AND DECIDE

What is the correct interpretation?

A Mutually exclusive

The error is stating a hard constraint, not a misconfiguration. A weighted alias cannot carry provisioned concurrency, so one of the two requirements must give.

B Increase provisioned concurrency

Reads a constraint as a capacity problem. No amount of provisioned concurrency makes the combination valid.

C Point at $LATEST

Aliases cannot point at $LATEST at all, and weighted routing specifically requires two published versions.

D Reserved concurrency first

Reserved and provisioned concurrency are different features. Neither enables weighted routing.

THE SIGNAL

Some exam answers are simply 'this combination is not supported'. Recognising a hard constraint rather than hunting for a workaround is the skill being tested.

> A team's CI/CD pipeline has published a version on every deployment for two years across roughly 60 functions. Deployments now fail with CodeStorageExceededException, and no new functions can be created in the Region.
>
> Rollback capability to recent releases must be retained.

What should the team do?

A. Request a quota increase for Lambda concurrent executions

B. Stop publishing versions and deploy to $LATEST instead

C. Add a lifecycle rule to expire old function versions automatically

D. Delete old versions that no alias points at, keeping the most recent few per function

◆ PAUSE AND DECIDE

What should the team do?

A Concurrency quota increase

Concurrency and code storage are unrelated quotas. This error is about stored bytes, not simultaneous executions.

B Deploy to $LATEST

Frees storage by abandoning versioning, which discards rollback, canary deployments and provisioned concurrency in one move.

C A lifecycle rule

No such feature exists for Lambda versions. Nothing expires automatically, which is why the storage accumulated.

D Delete unreferenced old versions

The 300 GB per-Region quota covers functions and layers. Deleting versions no alias points at frees space while keeping recent rollback targets.

THE SIGNAL

Option C is worth remembering as a false feature. The absence of automatic expiry is exactly why this problem happens at all.

A function reads an API endpoint from an environment variable. A developer updates that variable to point at a new endpoint and confirms the change is saved in the console. Traffic continues going to the old endpoint.

The function is invoked through an alias named live, which points at version 7. The deployment pipeline was not run.

Why has the change not taken effect?

A Environment variable changes require the function to be restarted

B The variable changed on $LATEST only. Version 7 retains the value captured when it was published

C The alias is caching the old configuration and needs to be recreated

D Environment variables cannot be changed after a version is published

◆ PAUSE AND DECIDE

Why has the change not taken effect?

A Needs a restart

Lambda has no restart concept, and new execution environments start constantly. They would pick up a change if the version had one.

B Changed on $LATEST only

A version freezes its environment variables at publish time. Editing them changes $LATEST, and version 7 is unaffected.

C Alias caching

An alias is a pointer, not a cache. It resolves to version 7, which genuinely holds the old value.

D Cannot be changed after publishing

Half right in a misleading way. You cannot change them on a published version, and you can change them on $LATEST, which is what happened.

THE SIGNAL

Option D states a true fact and draws the wrong conclusion. The console let the edit succeed because it applied to $LATEST, not to version 7.

What to carry forward

- **Versions are immutable, aliases move.** Clients hold the alias, never the version.
- **An unqualified ARN means $LATEST**, so every deployment is instant and irreversible.
- **A version freezes environment variables and layer versions**, not just code.
- Weighted aliases: two published versions maximum, and never with provisioned concurrency.
- 300 GB of code storage per Region, and nothing expires on its own.

> ON THE EXAM
>
> That closes the deployment services. Next: CloudWatch logs, metrics and alarms, opening the observability run into Domain 4.

MODULE 25 · DOMAIN 4 - TROUBLESHOOTING AND OPTIMIZATION

CloudWatch – Logs, Metrics and Alarms

Metric filters, subscription filters, embedded metrics - and why an alarm on a quiet function never fires.

DOMAIN 4 of 4 EXAM WEIGHT 18% MODULE 25 of 34

admin@sthithapragna.com

Logs are **events**. Metrics are **numbers over time**. Most CloudWatch questions are about moving between the two.

Log groups and log streams

- A **log group** is the container. Retention, encryption and metric filters are set on it.
- A **log stream** is a sequence of events from one source, within a group.
- Lambda creates one group per function, and a stream per execution environment.
- So one stream contains many invocations, and one invocation never spans streams.
- Every event has a timestamp and a message. That is the whole model.

> ON THE EXAM
>
> The Lambda mapping is worth internalising: a stream per execution environment means a cold start creates a new stream, which is why streams appear in bursts after a deployment.

Retention

- **Log groups default to Never Expire.** Logs accumulate and are billed forever.
- Retention is set per group, from **1 day to 10 years**, or never.
- A group created automatically by a service also defaults to never expiring.
- **Export to S3** for long-term archival at a much lower storage cost.
- Changing retention applies to existing events too, so old logs are deleted.

> ON THE EXAM
>
> This is one of the most common sources of unexplained AWS spend. Every Lambda function you ever deployed still has its logs, unless somebody set a retention period.

Currency note: log classes

- **The examinable model:** one kind of log group, priced per GB ingested.
- **Infrequent Access** is a cheaper ingestion class for logs queried only occasionally.
- It supports Logs Insights, export to S3 and encryption, at a lower price per GB.
- It does not support metric filters, subscription filters, Live Tail or alarms.
- **A log group's class cannot be changed after creation.** It is a one-time decision.

> ON THE EXAM
>
> The trade is specific: cheaper ingestion in exchange for losing exactly the features that turn logs into metrics and alerts. Right for forensic logs, wrong for operational ones.

How logs get in

- **AWS services publish directly.** Lambda, API Gateway and others need only permission.
- The **CloudWatch agent** ships system and application logs from EC2 and on-premises servers.
- The **PutLogEvents** API, used directly or through a logging library.
- For Lambda, the execution role needs **logs:CreateLogStream** and **logs:PutLogEvents**.
- A function with no logs at all is usually a permissions problem, not a code problem.

> ON THE EXAM
>
> That last line is a real diagnostic. If a function runs but produces no log output whatsoever, check the execution role before reading any code.

Structured logging

- Log a **JSON object** per event rather than an interpolated string.
- Logs Insights can then filter and aggregate on fields with no **parse** step.
- Include a **correlation ID** on every line, so one request can be traced across functions.
- Lambda's JSON log format adds requestId and log level automatically.
- An interpolated message is a string. Nothing can query inside it reliably.

> ON THE EXAM
>
> The correlation ID is the part people skip and later wish they had. Without it, following one request across five functions means guessing from timestamps.

Lambda logging in detail

- **Log format** can be plain text or **structured JSON**, configured on the function.
- **Application log level** filters your own logs at the platform, before ingestion.
- So debug logging can be left in the code and switched on without redeploying.
- **System log level** controls the platform's own START, END and REPORT lines.
- A custom **log group** can be set, so several functions share one group.

> ON THE EXAM
>
> Filtering at the platform means unwanted log lines are never ingested, so you are not billed for them. Filtering in code still costs you the CPU time.

Metric filters

- A **filter pattern** on a log group matches events and increments a **custom metric**.
- Patterns match terms, phrases, JSON fields, or a regular expression.
- The metric value can be a fixed **1** per match, or a **numeric field** from the event.
- You can then **alarm** on that metric, which you cannot do on logs directly.
- **Metric filters are not retroactive.** They only match events ingested after creation.

> ON THE EXAM
>
> This is the standard answer for 'alarm when this appears in the logs'. The non-retroactive point matters: creating a filter to investigate a past incident finds nothing.

Subscription filters

- Delivers matching log events in **near real time** to another destination.
- Destinations: **Kinesis Data Streams**, **Data Firehose**, **Lambda**, or OpenSearch.
- Firehose to S3 or OpenSearch is the standard centralised-logging pattern.
- **Cross-account** delivery is supported, using a destination in the receiving account.
- Events arrive **base64-encoded and gzip-compressed**, so a Lambda consumer must decompress.

> ON THE EXAM
>
> That encoding detail catches people writing their first log-processing function: the payload is not readable JSON until you have decoded and decompressed it.

Metric filter or subscription filter?

METRIC FILTER

Count it and alarm

- Produces a CloudWatch metric
- Stays inside CloudWatch
- The basis for an alarm
- No destination to run
- Not retroactive

SUBSCRIPTION FILTER

Send it somewhere

- Delivers the events themselves
- Kinesis, Firehose, Lambda, OpenSearch
- For search, archival or analysis
- Cross-account capable
- You run the consumer

THE TIEBREAKER

Ask whether the scenario wants a number or the events. 'Alert when errors exceed 10' is a metric filter; 'centralise logs for searching' is a subscription filter.

CloudWatch Logs Insights

- An interactive query engine over log groups. No indexing to configure.
- Core commands: **fields**, **filter**, **stats**, **sort**, **limit**, **parse**.
- **parse** extracts fields from unstructured text. Structured JSON logs need no parsing.
- Results can be added to a dashboard, and saved for reuse.
- Billed by **data scanned**, so a narrow time range costs less than a wide one.

> ON THE EXAM
>
> The billing model shapes usage: always narrow the time range first. Querying a week when you needed an hour costs roughly 168 times as much.

Queries worth knowing

- **filter @message like /ERROR/** narrows to matching events.
- **stats count() by bin(5m)** produces a time series from log events.
- **stats avg(duration), pct(duration, 99) by operation** compares latency across operations.
- **sort @timestamp desc | limit 20** is the fastest way to see what just happened.
- **@message**, **@timestamp** and **@logStream** are always available as fields.

> ON THE EXAM
>
> stats with bin is the one to remember: it turns log events into a chart, which is often all an investigation needs before deciding where to look next.

Currency note: three query languages

- **The examinable language** is Logs Insights QL, with its pipe-separated commands.
- Logs Insights now also accepts **OpenSearch PPL** and **OpenSearch SQL**.
- SQL brings **JOINs across log groups** and subqueries, which QL cannot do.
- PPL is pipe-based and will feel familiar if you know Logs Insights QL.
- Concurrency differs: 100 concurrent QL queries per account, but only 15 for SQL or PPL.

> ON THE EXAM
>
> Answer Logs Insights QL on the exam. The addition is genuinely useful in practice, because joining two log groups was previously impossible.

Live Tail

- Streams log events into the console **as they are ingested**, with a filter pattern.
- Useful during a deployment or while reproducing a bug deliberately.
- Several log groups can be tailed at once.
- Billed **per minute of tailing**, so it is not something to leave running.
- Standard log class only. Not available on Infrequent Access groups.

> ON THE EXAM
>
> Live Tail fills the gap between an alarm firing and a Logs Insights query: it shows what is happening now, rather than what happened.

Metrics

- A metric is identified by a **namespace**, a **name** and a set of **dimensions**.
- **Each unique dimension combination is a separate metric**, and is billed as one.
- AWS namespaces look like **AWS/Lambda**. Custom ones are anything else.
- Dimensions are how you slice: by function name, by alias, by executed version.
- High-cardinality dimensions such as user ID create an unbounded number of metrics.

> ON THE EXAM
>
> That last point is the classic custom-metric cost mistake. A dimension with a million possible values produces a million billable metrics.

Resolution and statistics

- **Standard resolution** is one-minute granularity. **High resolution** is one second.
- High resolution alarms can evaluate at 10 or 30 seconds, at higher cost.
- Statistics: **Sum**, **Average**, **Minimum**, **Maximum**, **SampleCount** and percentiles.
- **Average hides outliers.** A p99 alarm catches what an average alarm misses.
- Metrics are retained 15 months, aggregated to coarser periods as they age.

> ON THE EXAM
>
> Alarming on average latency is a common mistake. If 1% of requests take 30 seconds, the average barely moves and the p99 tells you immediately.

Custom metrics

- **PutMetricData** publishes a value into a namespace you choose.
- It is a **synchronous API call**, so it adds latency to whatever path calls it.
- It is billed per call, and per metric stored.
- Values can be sent as a **statistic set**, pre-aggregated, to reduce call volume.
- In a Lambda function, a failed PutMetricData call can fail the invocation.

> ON THE EXAM
>
> The synchronous cost is the reason the next slide exists. Adding a metric call to a hot path means every request pays for the observability.

Embedded Metric Format

- Write a specially structured **JSON log line**, and CloudWatch extracts metrics from it.
- No synchronous API call, so no added latency and no failure path.
- The log line keeps **full context** alongside the metric values.
- So you can alarm on the metric and then read the exact events behind it.
- High-cardinality fields can be logged as properties without becoming dimensions.

> READ THIS TWICE
>
> That last point is the real advantage. Log the user ID as a property so it is queryable, without creating a billable metric per user.

PutMetricData or EMF?

PUTMETRICDATA

Direct API call

- Immediate, explicit publication
- Works from anywhere with credentials
- Adds latency to the calling path
- Billed per API call
- No surrounding context

EMBEDDED METRIC FORMAT

A structured log line

- Asynchronous, via the log pipeline
- No latency and no failure path
- Context preserved with the metric
- Billed as log ingestion
- The recommended default in Lambda

THE TIEBREAKER

For anything running in Lambda, EMF is almost always right. PutMetricData suits a process publishing occasional aggregate values rather than per-request data.

Alarms

- A metric, a **statistic**, a **period**, a **threshold** and a comparison operator.
- **Evaluation periods** and **datapoints to alarm** give you M-of-N behaviour.
- So 3 of 5 datapoints breaching avoids alerting on a single spike.
- States are **OK**, **ALARM** and **INSUFFICIENT_DATA**.
- Actions can fire on entering any state, not only ALARM.

> ON THE EXAM
>
> M-of-N is the fix for a flapping alarm. Widening the period smooths the data; requiring 3 of 5 datapoints tolerates a spike without missing a sustained problem.

Missing data

- **missing** is the default. The datapoint is ignored and the state does not change.
- **notBreaching** treats missing data as good, so the alarm goes OK.
- **breaching** treats missing data as bad, so the alarm fires.
- **ignore** keeps the current state indefinitely.
- A metric with **no data at all** means no alarm. Errors publish nothing when nothing runs.

> ON THE EXAM
>
> This is why an error alarm on a low-traffic function can stay silent through an outage: with no invocations there are no datapoints, so there is nothing to breach.

Anomaly detection

- CloudWatch learns a metric's normal **band** from up to two weeks of history.
- The alarm fires when a value falls outside that band, not outside a fixed number.
- It accounts for **daily and weekly seasonality**, so quiet nights are expected.
- Useful where the right threshold genuinely changes over time.
- It needs history to learn from, so it is useless on a brand-new metric.

> ON THE EXAM
>
> The case for it is traffic that varies by time of day: a fixed threshold is either too high at 3am or too low at midday, and anomaly detection is neither.

Composite alarms

- An alarm whose state is a **boolean expression** over other alarms.
- Alert only when errors are high **and** latency is high, rather than on either.
- Suppress child alarms during a known maintenance window with a suppressor alarm.
- One notification for a correlated failure, instead of twenty.
- Composite alarms cannot invoke every action type. Check what the destination supports.

> ON THE EXAM
>
> The suppressor is the underused feature. It stops a single root cause producing a page from every downstream alarm that noticed it.

Alarm actions

- **SNS notification**, which is how alarms reach people or other systems.
- **Auto Scaling** actions, and EC2 actions such as stop, terminate, reboot or recover.
- **Systems Manager** actions, including creating an OpsItem or an incident.
- Alarms also emit **EventBridge events**, which reach anything at all.
- This is the same alarm mechanism CodeDeploy, AppConfig and CloudFormation use for rollback.

> ON THE EXAM
>
> Worth noticing how much of this course routes through one CloudWatch alarm: deployment rollback, configuration rollback and scaling all read the same signal.

CloudWatch Synthetics

- A **canary** is a scripted Lambda function that runs on a schedule against your endpoint.
- It checks availability and latency the way a user would, from outside the system.
- Blueprints cover heartbeat checks, API checks and broken-link checks.
- Failures produce metrics you can alarm on, plus screenshots and a HAR file.
- Distinct from a canary **deployment**. The same word, two unrelated meanings.

> ON THE EXAM
>
> Synthetics catches the outage your internal metrics miss: DNS, certificates and the load balancer are all outside the code that publishes your metrics.

Dashboards and Metrics Insights

- Dashboards are global, and can display metrics from **several Regions and accounts**.
- **Metric math** computes derived values, such as an error rate from two metrics.
- **Metrics Insights** is a SQL-like query language over metrics, evaluated at query time.
- It can rank and aggregate across dimensions without listing every metric.
- Logs Insights query results can be added to a dashboard alongside metric widgets.

> ON THE EXAM
>
> Metric math is worth knowing by name: error count and invocation count are separate metrics, and an error rate alarm needs the ratio of the two.

Controlling CloudWatch cost

- **Log ingestion** is usually the largest line. Filter at the source, not after ingestion.
- **Retention set to never expire** is the second. Set a period on every log group.
- **Custom metrics** are billed per metric, and dimensions multiply the count.
- Lambda's application log level filters before ingestion, so unwanted lines cost nothing.
- Infrequent Access reduces ingestion cost where the features are not needed.

> ON THE EXAM
>
> The order matters: ingestion, then retention, then custom metrics. Debug logging left on in production is usually the single biggest contributor.

Ask first: does the scenario want **a number**, or does it want **the events**?

> An application writes a log line containing PaymentDeclined whenever a card is refused. The operations team wants an alert when more than 50 declines occur in five minutes, and wants to see the trend on a dashboard.
>
> The application code cannot be modified, and the log group is in the Standard class.

How should this be implemented?

A. A subscription filter to a Lambda function that counts occurrences and sends an SNS message

B. A metric filter on the log group matching PaymentDeclined, with a CloudWatch alarm on the resulting metric

C. A scheduled Logs Insights query that counts matching lines and publishes a metric

D. Modify the application to call PutMetricData when a decline occurs

◆ PAUSE AND DECIDE

How should this be implemented?

A. Subscription filter plus Lambda

Rebuilds a metric filter by hand, including the counting window and the alerting, with a function to run and pay for.

B. Metric filter plus alarm

Exactly what metric filters exist for. The pattern produces a metric, the alarm watches it, and the metric charts on a dashboard for free.

C. Scheduled Logs Insights query

Logs Insights is interactive and billed by data scanned. Running it on a schedule is expensive and adds delay before the alert.

D. Modify the application

The scenario states the code cannot be modified. It would also add a synchronous API call to the payment path.

THE SIGNAL

Three requirements met by one feature: match a pattern, produce a metric, alarm on it. The 'cannot modify code' clause rules out the only alternative that would be reasonable.

> A company runs 200 Lambda functions across 12 accounts. A central security team needs all log events searchable in one Amazon OpenSearch domain within a minute of being written, for correlation and investigation.
>
> They do not need alarms on the logs themselves, and the volume is high.

What should be configured?

A. A metric filter on each log group, publishing to a shared namespace

B. A nightly export task from each log group to S3, loaded into OpenSearch

C. Subscription filters on the log groups, delivering to Data Firehose and on to OpenSearch

D. Move all log groups to the Infrequent Access class and query them with OpenSearch SQL

◆ PAUSE AND DECIDE

What should be configured?

A. Metric filters

Produces counts, not events. The security team needs to read and correlate the log lines themselves, which a metric cannot give them.

B. Nightly export

The requirement is within a minute. A nightly job is off by three orders of magnitude.

C. Subscription filters to Firehose

Near real-time delivery of the events themselves, supports cross-account destinations, and Firehose handles buffering and delivery at high volume.

D. Infrequent Access plus SQL

Leaves logs in each account rather than centralising them, and IA does not support subscription filters at all.

THE SIGNAL

'Searchable' plus 'within a minute' means the events, delivered in near real time. That is the subscription filter definition almost word for word.

A Lambda function processing orders should record order value, processing duration and the customer tier for each order. The team wants to alarm on average order value by tier, and to investigate individual slow orders by customer ID afterwards.

Latency is critical and there are around 2,000 invocations per second.

How should the instrumentation be done?

A. Emit Embedded Metric Format log lines, with tier as a dimension and customer ID as a property

B. Call PutMetricData once per invocation, with customer ID and tier as dimensions

C. Write plain log lines and create metric filters for each customer tier

D. Call PutMetricData asynchronously in a background thread after returning the response

◆ PAUSE AND DECIDE

How should the instrumentation be done?

A EMF, tier as dimension, customer as property

No API call so no added latency, tier is a low-cardinality dimension you can alarm on, and customer ID stays queryable in Logs Insights without becoming a metric.

B PutMetricData with customer as a dimension

Two problems: a synchronous call on every invocation at 2,000 per second, and customer ID as a dimension creates a billable metric per customer.

C Metric filters per tier

Workable for counts, and it means a filter per tier to maintain, and extracting numeric values like order value is awkward.

D Background thread

Lambda freezes the execution environment after the handler returns, so background work may never run. Module 3 covered this.

THE SIGNAL

The dimension-versus-property distinction is the whole question. Tier has a handful of values and belongs as a dimension; customer ID has millions and belongs as a property.

> An alarm watches the Errors metric of a Lambda function that runs a few times an hour. During a recent incident the function failed on every invocation for three hours, and the alarm never left the INSUFFICIENT_DATA state.
>
> The alarm uses a one-minute period with one datapoint to alarm, and default missing data treatment.

What is the correct fix?

A. Set missing data treatment to breaching so the alarm fires when no data arrives

B. Switch the metric to high resolution so datapoints arrive every second

C. Add a composite alarm combining Errors with Duration

D. Widen the period to match the invocation rate, and alarm on an error rate using metric math

◆ PAUSE AND DECIDE

What is the correct fix?

A Missing data as breaching

The tempting one. It would fire during the incident and also fire every quiet minute, because a function invoked a few times an hour is missing data most of the time.

B High resolution

Publishes datapoints more often only when there is data. A function that is not invoked still produces nothing.

C Composite alarm

Combines alarms that already work. Both underlying metrics have the same gap, so combining them changes nothing.

D Widen the period, alarm on error rate

A period long enough to contain invocations produces datapoints, and an error rate from Errors divided by Invocations is meaningful at any volume.

THE SIGNAL

The mismatch is between a one-minute period and an hourly invocation rate. Option A treats the symptom and would page the on-call engineer all night.

What to carry forward

- **Metric filter for a number, subscription filter for the events.** That splits most questions.
- **EMF over PutMetricData in Lambda:** no latency, no failure path, context preserved.
- **Dimensions are billable and multiply.** High-cardinality values belong as properties.
- Missing data treatment explains an alarm that never fires on a quiet resource.
- Log groups never expire by default, which is the commonest source of surprise cost.

> ON THE EXAM
>
> Next: tracing with X-Ray and Application Signals, which answers the question metrics and logs cannot - where in the call chain the time actually went.

MODULE 26 · DOMAIN 4 - TROUBLESHOOTING AND OPTIMIZATION

Tracing with X-Ray and Application Signals

Segments, sampling, annotations - and the question that logs and metrics genuinely cannot answer.

DOMAIN 4 of 4 EXAM WEIGHT 18% MODULE 26 of 34

admin@sthithapragna.com

Logs say **what a service said**. Metrics say **how much and how often**. Only a trace says **where the time went**.

Traces, segments and subsegments

- A **trace** is the complete path of one request through your system, with one trace ID.
- A **segment** is one service's contribution to that trace, with timing and status.
- A **subsegment** is finer detail inside a segment: a database call, an HTTP request, a code block.
- Subsegments are what let you see that the time went to DynamoDB rather than your code.
- Every segment carries the same trace ID, which is how the pieces are reassembled.

> ON THE EXAM
>
> The subsegment level is where the answer usually is. A segment tells you a service was slow; a subsegment tells you which call inside it was slow.

How the trace ID propagates

- The **X-Amzn-Trace-Id** header carries the trace ID between services.
- The first service to see a request creates the ID. Everything downstream reuses it.
- The instrumented SDK adds the header to outbound calls automatically.
- A service that does not forward the header **breaks the trace** into disconnected pieces.
- **W3C Trace Context** is now supported alongside it, which matters for OpenTelemetry.

> ON THE EXAM
>
> A trace that stops halfway through the call chain is almost always a service that did not propagate the header, often because it was not instrumented at all.

The service map

- A generated diagram of your services, built from the segments X-Ray received.
- Each node is a service. Each edge is a call, labelled with latency and error rate.
- Nodes turn **yellow for 4xx** and **red for 5xx**, so failures are visible immediately.
- Clicking a node filters to the traces behind it.
- Nothing appears on the map unless it is instrumented, so gaps mean missing instrumentation.

> ON THE EXAM
>
> During an incident the service map is the fastest starting point: it shows which node is red before you have read a single log line.

Sampling

- Tracing everything would be expensive and would add overhead. X-Ray samples instead.
- The default rule: the **first request each second**, plus **5% of additional requests**.
- That one per second is the **reservoir**, guaranteeing at least one trace while traffic flows.
- At 1,000 requests per second that yields roughly 51 traces, about 5%.
- **So a specific failing request may simply not have been traced.**

> ON THE EXAM
>
> That last point causes real frustration. A customer reports a failure, you search for their trace, and it was never sampled - which is exactly what Transaction Search addresses.

Reading a trace

- The trace view is a **waterfall**: each segment and subsegment as a bar on a timeline.
- A long bar with short children means the time went to your own code.
- A long child bar means the time went to whatever that subsegment called.
- A **gap** between bars is time nothing accounted for, often queueing or a cold start.
- Status and response codes appear per segment, so a failure is located immediately.

> ON THE EXAM
>
> The gaps are the underrated part. Time that appears in the parent but in no child is unaccounted for, and that is usually where the surprise is.

Custom sampling rules

- Rules match on service name, host, HTTP method and **URL path**.
- Each sets a **reservoir size** and a **fixed rate**, and has a **priority**.
- Lower priority numbers win. The default rule sits at priority 10,000, so anything overrides it.
- Trace 100% of a payment endpoint, and 1% of health checks, from two rules.
- Rules are configured centrally and **fetched by the SDK**, so no redeployment is needed.

> ON THE EXAM
>
> The central configuration is the useful part: raising sampling on a problematic endpoint during an incident takes effect without shipping any code.

Annotations or metadata?

ANNOTATIONS

Indexed and searchable

- Key-value pairs, indexed by X-Ray
- Filterable in the console and API
- Strings, numbers and booleans only
- Up to 50 per trace
- Use for anything you will search on

METADATA

Stored, not indexed

- Arbitrary JSON of any shape
- Cannot be filtered on
- No practical count limit
- Read manually when viewing a trace
- Use for larger debug payloads

THE TIEBREAKER

The whole distinction is searchability. An order ID in metadata cannot be found; the same value as an annotation can be filtered on directly.

Tracing Lambda

- Enable **active tracing** on the function. No agent and no daemon required.
- The execution role needs the **AWSXRayDaemonWriteAccess** managed policy.
- That alone gives you the invocation segment, showing initialisation and invocation time.
- Adding the SDK or ADOT gives **subsegments** for downstream AWS and HTTP calls.
- Active tracing without instrumentation shows the function as a single opaque block.

> ON THE EXAM
>
> The two levels matter. Active tracing tells you the function took 900 ms; instrumentation tells you 850 ms of that was one DynamoDB query.

Tracing ECS and EC2

- On EC2 and ECS, something must **collect and forward** segments to X-Ray.
- The **X-Ray daemon** listens on **UDP port 2000** and batches segments to the service.
- On ECS it runs as a **sidecar container** in the same task definition.
- On EC2 it runs as a process on the instance, usually installed via user data.
- The **task role** or instance profile needs permission to write segments.

> ON THE EXAM
>
> This is the usual reason ECS services are missing from a service map: the application is instrumented, and nothing is listening on port 2000 to forward the segments.

Currency note: SDKs and daemon frozen

- **The examinable answer:** instrument with the X-Ray SDK, and run the daemon on ECS and EC2.
- On **25 February 2026** the X-Ray SDKs and the daemon entered **maintenance mode**.
- Security fixes only, no new features, and **end of support in February 2027**.
- **The X-Ray service itself is fully supported** and still being enhanced.
- The recommended path is **AWS Distro for OpenTelemetry**, or native OpenTelemetry.

> READ THIS TWICE
>
> Read the scope carefully. Only the instrumentation SDKs and the daemon are frozen. X-Ray, the service, is where Transaction Search and Application Signals are being built.

ADOT and OpenTelemetry

- **OpenTelemetry** is a vendor-neutral standard for traces, metrics and logs.
- **ADOT** is AWS's supported distribution of it, with an X-Ray exporter.
- Available as a **Lambda layer**, or as a collector sidecar on ECS and EC2.
- The same instrumentation can send data to X-Ray and to a third-party backend.
- **Answer X-Ray SDK on the exam.** ADOT postdates exam guide v2.1.

> ON THE EXAM
>
> The practical argument for OpenTelemetry is portability: instrumentation is no longer tied to one vendor's backend, which is why AWS itself moved to it.

What gets traced automatically

- **API Gateway** can enable tracing per stage, becoming the first segment.
- **Lambda** with active tracing, as covered. **Step Functions** per state machine.
- **SNS and SQS** propagate the trace header, so a trace survives an asynchronous hop.
- The instrumented SDK traces AWS API calls as subsegments automatically.
- Anything not instrumented appears as a gap, not as an error.

> ON THE EXAM
>
> The SQS propagation is worth knowing: without it, an asynchronous architecture would produce a separate disconnected trace for every queue hop.

Finding the trace you want

- Filter on **service()**, **http.status**, **responsetime**, **error**, **fault** and **throttle**.
- Filter on annotations directly: **annotation.orderId = "12345"**.
- **Error** is a 4xx, **fault** is a 5xx, and **throttle** is a 429. They are distinct.
- **Groups** save a filter expression and produce their own CloudWatch metrics.
- So you can alarm on the fault rate of one named group of traces.

> ON THE EXAM
>
> The error, fault and throttle distinction appears throughout X-Ray, including on the service map. Fault means your service failed; error means the caller did.

X-Ray Insights

- Detects anomalies in fault rates automatically, without a threshold being set.
- It builds a **baseline** of expected error rates and flags deviations from it.
- An insight identifies the affected service, the root cause node and the impact.
- Insights can notify through **EventBridge**, so they can page or open a ticket.
- Enabled per group, and it needs traffic to learn from.

> ON THE EXAM
>
> The value is identifying the root cause node rather than the symptomatic one. A slow database surfaces as errors in five services; Insights points at the database.

Limits worth knowing

- Traces are retained for **30 days**. There is no configurable retention.
- Maximum trace size is **500 KB**. A segment document is capped at **64 KB**.
- Up to **50 annotations** per trace.
- Encryption at rest uses a default key, or a KMS key you choose.
- Traces can be shared **across accounts** for a central observability account.

> ON THE EXAM
>
> The 30-day retention is the one that shapes practice: X-Ray is for investigating now, not for historical analysis. Long-term trace data belongs in logs.

X-Ray or CloudTrail?

AWS X-RAY

The data plane

- Traces application requests
- Where did this request spend time?
- Segments across your services
- Sampled, retained 30 days
- For performance and failure analysis

AWS CLOUDTRAIL

The control plane

- Logs AWS API calls
- Who deleted this bucket?
- One event per API call
- Complete, not sampled
- For audit and security investigation

THE TIEBREAKER

A named exam trap. They sound similar and answer opposite questions: X-Ray follows your traffic, CloudTrail records changes to your account.

Connecting traces to logs

- Lambda writes the **trace ID into the log line** when active tracing is on.
- So a slow trace links directly to the log events from that exact invocation.
- For your own services, log the trace ID yourself on every line.
- The X-Ray console links to CloudWatch Logs for the segment you are viewing.
- That closes the loop: alarm to metric, metric to trace, trace to log line.

> ON THE EXAM
>
> This is the payoff for structured logging from module 25. With a trace ID field, one Logs Insights filter retrieves every log line for a single request.

CloudWatch Application Signals

- Auto-instruments applications and produces a curated **application performance** view.
- Standard **golden metrics** per service and operation: latency, errors, faults, request volume.
- It discovers services automatically, including ones you have not instrumented.
- Built on X-Ray traces, so it is a layer above rather than a replacement.
- It defaults to **5% trace sampling**, adjustable up or down.

> ON THE EXAM
>
> The distinction from raw X-Ray is opinionated defaults. X-Ray gives you traces and expects you to know what to ask; Application Signals gives you the standard questions already answered.

X-Ray or Application Signals?

AWS X-RAY

Traces, and your questions

- Individual traces and the service map
- You choose what to instrument
- Filter expressions you write
- Annotations you decide on
- For investigating a specific problem

APPLICATION SIGNALS

Curated application view

- Golden metrics per operation
- Auto-instrumentation and discovery
- SLOs and error budgets
- Standard dashboards out of the box
- For knowing whether you are healthy

THE TIEBREAKER

They are layers, not alternatives. Application Signals reads X-Ray data, so enabling it does not replace anything you already have.

SLOs and error budgets

- An **SLO** is a target for a service level indicator over a window, such as p99 under 300 ms.
- The gap between target and 100% is the **error budget** you may spend.
- **Burn rate** is how fast you are consuming it. A rate of 1.0 exactly exhausts it over the window.
- Fast burn pages somebody. Slow burn opens a ticket. Two alarms, two urgencies.
- SLOs attach to a service or an operation, and track attainment automatically.

> ON THE EXAM
>
> Burn rate alarms fix the two failures of threshold alarms: paging for a brief spike that costs nothing, and staying silent through a slow degradation that costs everything.

Transaction Search

- Stores **100% of spans** as structured logs in a log group named **aws/spans**.
- Written in OpenTelemetry semantic convention format, queryable with Logs Insights.
- A configurable percentage, **1% by default**, is still indexed as X-Ray trace summaries.
- So a specific customer's failing request can be found, even though it was not sampled.
- Because spans land in Logs, metric filters and data masking work on trace data.

> ON THE EXAM
>
> This is the direct fix for 'the trace I need was never sampled'. Every span is stored; only the summary indexing is sampled.

Which observability tool?

THE SCENARIO SAYS	THE INTENDED ANSWER
Where did the latency go across services?	AWS X-Ray, and the service map
Which service is failing right now?	X-Ray service map, or Application Signals
Find the trace for order 12345	An X-Ray annotation, filtered on
Who changed this AWS resource?	AWS CloudTrail
Alert when errors exceed a count	A CloudWatch metric filter and alarm
Is the endpoint up, as a user sees it?	CloudWatch Synthetics canary
Are we meeting our latency target?	Application Signals SLO

ON THE EXAM

Worth screenshotting. Most Domain 4 selection questions reduce to one row, and the X-Ray against CloudTrail row is the one most often got wrong.

Ask first: does the scenario need **one request followed**, or **everything counted**?

> A checkout request passes through API Gateway, a Lambda function, an SQS queue, a second Lambda function and DynamoDB. Users report that checkout sometimes takes eight seconds. Every service reports healthy metrics and no errors appear in any log.
>
> The team needs to know which component is responsible before they can fix it.

What should they use?

A. A CloudWatch dashboard combining the Duration metric of both Lambda functions

B. A Logs Insights query across all the log groups, sorted by timestamp

C. CloudTrail, to see which API calls were made during the slow requests

D. AWS X-Ray with active tracing enabled across the chain, and the service map

◆ PAUSE AND DECIDE

What should they use?

A Dashboard of Duration metrics

Averages across all requests. The slow ones are a minority, so the metric barely moves, which is why everything already looks healthy.

B Logs Insights across log groups

Log lines from different services cannot be tied to one request without a shared trace or correlation ID. You would be matching on timestamps and guessing.

C CloudTrail

Records AWS API calls for audit. It has no visibility into how long an application request spent in each component.

D X-Ray and the service map

A trace follows one request through every hop with timing per segment, which is exactly the question. The service map shows which edge is slow.

THE SIGNAL

'Every service reports healthy metrics' is the clue: aggregate metrics cannot see a minority of slow requests, which is precisely the gap tracing fills.

> A support team needs to investigate individual failed orders. Given an order ID, they must retrieve the X-Ray trace for that specific order. The application currently attaches the order ID and the full order payload to each segment using put_metadata.
>
> Searching for the order ID in the X-Ray console returns nothing.

What change is needed?

A. Increase the sampling rate to 100% so all orders are traced

B. Record the order ID as an annotation instead, keeping the payload as metadata

C. Move the order ID into the segment name so it appears on the service map

D. Export traces to S3 and query them with Athena

◆ PAUSE AND DECIDE

What change is needed?

A 100% sampling

Would trace every order, and the traces still could not be searched by order ID, because metadata is not indexed. It also multiplies cost.

B Order ID as an annotation

Annotations are indexed and filterable, so annotation.orderId = the value retrieves the trace. The bulky payload correctly stays as metadata.

C Order ID in the segment name

Segment names identify services, not requests. A unique name per order would create thousands of nodes on the service map.

D Export and query with Athena

Builds a search system to work around an indexing feature that already exists and needs one line changed.

THE SIGNAL

Note that B keeps the payload as metadata. The fix is not moving everything to annotations, but putting the searchable field in the searchable place.

> An application runs as an ECS service on Fargate. The code is instrumented with the X-Ray SDK and the same code traces correctly when run as a Lambda function. On ECS, no segments appear in X-Ray at all and the service is missing from the service map.
>
> The task role has the AWSXRayDaemonWriteAccess policy attached.

What is missing?

A Active tracing must be enabled on the ECS service, as it is on Lambda

B The task execution role needs X-Ray permissions rather than the task role

C The X-Ray daemon is not running as a sidecar container in the task definition

D Fargate does not support X-Ray tracing

◆ PAUSE AND DECIDE

What is missing?

A Enable active tracing

Active tracing is a Lambda function setting. ECS services have no equivalent toggle, because collection is the daemon's job.

B Task execution role

Module 14's distinction: the execution role starts the container, the task role is what application code uses. Writing segments is application code.

C No daemon sidecar

The SDK sends segments to UDP port 2000 and something must listen there. On ECS that is a daemon sidecar in the task definition, which is absent.

D Fargate unsupported

Fargate supports X-Ray fully, using a sidecar container exactly as EC2-launch-type tasks do.

THE SIGNAL

'The same code works on Lambda' is the clue: the instrumentation is fine, so the difference is collection. Lambda collects for you; ECS does not.

A high-traffic API is traced with default sampling. When a customer reports a failure with a specific request ID, the trace almost never exists, because that request was not sampled. Raising sampling to 100% was rejected as too expensive.

The team needs to investigate individual reported failures, and still wants the normal service map and trace summaries.

What should they enable?

A CloudWatch Transaction Search, which stores 100% of spans in the aws/spans log group

B A custom sampling rule with a larger reservoir on the failing endpoint

C X-Ray Insights, to detect the anomalies automatically

D A CloudWatch metric filter counting failures by request ID

◆ PAUSE AND DECIDE

What should they enable?

A Transaction Search

Stores every span as structured logs while indexing only a small percentage as summaries. Individual requests become findable, and the normal experience is unchanged.

B Larger reservoir

Raises the proportion sampled without guaranteeing any particular request. The reported one may still be missing, which is the actual complaint.

C X-Ray Insights

Finds anomalies across traffic. It cannot retrieve a trace that was never recorded.

D Metric filter by request ID

Request ID is unbounded cardinality, so this would create a metric per request. Module 25 ruled that out explicitly.

THE SIGNAL

The requirement is a specific request, not a representative sample. Only storing everything satisfies that, and Transaction Search separates storing from indexing.

What to carry forward

- **Only a trace answers where the time went.** Metrics average it away.
- **Annotations are indexed, metadata is not.** Anything you will search on is an annotation.
- **Lambda needs only active tracing. ECS and EC2 need the daemon**, as a sidecar or a process.
- Default sampling is 1 per second plus 5%, so a given request may not be traced.
- X-Ray is the data plane, CloudTrail is the control plane. A named exam trap.

> ON THE EXAM
>
> Next: troubleshooting errors, throttling and retries - which uses everything from these two modules on the failures the exam actually asks about.

MODULE 27 · DOMAIN 4 - TROUBLESHOOTING AND OPTIMIZATION

Troubleshooting – Errors, Throttling and Retries

Which errors to retry, how to back off, and why the same fix makes some outages worse.

DOMAIN 4 of 4 EXAM WEIGHT 18% MODULE 27 of 34

admin@sthithapragna.com

Retrying a **transient** failure is correct. Retrying a **permanent** one wastes time, and retrying a **throttle** badly makes the outage worse.

The error taxonomy

- **4xx means the client was wrong.** Bad parameters, missing permission, resource not found.
- **5xx means the service failed.** The same request might succeed on the next attempt.
- **429 is the exception**, a 4xx that should be retried, because the request was valid.
- **408 request timeout** is the other 4xx worth retrying.
- So the rule is: retry 5xx, retry 429 and 408, do not retry other 4xx.

> ON THE EXAM
>
> Retrying a 403 is the most common wasted retry. The permission will not appear between attempts, and the retries delay the error reaching whoever can fix it.

Retryable and not

CATEGORY	EXAMPLES
Retry: throttling	ThrottlingException, TooManyRequests, SlowDown
Retry: transient service	500, 502, 503, 504, InternalServerError
Retry: transient network	Connection reset, socket timeout, DNS failure
Do not retry: client	ValidationException, InvalidParameterValue
Do not retry: auth	AccessDenied, expired credentials
Careful: conditional	ConditionalCheckFailedException

ON THE EXAM

The last row matters: a conditional check failure is not an error to retry blindly. Something else changed the item.

Exponential backoff

- Wait longer after each failed attempt: roughly 50 ms, 100 ms, 200 ms, doubling each time.
- **Without jitter, every client retries at the same moment.** The overload repeats in waves.
- **Jitter** randomises each delay, spreading retries across the window.
- Full jitter picks a random value between zero and the computed backoff.
- A **maximum backoff** caps the delay so it does not grow indefinitely.

> ON THE EXAM
>
> Backoff alone is not enough. A thousand clients all doubling from the same start still retry in synchronised waves; jitter is what breaks the synchronisation.

The SDK already does this

- Every AWS SDK retries retryable errors automatically, with exponential backoff and jitter.
- So a visible failure has usually **already been retried** two or three times.
- Wrapping SDK calls in your own retry loop **multiplies** the attempts.
- Three SDK attempts inside three of your own is nine, with unpredictable total latency.
- Configure the SDK instead: retry mode and max attempts are both settings.

> ON THE EXAM
>
> The multiplication is the practical danger. A Lambda function timing out at 30 seconds can spend all of it inside nested retry loops that were each individually reasonable.

The three retry modes

MODE	BEHAVIOUR
`standard`	Exponential backoff with jitter, plus a retry quota. The recommended default
`adaptive`	Everything in standard, plus client-side rate limiting. Specialised, and experimental
`legacy`	Older, SDK-specific behaviour with no retry quota. For backward compatibility only

ON THE EXAM

Adaptive is not simply 'better standard'. It rate-limits the client itself, so in a multi-tenant application one tenant's throttling can delay another's requests.

Max attempts

- **Max attempts defaults to 3**: one initial request plus two retries.
- Setting it to **1 disables retries entirely**, which is occasionally what you want.
- **DynamoDB clients default to 4**, with a shorter base delay, matching its low-latency profile.
- Configurable in code, by **AWS_MAX_ATTEMPTS**, or in the shared config file.
- Code beats environment variable, which beats the config file.

> ON THE EXAM
>
> Max attempts counts total attempts, not retries. A value of 3 means three requests reach the service at most, which is a common off-by-one misreading.

The retry quota

- Standard mode holds a **token bucket** for retries across the client.
- Each retry deducts tokens. Successful requests replenish them.
- When tokens run out, the SDK **returns the error without retrying**.
- So during a sustained outage the application fails fast rather than waiting through retries.
- **Legacy mode has no retry quota**, which is the main reason it is not recommended.

> ON THE EXAM
>
> The quota converts a slow, retrying failure into a fast one. During a real outage, failing in 100 ms is far better than every request taking six seconds to give up.

Currency note: retry behaviour is changing

- **The examinable model:** exponential backoff with jitter, three attempts, standard mode.
- AWS announced updated retry behaviour, opt-in now via **AWS_NEW_RETRIES_2026**.
- It changes backoff timing, retry quota costs and the DynamoDB defaults.
- It also switches **Java, Python, Ruby, PHP, C++ and the CLI** from legacy to standard.
- **Default rollout is November 2026.** Explicitly set values are preserved.

> ON THE EXAM
>
> The principle does not change: exponential backoff with jitter, and do not retry client errors. Only the specific defaults are moving.

How each service tells you it is throttling

SERVICE	WHAT YOU GET
AWS Lambda	429 TooManyRequestsException, with a Throttles metric
Amazon DynamoDB	ProvisionedThroughputExceededException, or ThrottlingException
Amazon API Gateway	429 Too Many Requests
Amazon S3	503 Slow Down
AWS KMS	ThrottlingException. Quotas are per account and Region
Amazon SQS and Kinesis	ProvisionedThroughputExceededException on Kinesis shards
AWS STS	ThrottlingException on high AssumeRole volume

ON THE EXAM

S3 is the odd one: it signals throttling with a 503 rather than a 429, so it looks like a server error and is retried as one. The effect is the same.

What to do about throttling

- **Retry with backoff and jitter first.** Brief throttling is normal and self-correcting.
- **Reduce the request rate:** batch, cache, or buffer through a queue.
- **Spread the load:** better partition keys, more prefixes, more shards.
- **Raise the limit:** provisioned capacity, reserved concurrency, or a quota increase.
- Retrying harder without any of the above turns a throttle into a sustained outage.

> ON THE EXAM
>
> The order is deliberate. Retrying is the immediate response, and if throttling is sustained the answer is fewer requests or more capacity, never more retries.

Idempotency

- **Every retry mechanism can process the same request twice.** That is not a bug.
- A timeout is the worst case: you do not know whether the operation succeeded.
- An **idempotency key** lets the receiver recognise and ignore a repeat.
- In DynamoDB, a **conditional write** gives idempotency directly.
- Store the key and the result, so a repeat returns the original outcome.

> ON THE EXAM
>
> The timeout case is why this matters. The request may have succeeded and the response been lost, so a retry without an idempotency key charges the customer twice.

Connection pool exhaustion

- A pool of connections is finite. Under load, requests queue waiting for one.
- That surfaces as a **timeout**, not as a connection error, which misdirects the investigation.
- For Lambda and a relational database, this is module 10's problem: **RDS Proxy** or the Data API.
- A connection **not returned** to the pool leaks one each time, so it degrades gradually.
- The signature is errors that scale with concurrency and vanish when traffic drops.

> ON THE EXAM
>
> 'Fine in testing, fails in production' is usually this. A single-threaded test never exhausts a pool, so the fault only exists at concurrency.

Timeouts are layered

- A request passes through several timeouts, each set independently.
- Client, then API Gateway, then Lambda, then the SDK, then the downstream service.
- **An inner timeout longer than an outer one is useless.** The outer fires first.
- An SDK timeout of 60 seconds inside a 30-second function never takes effect.
- Each layer should time out **before** the layer outside it, leaving room to handle the failure.

> READ THIS TWICE
>
> This is the commonest timeout misconfiguration, and it is invisible until something is slow. The inner timeout was meant to produce a clean error and never gets the chance.

The timeout numbers

WHERE	LIMIT
AWS Lambda	15 minutes maximum, 3 seconds default
API Gateway integration	29 seconds. See module 11 for what has changed
Application Load Balancer idle	60 seconds by default, configurable
Step Functions Standard	1 year. Express, 5 minutes
SQS visibility timeout	12 hours maximum, 30 seconds default
SDK socket timeout	SDK-specific, and configurable per client

ON THE EXAM

The 3-second Lambda default and the 30-second SQS default are both shorter than people expect, and both cause failures that look like something else.

Timeout, cold start or throttle?

- A **timeout** shows Task timed out after N seconds in the logs, and Duration equals the limit.
- A **cold start** shows an Init Duration in the REPORT line, and only on some invocations.
- A **throttle** produces no function logs at all, and increments the Throttles metric.
- Throttling is the confusing one, because the function never ran and wrote nothing.
- So absence of logs is itself a diagnostic signal, not a logging problem.

> ON THE EXAM
>
> That last line is worth holding onto. When invocations are clearly failing and the log group is empty, look at Throttles before you look at the code.

Poison pills

- A message that always fails is retried until it expires, or forever on an ordered source.
- On a **Kinesis or DynamoDB Stream shard**, it blocks every message behind it.
- **maxRecordAge** and **maxRetryAttempts** on the event source mapping bound it.
- **bisectBatchOnFunctionError** splits a failing batch to isolate the bad record.
- For SQS, **maxReceiveCount** and a DLQ do the same job.

> ON THE EXAM
>
> The ordered-source case is the severe one: an SQS poison pill delays one message, while a Kinesis poison pill stops the entire shard until it is bounded.

Failing fast

- A **circuit breaker** stops calling a dependency that is consistently failing.
- After a threshold of failures it opens, rejecting calls immediately for a period.
- It then half-opens, allowing one probe through to test recovery.
- The SDK's **retry quota** is a simpler form of the same idea.
- A **fallback** returns degraded but useful data instead of an error.

> ON THE EXAM
>
> The argument is that during a dependency outage, retrying consumes your own capacity too. Failing fast preserves the parts of your system that still work.

Service quotas

- **Service Quotas** lists every limit, whether it is adjustable, and your current usage.
- Some quotas are **hard** and cannot be raised. API Gateway's burst is one.
- Adjustable quotas are raised through a request, which is not instant.
- A **CloudWatch alarm on quota utilisation** warns before you hit the ceiling.
- Quotas are usually **per account and Region**, so one workload can starve another.

> ON THE EXAM
>
> Alarming on quota utilisation is the underused part. Discovering a hard limit during an incident is far worse than discovering it at 80% on a Tuesday.

Where to look

- **Metrics first.** Errors, Throttles and Duration tell you the category in seconds.
- **Then the trace.** X-Ray shows which component and which call.
- **Then the logs** for that specific request, found by trace or correlation ID.
- Starting with logs is the slow path, because you do not yet know where to look.
- **CloudTrail** if the question is what changed, rather than what is failing.

> ON THE EXAM
>
> Metrics, trace, logs, in that order. Each step narrows the search, and jumping straight to logs means searching everything without knowing what for.

Symptom to cause

SYMPTOM	USUAL CAUSE
Failing invocations, empty log group	Throttling, or missing logs permission
Duration exactly equals the timeout	A timeout, not an application error
Intermittent slowness, only some requests	Cold starts, or a hot partition
Every request fails after a deployment	Configuration or permission, not capacity
Errors rise only under load	Throttling, or a connection pool exhausted
Duplicate records downstream	Retries without idempotency
One consumer stuck, others fine	A poison pill on an ordered shard

ON THE EXAM

Worth screenshotting. Most troubleshooting questions on this exam describe one of these rows, and the wrong answers are usually a different row's cause.

The failures that are not capacity

- **Permission failures** from module 15: AccessDenied, and no amount of retrying helps.
- Broad permissions still failing means an explicit deny, usually an SCP or a boundary.
- **Networking failures** in a VPC: no route, no NAT gateway, or a security group blocking.
- These typically present as a **connection timeout**, not a permission error.
- A VPC endpoint is needed to reach an AWS service privately without internet egress.

> ON THE EXAM
>
> The confusable pair: a permission failure returns AccessDenied quickly, while a networking failure hangs until the socket times out. The timing tells you which.

Retry storms

- A brief failure causes retries, which add load, which causes more failures.
- **Nested retries multiply**: three layers of three attempts is twenty seven requests.
- Jitter, retry quotas and circuit breakers all exist to break this loop.
- Retry at **one layer**, ideally the outermost one that can act on the failure.
- A queue in front of a struggling service converts a retry storm into a backlog.

> ON THE EXAM
>
> That last point is the architectural answer. A backlog is a problem you can watch drain; a retry storm is a problem that grows while you watch it.

Six scenarios. For each, decide first: is this capacity, configuration, or correctness?

> A DynamoDB table in on-demand mode serves an event ticketing application. During a popular release, writes fail with ProvisionedThroughputExceededException even though the table has no provisioned capacity configured.
>
> All writes during the spike use the event ID as the partition key, and one event accounts for almost all of them.

What is the cause?

A On-demand mode has a hard account-wide write limit that has been reached

B The SDK is retrying too aggressively and causing its own throttling

C A hot partition. All writes target one partition key, exceeding the per-partition limit

D The table needs a global secondary index to spread the writes

◆ PAUSE AND DECIDE

What is the cause?

A Account-wide limit

On-demand scales to very high throughput and this is nowhere near an account ceiling. It also would not correlate with one event ID.

B Aggressive SDK retries

Retries amplify throttling but cannot cause it from nothing. Something has to be exceeding a limit first.

C A hot partition

Per-partition limits apply regardless of capacity mode. All writes hitting one partition key exceed 1,000 WCUs for that partition while the table is barely loaded.

D Add a GSI

A GSI serves different query patterns. It does not redistribute writes to the base table, and it would add write cost.

THE SIGNAL

The clue is that on-demand mode does not remove per-partition limits. Module 7 made this point: capacity mode changes the table's ceiling, never the partition's.

> A Lambda function consumes an SQS queue and charges a customer's card for each message. Finance reports that a small number of customers were charged twice on the same day. The function has no errors in CloudWatch and the DLQ is empty.
>
> The payment provider's API occasionally takes longer than usual to respond.

What is the most likely cause and fix?

A. The charge is not idempotent. A timed-out call succeeded and was retried, so add an idempotency key

B. The DLQ is misconfigured and is silently replaying messages

C. SQS is delivering duplicates because the queue is not FIFO

D. The function timeout is too high and should be reduced

◆ PAUSE AND DECIDE

What is the most likely cause and fix?

A Not idempotent, add a key

A slow payment call times out on the client side after the charge succeeded. The retry charges again. An idempotency key makes the second call return the first result.

B DLQ replaying

A DLQ receives messages, it does not replay them. The scenario also states it is empty.

C Standard queue duplicates

The strongest distractor, and it is only half the story. At-least-once delivery is real, and the fix is still idempotency rather than FIFO, which would cripple throughput.

D Reduce the timeout

Would cause more timeouts and therefore more double charges, since the underlying operation still succeeds before the client gives up.

THE SIGNAL

The clue is the slow third-party API. A timeout is the one failure where you genuinely do not know whether the operation happened, which is exactly what idempotency exists for.

> An API Gateway REST API integrates with a Lambda function that calls a slow reporting database. Users report that some requests return 504 Gateway Timeout. The Lambda function's timeout is set to 60 seconds and its Duration metric shows successful invocations completing in up to 45 seconds.
>
> The function itself reports no errors.

What is happening?

A The Lambda function is being throttled and never runs

B The Lambda function is returning a malformed response

C The database connection is being refused

D API Gateway's 29-second integration timeout fires while the function is still running

◆ PAUSE AND DECIDE

What is happening?

A Throttling

Throttling returns 429, not 504. The Duration metric also shows the function running, so it clearly is being invoked.

B Malformed response

Module 11's fact: a bad proxy response produces 502, not 504. The two codes mean different things.

C Connection refused

Would produce a function error and appear in the logs. The scenario says the function reports no errors.

D The 29-second gateway timeout

Invocations run up to 45 seconds, well past the 29-second integration timeout. API Gateway gives up and returns 504 while the function carries on.

THE SIGNAL

The layered timeout problem from earlier. The function timeout of 60 seconds is longer than the gateway's 29, so the inner limit never takes effect.

> A fleet of 500 EC2 instances calls a downstream service. A brief 30-second outage of that service turned into a 20-minute one. Metrics show request volume to the service increased tenfold during the incident and stayed high after the underlying fault was fixed.
>
> Each instance retries failed calls five times with a fixed one-second delay.

What should be changed?

A. Increase the retry count so requests eventually succeed

B. Use exponential backoff with jitter, and add a retry quota or circuit breaker

C. Remove retries entirely so failures surface immediately

D. Add more instances to the fleet to spread the load

◆ PAUSE AND DECIDE

What should be changed?

A More retries

Directly worsens it. The tenfold volume increase is caused by retries, so adding more amplifies the storm that extended the outage.

B Backoff with jitter, plus a quota

Fixed delays synchronise 500 clients into waves. Jitter spreads them, backoff reduces the rate, and a quota or breaker stops retrying into a sustained failure.

C No retries

Overcorrects. Transient failures that would have self-corrected now become user-visible errors.

D More instances

Increases the number of clients generating retries. It makes the storm larger.

THE SIGNAL

'Fixed one-second delay' across 500 instances is the whole problem: every client retries at the same instant, five times, recreating the overload in waves.

> A Lambda function processes records from a Kinesis Data Stream. One shard has stopped making progress entirely, while the other shards continue normally. The iterator age for the stuck shard has been climbing for hours.
>
> The function throws an exception on one particular malformed record.

What should the team configure?

A. Increase the function's reserved concurrency so it can keep up

B. Add more shards to the stream to spread the load

C. Set maxRetryAttempts and maxRecordAge on the event source mapping, and enable bisectBatchOnFunctionError

D. Switch the stream to enhanced fan-out

◆ PAUSE AND DECIDE

What should the team configure?

A More concurrency

The shard is blocked, not slow. Kinesis processes one batch at a time per shard, so extra concurrency cannot help a shard that never advances.

B More shards

Redistributes future records and leaves the existing poison pill exactly where it is, still blocking its shard.

C Bound the retries and bisect the batch

maxRetryAttempts and maxRecordAge stop the infinite retry, and bisecting isolates the one bad record so the rest of the batch succeeds.

D Enhanced fan-out

Gives consumers dedicated throughput. It changes how records are delivered, not what happens when one always fails.

THE SIGNAL

'One shard stuck, others fine' is the poison pill signature. Ordering guarantees mean the shard cannot skip the failing record, so it retries forever until bounded.

> A newly deployed Lambda function fails on every invocation. The CloudWatch log group for the function exists but contains no log streams at all. The Errors metric is zero and the Invocations metric is zero, but the upstream caller reports failures.
>
> The function was working in the development account with an identical deployment.

Where should the team look first?

A Reserved concurrency set to zero, or an account concurrency limit, throttling every invocation

B A syntax error in the handler causing an initialisation failure

C The function timeout being too short for the workload

D A missing VPC endpoint preventing the function reaching DynamoDB

◆ PAUSE AND DECIDE

Where should the team look first?

A Concurrency at zero, or throttling

Invocations is zero and Errors is zero, so the function is not running at all. Reserved concurrency of zero throttles every call, which is exactly this signature.

B Syntax error

An initialisation failure still counts as an invocation and an error, and writes a log stream containing the stack trace. All three are absent.

C Timeout too short

A timeout increments Invocations and Errors and writes a log line. Nothing here is being recorded.

D Missing VPC endpoint

The function would run, fail to connect, and log a timeout. That produces invocations and errors, not silence.

THE SIGNAL

Invocations at zero is the decisive metric. Every other option requires the function to have run, and running always produces an invocation and a log stream.

What to carry forward

- **Retry 5xx, 429 and 408. Never retry other 4xx.** The permission will not appear.
- **Backoff without jitter still synchronises clients.** Jitter is what breaks the waves.
- **An inner timeout longer than an outer one never fires.** Layers must agree.
- Retries make duplicates inevitable, so idempotency is a requirement, not a refinement.
- **Invocations at zero means it never ran.** Look at throttling before looking at code.

> ON THE EXAM
>
> Next: performance and cost optimisation, which closes Domain 4 and the teaching content.

MODULE 28 · DOMAIN 4 - TROUBLESHOOTING AND OPTIMIZATION

Performance and Cost Optimisation

Why more memory is often cheaper, where a serverless bill actually goes, and the line item nobody predicted.

DOMAIN 4 of 4 EXAM WEIGHT 18% MODULE 28 of 34

admin@sthithapragna.com

On servers, speed costs money. In serverless, you pay for time, so faster is usually cheaper.

Memory and CPU in Lambda

- **Memory is the only performance setting.** CPU scales proportionally with it.
- At 128 MB a function gets a fraction of a vCPU. At **1,769 MB** it gets a full one.
- Network bandwidth scales with memory too, so uploads and downloads speed up.
- **You pay per GB-second**, so doubling memory doubles the rate and may halve the duration.
- **The cheapest setting is often not the lowest.** That is the counterintuitive part.

> READ THIS TWICE
>
> The arithmetic: 128 MB for 800 ms is 0.1 GB-seconds. 512 MB for 150 ms is 0.075 GB-seconds. Four times the memory, and it costs less.

Lambda Power Tuning

- A Step Functions state machine that runs your function at several memory settings.
- It plots duration and cost against memory, so the optimum is visible rather than guessed.
- Strategies: optimise for **cost**, for **speed**, or for a **balanced** weighting.
- The optimum is almost never where a team first set it, which is usually 128 MB by default.
- Re-run it after significant code changes. The curve moves.

> ON THE EXAM
>
> It is a Step Functions application you deploy from the Serverless Application Repository, which makes it the answer when a scenario asks how to determine the right memory setting.

Graviton

- Setting the architecture to **arm64** runs the function on AWS Graviton2.
- Duration charges are **20% lower**, and performance is up to 19% better.
- AWS quotes up to **34% better price performance** from the two combined.
- Most runtimes need no code change at all. Interpreted languages are the easiest.
- Native binaries and architecture-specific dependencies must be rebuilt for ARM.

> ON THE EXAM
>
> This is the rare change that costs nothing to make and reduces the bill immediately. The only real risk is a compiled dependency that has no ARM build.

Where a serverless bill actually goes

- **Lambda compute is often only 20 to 40% of a serverless bill.**
- **CloudWatch Logs** ingestion and retention are frequently larger than the compute.
- **NAT gateway** charges appear for any VPC-attached function reaching AWS services.
- **API Gateway** at 3.50 dollars per million requests can exceed the Lambda cost behind it.
- Data transfer sits under **EC2-Other** in Cost Explorer, which hides it from casual reading.

ON THE EXAM

Optimising Lambda memory on a bill where logs cost more than compute is effort in the wrong place. Look at the breakdown before choosing a lever.

Batching and payload size

- **BatchWriteItem** and **BatchGetItem** reduce DynamoDB request overhead.
- **SQS batch send and receive** handle up to 10 messages per API call.
- SQS bills per request in **64 KB chunks**, so a large message is several billable requests.
- A Lambda event source mapping with a larger **batch size** means fewer invocations.
- Larger batches raise the cost of a failure, since a whole batch retries together.

> ON THE EXAM
>
> The trade-off is real: bigger batches are cheaper per item and more expensive per failure, which is why partial batch responses matter.

Cold starts, revisited as cost

- Initialisation time is **billed** since August 2025, so a slow init costs money on every cold start.
- Smaller deployment packages initialise faster. Trim dependencies rather than adding memory.
- Instantiate SDK clients **outside the handler**, so they are reused across invocations.
- **SnapStart** removes most init time for supported runtimes, at no extra charge.
- **Provisioned concurrency** removes cold starts by paying for idle capacity.

> ON THE EXAM
>
> The ordering matters: trim the package, move client creation out of the handler, then consider SnapStart, and only then pay for provisioned concurrency.

The provisioned concurrency trade

- You pay for provisioned environments **whether or not they are used**.
- At steady high utilisation the per-invocation rate is lower, so it can be cheaper.
- At low or spiky utilisation you are paying for idle capacity, so it is dearer.
- **Scheduled scaling** provisions only during known busy hours, which fits predictable traffic.
- Buying it to fix occasional cold starts on a quiet function is the classic overspend.

> ON THE EXAM
>
> The question to ask is whether the traffic is predictable. Provisioned concurrency rewards predictability and punishes everything else.

DynamoDB cost levers

- **On-demand** for unpredictable traffic. **Provisioned with auto scaling** for steady traffic.
- Provisioned is substantially cheaper at consistent utilisation, and **reserved capacity** cheaper still.
- **Projections** matter: a KEYS_ONLY index costs far less to maintain than ALL.
- **TTL** deletes expired items automatically, and the deletions are free.
- Eventually consistent reads cost **half** what strongly consistent reads cost.

> ON THE EXAM
>
> TTL is the underused one. It removes both storage cost and the read cost of scanning past expired items, and it charges nothing for the deletions.

S3 cost levers

- **Lifecycle rules** transition to cheaper classes and expire objects on a schedule.
- **Intelligent-Tiering** moves objects automatically based on access, with no retrieval fees.
- **AbortIncompleteMultipartUpload** removes storage you are billed for and cannot see.
- **S3 Bucket Keys** cut SSE-KMS request costs by up to 99%.
- Expiring **noncurrent versions** is essential on any versioned bucket.

> ON THE EXAM
>
> Intelligent-Tiering is the answer when access patterns are unknown or changing. Lifecycle rules are better when you already know the pattern.

CloudWatch cost levers

- **Ingestion is around 0.50 dollars per GB**, and storage around 0.03 dollars per GB per month.
- **Log groups never expire by default**, so every log line ever written is still billed.
- Lambda's **application log level** filters before ingestion, so filtered lines cost nothing.
- Set a retention period on **every** log group, and export to S3 for long-term archival.
- High-cardinality **custom metric dimensions** multiply into thousands of billable metrics.

> ON THE EXAM
>
> Debug logging left on in production is the most common single line item people are surprised by, and filtering at the platform is free to configure.

The front door and the plumbing

- **HTTP APIs cost around 71% less** than REST APIs. Module 11 listed what you give up.
- An **Application Load Balancer** is cheaper still at high volume, with fewer features.
- **API Gateway caching** is billed hourly whether or not it gets hits.
- **EventBridge** costs more per million events than **SQS**. Direct S3 triggers are free.
- SQS long polling removes the empty receives you would otherwise pay for.

> ON THE EXAM
>
> The front door is often overlooked because it feels like plumbing. At high request volume it can cost more than the compute it fronts.

Data transfer

- Inbound data transfer is free. **Outbound to the internet is charged**, by the gigabyte.
- **Cross-Availability-Zone traffic is charged in both directions**, even inside one VPC.
- **Cross-Region traffic costs more.** Replication is a recurring charge, not a one-off.
- **CloudFront** reduces internet egress cost as well as latency.
- Keeping chatty components in the same Availability Zone removes a real line item.

> ON THE EXAM
>
> The cross-AZ charge surprises people because it is invisible in the architecture diagram. Two services talking constantly across zones pay for every byte, twice.

NAT gateways and VPC endpoints

- A NAT gateway costs roughly **32 dollars a month per Availability Zone**, plus per-GB processing.
- A VPC-attached function reaching S3 or DynamoDB routes through it by default.
- **Gateway endpoints** for S3 and DynamoDB are **free**, with no per-GB charge.
- **Interface endpoints** for other services cost roughly 7 dollars a month plus a small per-GB fee.
- If a function only calls AWS services, the NAT gateway can be removed entirely.

> ON THE EXAM
>
> The gateway endpoint being free is the detail to remember. A VPC-attached function talking to S3 through a NAT gateway is paying for something available at no cost.

Caching as a cost lever

- Every cache hit is a request the backing service never sees, and never bills for.
- **DAX** removes DynamoDB read cost as well as read latency.
- **CloudFront** removes origin requests and reduces egress charges.
- **API Gateway caching** removes backend invocations entirely for cached responses.
- A cache has its own standing cost, so it pays off only above a certain hit rate.

> ON THE EXAM
>
> The break-even is the honest part: a cache that costs 100 dollars a month must remove more than 100 dollars of backend calls, or it is a net loss.

The cost levers, in order of usual impact

LEVER	TYPICAL EFFECT
Set log retention on every group	Often the single largest saving, and free to do
Reduce log verbosity in production	Removes ingestion cost, not just storage
Tune Lambda memory	Frequently 30 to 40% off duration cost
Switch to arm64	20% off duration, one configuration change
Replace NAT gateway with VPC endpoints	Removes a fixed monthly charge entirely
Right-size or commit on steady compute	Savings Plans, reserved capacity, provisioned throughput
Cache what is read repeatedly	Removes requests, above a break-even hit rate

ON THE EXAM

Worth screenshotting. The first two are configuration changes with no engineering effort, which is why they belong at the top.

Right-sizing and commitments

- **Compute Optimizer** recommends instance and Lambda memory sizes from actual usage.
- **Savings Plans** discount compute for a one or three year commitment, including Lambda and Fargate.
- **Spot** for interruption-tolerant work, at a large discount.
- **Reserved capacity** exists for DynamoDB, and reserved nodes for ElastiCache.
- Commitments reward predictability. Committing to capacity you stop using is pure loss.

> ON THE EXAM
>
> Right-size before you commit. A Savings Plan on an oversized fleet locks in the wrong number for a year.

Seeing the cost

- **Cost Explorer** breaks spend down by service, tag, account and time.
- **AWS Budgets** alerts on forecast or actual spend crossing a threshold.
- **Cost Anomaly Detection** flags unusual spend automatically, without a threshold.
- **Trusted Advisor** surfaces idle and underused resources across the account.
- Cost data lags by hours, so none of these catches a runaway within minutes.

> ON THE EXAM
>
> That lag is why a CloudWatch alarm on the underlying metric beats a budget alert for catching a runaway loop. Budgets tell you afterwards.

Attributing the cost

- **Cost allocation tags** must be activated in the billing console before they appear in reports.
- **Activation is not retroactive.** Tags only apply to costs incurred afterwards.
- Tag by team, environment and application to attribute shared-account spend.
- **Tag policies** in Organizations enforce consistent keys and values.
- An IAM condition on **aws:RequestTag** can require tags at creation time.

> ON THE EXAM
>
> The non-retroactive point matters: activating tags today does not explain last month's bill. Tagging has to be in place before the spend happens.

When performance and cost conflict

- **Provisioned concurrency** buys latency with money spent on idle capacity.
- **DAX and ElastiCache** buy latency with a standing cluster cost.
- **Multi-Region** buys resilience and latency with duplicated everything.
- **High-resolution metrics and 100% tracing** buy visibility with ingestion cost.
- In each case the question is whether the latency or visibility is worth the standing charge.

> ON THE EXAM
>
> Not everything aligns. Where the two genuinely conflict, the exam expects you to notice which one the scenario prioritises rather than assuming cheapest wins.

What the exam actually asks

- **Most cost-effective** means the cheapest option that still meets every stated requirement.
- It does **not** mean cheapest overall. An option that drops a requirement is wrong.
- **Least operational overhead** means the most managed option, even if it costs more.
- **Minimal code change** rules out rewrites, however elegant.
- Two options often both work. The qualifier in the question decides between them.

> ON THE EXAM
>
> These phrases are not decoration. They are the tiebreaker, and reading past them is how you pick a defensible answer that is not the intended one.

Ask first: is the money going to compute, to logs, to networking, or to requests?

> An image processing Lambda function is configured with 128 MB of memory and takes about 8 seconds per image. The team wants it faster, and has been told to keep costs down, so they have avoided raising the memory.
>
> The function is CPU-bound, doing image resizing in memory.

What should the team do?

A Keep 128 MB and split the work across several functions invoked in parallel

B Run Lambda Power Tuning, and expect a higher memory setting to be both faster and cheaper

C Enable provisioned concurrency so the function is always warm

D Move the workload to a small EC2 instance running continuously

◆ PAUSE AND DECIDE

What should the team do?

A Split across parallel functions

Each function still gets a fraction of a vCPU, so each piece is still slow. It adds orchestration to avoid a configuration change.

B Power Tuning, expect higher memory

CPU scales with memory, so a CPU-bound function at 128 MB is severely constrained. Finishing several times faster usually costs less in GB-seconds.

C Provisioned concurrency

Removes cold start latency. The 8 seconds is execution time, not initialisation, so this changes nothing and adds cost.

D Move to EC2

Trades per-invocation billing for a continuously running instance. For intermittent image processing that is almost certainly dearer.

THE SIGNAL

The premise is the error: avoiding memory to save money. For a CPU-bound function the opposite is usually true, and Power Tuning is how you prove it.

> A set of Lambda functions is attached to a VPC to reach a private RDS database. They also read and write to Amazon S3 and DynamoDB. The monthly bill shows a substantial NAT gateway charge, with high data processing costs.
>
> The functions do not call anything outside AWS.

How should the networking cost be reduced?

A Remove the functions from the VPC entirely

B Move the NAT gateway to a single Availability Zone

C Replace the NAT gateway with a NAT instance on a small EC2 host

D Add gateway VPC endpoints for S3 and DynamoDB, and remove the NAT gateway

◆ PAUSE AND DECIDE

How should the networking cost be reduced?

A Leave the VPC

The functions need a private RDS database, which requires VPC attachment. This removes the cost by breaking the requirement.

B Single-AZ NAT gateway

Halves or thirds the fixed charge and keeps all the per-GB processing. It also creates an availability single point of failure.

C NAT instance

Cheaper per hour and now something you patch, scale and monitor. It also still processes every byte.

D Gateway endpoints, drop the NAT

Gateway endpoints for S3 and DynamoDB are free with no per-GB charge, and with no external calls the NAT gateway is unnecessary entirely.

THE SIGNAL

'Do not call anything outside AWS' is the clause that allows removing the NAT gateway altogether, rather than merely making it cheaper.

A company's AWS bill shows CloudWatch costing more than all their Lambda compute combined. Investigation finds debug-level logging enabled in production across 300 functions, and no log group has a retention period set.

The team wants the cost down without losing the ability to debug production issues when they arise.

What should they do?

A Set retention on every log group, and set the Lambda application log level to INFO so debug lines are never ingested

B Delete the existing log groups and let them be recreated

C Move all log groups to the Infrequent Access class

D Remove the debug logging statements from the code and redeploy

◆ PAUSE AND DECIDE

What should they do?

A Retention plus application log level

Retention removes accumulated storage; the log level stops debug lines being ingested at all. Both are configuration, and the level can be raised again when debugging.

B Delete the log groups

Frees storage once and changes nothing about ingestion, so the cost returns. It also destroys the history.

C Infrequent Access

Reduces the ingestion rate and loses metric filters, subscription filters and alarms. It also cannot be changed back.

D Remove the statements

Requires redeploying 300 functions, and permanently removes the debug logging the team explicitly wants available.

THE SIGNAL

Option D is the near miss. It fixes the cost and destroys the capability the scenario says must be retained; the log level does both without a deployment.

> A DynamoDB table serving a retail catalogue runs in provisioned mode with capacity set for the Black Friday peak. For most of the year utilisation sits below 10%, and the team pays for the peak capacity every month.
>
> Traffic is highly predictable day to day, with two known annual spikes.

What is the best change?

A. Switch permanently to on-demand mode

B. Add a DAX cluster to absorb the peak reads

C. Enable provisioned auto scaling for the daily pattern, and buy reserved capacity for the steady baseline

D. Reduce provisioned capacity to the average and accept throttling at the peak

◆ PAUSE AND DECIDE

What is the best change?

A On-demand permanently

The strongest distractor. On-demand suits unpredictable traffic, and this traffic is described as highly predictable, where provisioned is substantially cheaper.

B Add DAX

Reduces read capacity needs and adds a standing cluster cost. It does nothing about paying for peak write capacity all year.

C Auto scaling plus reserved capacity

Auto scaling follows the predictable daily pattern, and reserved capacity discounts the baseline that is always there. It matches both facts in the scenario.

D Average capacity, accept throttling

Saves money by failing customers at exactly the moment revenue is highest. Throttling on Black Friday is not an acceptable trade.

THE SIGNAL

'Highly predictable' is the deciding phrase. On-demand is right for unpredictable traffic; predictability is precisely what provisioned mode and commitments reward.

What to carry forward

- **In serverless, faster is usually cheaper.** More memory can reduce the bill.
- **Lambda compute is often a minority of a serverless bill.** Check logs and networking first.
- **Gateway endpoints for S3 and DynamoDB are free.** A NAT gateway is not.
- arm64 is 20% off duration for one configuration change and no code change.
- On-demand for unpredictable, provisioned and committed for predictable. Predictability decides.

> ON THE EXAM
>
> That closes the teaching content. Next: confusion pairs, how DVA-C02 questions are written, and the revision gap.

MODULE 29 · EXAM STRATEGY - ALL DOMAINS

Confusion Pairs

Forty-eight pairs across this course, sorted into the five patterns that generate almost all of them.

TYPE Synthesis MODULE 29 of 34 PAIRS 48 built

admin@sthithapragna.com

Most confusion pairs are the same few shapes wearing different service names. Learn the shapes, not the list.

The five patterns

PATTERN	THE QUESTION THAT RESOLVES IT
Grants against ceilings	Does this give permission, or only cap it?
Push against pull	Who initiates: the source, or the consumer?
Identifier against attribute	Do I need the name, or a property of the thing?
Capability against cost	Which feature does the scenario actually require?
Same word, two meanings	Which layer is this word being used at?

READ THIS TWICE

Five questions. Every pair in this course falls into at least one of them, and several fall into two.

Pattern one: grants against ceilings

- **Identity and resource policies grant.** They are the only two that do.
- **Permissions boundaries, SCPs and session policies cap.** None of them grants anything.
- **Reserved concurrency caps** a function's maximum. It also guarantees that much.
- **Provisioned concurrency allocates** warm environments. It is not a limit.
- The tell: if adding it alone changes nothing, it is a ceiling, not a grant.

> ON THE EXAM
>
> This pattern explains why 'we gave them administrator access and it still fails' is always a ceiling. Adding grants cannot defeat something that caps.

Pattern one, across the course

GRANTS	CAPS
Identity policy, resource policy	Permissions boundary, SCP, session policy
Provisioned concurrency	Reserved concurrency
Provisioned throughput on a table	Per-partition limits, which no mode removes
A KMS key policy statement	A grant constraint, or an encryption context condition
A usage plan's rate	Account-level API Gateway throttling, always applied

ON THE EXAM

Note the third row. Raising provisioned throughput never lifts a per-partition limit, which is why hot partitions throttle a table that looks under-used.

Pattern two: push against pull

- **Event source mappings poll.** Lambda reads from SQS, Kinesis and DynamoDB Streams.
- **Event notifications push.** S3, SNS and EventBridge invoke Lambda directly.
- That decides where retries live, and therefore what a DLQ is attached to.
- Polling means the consumer controls the rate. Pushing means the source does.
- Same shape: SQS consumers pull, SNS subscribers are pushed to.

> ON THE EXAM
>
> This pattern determines whether a failure is retried by Lambda or by the source, which is why destinations work for async and not for event source mappings.

Pattern two, across the course

PULL	PUSH
SQS, Kinesis, DynamoDB Streams	S3 events, SNS, EventBridge
Event source mapping, with a batch	Asynchronous invocation, one event
Retries handled by the mapping	Retries handled by Lambda, twice, then a DLQ
maxReceiveCount and a DLQ on the queue	Destinations, on success or on failure
The consumer sets the pace	The source sets the pace, so throttling matters

ON THE EXAM

The fourth row is the practical one. Destinations require asynchronous invocation, which is why they silently do nothing on an SQS-triggered function.

Pattern three: identifier against attribute

- **Ref returns the physical ID. GetAtt returns a named attribute.**
- **An annotation is indexed** and searchable. **Metadata is stored** and is not.
- An IAM role's Ref gives its **name**. Most places need its **ARN**, which is GetAtt.
- An S3 **bucket ARN** and its **object ARN** are different strings. Listing needs one, reading the other.
- The tell: if the scenario needs to **find** or **reference** something, ask which string it needs.

> ON THE EXAM
>
> Every pair here fails the same way: the value is present, has the right shape, and is the wrong string. The error usually names what it expected.

Pattern three, in reverse

- Not every case wants the attribute. Sometimes the identifier is correct.
- A **bucket ARN** is right for ListBucket. The **object ARN** is right for GetObject.
- A **queue URL** is what SendMessage takes. The **queue ARN** is what a policy names.
- An **alias ARN** is right for a client. A **version ARN** is right for a deliberate pin.
- So the question is never which is better, only which one this operation expects.

> ON THE EXAM
>
> The pattern is not 'always use GetAtt'. It is 'know which string the operation takes', and the documentation states it for every action.

Pattern three, across the course

THE IDENTIFIER	THE ATTRIBUTE YOU USUALLY WANT
Ref on an IAM role gives the name	GetAtt Arn gives the ARN
Ref on an RDS instance gives an ID	GetAtt Endpoint.Address gives the hostname
Ref on an SQS queue gives the URL	GetAtt Arn gives the ARN
Ref on an SNS topic gives the ARN	Already the ARN. The inconsistency the exam likes
X-Ray metadata is stored	An annotation is indexed and filterable
A version ARN never moves	An alias ARN follows your deployments

ON THE EXAM

The SNS and SQS rows together are a favourite: Ref gives a topic ARN but a queue URL. There is no principle behind it, so it has to be remembered.

Pattern four: capability against cost

- **REST or HTTP API** turns on six features, not on preference.
- **Secrets Manager or Parameter Store** turns on rotation, sharing and replication.
- **Standard or Express** turns on five minutes and exactly-once.
- **Provisioned or on-demand** turns on predictability.
- In each pair the cheaper option is correct **unless** a named capability forces the other.

> ON THE EXAM
>
> The reasoning direction matters. Do not ask which is better; ask whether the scenario names a capability only one of them has. If not, the cheaper one wins.

Pattern four: what forces the dearer option

PAIR	THE CAPABILITY THAT DECIDES
REST or HTTP API	API keys, usage plans, caching, validation, mapping templates, WAF
Secrets Manager or Parameter Store	Automatic rotation, cross-account, cross-Region
Standard or Express workflow	Over 5 minutes, exactly-once, or the callback pattern
Provisioned or on-demand capacity	Predictable traffic makes provisioned cheaper
DAX or ElastiCache	DAX only fronts DynamoDB, with no code change
KMS or CloudHSM	Single-tenant hardware, or a PKCS#11 interface

ON THE EXAM

Memorising the middle column is worth more than memorising the pairs. The capability is what a question names; the pair is only the two options offered.

Pattern five: same word, two meanings

- **Canary** is a traffic-shifting deployment, and also a CloudWatch Synthetics script.
- **Version** is a Lambda snapshot, an S3 object version, and a CloudFormation template version.
- **Alias** points at a Lambda version, and also names a KMS key.
- **Grant** is a KMS delegation, and also an ordinary English verb in policy documents.
- **Execution role** is Lambda's identity, and an **ECS task execution role** is something else entirely.

> ON THE EXAM
>
> These are the ones that catch people who know the material. The word is familiar, the meaning is from a different module, and the answer looks obviously right.

The near-synonyms

- **Error**, **fault** and **throttle** in X-Ray mean 4xx, 5xx and 429 specifically.
- **Authentication** proves identity. **Authorisation** decides what is permitted.
- **Reserved** and **provisioned** are opposites in Lambda and near-synonyms elsewhere.
- **Quota**, **limit** and **throttle**: the first two are the ceiling, the third is hitting it.
- Precision here is cheap to acquire and reliably worth a mark or two.

> ON THE EXAM
>
> The reserved and provisioned pair is the nastiest, because the words behave one way in Lambda and the opposite way in DynamoDB and EC2.

Pattern five, in full

WORD	THE TWO MEANINGS
Canary	Traffic-shifting deployment, or a Synthetics availability script
Execution role	Lambda's own identity, or ECS's role for starting a container
Alias	A movable pointer to a Lambda version, or a friendly name for a KMS key
Endpoint	A service URL, a VPC endpoint, or an RDS hostname
Policy	IAM policy, bucket policy, key policy, stack policy, scaling policy
Destination	Lambda's async destination, or a CloudWatch Logs cross-account destination
Stage	An API Gateway stage, or a CodePipeline stage

ON THE EXAM

When a familiar word appears in an unfamiliar context, slow down. The meaning is set by the service, not by what you learned first.

The pairs that blur between modules

THESE LOOK SIMILAR	BECAUSE THEY ARE AT DIFFERENT LAYERS
API Gateway canary and Lambda alias weights	The gateway shifts between deployments; the alias between versions
Beanstalk immutable and CodeDeploy blue/green	Both replace rather than update. Different services, same idea
CloudFormation change set and cdk diff	Both preview. One is the service, one is the tool
Step Functions Retry and SDK retries	One is workflow configuration, one is inside the API client
SQS DLQ and Lambda DLQ	One belongs to the queue, one to the function
Cognito authoriser and Lambda authoriser	One validates a token; one runs your code

ON THE EXAM

Every row is two correct mechanisms operating at different layers. A question naming one layer is never answered by the other.

The pairs decided by one number

- **15 minutes:** Lambda's ceiling. Past it, Fargate, ECS or Step Functions.
- **29 seconds:** API Gateway's integration timeout, and the reason for 504.
- **5 GB:** the S3 single-PUT limit, above which multipart is mandatory.
- **4 KB:** the KMS Encrypt limit, and why envelope encryption exists.
- **400 KB:** a DynamoDB item, and why large objects go to S3 with a pointer.

> ON THE EXAM
>
> Five numbers that each decide an entire class of question. If a scenario states a size or a duration, check it against this list first.

The numbers that separate a pair

NUMBER	WHAT IT SEPARATES
256 KB	A Step Functions state payload, and an SQS or SNS message
6 MB	A Lambda synchronous response, and its asynchronous payload
10 MB	An API Gateway payload, above which streaming or S3 is needed
1 MB	A DynamoDB Query or Scan page, and a Kinesis record batch
300 seconds	API Gateway's raised timeout ceiling, against the 29-second default
12 hours	An SQS visibility timeout maximum, and an STS session maximum

ON THE EXAM

Several of these numbers appear in more than one service, which is exactly why a question can state a size and expect you to know which limit it is testing.

The pairs decided by direction

- **Identity policy:** what my principals may do. **Resource policy:** who may touch my resource.
- **Trust policy:** who may assume this role. **Permissions policy:** what the role may then do.
- **X-Ray:** requests flowing through your services. **CloudTrail:** API calls changing your account.
- **ID token:** who the user is. **Access token:** what they may do.
- The tell: ask whether the scenario is about the caller or about the thing being called.

> ON THE EXAM
>
> Direction pairs are the easiest to resolve once noticed and the easiest to answer wrongly at speed, because both options are real and both sound relevant.

ALL DOMAINS / AND A CLAUSE DECIDES

The pairs where both are right

- Several pairs have **no wrong answer in general**, only a wrong answer for this scenario.
- RDS Proxy and the Data API both solve Lambda's connection problem, differently.
- SAM and the CDK both produce CloudFormation. So does CloudFormation.
- Redis and Memcached both cache. Fargate and EC2 both run containers.
- In these, **find the disqualifying clause** rather than comparing the options.

> ON THE EXAM
>
> Modules 10 and 14 built this habit deliberately. If you cannot name a clause that eliminates your second choice, you have not found the real constraint yet.

The pairs most often got wrong

PAIR	THE SINGLE FACT THAT RESOLVES IT
Task role or task execution role	Failure before the container starts is the execution role
Ref or GetAtt	Need an ARN or an endpoint? GetAtt
Metric filter or subscription filter	A number, or the events themselves?
Annotations or metadata	Only annotations are indexed
User pool or identity pool	Calling an AWS service directly needs both
Reserved or provisioned concurrency	Reserved is a limit and costs nothing
502 or 504	Answered wrongly, or did not answer in time

ON THE EXAM

Worth screenshotting. Seven pairs, seven single facts, and between them a meaningful share of the marks the exam puts on this kind of distinction.

How to use a pair under pressure

- **Name the pattern first.** Grant or ceiling, push or pull, identifier or attribute?
- **Then find the clause** in the scenario that only one option satisfies.
- If both still fit, you have missed a clause. Reread rather than guessing.
- Check the qualifier: most cost-effective, least operational overhead, minimal code change.
- Eliminate options belonging to a different **layer** or a different **platform**.

> ON THE EXAM
>
> That last step is fast and reliable. ValidateService in a Lambda question, or a queue URL where an ARN is needed, are eliminations you can make without deep thought.

Four scenarios, each built on a pair. Name the pattern before you name the answer.

admin@sthithapragna.com

> A media company distributes large video files to paying subscribers worldwide, and separately lets those subscribers upload their own footage. Both must be restricted to authenticated users, and the bucket must stay private.
>
> Downloads must remain available for 30 days after purchase. Uploads happen immediately after the user selects a file.

What should be used for each direction?

A. CloudFront signed URLs for downloads, and S3 presigned URLs for uploads

B. S3 presigned URLs for both directions

C. CloudFront signed URLs for both directions

D. A public bucket with a bucket policy restricting access by referer header

◆ PAUSE AND DECIDE

What should be used for each direction?

A CloudFront down, presigned up

Downloads need 30 days and global distribution, which only CloudFront gives. Uploads must be presigned, because CloudFront cannot sign an upload at all.

B Presigned for both

Presigned URLs cap at 7 days with IAM user credentials, and far less from a role. 30 days is impossible, and there is no edge caching.

C CloudFront for both

CloudFront signed URLs are download only. There is no upload equivalent, so this cannot be built as described.

D Public bucket, referer check

Contradicts 'the bucket must stay private', and a referer header is trivially forged. Making a bucket public is essentially always wrong.

THE SIGNAL

A capability pair with a direction twist. The two requirements point at different services, and the giveaway is that only one of the pair can sign an upload at all.

An account runs 40 Lambda functions sharing a 1,000 concurrent execution limit. A batch reporting function occasionally scales to hundreds of concurrent executions and starves the customer-facing checkout function, which then throttles.

The checkout function must always have capacity available, and the batch job may run slower.

What should be configured?

A Provisioned concurrency on the checkout function

B Reserved concurrency on the batch function, capping it well below the account limit

C Request an account concurrency increase and change nothing else

D Provisioned concurrency on the batch function so it runs faster and finishes sooner

◆ PAUSE AND DECIDE

What should be configured?

A. Provisioned concurrency on checkout

The near miss. It reserves warm environments for checkout and costs money continuously, and it does not stop the batch job consuming the shared pool.

B. Reserved concurrency on the batch function

Reserved concurrency caps the batch job, so it can never consume more than its share. It costs nothing, and the batch simply runs slower.

C. Raise the account limit

Buys headroom until the batch job scales further. It does not prevent one function starving another, which is the actual problem.

D. Provisioned concurrency on the batch

Makes the batch function start faster, which makes it consume the shared pool sooner. It worsens the problem.

THE SIGNAL

A grant-or-ceiling pair. The requirement is to cap the noisy neighbour, and only reserved concurrency caps. Provisioned concurrency allocates, which is the opposite.

> A compliance team must retain every application log line for seven years in a searchable archive outside CloudWatch. Separately, the operations team wants a page when the phrase FATAL appears more than five times in ten minutes.
>
> Both requirements apply to the same log group.

What should be configured?

A Two metric filters, one counting FATAL and one counting all lines

B A subscription filter for both, with a Lambda function that counts and alerts

C A subscription filter to Firehose for the archive, and a metric filter with an alarm for FATAL

D Set retention to seven years and create a CloudWatch alarm on the log group

◆ PAUSE AND DECIDE

What should be configured?

A Two metric filters

Metric filters produce counts. A count cannot be archived and searched, so the compliance requirement is unmet.

B Subscription filter for both

Delivers the events, then rebuilds counting and alerting in a function that a metric filter and alarm already do for nothing.

C Subscription for archive, metric filter for the alarm

Two requirements, two mechanisms. The subscription filter delivers events for archival; the metric filter produces the number the alarm needs.

D Retention plus a log group alarm

You cannot alarm on a log group directly, only on a metric derived from it. Seven years in CloudWatch is also expensive and not outside CloudWatch.

THE SIGNAL

The pair does not resolve to one option. Two requirements need two mechanisms, and recognising that is what the question tests.

A mobile app signs users in with a Cognito user pool. Its backend API, behind API Gateway with a Cognito authoriser, needs to know the caller's identity and their group membership. The developer currently sends the ID token in the Authorization header.

A security review objects that user attributes including email and date of birth are being transmitted to a service that does not need them.

What should change?

A Move to an identity pool, so the API receives AWS credentials instead

B Remove the attributes from the user pool schema

C Encrypt the ID token before sending it

D Send the access token instead, which carries scopes and cognito:groups but not user attributes

◆ PAUSE AND DECIDE

What should change?

A Move to an identity pool

Identity pools issue AWS credentials for calling AWS services directly. The app calls its own API, so a user pool alone is correct.

B Remove the attributes

The application genuinely needs email and date of birth. Deleting them from the pool solves the transmission by destroying the data.

C Encrypt the token

It already travels over TLS, and the objection is that the API receives the attributes at all, not that they are intercepted.

D Send the access token

The access token carries scopes and cognito:groups, which is exactly what the API needs, without the user attributes it does not.

THE SIGNAL

A direction pair. The ID token says who the user is, for your application; the access token says what they may do, for an API. Sending the wrong one is the classic mistake.

What to carry forward

- **Five patterns generate almost every pair.** Name the pattern before the answer.
- **Grants against ceilings** explains why more permissions sometimes change nothing.
- **Push against pull** decides where retries live and what a DLQ attaches to.
- For capability pairs, the cheaper option wins unless a named capability forces the other.
- Some scenarios use **both** halves of a pair. Two requirements can need two mechanisms.

> ON THE EXAM
>
> Next: how DVA-C02 questions are written, which turns these habits into a reading technique for the exam itself.

MODULE 30 · EXAM STRATEGY - ALL DOMAINS

How DVA-C02 Questions Are Written

The anatomy of a question, the six kinds of wrong answer, and the clause that decides.

TYPE Exam technique MODULE 30 of 34 SCORED 50 of 65

admin@sthithapragna.com

A hard question is rarely testing a hard fact. It is testing whether you noticed **which** fact the scenario made relevant.

The mechanics

	VALUE
Questions	65 total. 50 scored, 15 unscored and not identified
Duration	130 minutes. Exactly 2 minutes per question
Score	Scaled 100 to 1,000. Pass at 720
Scoring model	Compensatory. No per-domain pass mark
Guessing	No penalty. Never leave a question blank
Domains	Development 32%, Security 26%, Deployment 24%, Troubleshooting 18%
ESL	An extra 30 minutes, requested before you schedule

ON THE EXAM

The 15 unscored questions are the reason a question that feels unreasonably obscure may simply not count. Answer it, and move on without letting it unsettle you.

What compensatory scoring means

- You need **720 overall**. There is no minimum in any individual domain.
- So a weak domain can be carried by strong ones. Nothing is a knockout.
- But the weights are not equal. **Development is 32%** and Troubleshooting is 18%.
- Improving your weakest large domain is worth more than perfecting a small one.
- The scaled score is **not a percentage**. 720 does not mean 72% of questions correct.

> ON THE EXAM
>
> The last line matters for your expectations. Candidates commonly report needing roughly 70 to 75% correct, but the mapping is not published and varies by exam form.

The two question types

- **Multiple choice:** one correct answer, three distractors. The majority of the exam.
- **Multiple response:** two or more correct, usually from five options.
- The question **states how many to choose**. Choose TWO means exactly two.
- **There is no partial credit.** Two of three correct scores zero, exactly as zero of three would.
- So on a multiple response question, treat each option as its own true-or-false decision.

> ON THE EXAM
>
> The no-partial-credit rule changes the strategy. On a Choose TWO you need both right, so it is worth more time than a single-answer question, not less.

The anatomy of a question

- **The setting:** what the system is and does. Often the longest part, and often the least useful.
- **The constraints:** what must be true. This is where the deciding clause hides.
- **The ask:** the actual question, usually one sentence at the end.
- **The qualifier:** most cost-effective, least operational overhead, and so on.
- The setting is padding as often as it is signal. The constraints rarely are.

> ON THE EXAM
>
> Recognising these four parts is what lets you read a long scenario quickly: you are scanning for constraints and the qualifier, not absorbing the whole story.

ALL DOMAINS / NOT TOP TO BOTTOM

Reading order

- **Read the ask first**, at the bottom. It tells you what to look for.
- **Then read the scenario**, now knowing which details matter.
- **Then form an answer** before looking at the options.
- Only then read the options, checking whether your answer is among them.
- Reading options first anchors you. Every one will then look partly plausible.

> READ THIS TWICE
>
> Forming an answer before reading the options is the single highest-value habit here. It converts four plausible options into one match and three rejections.

The qualifiers, and what each selects

THE QUALIFIER	IT SELECTS
MOST cost-effective	The cheapest option that still meets every stated requirement
LEAST operational overhead	The most managed option, even where it costs more
MINIMAL code change	Configuration over rewriting, however elegant the rewrite
MOST secure	Least privilege, no long-lived credentials, encryption by default
MOST highly available	Multi-AZ or multi-Region, accepting the cost
MOST likely cause	Diagnosis. The other options may be possible but not probable
BEST practice	The AWS-idiomatic answer, not the one you would build

ON THE EXAM

Worth screenshotting. When two options both work, the qualifier is the tiebreaker, and reading past it is how you choose a defensible answer that is not the intended one.

The deciding clause

- Most scenarios contain a sentence that **eliminates all but one option**.
- It is often short, and rarely in the first paragraph.
- Examples: cannot modify the code, must not reduce capacity, users must not reset passwords.
- A negative statement is frequently the clue. **Versioning is not enabled** exists to rule an option out.
- If you cannot name the clause, you have probably not found the real constraint.

> ON THE EXAM
>
> This course used that construction deliberately in every module. When a scenario states something apparently incidental, ask which option it was written to eliminate.

How this course was built

- Every worked scenario in this course used the same construction deliberately.
- A setting, constraints, an ask, and one clause that eliminates the strongest distractor.
- Each solution named that clause explicitly, under **The signal**.
- Several modules built a **near miss** option: correct engineering, wrong for the scenario.
- That was practice for this, not padding. The habit is what transfers.

> ON THE EXAM
>
> If you have been naming the deciding clause before checking the answer, you have already been doing the thing this module describes.

The six kinds of wrong answer

TYPE	WHAT IT LOOKS LIKE
Right service, wrong feature	DAX for an RDS cache, or a GSI to spread writes
Right for a different scenario	A perfectly good answer to a question that was not asked
Solves it by dropping a requirement	Make the bucket public, or remove the audit trail
Works, but ignores the qualifier	Correct engineering, at ten times the cost or effort
Belongs to another platform	ValidateService on Lambda, or active tracing on ECS
Formerly correct	The pre-2020 answer, still plausible if you learned it once

ON THE EXAM

The fourth type is the most dangerous, because nothing about it is factually wrong. It loses only against the qualifier you were meant to read.

The three easy eliminations

- **Wrong platform.** An EC2-only hook in a Lambda question is gone immediately.
- **Drops a requirement.** Making a bucket public, or abandoning the audit trail.
- **Rebuilds a managed feature by hand.** A Lambda function counting what a metric filter counts.
- These three take seconds and typically remove one or two options.
- That converts a four-way guess into a two-way decision, which is worth real marks.

> ON THE EXAM
>
> Elimination is not a substitute for knowing the answer, and it is what rescues the questions where you do not. Two options at 50% beats four at 25%.

When two options both work

- This is normal, not a sign you have misread. Distractors are meant to be defensible.
- **Check the qualifier first.** It exists precisely for this case.
- **Then look for a constraint** that one option violates and the other does not.
- Prefer the **managed** option, the one with less code, and the one AWS documents.
- Prefer the simpler option. A complex answer is rarely the intended one at associate level.

> ON THE EXAM
>
> The AWS-idiomatic tiebreaker is genuinely predictive: managed over self-built, configuration over code, loosely coupled over integrated.

Weak signals worth knowing

- Options containing **always** or **never** are more often wrong than right.
- An option naming a service that appears nowhere in the scenario is often filler.
- **The longest option is not reliably correct.** That heuristic does not survive contact with AWS.
- Neither is any pattern in the letter. Answer positions are randomised.
- Use these only to break a genuine tie, never in place of reasoning.

> ON THE EXAM
>
> Stated honestly because the internet repeats the length heuristic constantly. It is not reliable, and building a habit on it will cost you marks.

Time management

- **130 minutes, 65 questions, exactly 2 minutes each.** That is the whole budget.
- **At 2.5 minutes, flag it and move on.** A long question is worth the same as a short one.
- Check the clock at question 20 and question 40, not constantly.
- Aim to reach the end with 15 to 20 minutes left for flagged questions.
- Running out of time with questions unanswered is the avoidable way to fail.

> ON THE EXAM
>
> Every question is worth the same. Spending six minutes on a hard one costs you two easy ones you never reached, which is a net loss of marks.

Flagging and reviewing

- **Always answer before flagging.** A flagged blank becomes a blank if time runs out.
- Flag two kinds: ones you could not decide, and ones you rushed.
- Later questions sometimes remind you of a fact an earlier one needed.
- **Change an answer only for a reason you can name.** Not because it feels wrong now.
- Reviewing everything is not possible in the time. Review the flags.

> ON THE EXAM
>
> The named-reason rule matters. Second-guessing on a feeling changes correct answers to incorrect ones more often than the reverse.

When you genuinely do not know

- **Eliminate the wrong platform and the requirement-dropping options.** Usually one or two go.
- **Apply the qualifier** to whatever remains.
- Prefer managed over self-built, and configuration over code.
- Pick one, flag it, move on. Do not spend a third minute.
- It may be one of the 15 unscored questions. You will never know which.

> ON THE EXAM
>
> That last point is worth internalising before exam day. An unreasonably obscure question may simply not count, so it deserves no more than its two minutes.

What the exam does not do

- **It does not require memorised pricing.** Relative cost, yes. Dollar figures, no.
- It does not require writing code. Snippets appear, usually Python with boto3.
- It does not test CLI flags or console click paths.
- It does not use trick questions. Ambiguity is a badly written question, not a trap.
- It does expect **limits and defaults**: 15 minutes, 29 seconds, 400 KB, 5 GB.

> ON THE EXAM
>
> The last line is where revision effort pays. Numbers decide questions outright, and there are perhaps thirty that matter across the whole exam.

These four are different: each shows you a question and asks how it works.

> A question describes a team running scheduled reports on EC2, and asks for the MOST cost-effective way to run them, given they take 40 minutes and run twice a day. Every option would technically work.
>
> One option is a Fargate task on a schedule. One is a Lambda function. One is a persistent EC2 instance. One is an EC2 instance started and stopped by a script.

What is the fastest correct way to resolve this question?

A Compare the hourly rates of Fargate and EC2 and pick the cheaper

B Eliminate Lambda on the 15-minute limit, then apply the qualifier to what remains

C Choose the longest option, since detailed options are usually correct

D Choose the persistent EC2 instance, as it has the lowest per-hour price

◆ PAUSE AND DECIDE

What is the fastest correct way to resolve this question?

A **Compare hourly rates**

The exam does not require memorised pricing, and it would not eliminate Lambda, which fails on a hard limit rather than on price.

B **Eliminate on the limit, then qualify**

40 minutes exceeds Lambda's 15-minute ceiling, so that option is gone on a fact. The qualifier then chooses between the remaining three on idle cost.

C **Longest option**

The length heuristic is unreliable and is not a reasoning method. It happens to be wrong here as often as right.

D **Persistent EC2**

The lowest hourly rate for something running 24 hours a day to do 80 minutes of work. Cheapest per hour is not cheapest overall.

THE SIGNAL

Two steps in order: a hard limit eliminates, then the qualifier chooses. The stated duration is doing the first job, and 'MOST cost-effective' the second.

> A question asks how to give a third-party auditor read access to CloudTrail logs in an S3 bucket. The requirements state the bucket must not be publicly accessible and access must be revocable.
>
> One option reads: Enable public read access on the bucket and share the URL with the auditor.

What kind of distractor is that option?

A Right service, wrong feature

B Formerly correct, now outdated

C Works, but ignores the qualifier

D Solves the problem by dropping a stated requirement

◆ PAUSE AND DECIDE

What kind of distractor is that option?

A Right service, wrong feature

That type uses a real feature of the right service incorrectly, like DAX in front of RDS. Public access is not a misused feature here, it is a violated constraint.

B Formerly correct

Public buckets were never the documented way to share audit logs. This was not good practice at any point.

C Ignores the qualifier

That type is technically valid but loses on cost or overhead. This option fails a hard requirement, not a preference.

D Drops a stated requirement

The scenario says the bucket must not be publicly accessible. The option achieves access by removing that requirement, which is never permitted.

THE SIGNAL

Naming the type is faster than evaluating the option. Requirement-dropping distractors can be eliminated the moment you match them against the constraints.

> A question describes a Lambda function that must read a database credential rotating every 30 days. Two options remain after elimination: store it in Secrets Manager and fetch it per invocation, or store it in Secrets Manager and fetch it during initialisation.
>
> The question asks for the approach with the LOWEST latency. The scenario does not mention rotation timing beyond the 30-day period.

How should this be resolved?

A. Fetch during initialisation. The qualifier is latency, and a 30-day rotation tolerates a warm environment's lifetime

B. Fetch per invocation, because correctness always outranks latency

C. Neither. The question is ambiguous and should be flagged and skipped

D. Fetch per invocation, since it is the option that appears more often in AWS documentation

◆ PAUSE AND DECIDE

How should this be resolved?

A Initialisation, following the qualifier

The qualifier is LOWEST latency, and nothing in the scenario makes staleness a stated constraint. A 30-day rotation comfortably exceeds any environment's lifetime.

B Per invocation, correctness first

Imports a priority the question did not state. When the qualifier names latency, latency is the tiebreaker.

C Flag and skip

The question is not ambiguous once the qualifier is read. Skipping a resolvable question costs a mark you could have had.

D Documentation frequency

Not a reasoning method, and not true. Fetching at initialisation is the documented pattern for exactly this case.

THE SIGNAL

Module 19's scenario 4 had the opposite answer, because it stated a 7-day rotation and required rotated credentials to be picked up. The qualifier and the constraints changed, so the answer did.

> You are 95 minutes into the exam and have reached question 48. Eleven questions are flagged. Question 48 is a Choose TWO about DynamoDB indexes, and you are confident about one option and torn between two others.
>
> You have 35 minutes remaining.

What is the best use of the remaining time?

A Spend as long as needed on question 48, since Choose TWO questions are worth more

B Skip question 48 entirely and use the time on the eleven flagged questions

C Answer 48 with your best pair, flag it, finish the remaining questions, then review flags

D Change your earlier flagged answers to whichever now feels more likely

◆ PAUSE AND DECIDE

What is the best use of the remaining time?

A. Take as long as needed

A Choose TWO is worth the same as any other question. 35 minutes for 18 questions is already under two minutes each, so overspending here loses easy marks later.

B. Skip it entirely

Leaves a blank on a question you partly know, and there is no penalty for guessing. Never leave anything unanswered.

C. Answer, flag, finish, then review

Records a scored answer, preserves the option to reconsider, and prioritises reaching the 17 unseen questions over perfecting one.

D. Revise flagged answers on feeling

Changing without a nameable reason turns correct answers into incorrect ones more often than the reverse. It also abandons unseen questions.

THE SIGNAL

Unseen questions outrank flagged ones. A question you have not read is a guaranteed zero if you never reach it; a flagged one already has an answer recorded.

What to carry forward

- **Read the ask first, form an answer, then read the options.** The highest-value habit.
- **Find the deciding clause.** If you cannot name it, you have not found the constraint.
- **The qualifier is the tiebreaker** when two options both work. It is not decoration.
- Eliminate wrong platform, requirement-dropping and hand-built options first. It is fast.
- **Two minutes each.** Flag at 2.5, always answer before flagging, never leave a blank.

> ON THE EXAM
>
> Next: the revision gap, which gathers every currency note in this course into one place. Then three practice exams.

MODULE 31 · EXAM STRATEGY - ALL DOMAINS

The Revision Gap

Everything AWS changed after exam guide v2.1 - and what to answer anyway.

TYPE Currency MODULE 31 of 34 NOTES 22 across the course

admin@sthithapragna.com

When the documentation and the exam disagree, **answer the exam**. Then know why it disagrees, so you are not talked out of it.

Why the gap exists

- Exam questions are written against a **published guide**, then reviewed and validated.
- That process takes months, so the bank always trails the service by some distance.
- A change announced after the guide **cannot be the intended answer** to any question.
- So a newer fact is never wrong to know. It is simply not what is being asked.
- Most changes **raise a limit** or **add an option**, leaving the old answer still correct.

> ON THE EXAM
>
> That last point is why this list is shorter and less threatening than it looks. Very few changes make a previously correct answer actively wrong.

How this course flagged them

- Every currency note opened with **The examinable answer**, stated first.
- Then what changed, with a date, so you can check it against any source.
- Then an explicit instruction: **answer the old value on the exam**.
- They were placed at the point of teaching, not collected at the end.
- That ordering was deliberate. The exam answer comes first so it is what sticks.

> ON THE EXAM
>
> If you remember the exam answer and vaguely recall that something changed, the convention worked. That is exactly the state you want on exam day.

Changes that raise a limit

THE EXAMINABLE ANSWER	WHAT IS TRUE NOW
S3 object maximum is 5 TB	50 TB, since 2 December 2025
SQS message maximum is 256 KB	1 MiB, since 4 August 2025
Lambda async payload is 256 KB	1 MB, since January 2026
API Gateway integration timeout is 29 seconds	Raisable to 300 seconds, since June 2024
KMS automatic rotation is annual	90 days to 2,560 days, since 12 April 2024
DynamoDB transactions handle 25 items	100 items, since September 2022

ON THE EXAM

Every row here is safe. The old number is a subset of the new one, so answering the exam value is never contradicted by a scenario built on it.

The one that raises a limit and costs money

- **The examinable answer:** an SQS message caps at **256 KB**.
- Since 4 August 2025 it is **1 MiB**, on standard and FIFO queues alike.
- That removes the usual workaround of storing the body in S3 and sending a pointer.
- **Billing still meters in 64 KB chunks**, so a 1 MiB message is roughly 16 requests.
- The limit moved. The pricing model did not.

> ON THE EXAM
>
> Worth separating from the other limit changes because it has a cost consequence the others do not. A larger message is not one cheap request.

Changes that add an option

THE EXAMINABLE ANSWER	WHAT WAS ADDED
ElastiCache runs Redis or Memcached	Valkey, October 2024, now AWS's recommendation
Cognito uses the hosted UI	Managed Login, plus Lite/Essentials/Plus tiers, November 2024
Step Functions uses JSONPath	JSONata and Variables, 22 November 2024
Logs Insights has one query language	Plus OpenSearch PPL and SQL, December 2024
CloudWatch log groups have one class	Plus Infrequent Access, cheaper and more limited
ECS blue/green uses CodeDeploy	ECS native blue/green, 2025
Instrument tracing with the X-Ray SDK	ADOT and OpenTelemetry, now the recommended path

ON THE EXAM

The left column remains correct in every row. The addition is an alternative, not a replacement, which is why none of these threatens a mark.

The rename that matters

- **The exam calls it** advanced security features.
- **The console calls it** threat protection, since November 2024.
- Same capability: adaptive authentication, and compromised credentials detection.
- It now sits in the **Plus** tier rather than being a toggle.
- A rename is the most confusing kind of change, because both names describe real things.

> ON THE EXAM
>
> Renames are worse than new features for revision, because searching the current documentation for the exam's term returns nothing useful.

The other renames

THE OLDER NAME	THE CURRENT NAME
AWS CodeStar Connections	AWS CodeConnections
AWS SSO	AWS IAM Identity Center
Application Composer	Infrastructure Composer, October 2024
Amazon Kinesis Data Firehose	Amazon Data Firehose
Cognito advanced security features	Threat protection, in the Plus tier

ON THE EXAM

Exam questions may use either name. Recognising both is the whole task, and none of these represents a change in behaviour.

The change that costs money

- **The examinable answer:** initialisation is not billed for ZIP functions on managed runtimes.
- Since **1 August 2025**, INIT duration **is** billed for those functions.
- Container images and custom runtimes were already billed for it.
- So a slow cold start now costs money as well as latency.
- This one genuinely changes an answer to a cost question, so hold both facts.

> ON THE EXAM
>
> Unlike the limit increases, this changes the economics rather than the ceiling. It is the note most likely to make an old answer feel wrong.

Services that closed, and one that reopened

SERVICE	STATUS
S3 Select and Glacier Select	Closed to new customers, 25 July 2024. Still a correct exam answer
AWS Cloud9	Closed to new customers. Not reversed
Aurora Serverless v1	End of life, 31 March 2025. Clusters upgraded to v2
Amazon Linux 2	End of life, 30 June 2026, taking Beanstalk AL2 platforms with it
AWS CodeCommit	Closed June 2024, then REOPENED to GA in November 2025

READ THIS TWICE

The CodeCommit row is why you should be careful with 2024 and 2025 material: a great deal of it confidently describes a service that is now actively developed again.

Why the CodeCommit reversal matters

- AWS closed CodeCommit to new customers in **June 2024**.
- A great deal of 2024 and 2025 material describes it as dying or already dead.
- It was **reopened to general availability in November 2025**, with an active roadmap.
- So it is a live, developed service again, and a perfectly valid source action.
- **Answer normally on the exam.** Nothing about the exam's position changed.

> ON THE EXAM
>
> This is the one case where the intermediate material is confidently wrong in both directions. Trust the current position, and answer the exam normally.

The deprecation with dates attached

- **The examinable answer:** instrument with the X-Ray SDK, run the daemon on ECS and EC2.
- On **25 February 2026** the SDKs and daemon entered **maintenance mode**.
- Security fixes only, and **end of support in February 2027**.
- **The X-Ray service itself is fully supported** and actively being enhanced.
- Transaction Search and Application Signals are being built on it right now.

> ON THE EXAM
>
> Read the scope precisely. Only the instrumentation tooling is frozen, which is a much narrower statement than 'X-Ray is deprecated'.

Announced but not examinable

ANNOUNCEMENT	WHY IT IS NOT THE ANSWER
Lambda Durable Functions, Dec 2025	Long workflows in code. Still Step Functions
Lambda Managed Instances, Dec 2025	EC2-backed Lambda. Not in the guide
ECS native blue/green, 2025	Real. CodeDeploy remains examinable
CloudFormation IaC generator, 2024	Templates from existing resources
Stack refactoring, Feb 2025	Moves resources between stacks

ON THE EXAM

Durable Functions is the most dangerous row. It genuinely overlaps Step Functions, so a well-read candidate could talk themselves into the wrong answer.

The change still in progress

- **The examinable model:** exponential backoff with jitter, three attempts, standard mode.
- AWS has announced updated retry behaviour, opt-in now via **AWS_NEW_RETRIES_2026**.
- It changes backoff timing, retry quota costs and the DynamoDB defaults.
- It switches **Java, Python, Ruby, PHP, C++ and the CLI** from legacy to standard mode.
- **Default rollout is November 2026.** The principles are unchanged; only the numbers move.

> ON THE EXAM
>
> Included because it is the only change on this list that has not fully landed. The exponential-backoff-with-jitter principle is not going anywhere.

The timeline: 2024

WHEN	WHAT
April 2024	KMS rotation becomes configurable
June 2024	API Gateway timeout raisable to 300s. CodeCommit closes
July 2024	S3 Select closed to new customers
October 2024	ElastiCache adds Valkey
November 2024	Cognito Managed Login and tiers
November 2024	Step Functions JSONata. Logs Insights query languages

ON THE EXAM

November 2024 is the densest month, and it lands just before the guide was published on 12 December 2024.

The timeline: 2025 and 2026

WHEN	WHAT
March 2025	Aurora Serverless v1 end of life
August 2025	Lambda INIT billed. SQS payloads to 1 MiB
November 2025	API Gateway REST streaming. CodeCommit reopens
December 2025	S3 objects to 50 TB. Durable Functions announced
January 2026	Lambda async payloads to 1 MB
February 2026	X-Ray SDKs and daemon frozen. June: Amazon Linux 2 ends

ON THE EXAM

Everything on this slide postdates the guide entirely, which is why none of it can be the intended answer to a question.

What has not changed

- **The numbers that decide questions:** 15 minutes, 29 seconds, 5 GB, 400 KB, 4 KB.
- **The IAM evaluation algorithm.** Explicit deny wins, and cross-account needs both sides.
- **DynamoDB key design.** Partitions, hot keys, and the cost of a Scan.
- **Versions and aliases**, and everything built on traffic shifting.
- Almost everything this course spent its time on is in this list, not the previous one.

> ON THE EXAM
>
> The stable core is where the marks are. Twenty-two currency notes across thirty modules is a small fraction of the material, and none of it is foundational.

What is likely to change next

- **Limits rise.** Payload sizes, timeouts and object sizes have all moved and will again.
- **Instrumentation converges on OpenTelemetry.** The X-Ray SDK freeze is part of a pattern.
- **Managed replaces self-managed.** ECS native blue/green over CodeDeploy is one example.
- **Fundamentals do not move.** IAM evaluation, key design and the consistency model are stable.
- None of this is examinable. It is for the year after you pass.

> ON THE EXAM
>
> Included because a certification is a snapshot and the work is not. Knowing the direction of travel is what keeps the knowledge useful afterwards.

How to revise from current sources

- **AWS documentation is current, not examinable.** It describes today, not the guide.
- Blog posts dated 2024 or 2025 may describe a state that has since reversed.
- Check the **date** on anything you read, and check it against this timeline.
- If a source contradicts this course on a flagged item, it is probably describing now.
- The exam guide itself is the authority on what is in scope. It is short and worth reading.

> ON THE EXAM
>
> The practical habit: when documentation surprises you, check whether the surprise is on the timeline. If it is, you already know both answers.

The three that could actually cost you

- **Durable Functions**. A long workflow is **still Step Functions**. This is the likeliest trap.
- **S3 Select**. Still the correct answer, despite being unpurchasable since July 2024.
- **Cognito threat protection**. The exam says advanced security features. Same thing.
- Each is a case where a well-read candidate is more at risk than an under-read one.
- Everything else on the timeline either raises a limit or adds an option.

> READ THIS TWICE
>
> The common feature is that reading current material makes you less likely to answer correctly, which is exactly why the course flagged them rather than ignoring them.

A note on confidence

- Twenty-two flagged changes sounds like a lot. It is **22 out of several hundred facts**.
- Only three could plausibly cost a mark, and you have just read all three.
- The rest are safe: a raised limit never contradicts a question built on the old one.
- Do not let currency anxiety pull you away from the material that actually carries the marks.
- If you are unsure on exam day, answer what the guide would say. That is what is scored.

> ON THE EXAM
>
> This module exists so you are not surprised, not so you are worried. The stable core is where revision time belongs.

Every note, and where it was taught

MODULE	THE CURRENCY NOTE
03, 04	Lambda INIT billing. Scaling rebuilt. Managed Instances
06, 13	REST API streaming. Durable Functions, flagged twice
09	S3 objects to 50 TB. S3 Select closed
10, 11	Aurora Serverless v1 EOL. Valkey. The 29-second limit
12, 14	SQS and Lambda payloads. Amazon Linux 2 EOL
17, 18	Cognito Managed Login and tiers. KMS rotation
20, 22, 23	IaC generator. Pipeline V2. CodeCommit. ECS blue/green
25, 26, 27	Log classes and query languages. X-Ray freeze. Retry behaviour

ON THE EXAM

If any row is unfamiliar, that module is where to go back to. Each note sits beside the material it qualifies, which is where it makes sense.

Answer the guide. Know the gap. Do not let the second talk you out of the first.

What to carry forward

- **Answer the exam guide**, which is what the question bank was written against.
- **Most changes raise a limit or add an option**, leaving the old answer correct.
- **Three could cost you:** Durable Functions, S3 Select, and the threat protection rename.
- Lambda INIT billing is the one change that alters the economics rather than a ceiling.
- The stable core is unchanged, and it is where almost all the marks live.

> ON THE EXAM
>
> That completes the teaching and revision content. What remains is three practice exams, and the two opening modules recorded last.

MODULE 32 · EXAM STRATEGY - ALL DOMAINS

Practice Exam 1

Twenty questions, weighted to the real domain mix. Nine seconds to answer each.

TYPE Practice exam QUESTIONS 20 PAUSE 9 seconds

admin@sthithapragna.com

How to use this

- Each question gives you **nine seconds** before the answer. Pause if you need longer.
- **Commit to an answer out loud** before the walkthrough. A guess you did not commit to teaches nothing.
- Apply module 30: read the ask, find the deciding clause, check the qualifier.
- Track your score. Under 14 out of 20 means revisit before booking.
- The real exam gives you 2 minutes per question. Nine seconds is deliberately tight.

> ON THE EXAM
>
> The nine-second pause is for recognition, not deliberation. If a question needs longer, pause the video - that is a signal about where to revise.

A Lambda function backing a checkout API is invoked in bursts after marketing emails. p50 latency is 80 ms but p99 is 2.4 seconds. CloudWatch shows Init Duration present on the slow invocations only.

Traffic is predictable: the emails go out at 09:00 each weekday. Cost matters.

What should the team do?

A Configure provisioned concurrency with scheduled scaling for the 09:00 window

B Increase the function's reserved concurrency

C Increase the function timeout so slow invocations complete

D Enable provisioned concurrency permanently at the peak level

◆ PAUSE AND DECIDE

What should the team do?

A Scheduled provisioned concurrency

Init Duration on slow invocations identifies cold starts. Predictable timing means scheduled scaling pays only for the window that needs it.

B Reserved concurrency

Reserved concurrency is a cap, not an allocation of warm environments. It does nothing about cold starts.

C Longer timeout

The invocations are completing, just slowly. A timeout is not being hit.

D Permanent provisioned concurrency

Works, and pays for warm environments 24 hours a day to serve one predictable window. The cost clause rules it out.

THE SIGNAL

Two clues: Init Duration names the cause, and 'predictable' plus 'cost matters' selects scheduled over permanent.

A DynamoDB table storing IoT readings uses deviceId as the partition key and timestamp as the sort key. One device produces 80% of all traffic. The table is in on-demand mode and reports throttling, while overall consumed capacity is low.

Reads always request the most recent hour for a single device.

What change resolves the throttling?

A Switch to provisioned capacity with auto scaling

B Add a global secondary index on timestamp

C Add a write-sharding suffix to the partition key for the high-volume device

D Enable DynamoDB Accelerator in front of the table

◆ PAUSE AND DECIDE

What change resolves the throttling?

A Provisioned with auto scaling

Changes the table's capacity model. Per-partition limits apply in both modes, so the hot partition still throttles.

B GSI on timestamp

Timestamp is a poor partition key and the reads are already efficient. A GSI does not redistribute writes to the base table.

C Write sharding on the partition key

Spreads one device's writes across several partition keys, which is the only change that addresses a per-partition limit.

D DAX

Caches reads. The throttling is on writes concentrated in one partition, which a read cache cannot help.

THE SIGNAL

'Throttling while overall capacity is low' is the hot partition signature. No capacity mode removes a per-partition limit.

A web application lets authenticated users upload files of up to 2 GB. Uploads currently pass through an EC2 fleet, which is running out of disk. The S3 bucket must remain private and users must not receive long-lived credentials.

Uploads should not traverse the application servers at all.

What is the correct approach?

A Make the bucket public for PUT and validate uploads afterwards

B Have the application return a presigned PUT URL, and upload directly using multipart

C Issue each user an IAM user with scoped credentials

D Use a CloudFront signed URL for the upload

◆ PAUSE AND DECIDE

What is the correct approach?

A **Public bucket**

Contradicts 'must remain private'. Any option that solves a problem by making a bucket public is essentially always wrong.

B **Presigned PUT plus multipart**

The bucket stays private, no long-lived credentials are issued, the bytes bypass the servers, and multipart handles 2 GB with resumable parts.

C **An IAM user per user**

IAM users are long-lived credentials, which the scenario forbids, and the 5,000-user quota makes it unworkable.

D **CloudFront signed URL**

CloudFront signed URLs are download only. They cannot sign an upload at all.

THE SIGNAL

Three requirements, one mechanism. Option D is the direction trap: the pair splits by upload versus download.

> An API Gateway REST API with a Lambda proxy integration returns 502 Bad Gateway for every request. CloudWatch shows the function completing in 30 ms with no errors logged.
>
> The handler ends by returning a plain object containing the result data.

What is the cause?

A The function has been throttled

B The integration timeout has been exceeded

C API Gateway lacks permission to invoke the function

D The response does not match the proxy integration format

◆ PAUSE AND DECIDE

What is the cause?

A Throttled

Throttling returns 429 and the function would not run. This one completes in 30 ms.

B Timeout

A timeout returns 504, not 502, and 30 ms is nowhere near 29 seconds.

C Missing permission

Produces 500 with a permission error, and the function would never be invoked.

D Wrong response shape

Proxy integration requires statusCode and a string body. A bare object leaves the gateway unable to build a response, which is exactly what 502 means.

THE SIGNAL

The function succeeded and the API still failed, which places the fault between them. That is the definition of 502.

A Lambda function consumes an SQS queue. Its timeout is 90 seconds and processing typically takes 70. The queue's visibility timeout is 30 seconds and maxReceiveCount is 4.

Downstream systems report each record created four times, and the dead-letter queue is filling steadily despite almost no Lambda errors.

What should be changed?

A Raise the visibility timeout to at least six times the function timeout

B Increase maxReceiveCount so messages are not sent to the DLQ

C Switch the queue to FIFO to prevent duplicate delivery

D Reduce the function timeout to match the visibility timeout

◆ PAUSE AND DECIDE

What should be changed?

A Raise the visibility timeout

70 seconds of work under a 30-second visibility timeout means the message reappears mid-flight and is redelivered. Four duplicates matching maxReceiveCount of 4 confirms it.

B Raise maxReceiveCount

Would produce more duplicates, not fewer. It raises the threshold without addressing why messages are redelivered.

C Switch to FIFO

FIFO suppresses duplicate sends within five minutes. It does not stop redelivery caused by a short visibility timeout.

D Reduce the function timeout

Processing takes 70 seconds, so a 30-second timeout would fail every invocation outright.

THE SIGNAL

Four duplicates and maxReceiveCount of 4 is not a coincidence. The arithmetic identifies the cause before you read the options.

A loan approval workflow must pause after automated checks until a human underwriter approves or rejects it. Underwriters typically respond within a day but occasionally take four days.

No compute should run while waiting, and the workflow must resume automatically on the decision.

How should the pause be implemented?

A A Step Functions Wait state of 96 hours followed by a check

B A Step Functions Task using .waitForTaskToken, with the token sent to the underwriter

C An Express workflow that polls an approvals table

D A Lambda function that sleeps until approval arrives

◆ PAUSE AND DECIDE

How should the pause be implemented?

A Wait 96 hours then check

Every loan waits four days even when approved in ten minutes, and it fails if an underwriter takes slightly longer.

B .waitForTaskToken

The execution pauses with no compute billed, appears as awaiting callback, and resumes the moment SendTaskSuccess arrives. Standard workflows wait up to a year.

C Express polling

Express workflows stop at five minutes, so this cannot span four days.

D A sleeping Lambda

Lambda stops at 15 minutes and bills every second of sleeping. Paying a function to wait is the anti-pattern callbacks remove.

THE SIGNAL

Three requirements: no compute while waiting, an unpredictable duration, and automatic resumption. Only the callback pattern gives all three.

> Account A must read objects from a bucket in Account B encrypted with SSE-KMS using a customer managed key. The bucket policy allows the role in Account A, and that role's identity policy permits s3:GetObject and kms:Decrypt on the key ARN.
>
> Downloads fail with AccessDenied originating from KMS.

What is missing?

A Block Public Access must be disabled on the bucket

B The key must be converted to a multi-Region key

C The role must be granted s3:ListBucket as well

D The KMS key policy in Account B must explicitly allow the role in Account A

◆ PAUSE AND DECIDE

What is missing?

A Disable Block Public Access

It blocks anonymous and public access. A named role in a named account is neither.

B Multi-Region key

Multi-Region solves cross-Region, not cross-account. Nothing here involves two Regions.

C Add ListBucket

ListBucket is needed to enumerate objects, not to read one. The error also comes from KMS, not S3.

D The key policy must allow it

KMS uniquely requires an explicit allow in the key policy. An identity policy alone is never sufficient, even in the same account.

THE SIGNAL

The error originating from KMS rather than S3 is the clue: bucket access already works, and the key is refusing separately.

A mobile app must let users sign up with email or Google, then upload images directly to Amazon S3 from the device, with each user confined to their own prefix.

The team does not want to build a user directory and does not want uploads passing through their servers.

What should the team use?

A. A Cognito user pool only, sending the ID token to S3

B. A Cognito identity pool only, federating Google

C. A user pool for sign-in, feeding an identity pool whose role uses policy variables

D. An IAM role assumed directly by the mobile app using embedded credentials

◆ PAUSE AND DECIDE

What should the team use?

A User pool only

S3 does not accept Cognito JWTs. It requires SigV4-signed requests with AWS credentials, which a user pool does not issue.

B Identity pool only

Would cover Google sign-in, and email sign-up needs a directory. Identity pools hold no users of their own.

C Both pools, with policy variables

The user pool provides the directory and Google federation; the identity pool exchanges the token for credentials, scoped per user by policy variables.

D Embedded credentials

Anything shipped to a device can be extracted. Embedding AWS credentials in a mobile app is never acceptable.

THE SIGNAL

Two requirements force both halves: email sign-up needs a directory, and direct S3 access needs AWS credentials. Neither service does both.

> Compliance requires an Amazon RDS master password to rotate every 30 days. Several applications connect to the database and none may fail during the change. The team wants to write as little code as possible.
>
> Applications read the credential at startup and cache it.

How should this be implemented?

A. A scheduled Lambda function that changes the password and updates a Parameter Store SecureString

B. Secrets Manager managed rotation on a 30-day schedule, using the alternating users strategy

C. Secrets Manager managed rotation using the single user strategy

D. A Parameter Store advanced parameter with a no-change notification after 30 days

◆ PAUSE AND DECIDE

How should this be implemented?

A Scheduled Lambda

Rebuilds rotation by hand without staging labels or a test step, and the scenario asks for minimal code.

B Managed rotation, alternating users

RDS is covered by managed rotation so no code is needed, and alternating users keeps the previous credential valid, so cached credentials keep working.

C Managed rotation, single user

The strongest distractor. Single user rotation changes the password in place, leaving a window where a cached credential fails.

D No-change notification

Notifies that the password is stale. It does not rotate anything, which is the entire requirement.

THE SIGNAL

'Applications cache the credential' plus 'none may fail' is what selects alternating users over single user. Both are managed rotation.

> A developer's role has the AdministratorAccess policy attached. They work normally in eu-west-1 but every action in us-east-1 fails with AccessDenied. The account belongs to an AWS Organization.
>
> Attaching further policies to the role changes nothing.

What is the cause?

A. A service control policy denies actions outside approved Regions

B. AdministratorAccess does not cover us-east-1 by default

C. us-east-1 must be enabled for the account

D. The role's trust policy is missing the us-east-1 endpoint

◆ PAUSE AND DECIDE

What is the cause?

A An SCP denying Regions

Administrator access failing, and more permissions not helping, is the signature of an explicit deny. An SCP is the only thing that overrides AdministratorAccess.

B Region-limited policy

AdministratorAccess is Action star on Resource star with no Region condition. It covers every Region.

C Region not enabled

us-east-1 is enabled by default in every account. Some newer Regions are opt-in; this one is not.

D Trust policy

Trust policies govern who may assume a role, not where it may act. The role is clearly already assumed.

THE SIGNAL

Adding permissions changing nothing is the defining property of an explicit deny. Inside an Organization, that means an SCP.

> An ECS task on Fargate assumes a cross-account role to read a partner's S3 bucket. The role's maximum session duration is 12 hours and the code requests 12 hours. The same code run locally with a developer's IAM user credentials succeeds.
>
> In Fargate the AssumeRole call fails with an error about the requested duration.

What explains the difference?

A Fargate tasks cannot assume cross-account roles

B The task execution role needs sts:AssumeRole rather than the task role

C The task already runs as a role, so this is role chaining, which caps sessions at one hour

D The partner account must raise its maximum session duration

◆ PAUSE AND DECIDE

What explains the difference?

A Fargate cannot assume

It can, and does routinely. Nothing about Fargate prevents cross-account assumption.

B Execution role instead

The execution role starts the container; the task role is what application code uses. This would also produce access denied, not a duration error.

C Role chaining caps at 1 hour

The task runs as its task role, so assuming another role is a second hop. Chaining caps at one hour and requesting more fails outright rather than truncating.

D Partner raises the duration

It is already 12 hours. The cap being hit is the chaining cap, which no role setting can raise.

THE SIGNAL

Two clues: the error names duration rather than permission, and it works from an IAM user but not from a role. Only chaining explains both.

A CloudFormation template creates an IAM role and a Lambda function. The function's Role property is set with !Ref ExecutionRole. Stack creation fails with an error stating the role ARN is invalid.

The role itself is created successfully.

What is wrong?

A The role needs a DependsOn attribute so it is created first

B The role's trust policy is missing the Lambda service principal

C IAM roles cannot be referenced from a Lambda resource in the same template

D Ref on an IAM role returns the role name. The property needs !GetAtt ExecutionRole.Arn

◆ PAUSE AND DECIDE

What is wrong?

A Missing DependsOn

The Ref already creates an implicit dependency, and the role was created successfully. Ordering is not the issue.

B Missing service principal

That fails at invocation with a permission error, not at creation with a message about ARN format.

C Cannot reference in one template

This is the normal pattern and works fine. Templates reference roles from functions constantly.

D Ref gives the name, use GetAtt

The Role property requires an ARN. Ref on an IAM role returns the role name, so a bare name is passed where an ARN is expected.

THE SIGNAL

The error names the problem: it says ARN. Ref returned a name, and that distinction is the single most tested CloudFormation detail.

A payments Lambda function is deployed by AWS SAM. Deployments currently replace all traffic at once, and a recent bad release caused fifteen minutes of failures before anyone noticed.

The team wants a small share of traffic on the new version first, automatic reversion if an error alarm fires, and the smallest possible template change.

What should be added?

A AutoPublishAlias and a DeploymentPreference of Canary10Percent5Minutes, with the alarm listed

B An API Gateway canary release deployment on the stage

C Provisioned concurrency so new code warms before receiving traffic

D A second function and a Route 53 weighted record between them

◆ PAUSE AND DECIDE

What should be added?

A AutoPublishAlias plus DeploymentPreference

Four lines. SAM creates a CodeDeploy deployment group, shifts 10% for five minutes, and rolls back automatically when the alarm fires.

B API Gateway canary

Shifts traffic between API deployments, not Lambda versions, and does not roll back on a CloudWatch alarm.

C Provisioned concurrency

Addresses cold starts. Warm bad code fails exactly as reliably as cold bad code.

D Two functions and weighted DNS

Builds traffic shifting by hand, with DNS TTLs making rollback slow rather than immediate.

THE SIGNAL

Option B is the layer trap: a gateway canary and Lambda alias weights are different mechanisms at different layers.

A new CodeDeploy deployment group targets EC2 instances that previously ran a different application deployed by CodeDeploy. The first deployment fails immediately, reporting that the ApplicationStop lifecycle event failed on every instance.

The new revision's scripts have been tested and work correctly.

What is the cause?

A The CodeDeploy agent is not installed

B The new appspec.yml contains invalid YAML

C The instance profile cannot read the artifact bucket

D ApplicationStop runs the previous revision's script, which is stale or broken

◆ PAUSE AND DECIDE

What is the cause?

A Agent not installed

Without the agent the deployment hangs and times out rather than reporting a specific lifecycle event failure. Naming ApplicationStop proves the agent is running.

B Invalid YAML

That fails before any lifecycle event runs, and it contradicts the statement that the new scripts work.

C Bucket permissions

That fails at DownloadBundle, which comes after ApplicationStop. The failure would name a different event.

D Previous revision's script

ApplicationStop runs from the last successfully deployed revision, because the new bundle is not on the instance yet. Old scripts are the cause.

THE SIGNAL

'Previously ran a different application deployed by CodeDeploy' is the planted clue. Those instances still hold that application's stop script.

A pipeline has a source stage, a CodeBuild stage and a CodeDeploy stage. The build reports success and all tests pass. The deploy stage then fails, reporting that the input artifact contains no deployable files.

The build writes compiled output to a build directory and the appspec file to the repository root.

What is the most likely cause?

A The CodeDeploy service role cannot read the artifact bucket

B The build and deploy stages are in different Regions

C The buildspec artifacts section does not list the build directory and the appspec file

D The build stage timed out before writing its output

◆ PAUSE AND DECIDE

What is the most likely cause?

A **Role cannot read the bucket**

That produces access denied, not a report that the artifact is empty. The artifact was read successfully.

B **Different Regions**

A cross-Region artifact problem fails on retrieval with a bucket error, and would have failed pipeline validation earlier.

C **Artifacts section incomplete**

CodeBuild uploads only what the buildspec artifacts section lists. A green build producing an empty artifact is this, essentially every time.

D **Build timed out**

A timeout fails the build. This build reported success, and CodeBuild does not upload partial output.

THE SIGNAL

'Build green, deploy finds nothing' is the single most recognisable CI/CD symptom. The build did its job and was not asked to keep anything.

> An API Gateway integration points at a Lambda function using the unqualified function ARN. A developer deploys a code fix, and production begins returning errors within seconds. Reverting requires redeploying the previous code, taking several minutes.
>
> The team wants deployments to reach production only when they choose.

What change prevents this?

A Pin the integration to a specific published version number

B Publish a version on each deployment and point the integration at an alias, updated separately

C Enable provisioned concurrency so the previous code stays warm

D Add reserved concurrency to isolate the production function

◆ PAUSE AND DECIDE

What change prevents this?

A Pin to a version

Fixes the immediacy and creates a new problem: the integration never moves, so every release means editing and redeploying the API.

B Publish a version, point at an alias

Code updates change $LATEST only, so nothing reaches production until the alias moves. Reverting is one UpdateAlias call.

C Provisioned concurrency

Addresses cold starts. It has no bearing on which code version is invoked.

D Reserved concurrency

Caps how many invocations run at once, not which version they run.

THE SIGNAL

An unqualified ARN resolves to $LATEST, so every deployment is instant and irreversible. The alias is what sits between deployment and production.

> An application writes a log line containing OrderFailed whenever an order cannot be processed. Operations want an alert when more than twenty occur in five minutes, and want the trend on a dashboard.
>
> The application code cannot be modified.

What should be configured?

A. A subscription filter to a Lambda function that counts occurrences and publishes an SNS message

B. A scheduled Logs Insights query that counts matching lines

C. Modify the application to call PutMetricData on each failure

D. A metric filter on the log group, with a CloudWatch alarm on the resulting metric

◆ PAUSE AND DECIDE

What should be configured?

A Subscription filter plus Lambda

Rebuilds a metric filter by hand, including counting and alerting, with a function to run and pay for.

B Scheduled Logs Insights

Logs Insights is interactive and billed by data scanned. Running it on a schedule is expensive and delays the alert.

C Modify the application

The scenario states the code cannot be modified.

D Metric filter plus alarm

A pattern produces a metric, the alarm watches it, and the metric charts on a dashboard with no further work.

THE SIGNAL

The question wants a number, not the events. That is the metric filter side of the pair every time.

A support team must retrieve the X-Ray trace for a specific order when a customer reports a problem. The application attaches the order ID and the full order payload to each segment using put_metadata.

Searching for the order ID in the X-Ray console returns nothing.

What change is needed?

A Record the order ID as an annotation, keeping the payload as metadata

B Increase the sampling rate to 100% so every order is traced

C Move the order ID into the segment name

D Export traces to S3 and query them with Amazon Athena

◆ PAUSE AND DECIDE

What change is needed?

A Order ID as an annotation

Annotations are indexed and filterable, so the trace becomes retrievable. The bulky payload correctly stays as metadata.

B 100% sampling

Traces every order, and the traces still cannot be searched by order ID, because metadata is not indexed.

C Order ID in the segment name

Segment names identify services, not requests. A unique name per order would create thousands of service map nodes.

D Export and query with Athena

Builds a search system to work around an indexing feature that already exists and needs one line changed.

THE SIGNAL

Only annotations are indexed. The fix is not moving everything across, but putting the searchable field in the searchable place.

> A fleet of 300 instances calls a downstream service. A brief 20-second outage became a 15-minute one. Request volume increased eightfold during the incident and stayed high after the original fault was fixed.
>
> Each instance retries failed calls five times with a fixed 500 ms delay.

What should change?

A Increase the retry count so requests eventually succeed

B Use exponential backoff with jitter, and add a retry quota or circuit breaker

C Remove retries entirely so failures surface immediately

D Add instances to spread the load

◆ PAUSE AND DECIDE

What should change?

A More retries

Directly worsens it. The eightfold volume increase is caused by retries, so adding more amplifies the storm.

B Backoff with jitter plus a quota

A fixed delay synchronises 300 clients into waves. Jitter spreads them, backoff reduces the rate, and a quota or breaker stops retrying into a sustained failure.

C No retries

Overcorrects. Transient failures that would have self-corrected now become user-visible errors.

D More instances

Increases the number of clients generating retries, making the storm larger.

THE SIGNAL

'Fixed 500 ms delay' across 300 instances is the whole diagnosis: every client retries at the same instant, five times.

> An image resizing Lambda function is configured with 128 MB of memory and takes about 6 seconds per image. The function is CPU-bound. The team wants it faster and has been told to control costs, so they have avoided raising the memory.
>
> The function runs about 50,000 times a day.

What should the team do?

A. Keep 128 MB and invoke several functions in parallel per image

B. Enable provisioned concurrency so the function is always warm

C. Run Lambda Power Tuning and expect a higher memory setting to be faster and cheaper

D. Move the workload to a continuously running EC2 instance

◆ PAUSE AND DECIDE

What should the team do?

A Parallel functions

Each function still gets a fraction of a vCPU, so each piece is still slow. It adds orchestration to avoid a configuration change.

B Provisioned concurrency

Removes cold start latency. The 6 seconds is execution time, so this changes nothing and adds cost.

C Power Tuning, expect higher memory

CPU scales with memory, so a CPU-bound function at 128 MB is severely constrained. Finishing several times faster usually costs less in GB-seconds.

D Continuous EC2

Trades per-invocation billing for an always-running instance to do roughly 83 minutes of work a day.

THE SIGNAL

The premise is the error. In serverless you pay for time, so for a CPU-bound function more memory is frequently cheaper.

Scoring this exam

- **17 or more:** comfortable. Move to practice exam 2.
- **14 to 16:** passing territory. Review the domains you dropped marks in.
- **Below 14:** revisit the relevant modules before booking.
- Which domain you missed matters more than the total. Development is 32% of the real exam.
- Re-read any answer you got right but could not justify. That is a guess, not knowledge.

> ON THE EXAM
>
> Twenty questions is a small sample, so treat the domain pattern as the signal rather than the exact number. Two more exams follow.

MODULE 33 · EXAM STRATEGY - ALL DOMAINS

Practice Exam 2

Twenty fresh questions, same domain weighting, no repeats from exam 1.

TYPE Practice exam QUESTIONS 20 PAUSE 9 seconds

admin@sthithapragna.com

Before you start

- **Nine seconds** per question before the answer. Pause if you need longer.
- **Commit out loud.** An uncommitted guess teaches nothing.
- Read the ask, find the deciding clause, check the qualifier.
- Compare your domain breakdown against exam 1, not just the total.
- Two of these questions turn on a number. If you do not know it, note which.

> ON THE EXAM
>
> Comparing domain breakdowns across two exams is more informative than either score alone: a consistent weak domain is a revision target, a one-off dip is noise.

> A Lambda function processes batches of 100 records from an SQS queue. When one record fails, the entire batch returns to the queue and every record is reprocessed, causing duplicate downstream writes.
>
> The team wants only genuinely failed records retried, without reducing the batch size.

What should be configured?

A Set maxReceiveCount to 1 so failed batches go straight to the DLQ

B Enable partial batch responses and return ReportBatchItemFailures from the function

C Switch the queue to FIFO so ordering prevents reprocessing

D Increase the visibility timeout so the batch is not redelivered

◆ PAUSE AND DECIDE

What should be configured?

A maxReceiveCount of 1

Sends the whole batch to the DLQ on first failure, including the 99 records that succeeded. It loses data rather than retrying selectively.

B ReportBatchItemFailures

The function returns the identifiers of failed records only, and Lambda deletes the rest. Exactly the stated requirement, with no batch size change.

C FIFO

FIFO governs ordering and duplicate sends. It does not change how a partially failed batch is handled.

D Longer visibility timeout

Prevents premature redelivery. Here the batch is returned deliberately because it failed, which a longer timeout does not change.

THE SIGNAL

'Only genuinely failed records' with the batch size fixed describes partial batch responses precisely. Everything else trades one problem for another.

A DynamoDB table stores support tickets with customerId as the partition key and ticketId as the sort key. A new requirement is to query a customer's tickets by status, always within that same customer.

The table already exists in production with 40 million items. Reads must be strongly consistent.

What should be created?

A A local secondary index on status

B A global secondary index with status as the partition key

C A global secondary index with customerId as the partition key and status as the sort key

D A new table, because the requirement cannot be met on the existing one

◆ PAUSE AND DECIDE

What should be created?

A An LSI on status

An LSI supports strong consistency and the same partition key, which fits perfectly. It can only be created when the table is created, and this table already exists.

B GSI with status as partition key

Status has very low cardinality, producing a hot partition, and it would query across all customers rather than within one.

C GSI on customerId and status

Wrong on the consistency requirement. GSI reads are eventually consistent only, which the scenario rules out.

D A new table

The only way to add an LSI is at table creation, so a new table with the LSI and a migration is the remaining option.

THE SIGNAL

The trap is that A is the correct index type and cannot be added. LSIs are creation-time only, which turns this into a migration question.

A finance team reports that an S3 bucket's storage bill is 35% higher than the total size of the objects it contains. A mobile application uploads large files to the bucket and frequently loses connectivity mid-upload.

Listing the bucket accounts only for the expected object storage. Versioning is not enabled.

What should be configured?

A A lifecycle rule with AbortIncompleteMultipartUpload

B A lifecycle rule expiring noncurrent versions

C A lifecycle transition to S3 Standard-Infrequent Access

D S3 Transfer Acceleration to reduce failed uploads

◆ PAUSE AND DECIDE

What should be configured?

A AbortIncompleteMultipartUpload

Interrupted uploads leave parts that are billed but invisible to a listing, which explains storage that listing cannot account for. The rule removes them and prevents recurrence.

B Expire noncurrent versions

The scenario states versioning is not enabled, so there are no noncurrent versions to expire.

C Transition to Standard-IA

Reduces cost per gigabyte and leaves the invisible storage in place. It treats the symptom while the leak keeps growing.

D Transfer Acceleration

Speeds uploads and costs more, not less. It also does nothing about parts already accumulated.

THE SIGNAL

Three clues converge: storage a listing cannot see, large uploads, and dropped connections. Only incomplete multipart parts fit all three.

> A company sells tiered access to a public API: free at 10 requests per second and 10,000 per month, and paid at 500 per second with no monthly cap. Excess requests must be rejected before the backend is invoked, and per-customer usage must be reportable.
>
> Customers already authenticate with a Cognito user pool.

What should be built?

A An HTTP API with a JWT authoriser and per-route throttling

B A Lambda authoriser counting requests per customer in DynamoDB

C A REST API with usage plans and API keys, alongside the Cognito authoriser

D AWS WAF rate-based rules, one per tier

◆ PAUSE AND DECIDE

What should be built?

A HTTP API with route throttling

HTTP APIs have no API keys and no usage plans, so per-customer quotas are impossible. Route throttling applies to everyone equally.

B Lambda authoriser counting

Rebuilds usage plans by hand, invokes a function on every request including rejected ones, and adds a DynamoDB write to the request path.

C REST API with usage plans

Usage plans express rate, burst and monthly quota per key, reject at the gateway, and produce per-key usage reports. Cognito still handles authentication.

D WAF rate-based rules

WAF rate rules work on IP addresses over five-minute windows. They cannot express a per-customer monthly quota.

THE SIGNAL

The Cognito detail is a distractor. Authentication and metering are separate jobs, and this API needs both.

> An order event must reach three independent consumers: fulfilment, analytics and a partner webhook. Each processes at a different speed. The analytics consumer is regularly offline for maintenance for up to two hours.
>
> No event may be lost for any consumer, and adding a fourth consumer must not require changing the publisher.

What architecture should be used?

A One SQS queue polled by all three consumers

B An SNS topic with all three Lambda functions subscribed directly

C The publisher calling each consumer in turn

D An SNS topic with a separate SQS queue subscribed for each consumer

◆ PAUSE AND DECIDE

What architecture should be used?

A One shared queue

A queue message is consumed by one consumer. Three consumers polling one queue means each event is processed once, by whichever gets there first.

B SNS direct to Lambda

The strongest distractor. SNS does not store messages, so two hours of analytics downtime loses those events once retries are exhausted.

C Direct calls

Recouples everything. The publisher now knows all three consumers, and a slow one blocks order publication.

D SNS fan-out to per-consumer queues

Each consumer gets a durable backlog, so the analytics outage is survivable, and a fourth consumer means a fourth queue with no publisher change.

THE SIGNAL

'Offline for two hours' and 'no event may be lost' together demand durability. SNS provides the fan-out; only the queues provide the backlog.

> A workflow must process roughly 8 million objects listed in an S3 bucket, running an ECS task on each. Corrupt objects should not stop the run, and the team needs a report of which failed.
>
> The object list is far too large to pass as workflow input.

How should this be orchestrated?

A A Distributed Map state reading the object list from S3, with a tolerated failure threshold

B An inline Map state over an array of all object keys

C A Parallel state with one branch per object

D An Express workflow triggered once per object by S3 event notifications

◆ PAUSE AND DECIDE

How should this be orchestrated?

A Distributed Map from S3

Reads items directly from S3 so nothing passes through state, scales to 10,000 child executions, and the tolerated failure threshold handles corrupt objects.

B Inline Map

The array would have to fit in the 256 KB state payload. Eight million keys is orders of magnitude past that.

C Parallel with one branch per object

Parallel branches are defined statically in the template. You cannot write eight million, and any failure fails the state.

D Express per object

Gives no aggregate view of the run and no failure report, and loses the tolerated failure behaviour entirely.

THE SIGNAL

'Far too large to pass as workflow input' names the 256 KB state payload limit without stating it. Only Distributed Map reads items from S3 instead.

A platform team wants application developers to create IAM roles for their own Lambda functions without waiting for approval. Security requires that no role a developer creates can ever exceed read access to S3 and write access to one DynamoDB table.

Developers must not be able to escalate to administrator, directly or indirectly.

How should this be implemented?

A Grant iam:CreateRole and review each created role manually

B Apply a service control policy to the account limiting all principals

C Grant iam:CreateRole with a condition requiring a permissions boundary on every role created

D Pre-create a fixed set of roles and let developers select one

◆ PAUSE AND DECIDE

How should this be implemented?

A CreateRole plus manual review

Manual review is not a control. A developer can create and use a privileged role before anyone looks at it.

B SCP on the account

Would cap every principal in the account, including the platform team and production workloads. Far too blunt for delegation.

C CreateRole conditioned on a boundary

Effective permissions are the intersection of the identity policy and the boundary, so a role carrying the boundary cannot exceed it whatever policy is attached.

D Fixed set of roles

Removes the escalation risk by removing the delegation, which reintroduces the approval wait the scenario is trying to eliminate.

THE SIGNAL

Delegation plus a hard ceiling on identities the developers create themselves is the textbook permissions boundary case.

A multi-tenant SaaS application uses a Cognito user pool behind an API Gateway Cognito authoriser. Every Lambda function currently queries DynamoDB on each request to discover the caller's tenant, adding latency and cost.

Tenant membership changes rarely. The value must reach the backend without a lookup and must be tamper-proof.

What should be implemented?

A Have the client send the tenant ID in a custom header

B Add a pre token generation trigger that injects the tenant ID as a claim

C Cache the tenant lookup in ElastiCache keyed by user ID

D Replace the Cognito authoriser with a Lambda authoriser that caches the lookup

◆ PAUSE AND DECIDE

What should be implemented?

A Client-supplied header

A client-controlled value is not tamper-proof. Any caller could claim another tenant, which is a cross-tenant data breach.

B Pre token generation trigger

The tenant becomes a signed claim in the token. No lookup at all, and the signature guarantees integrity. Rare membership changes make token lifetime acceptable.

C ElastiCache lookup

Faster than DynamoDB, and still a lookup on every request, plus a cache to run and invalidate.

D Lambda authoriser with caching

Works, and adds a function invocation and its own cache TTL to manage. More moving parts than putting the value in the token.

THE SIGNAL

'Tamper-proof' is the deciding word. A signed token claim cannot be altered by the client; a header can.

A team manages 4,000 configuration values across three environments: endpoints, timeouts, feature toggles and a handful of static third-party API keys. None of them rotates.

Applications must fetch all settings for their environment in one call, and the current cost of one Secrets Manager secret per value is unacceptable.

Where should these values live?

A Consolidate into a smaller number of large Secrets Manager secrets

B Environment variables set at deployment time by the pipeline

C Parameter Store advanced tier, for the higher parameter limit

D Parameter Store standard tier with a path hierarchy per environment, SecureString for the keys

◆ PAUSE AND DECIDE

Where should these values live?

A Fewer, larger secrets

Reduces cost by making configuration harder to change. Updating one timeout means rewriting a large JSON blob.

B Environment variables

Changing any value now requires a deployment, and the API keys end up readable by anyone who can describe the function.

C Advanced tier

Advanced costs about 5 cents per parameter per month, so 4,000 parameters is roughly 2,400 dollars a year. Worse than the problem.

D Standard tier with hierarchy

4,000 is within the 10,000 standard limit, standard is free, GetParametersByPath satisfies the one-call requirement, and SecureString covers the keys.

THE SIGNAL

Option C is the trap: 'thousands of parameters' sounds like it needs the higher tier, and 4,000 is well inside the standard limit of 10,000.

A company runs 250 servers in its own data centre. Each writes backups to Amazon S3 using an IAM user's access keys stored in a configuration file, unrotated for two years.

Security requires temporary, automatically rotating credentials. The servers cannot move to EC2, and the company operates its own certificate authority.

What should the company implement?

A IAM Roles Anywhere, with the existing CA registered as a trust anchor

B Store the access keys in AWS Secrets Manager and fetch them at startup

C Rotate the access keys automatically with a scheduled Lambda function

D Run an EC2 proxy that the on-premises servers call

◆ PAUSE AND DECIDE

What should the company implement?

A IAM Roles Anywhere

Exchanges an X.509 certificate for temporary credentials from STS. No stored keys, automatic rotation, and the CA already exists.

B Keys in Secrets Manager

Circular. Reading Secrets Manager requires AWS credentials, so the server still needs a stored credential to start.

C Automated key rotation

Improves a bad pattern rather than replacing it. The keys are still long-lived between rotations and still in a config file.

D EC2 proxy

Every backup byte now crosses a server you must run, scale and secure, to avoid a credential problem with a direct solution.

THE SIGNAL

Option B's circularity is the useful lesson: you cannot use an AWS service to bootstrap AWS credentials. The CA detail is the planted clue.

A Lambda function must encrypt 60 MB report files before writing them to Amazon S3, using a customer managed KMS key so that key use is auditable and revocable.

A first attempt called the KMS Encrypt API with the file contents and failed.

How should the encryption be implemented?

A Split the file into 4 KB chunks and call Encrypt on each

B Call GenerateDataKey, encrypt the file locally with the plaintext key, and store the encrypted data key alongside

C Use an asymmetric KMS key, which has no payload limit

D Request a quota increase for the KMS Encrypt payload size

◆ PAUSE AND DECIDE

How should the encryption be implemented?

A 4 KB chunks

60 MB is about 15,000 chunks, so 15,000 KMS calls per file. Slow, expensive, and it would throttle the account.

B Envelope encryption

One KMS call per file, encryption happens locally at full speed, and the stored encrypted data key is useless without KMS.

C Asymmetric key

Asymmetric keys have a smaller payload limit, not a larger one, and are far slower for bulk data.

D Quota increase

4 KB is a hard API limit, not an adjustable quota. No request changes it.

THE SIGNAL

Every wrong answer tries to make KMS encrypt the data. The design is that it never does: it encrypts the key that encrypts the data.

A team updated a CloudFormation stack to change a property on an RDS instance. CloudFormation replaced the instance, creating a new empty database and deleting the original. Several hours of data were lost.

The team wants certainty this cannot recur, and wants to see the impact of a change before applying it.

What should the team adopt?

A Enable termination protection on the stack

B Run drift detection before each deployment

C Create a change set before every update, and set both DeletionPolicy and UpdateReplacePolicy on the database

D Disable automatic rollback so failed updates leave resources in place

◆ PAUSE AND DECIDE

What should the team adopt?

A Termination protection

Protects against deleting the whole stack. This was an update, and the stack was never deleted.

B Drift detection

Finds changes made outside CloudFormation. This change came through CloudFormation, so there was no drift to find.

C Change set plus both policies

The change set shows Replacement: True in advance, and UpdateReplacePolicy: Snapshot preserves the old database even if replacement happens.

D Disable rollback

Affects what happens after a failure. This update succeeded exactly as CloudFormation intended, which is the problem.

THE SIGNAL

UpdateReplacePolicy is the crucial half. This was a replacement during an update, so DeletionPolicy alone would not have saved the data.

A security baseline stack containing IAM roles and Config rules must be deployed into 80 AWS accounts across 3 Regions. New accounts join the organisation most weeks and must receive the baseline without manual action.

All accounts belong to one AWS Organization.

How should this be deployed?

A. A service-managed StackSet targeting the OU, with automatic deployment enabled

B. A nested stack in the management account with one child per account

C. One stack in the management account exporting values the others import

D. A pipeline assuming a role in each account and running create-stack in a loop

◆ PAUSE AND DECIDE

How should this be deployed?

A Service-managed StackSet

One template across accounts and Regions, and automatic deployment adds new accounts to the OU without any action. Exactly the requirement.

B Nested stacks

Nested stacks are created in the same account and Region as the parent. They cannot span accounts at all.

C Exports and imports

Exports are Regional and same-account. Fn::ImportValue cannot read an export from another account.

D Pipeline looping create-stack

Rebuilds StackSets by hand, with 240 stacks to track, no failure tolerance and its own code to maintain.

THE SIGNAL

'New accounts must receive it without manual action' selects service-managed over self-managed. Only the Organizations model does automatic deployment.

An ECS service must run a full integration suite against new tasks using real HTTP requests before any production traffic reaches them. The tests take about five minutes.

Production traffic must be entirely unaffected while the tests run, and a failing test must abort the deployment.

How should this be configured?

A Run the tests in a CodeBuild stage before the deploy stage

B Use a Linear10PercentEvery1Minute configuration so failures affect few users

C Add a ValidateService hook that runs the integration suite

D Configure a test listener on a second port, with an AfterAllowTestTraffic hook running the tests

◆ PAUSE AND DECIDE

How should this be configured?

A CodeBuild stage before deploy

Tests a build artifact, not the deployed service. It cannot make real HTTP requests to tasks that do not exist yet.

B Linear traffic shifting

Limits the blast radius rather than preventing it. Production traffic reaches the new tasks immediately, which the scenario forbids.

C ValidateService hook

An EC2-only hook. It does not exist for ECS deployments.

D Test listener plus AfterAllowTestTraffic

The test listener serves the replacement task set on a separate port, so real requests reach it while production traffic does not. A failed hook aborts the deployment.

THE SIGNAL

'Real HTTP requests' plus 'production entirely unaffected' describes the test listener precisely. Option C is a hook from another platform.

A developer iterating on Lambda business logic waits about 80 seconds for sam build and sam deploy on every change. They make dozens of changes an hour and must test against real DynamoDB and SQS resources in a development account.

Local emulation is insufficient because the logic depends on real service behaviour.

What should the developer use?

A sam local start-api with the SDK pointed at the development account

B sam sync --watch, accepting that it bypasses CloudFormation during development

C A script calling the Lambda UpdateFunctionCode API directly

D Reduce the deployment package size to shorten the deploy

◆ PAUSE AND DECIDE

What should the developer use?

A sam local start-api

The scenario rules this out: local emulation is explicitly insufficient because the logic depends on real service behaviour.

B sam sync --watch

Purpose-built for this: pushes code changes in seconds, watches files automatically, and the CloudFormation bypass is acceptable in a development account.

C UpdateFunctionCode script

Hand-builds what sam sync does, without dependency handling or multi-function support, and with the same drift downside.

D Smaller package

Most of the 80 seconds is change set creation and execution, not upload. A smaller package barely moves it.

THE SIGNAL

'Development account' is what makes B acceptable. The same answer would be wrong for production, because bypassing CloudFormation is the mechanism.

A company publishes internal npm packages under a scoped name. A security audit warns that an attacker could publish a package with the same name to the public registry at a higher version, and builds might resolve to it.

Builds must still install genuine public dependencies normally.

What should the company implement?

A Pin every dependency to an exact version in the lockfile

B Remove the external connection and vendor all public packages manually

C CodeArtifact with an external connection, and package origin controls blocking upstream versions of internal names

D Run npm audit in a build phase and fail on any finding

◆ PAUSE AND DECIDE

What should the company implement?

A Pin exact versions

Helps, and breaks the moment anyone adds or upgrades a dependency. It is a discipline rather than a control.

B Vendor everything

Eliminates the risk by eliminating the convenience, and contradicts the requirement to install genuine public dependencies.

C CodeArtifact origin controls

The named defence for dependency confusion. Internal names can never be fetched from upstream, while genuine public packages resolve normally.

D npm audit

Reports known vulnerabilities in real packages. A malicious package impersonating an internal name is not a known vulnerability.

THE SIGNAL

This attack has a name, dependency confusion, and a specific AWS answer. Recognising the shape is what makes the question quick.

> A central security team needs every application log event from 150 Lambda functions across 8 accounts searchable in one OpenSearch domain within a minute of being written.
>
> They do not need alarms on the log content, and the volume is high.

What should be configured?

A Subscription filters on the log groups, delivering to Amazon Data Firehose and on to OpenSearch

B A metric filter on each log group publishing to a shared namespace

C A nightly export task from each log group to S3, loaded into OpenSearch

D Move all log groups to the Infrequent Access class and query with OpenSearch SQL

◆ PAUSE AND DECIDE

What should be configured?

A Subscription filters to Firehose

Near real-time delivery of the events themselves, cross-account destinations are supported, and Firehose handles buffering at high volume.

B Metric filters

Produces counts, not events. The team needs to read and correlate the log lines themselves.

C Nightly export

The requirement is within a minute. A nightly job is off by three orders of magnitude.

D Infrequent Access plus SQL

Leaves logs in each account rather than centralising them, and IA does not support subscription filters at all.

THE SIGNAL

'Searchable' plus 'within a minute' means the events, delivered in near real time. That is the subscription filter definition.

An alarm watches the Errors metric of a Lambda function invoked a few times an hour. During a recent incident the function failed on every invocation for four hours, and the alarm never left INSUFFICIENT_DATA.

The alarm uses a one-minute period, one datapoint to alarm, and default missing data treatment.

What is the correct fix?

A Set missing data treatment to breaching

B Widen the period to match the invocation rate, and alarm on an error rate using metric math

C Switch the metric to high resolution

D Create a composite alarm combining Errors and Duration

◆ PAUSE AND DECIDE

What is the correct fix?

A Missing data as breaching

The tempting one. It would fire during the incident and also fire every quiet minute, because a function invoked hourly is missing data most of the time.

B Widen the period, alarm on error rate

A period long enough to contain invocations produces datapoints, and an error rate from Errors divided by Invocations is meaningful at any volume.

C High resolution

Publishes datapoints more often only when there is data. A function not invoked still produces nothing.

D Composite alarm

Combines alarms that already work. Both underlying metrics have the same gap, so combining them changes nothing.

THE SIGNAL

The mismatch is a one-minute period against an hourly invocation rate. Option A treats the symptom and would page the on-call engineer all night.

> A Lambda function processing orders at 3,000 invocations per second must record order value and processing duration per customer tier, alarm on average value by tier, and allow investigation of individual slow orders by customer ID.
>
> Latency is critical.

How should the instrumentation be done?

A Call PutMetricData once per invocation with customer ID and tier as dimensions

B Write plain log lines and create a metric filter per tier

C Call PutMetricData in a background thread after returning the response

D Emit Embedded Metric Format log lines, with tier as a dimension and customer ID as a property

◆ PAUSE AND DECIDE

How should the instrumentation be done?

A PutMetricData with customer as a dimension

Two problems: a synchronous call on every invocation at 3,000 per second, and customer ID as a dimension creates a billable metric per customer.

B Metric filters per tier

Workable for counts, and it means a filter per tier to maintain, and extracting numeric values like order value is awkward.

C Background thread

Lambda freezes the execution environment after the handler returns, so background work may never run.

D EMF, tier as dimension, customer as property

No API call so no added latency, tier is a low-cardinality dimension you can alarm on, and customer ID stays queryable without becoming a metric.

THE SIGNAL

The dimension-versus-property distinction is the whole question. Tier has a handful of values; customer ID has millions.

> Lambda functions are attached to a VPC to reach a private Aurora cluster. They also read and write Amazon S3 and DynamoDB heavily. The bill shows a large NAT gateway charge with high data processing costs.
>
> The functions call nothing outside AWS.

How should the networking cost be reduced?

A Remove the functions from the VPC entirely

B Move the NAT gateway to a single Availability Zone

C Add gateway VPC endpoints for S3 and DynamoDB, and remove the NAT gateway

D Replace the NAT gateway with a NAT instance on a small EC2 host

◆ PAUSE AND DECIDE

How should the networking cost be reduced?

A Leave the VPC

The functions need a private Aurora cluster, which requires VPC attachment. This removes the cost by breaking the requirement.

B Single-AZ NAT gateway

Reduces the fixed charge and keeps all the per-GB processing. It also creates an availability single point of failure.

C Gateway endpoints, drop the NAT

Gateway endpoints for S3 and DynamoDB are free with no per-GB charge, and with no external calls the NAT gateway is unnecessary entirely.

D NAT instance

Cheaper per hour and now something you patch, scale and monitor, and it still processes every byte.

THE SIGNAL

'Call nothing outside AWS' is what allows removing the NAT gateway altogether rather than merely making it cheaper.

Scoring this exam

- **17 or more:** comfortable. Move to practice exam 3.
- **14 to 16:** passing territory. Review where you dropped marks.
- **Below 14:** revisit those modules before booking.
- **Compare domain breakdowns.** A domain weak in both exams is a real gap.
- A domain weak in only one is more likely a sampling artefact than a knowledge gap.

> ON THE EXAM
>
> Two exams give you a pattern rather than a number. The questions you got right without being able to justify are still the ones to revisit.

MODULE 34 · EXAM STRATEGY - ALL DOMAINS

Practice Exam 3

The last twenty. Same weighting, no repeats from either previous exam.

TYPE Practice exam QUESTIONS 20 PAUSE 9 seconds

admin@sthithapragna.com

Before you start

- **Nine seconds** per question. Pause if you need longer.
- **Commit out loud** before the walkthrough.
- This set leans slightly harder on diagnosis than the first two.
- Three questions turn on something being **impossible** rather than merely worse.
- Afterwards, average your three scores by domain. That is your real picture.

> ON THE EXAM
>
> Questions where an option is impossible rather than suboptimal are the ones worth slowing down on: a hard constraint eliminates faster than any judgement call.

A new serverless application needs Lambda functions to run SQL against an Aurora PostgreSQL cluster. The functions also call a third-party payment API over the internet on every request.

The data access layer is being written from scratch. The team wants no standing infrastructure cost while idle and short cold starts.

How should the functions reach the database?

A Attach the functions to the VPC and connect directly to the cluster endpoint

B Attach the functions to the VPC and connect through RDS Proxy

C Use the Aurora Data API over HTTPS, authorised with IAM

D Attach the functions to the VPC, add a NAT gateway, and use RDS Proxy

◆ PAUSE AND DECIDE

How should the functions reach the database?

A Direct VPC connection

Breaks the payment API call, because a VPC-attached function has no internet route without a NAT gateway. It also exhausts connections at concurrency.

B RDS Proxy in a VPC

Solves connections, still breaks the payment API, and adds a per-hour charge the team explicitly does not want while idle.

C Aurora Data API

No VPC, so internet access survives and cold starts stay short. No standing cost, and the code is being written from scratch anyway.

D RDS Proxy plus NAT gateway

Works fully, and adds two always-on billed components. It is the right answer to a question with different constraints.

THE SIGNAL

Three clauses each favour the Data API: an internet call, no idle cost, and code written from scratch. Option D is correct engineering for a different scenario.

> A DynamoDB table of 30 million orders is queried by a reporting job that filters on orderStatus and returns about 200 items. The job currently uses a Scan with a FilterExpression and consumes enormous read capacity.
>
> orderStatus has six possible values. The reporting job runs hourly.

What is the most efficient change?

A Create a GSI with orderStatus as the partition key and query it

B Add a FilterExpression on the Scan to reduce items returned

C Increase provisioned read capacity to absorb the Scan

D Enable DAX so repeated Scans are served from cache

◆ PAUSE AND DECIDE

What is the most efficient change?

A GSI on orderStatus

A Query on a GSI reads only matching items. Six values is low cardinality but acceptable for an hourly reporting job, and it replaces reading 30 million items with reading 200.

B Add a FilterExpression

It already has one. A filter is applied after items are read, so capacity is consumed on all 30 million regardless.

C More read capacity

Pays for the inefficiency rather than removing it. A Scan of 30 million items stays a Scan of 30 million items.

D DAX

Caches results, and the underlying Scan still runs on a cache miss. It also does not help an hourly job with changing data.

THE SIGNAL

The key fact is that FilterExpression runs after the read. Capacity is consumed on everything scanned, not on what is returned.

A single-page application uploads files to S3 using presigned PUT URLs generated by an API. Testing the same URL with curl succeeds every time. From the browser, the request fails before it reaches S3, and the browser console reports a cross-origin error.

The signature and expiry are confirmed correct.

What is missing?

A The presigned URL must be regenerated with a longer expiry

B The bucket must have Block Public Access disabled

C The signing role needs additional S3 permissions

D A CORS configuration on the bucket allowing the application's origin and the PUT method

◆ PAUSE AND DECIDE

What is missing?

A Longer expiry

The scenario confirms the expiry is correct, and curl succeeds with the same URL, which proves the signature is valid.

B Disable Block Public Access

A presigned URL carries the signer's permissions and is not public access. Block Public Access is not involved.

C More permissions

The same URL works in curl, which proves the signing role's permissions are sufficient.

D A CORS configuration

Works in curl but not in a browser is the CORS signature. The browser blocks the request itself, before S3 ever sees it.

THE SIGNAL

'Works in curl, fails in the browser' is diagnostic. Curl does not enforce CORS; the browser does, and the block happens client-side.

A developer adds request validation to an API Gateway REST API and changes an integration to a new Lambda alias. Both changes are saved and appear correctly when the method is reopened.

Calls to the production URL still reach the old alias and still accept invalid payloads. CloudWatch shows no errors of any kind.

What is the explanation?

A The API Gateway cache is serving stale responses

B The changes were saved but never deployed to the stage

C Request validation applies only to GET methods

D The Lambda alias needs a resource-based policy before it can be invoked

◆ PAUSE AND DECIDE

What is the explanation?

A Stale cache

Caching affects responses, not which alias is invoked, and it would not make invalid payloads pass validation.

B Never deployed

Configuration is saved against the API; a stage serves a deployment snapshot. Until a new deployment is created, the stage keeps serving the old one.

C Validation is GET-only

False. Body validation is most useful on POST and PUT, which are the methods carrying bodies.

D Missing resource policy

That produces visible errors and permission failures. The scenario states calls succeed against the old alias.

THE SIGNAL

Settings correct, behaviour unchanged, no errors anywhere. Nothing is broken - the change simply is not live.

> Clickstream events must be consumed by three independent systems: real-time personalisation, a fraud detector and an hourly warehouse loader. Each must see every event. After a bug, the team must be able to reprocess the last 24 hours from the beginning.
>
> Volume is around 20,000 events per second.

What should carry the events?

A One SQS standard queue polled by all three systems

B Three SQS queues, with the producer writing to each

C Amazon Kinesis Data Streams, with each system as a separate consumer

D An SQS FIFO queue to guarantee each system sees events in order

◆ PAUSE AND DECIDE

What should carry the events?

A One shared queue

A queue message is consumed once. Three consumers polling one queue means each event is processed by whichever gets there first.

B Three queues from the producer

Each system sees every event, and messages are deleted once consumed, so there is nothing to reprocess after a bug.

C Kinesis Data Streams

Records persist for the retention period, so multiple consumers each read the full stream and can replay from any point in the last 24 hours.

D FIFO queue

Adds ordering and caps at 3,000 messages per second with batching, well under 20,000. It also still deletes consumed messages.

THE SIGNAL

'Reprocess from the beginning' is the deciding clause. Queues delete on consumption; a stream retains, which is the whole difference.

> A telemetry pipeline runs 6,000 short workflows per second. Each runs four steps and finishes in under two seconds. Occasional duplicate processing is acceptable because the downstream store is idempotent.
>
> Cost at this volume is the primary concern.

Which Step Functions workflow type should be used?

A. A Standard workflow, one execution per event

B. A Standard workflow with a Distributed Map over batches

C. A Standard workflow with the callback pattern

D. An asynchronous Express workflow, one execution per event

◆ PAUSE AND DECIDE

Which Step Functions workflow type should be used?

A Standard per event

Standard is priced per state transition. Four transitions times 6,000 per second is an enormous bill for work finishing in two seconds.

B Distributed Map over batches

Distributed Map suits a large finite dataset, not a continuous stream. It adds orchestration overhead to a steady flow.

C Standard with callbacks

Callbacks are for waiting on an external decision. Nothing here waits, and it keeps the per-transition pricing.

D Asynchronous Express

Under five minutes, at-least-once is explicitly acceptable, and duration-based pricing is far cheaper at this volume. Exactly what Express is for.

THE SIGNAL

Every clause points at Express: under 5 minutes, duplicates acceptable, cost primary. 'At-least-once is fine' is the permission slip.

A monitoring vendor assumes a role in each of its customers' AWS accounts. It holds thousands of role ARNs. A review notes that if an attacker learns another customer's role ARN and persuades the vendor's platform to use it, the vendor's legitimate credentials would satisfy that trust policy too.

The vendor's platform runs from a fixed set of IP addresses.

What should each customer's trust policy require?

A. A condition on sts:ExternalId, with a unique secret issued per customer

B. A condition on aws:SourceIp restricting to the vendor's addresses

C. A condition on aws:MultiFactorAuthPresent

D. A permissions boundary on the role the vendor assumes

◆ PAUSE AND DECIDE

What should each customer's trust policy require?

A ExternalId per customer

Purpose-built for this. The vendor must present a secret only that customer issued, so knowing a role ARN alone is not enough.

B SourceIp restriction

The well-built distractor. The attack comes from the vendor's own platform at the vendor's own addresses, so an IP condition passes.

C Require MFA

MFA is for humans. An automated platform assuming roles thousands of times cannot present an MFA code.

D Permissions boundary

Caps what the role can do once assumed. It does nothing about who may assume it, which is the vulnerability.

THE SIGNAL

The mention of fixed IP addresses is planted to make B attractive. The attack originates from those very addresses, so the condition passes.

A mobile app authenticates with a Cognito user pool and calls a backend API behind an API Gateway Cognito authoriser. The API needs the caller's identity and group membership. The developer currently sends the ID token.

A security review objects that user attributes including email and date of birth reach a service that does not need them.

What should change?

A Remove the attributes from the user pool schema

B Send the access token instead, which carries scopes and cognito:groups

C Encrypt the ID token before transmission

D Switch to an identity pool so the API receives AWS credentials

◆ PAUSE AND DECIDE

What should change?

A Remove the attributes

The application genuinely needs email and date of birth. Deleting them solves the transmission by destroying the data.

B Send the access token

The access token carries scopes and cognito:groups, exactly what the API needs, without the user attributes it does not.

C Encrypt the token

It already travels over TLS. The objection is that the API receives the attributes at all, not that they are intercepted.

D Identity pool

Identity pools issue AWS credentials for calling AWS services directly. The app calls its own API, so a user pool alone is correct.

THE SIGNAL

A direction pair. The ID token says who the user is, for your app; the access token says what they may do, for an API.

An external auditor needs to decrypt a specific set of exported files for a three-week engagement. The files are encrypted with a customer managed KMS key whose policy is change-controlled and reviewed quarterly.

Access must be limited to decryption, revocable at any moment, and must not require modifying the key policy.

How should access be granted?

A Add a statement to the key policy and remove it after three weeks

B Share the plaintext data key with the auditor

C Create a KMS grant scoped to Decrypt, and retire it when the engagement ends

D Create a second KMS key and re-encrypt the files under it

◆ PAUSE AND DECIDE

How should access be granted?

A Edit the key policy

Directly contradicts the requirement, and puts a temporary change through a quarterly change-control process twice.

B Share the plaintext data key

Genuinely dangerous. It hands over something that cannot be revoked, cannot be audited, and protects every file that key encrypted.

C A KMS grant scoped to Decrypt

Grants exist for exactly this: temporary, operation-scoped access created and retired at runtime, with no key policy change.

D Second key and re-encrypt

Works, at the cost of re-encrypting the dataset and maintaining a second key. Disproportionate to the requirement.

THE SIGNAL

Three requirements - temporary, operation-scoped, no policy change - describe a grant precisely. That combination is the tell.

An application on EC2 reads from S3 using access keys stored in a configuration file. A penetration test found that a server-side request forgery flaw in the application could be used to read instance metadata.

Security wants long-lived credentials removed and the metadata attack surface closed.

What should be done?

A Move the access keys into AWS Secrets Manager and fetch them at startup

B Encrypt the configuration file at rest on the instance

C Rotate the access keys on a schedule with a Lambda function

D Attach an instance profile, delete the keys, and require IMDSv2 on the instance

◆ PAUSE AND DECIDE

What should be done?

A Keys in Secrets Manager

Circular. Reading Secrets Manager requires AWS credentials, so the instance still needs a stored credential to start.

B Encrypt the file

The application must decrypt it to use it, so anything running as the application can still read the keys.

C Scheduled rotation

Improves a bad pattern rather than replacing it, and does nothing about the metadata attack surface.

D Instance profile plus IMDSv2

The instance profile removes stored keys entirely, and IMDSv2 requires a PUT for a session token, which most SSRF attacks cannot perform.

THE SIGNAL

Two requirements, one answer. The instance profile addresses the credentials, IMDSv2 addresses the SSRF path to them.

A company has 300 project teams, each owning its own S3 buckets and DynamoDB tables. Each team must access only its own resources, and new teams appear weekly. Maintaining 300 near-identical IAM policies has become unmanageable.

All resources already carry a Project tag.

What should be implemented?

A One IAM group per team, with the existing policies attached to groups

B An ABAC policy comparing aws:PrincipalTag/Project to aws:ResourceTag/Project

C A permissions boundary per team applied to that team's roles

D A separate AWS account per team with cross-account roles

◆ PAUSE AND DECIDE

What should be implemented?

A Groups with the same policies

Improves how policies are attached, not how many exist. You still maintain 300 near-identical documents.

B ABAC on tags

One policy serves every team, and a new team needs only correct tags rather than a new policy. Exactly the problem described.

C A boundary per team

Boundaries cap, they do not grant, so you would still need the 300 granting policies. It adds 300 more documents.

D An account per team

Used at scale, and an enormous change adding 300 accounts to govern. Far beyond what the problem requires.

THE SIGNAL

'Already carry a Project tag' is the planted detail. ABAC needs disciplined tagging, and the scenario tells you it exists.

A CloudFormation stack creates an S3 bucket that an application fills with objects. Deleting the stack fails with DELETE_FAILED on the bucket. A separate requirement is to seed a DynamoDB table with reference data at creation time.

Both must be handled inside the template rather than by manual steps.

What should the template use?

A. A Lambda-backed custom resource that empties the bucket on Delete and seeds the table on Create

B. DeletionPolicy: Retain on the bucket, and a Mappings section for the seed data

C. A stack policy denying bucket deletion, and cfn-init for the seed data

D. Termination protection, and a DependsOn between the table and the application

◆ PAUSE AND DECIDE

What should the template use?

A Lambda-backed custom resource

The standard answer for both halves: emptying a bucket on Delete and running one-off setup on Create are things CloudFormation cannot do natively.

B Retain plus Mappings

Retain avoids the failure by orphaning a bucket that still exists and still costs money. Mappings hold literal values and cannot write to a table.

C Stack policy plus cfn-init

Stack policies govern updates, not deletion. cfn-init runs on EC2 instances, and there are none here.

D Termination protection plus DependsOn

Termination protection makes deletion harder rather than possible, and DependsOn only orders resources.

THE SIGNAL

Two requirements, one mechanism. 'CloudFormation cannot do this natively' is the custom resource signal, and emptying a bucket is its most cited example.

A pipeline deploys to staging automatically. Compliance requires a named release manager to authorise the production deployment explicitly, and to be notified rather than watching the console.

If nobody responds, the deployment must not proceed. The team does not want to build a custom approval application.

How should this be implemented?

A A Lambda invoke action polling a DynamoDB approvals table

B Disable the production stage and enable it manually per release

C A stage condition checking a pipeline variable

D A manual approval action in its own stage, publishing to an SNS topic

◆ PAUSE AND DECIDE

How should this be implemented?

A Lambda polling DynamoDB

Rebuilds manual approval by hand, including notification, timeout and audit trail. It is the custom application the team ruled out.

B Disable the stage

Not an approval, and it leaves no record of who authorised the release, which is what compliance actually asked for.

C Stage condition on a variable

Conditions evaluate automatically against rules. There is no human in the loop and no notification.

D Manual approval with SNS

The pipeline pauses, SNS notifies, PutApprovalResult permission names who may approve, and it fails after seven days if nobody responds.

THE SIGNAL

The compliance angle makes D right rather than merely convenient: the approval is recorded, attributed, and governed by an IAM permission.

An Elastic Beanstalk environment runs a customer-facing API on twelve instances. Deployments must never reduce capacity, must never serve a mix of old and new versions, and must be reversible within seconds.

The team accepts a longer deployment and a temporarily larger fleet.

Which deployment policy should be used?

A Rolling

B Rolling with additional batch

C Immutable

D All at once

◆ PAUSE AND DECIDE

Which deployment policy should be used?

A Rolling

Reduces capacity while batches update, and serves a mix of versions during the deploy. Fails two requirements.

B Rolling with additional batch

The closest wrong answer. It maintains capacity, and still serves a mix of versions mid-deploy, and rollback means deploying backwards.

C Immutable

New instances in a separate ASG, swapped only when healthy. Capacity maintained, no version mixing, and rollback is terminating the new instances.

D All at once

Takes the environment down during the deploy, failing the capacity requirement outright.

THE SIGNAL

Three requirements, and 'reversible within seconds' is what eliminates B. Rolling has already replaced the old instances; immutable has not.

A team uses the AWS CDK in TypeScript. They must set a newly released property on an S3 bucket that the L2 Bucket construct does not yet expose. CloudFormation supports the property fully.

They rely on the L2 construct's grant methods elsewhere in the stack and do not want to abandon it.

How should they set the property?

A Replace the L2 Bucket with a CfnBucket and set every property manually

B Access the underlying CfnBucket via node.defaultChild and use addPropertyOverride

C Define the bucket in a separate raw CloudFormation template

D Wait for a future CDK release to add support

◆ PAUSE AND DECIDE

How should they set the property?

A Drop to CfnBucket entirely

Works, and discards the defaults and grant methods the scenario says they want. An escape hatch keeps both.

B defaultChild plus addPropertyOverride

Exactly what escape hatches are for. The L2 construct is kept, and the property is written straight into the synthesised template.

C Separate raw template

Splits one bucket out of the stack that references it, creating a cross-stack dependency for a single property.

D Wait

Not a solution. Escape hatches exist so that CDK library lag never blocks you.

THE SIGNAL

'Do not want to abandon the L2 construct' separates B from A. Both set the property; only one keeps the grant methods.

A Lambda function reads an API endpoint from an environment variable. A developer updates the variable to a new endpoint and confirms it is saved in the console. Traffic continues going to the old endpoint.

The function is invoked through an alias named live, which points at version 12. The deployment pipeline was not run.

Why has the change not taken effect?

A The variable changed on $LATEST only. Version 12 retains the value captured at publish time

B Environment variable changes require the function to be restarted

C The alias caches configuration and must be recreated

D Environment variables cannot be changed after a version is published

◆ PAUSE AND DECIDE

Why has the change not taken effect?

A Changed on $LATEST only

A version freezes its environment variables at publish time. Editing them changes $LATEST, and version 12 is unaffected.

B Needs a restart

Lambda has no restart concept, and new execution environments start constantly. They would pick up a change if the version had one.

C Alias caching

An alias is a pointer, not a cache. It resolves to version 12, which genuinely holds the old value.

D Cannot change after publishing

Half right in a misleading way. You cannot change them on a published version, and you can on $LATEST, which is what happened.

THE SIGNAL

Option D states a true fact and draws the wrong conclusion. The console accepted the edit because it applied to $LATEST, not to version 12.

A company's CloudWatch bill exceeds its entire Lambda compute cost. Investigation finds debug-level logging enabled in production across 400 functions, and no log group with a retention period set.

The team must reduce cost without losing the ability to debug production issues when they arise.

What should they do?

A Delete the existing log groups and let them be recreated

B Move all log groups to the Infrequent Access class

C Set retention on every log group, and set the Lambda application log level to INFO

D Remove the debug logging statements from the code and redeploy

◆ PAUSE AND DECIDE

What should they do?

A Delete the log groups

Frees storage once and changes nothing about ingestion, so the cost returns. It also destroys the history.

B Infrequent Access

Reduces the ingestion rate and loses metric filters, subscription filters and alarms. The class also cannot be changed back.

C Retention plus application log level

Retention removes accumulated storage; the log level stops debug lines being ingested at all. Both are configuration, and the level can be raised again when debugging.

D Remove the statements

Requires redeploying 400 functions, and permanently removes the debug logging the team explicitly wants available.

THE SIGNAL

Option D is the near miss: it fixes the cost and destroys the capability the scenario says must be retained.

An application runs as an ECS service on Fargate, instrumented with the X-Ray SDK. The identical code traces correctly when deployed as a Lambda function. On ECS no segments appear and the service is absent from the service map.

The task role carries the AWSXRayDaemonWriteAccess policy.

What is missing?

A Active tracing must be enabled on the ECS service

B The task execution role needs the X-Ray permissions instead

C Fargate does not support X-Ray tracing

D The X-Ray daemon is not running as a sidecar container in the task definition

◆ PAUSE AND DECIDE

What is missing?

A Enable active tracing

Active tracing is a Lambda function setting. ECS services have no equivalent toggle, because collection is the daemon's job.

B Task execution role

The execution role starts the container; the task role is what application code uses. Writing segments is application code.

C Fargate unsupported

Fargate supports X-Ray fully, using a sidecar container exactly as EC2-launch-type tasks do.

D No daemon sidecar

The SDK sends segments to UDP port 2000 and something must listen there. On ECS that is a daemon sidecar in the task definition, which is absent.

THE SIGNAL

'The same code works on Lambda' is the clue: instrumentation is fine, so the difference is collection. Lambda collects for you; ECS does not.

> An API Gateway REST API integrates with a Lambda function calling a slow reporting database. Some requests return 504 Gateway Timeout. The function's timeout is 60 seconds and its Duration metric shows successful invocations taking up to 48 seconds.
>
> The function itself reports no errors.

What is happening?

A. API Gateway's 29-second integration timeout fires while the function is still running

B. The function is being throttled and never runs

C. The function returns a malformed proxy response

D. The database connection is being refused

◆ PAUSE AND DECIDE

What is happening?

A The 29-second gateway timeout

Invocations run up to 48 seconds, well past the 29-second integration timeout. API Gateway gives up and returns 504 while the function carries on.

B Throttling

Throttling returns 429, not 504, and the Duration metric shows the function running.

C Malformed response

A bad proxy response produces 502, not 504. The two codes mean different things.

D Connection refused

Would produce a function error in the logs. The scenario states the function reports no errors.

THE SIGNAL

The layered timeout problem: the function timeout of 60 seconds is longer than the gateway's 29, so the inner limit never takes effect.

> A DynamoDB table serving a retail catalogue is provisioned for the annual peak. For most of the year utilisation sits below 8%, and the team pays for peak capacity every month.
>
> Day-to-day traffic is highly predictable, with two known annual spikes.

What is the best change?

A. Switch permanently to on-demand mode

B. Enable provisioned auto scaling for the daily pattern, and buy reserved capacity for the baseline

C. Add a DAX cluster to absorb the peak

D. Reduce provisioned capacity to the average and accept throttling at the peak

◆ PAUSE AND DECIDE

What is the best change?

A On-demand permanently

The strongest distractor. On-demand suits unpredictable traffic, and this is described as highly predictable, where provisioned is substantially cheaper.

B Auto scaling plus reserved capacity

Auto scaling follows the predictable daily pattern, and reserved capacity discounts the baseline that is always there. It matches both stated facts.

C Add DAX

Reduces read capacity needs and adds a standing cluster cost. It does nothing about paying for peak write capacity all year.

D Average capacity, accept throttling

Saves money by failing customers at exactly the moment revenue is highest.

THE SIGNAL

'Highly predictable' is the deciding phrase, and it points away from on-demand. Predictability is what provisioned mode and commitments reward.

Scoring, and what comes next

- **17 or more across all three:** you are ready to book.
- **14 to 16 consistently:** passing territory, with a domain or two to firm up.
- **Below 14:** average your domain scores and revisit the weakest two modules.
- A domain weak in all three exams is a real gap. Weak in one is sampling noise.
- Sixty questions is a sample, not a simulation. The real exam has 65 and two minutes each.

> ON THE EXAM
>
> That is the end of the course. Module 31's rule is the one to carry into the exam room: answer the guide, know the gap, and do not let the second talk you out of the first.